
DATA SCIENCE

Série 1 — Volume V

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume V : Risque et assurance

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment utiliser ce cahier	1
Numérotation et niveaux	1
Données et environnement	2
Lancer le code	3
Vérifier son installation	3
Mini-diagnostic de départ	4
Plan du cahier	5
Chapitre 1 : Risque de crédit et scoring — exercices et applications	7
Applications	7
Application 1.1 — Construire la grille complète (section 1.1)	7
Application 1.2 — L'inférence des rejets (section 1.1.7)	9
Application 1.3 — Trois modèles, un verdict (section 1.2)	10
Application 1.4 — Performance, seuil et stabilité (section 1.3)	11
Application 1.5 — WOE, IV et fusion monotone (section 1.4)	12
Application 1.6 — LGD et CCF (section 1.5)	13
Application 1.7 — Perte attendue IFRS 9 (section 1.5)	15
Application 1.8 — Matrice de migration (section 1.6)	16
Exercices	17
Exercice 1.1 ★ — Échelle de points (section 1.1)	17
Exercice 1.2 ★ — WOE et IV à la main (section 1.1.4)	17
Exercice 1.3 ★★ — Fenêtres et fuite d'information (section 1.1.2)	18
Exercice 1.4 ★ — Perte attendue et taux minimal (section 1.5)	18
Exercice 1.5 ★★ — Contraintes de monotonie (section 1.2.4)	18
Exercice 1.6 ★ — AUC par comptage (section 1.3.1)	18
Exercice 1.7 ★★ — Le Gini ne dépend pas de la proportion de mauvais (section 1.3.2)	18
Exercice 1.8 ★★ — PSI à la main (section 1.3.6)	18
Exercice 1.9 ★ — IV avec une classe sans mauvais (section 1.4.4)	18
Exercice 1.10 ★★ — IV d'une variable de bruit (section 1.4.2)	19
Exercice 1.11 ★ — Perte attendue d'un prêt (section 1.5.4)	19
Exercice 1.12 ★★ — L'effet de falaise (section 1.5.4)	19
Exercice 1.13 ★ — Une matrice à trois états (section 1.6.4)	19
Exercice 1.14 ★★ — Chapman–Kolmogorov et conjoncture (section 1.6.5)	19
Corrigés	19
Corrigé 1.1	19
Corrigé 1.2	20
Corrigé 1.3	20

Corrigé 1.4	20
Corrigé 1.5	20
Corrigé 1.6	21
Corrigé 1.7	21
Corrigé 1.8	21
Corrigé 1.9	22
Corrigé 1.10	22
Corrigé 1.11	22
Corrigé 1.12	22
Corrigé 1.13	23
Corrigé 1.14	23
Pistes pour les « À vous » des applications	24
Chapitre 2 : Modélisation actuarielle — exercices et applications	25
Applications	25
Préparation commune	25
Application 2.1 — Compter les sinistres : exposition et Poisson (section 2.1.2)	26
Application 2.2 — Sur-dispersion et binomiale négative (section 2.1.3)	26
Application 2.3 — La queue et la charge annuelle (sections 2.1.5 et 2.1.6)	27
Application 2.4 — Un tarif par deux GLM (section 2.2.2)	28
Application 2.5 — Valider hors période et mesurer l'antisélection (sections 2.2.5 et 2.2.6)	29
Application 2.6 — Chain ladder, de la main au code (section 2.3)	29
Application 2.7 — Déviance et crédibilité (section 2.4)	30
Application 2.8 — Bornhuetter-Ferguson, Mack et bootstrap (section 2.5)	31
Application 2.9 — Un portefeuille de santé (section 2.6)	31
Exercices	32
Exercice 2.1 ★ — Moyenne et variance de la charge (section 2.1.6)	32
Exercice 2.2 ★★ — Estimer une fréquence avec exposition (section 2.1.2)	32
Exercice 2.3 ★★ — Poisson contre binomiale négative (section 2.1.3)	32
Exercice 2.4 ★ — De la prime pure au prix (section 2.2.1 et 2.2.4)	32
Exercice 2.5 ★★ — Lire des relativités (section 2.2.2)	33
Exercice 2.6 ★★★ — Un Gini à la main (section 2.2.5)	33
Exercice 2.7 ★ — Chain ladder sur un petit triangle (section 2.3.3)	33
Exercice 2.8 ★★ — Le facteur de queue (section 2.3.4)	33
Exercice 2.9 ★★ — Test du rapport de vraisemblance (section 2.4.1)	33
Exercice 2.10 ★★★ — Crédibilité de Bühlmann-Straub (section 2.4.4)	33
Exercice 2.11 ★ — Bornhuetter-Ferguson et chain ladder (section 2.5.2)	33
Exercice 2.12 ★★ — La variance de développement de Mack (section 2.5.3)	34
Exercice 2.13 ★★★ — Diagnostiquer un choc calendaire (section 2.5.5)	34
Exercice 2.14 ★★ — Sélection et comportement en santé (section 2.6.3)	34
Corrigés	34
Corrigé 2.1	34
Corrigé 2.2	34
Corrigé 2.3	35
Corrigé 2.4	35
Corrigé 2.5	35
Corrigé 2.6	35
Corrigé 2.7	35
Corrigé 2.8	36
Corrigé 2.9	36
Corrigé 2.10	36

Corrigé 2.11	36
Corrigé 2.12	37
Corrigé 2.13	37
Corrigé 2.14	37
Chapitre 3 : Mesures de risque et stress tests — exercices et applications	38
Applications	38
Application 3.1 — La VaR et l'ES du portefeuille, trois façons (sections 3.1.3 à 3.1.7)	38
Application 3.2 — EWMA et GARCH filtré : la couverture selon le régime (section 3.1.5)	39
Application 3.3 — Simulation de Monte-Carlo : la loi change la queue (section 3.1.4)	40
Application 3.4 — Bootstrap d'un chiffre de queue (section 3.1.9)	41
Application 3.5 — Un stress macroéconomique de crédit (sections 3.2.4 à 3.2.7)	41
Application 3.6 — Un choc de taux sur un petit portefeuille d'obligations (section 3.2.6)	42
Application 3.7 — Perte opérationnelle agrégée : avec et sans queue lourde (sections 3.3.1 à 3.3.3)	42
Application 3.8 — Un backtest complet (sections 3.4.1 à 3.4.6)	43
Exercices	44
Exercice 3.1 ★ — VaR et ES sur vingt pertes (sections 3.1.2 et 3.1.7)	44
Exercice 3.2 ★ — VaR normale d'un portefeuille de deux actifs (section 3.1.3)	44
Exercice 3.3 ★★ — Vérifier numériquement l'ES normale (section 3.1.7)	44
Exercice 3.4 ★★ — Sous-additivité avec trois prêts (section 3.1.8)	44
Exercice 3.5 ★★ — La règle de la racine avec de la mémoire (section 3.1.6)	44
Exercice 3.6 ★ — Taux de défaut et perte d'un scénario (section 3.2.5)	45
Exercice 3.7 ★★ — Duration et convexité d'une obligation (section 3.2.6)	45
Exercice 3.8 ★★★ — Stress inversé et sensibilité à la LGD (section 3.2.7)	45
Exercice 3.9 ★★ — Contributions au risque (section 3.3.4)	45
Exercice 3.10 ★★ — Perte agrégée d'un modèle fréquence-sévérité (sections 3.3.1 et 3.3.3)	45
Exercice 3.11 ★ — Kupiec et feu tricolore (sections 3.4.2 et 3.4.6)	45
Exercice 3.12 ★★★ — Contrôler une expected shortfall (section 3.4.7)	45
Corrigés	46
Corrigé 3.1	46
Corrigé 3.2	46
Corrigé 3.3	46
Corrigé 3.4	47
Corrigé 3.5	47
Corrigé 3.6	47
Corrigé 3.7	48
Corrigé 3.8	48
Corrigé 3.9	49
Corrigé 3.10	49
Corrigé 3.11	49
Corrigé 3.12	50
Chapitre 4 : Cadre réglementaire — exercices et applications	51
Préparation	51
Applications	52
Application 4.1 — Le capital IRB d'un portefeuille, de la PD au poids de risque	52
Application 4.2 — Simuler la perte au quantile 99,9 % : granularité et corrélation	53
Application 4.3 — La procyclicité du capital	54

Application 4.4 — Agréger les modules du capital de solvabilité	55
Application 4.5 — Marge de risque et résistance aux chocs	56
Application 4.6 — La marge sur services contractuels, année par année	57
Application 4.7 — Comparer des modèles de Takaful : où les deux parties s'y re- trouvent	58
Application 4.8 — Détecter des opérations suspectes, de bout en bout	60
Exercices	61
Exercice 4.1 ★ — La formule IRB à la main (section 4.1.4)	61
Exercice 4.2 ★ — Ratios et coussins (section 4.1.2)	61
Exercice 4.3 ★★ — La perte attendue du modèle de Vasicek (section 4.1.4)	61
Exercice 4.4 ★★ — À partir de quelle PD l'IRB devient-il plus sévère que le standard ? (section 4.1.5)	61
Exercice 4.5 ★★ — Sensibilités du capital (sections 4.1.5, 4.1.6)	61
Exercice 4.6 ★★ — Un SCR à trois modules, à la main (section 4.2.4)	62
Exercice 4.7 ★★ — Une marge de risque, à la main (section 4.2.3)	62
Exercice 4.8 ★★★ — Queues et agrégation (section 4.2.5)	62
Exercice 4.9 ★ — Une CSM à la main (section 4.4.1)	62
Exercice 4.10 ★★ — Le taux de wakala qui convient aux deux parties (section 4.5.2)	62
Exercice 4.11 ★★ — Régler une règle de fractionnement (section 4.6.3)	62
Exercice 4.12 ★★★ — Le graphe complet ne désigne personne (section 4.6.4)	62
Corrigés	62
Corrigé 4.1	63
Corrigé 4.2	63
Corrigé 4.3	63
Corrigé 4.4	63
Corrigé 4.5	64
Corrigé 4.6	64
Corrigé 4.7	64
Corrigé 4.8	64
Corrigé 4.9	65
Corrigé 4.10	65
Corrigé 4.11	65
Corrigé 4.12	66
Chapitre 5 : ☉ Assurance vie — exercices et applications	67
Applications	68
Application 5.1 — Table de période des hommes et comparaison à la vérité (section 5.1)	68
Application 5.2 — Lisser les taux d'un portefeuille : bruts, Gompertz–Makeham, Whittaker–Henderson (section 5.1.4)	68
Application 5.3 — Rapport réel/attendu par sexe, contrat et capital (section 5.1.5)	70
Application 5.4 — Grille de tarifs et coût d'une tarification unisexe (section 5.2)	70
Application 5.5 — Provisions : prospectif, rétrospectif, changement de base (sec- tion 5.2.4)	71
Application 5.6 — Lee–Carter pour les hommes : ajustement, vérité et projection (section 5.3)	72
Application 5.7 — Rentes : risque de tendance contre risque individuel (section 5.3.4)	73
Application 5.8 — Validation hors période : plusieurs dates de coupure (section 5.3.4)	74
Exercices	74
Exercice 5.1 ★ — Du taux à la probabilité (section 5.1.1)	74

Exercice 5.2 ★ — Une table à la main (section 5.1.2)	74
Exercice 5.3 ★★ — Gompertz et doublement (section 5.1.4)	75
Exercice 5.4 ★★ — Combien de décès pour un A/E précis ? (section 5.1.5)	75
Exercice 5.5 ★ — Prime d'une temporaire de 3 ans (section 5.2.1)	75
Exercice 5.6 ★★ — Capital et rente avec une mortalité constante (section 5.2.2)	75
Exercice 5.7 ★★ — Sensibilité de la rente au taux technique (section 5.2.5)	75
Exercice 5.8 ★★ — Provision après un an, deux méthodes (section 5.2.4)	75
Exercice 5.9 ★★ — Identifiabilité et contraintes de Lee-Carter (section 5.3.1)	75
Exercice 5.10 ★★ — Une SVD de rang 1 sur une petite matrice (section 5.3.2)	76
Exercice 5.11 ★★ — Incertitude sur la dérive (section 5.3.3)	76
Exercice 5.12 ★★ — Diversification et tendance (section 5.3.4)	76
Corrigés	76
Corrigé 5.1	76
Corrigé 5.2	77
Corrigé 5.3	77
Corrigé 5.4	77
Corrigé 5.5	77
Corrigé 5.6	78
Corrigé 5.7	78
Corrigé 5.8	79
Corrigé 5.9	79
Corrigé 5.10	79
Corrigé 5.11	80
Corrigé 5.12	80
Chapitre 6 : ☉ Réassurance — exercices et applications	81
Applications	81
Application 6.1 — Répartir huit sinistres entre trois traités (sections 6.1.2 et 6.1.3)	81
Application 6.2 — Le coefficient de variation selon la rétention (section 6.1.1)	82
Application 6.3 — Le <i>burning cost</i> d'une tranche (section 6.2.1)	83
Application 6.4 — La courbe d'exposition (section 6.2.2)	83
Application 6.5 — Ajuster une GPD : stabilité selon le seuil (section 6.2.3)	84
Application 6.6 — Bootstrap : par années ou par sinistres ? (section 6.2.4)	85
Application 6.7 — Une tranche de catastrophe (section 6.2.5)	85
Application 6.8 — La prime de risque (section 6.2.6)	86
Application 6.9 — Comparer des programmes (section 6.3.2)	86
Application 6.10 — Contrepartie et épuisement (section 6.3.4)	87
Exercices	88
Exercice 6.1 ★ — Quote-part avec commission (section 6.1.2)	88
Exercice 6.2 ★ — Excédent de plénitude (section 6.1.2)	88
Exercice 6.3 ★ — Une tranche par risque (section 6.1.3)	88
Exercice 6.4 ★★ — Réintégrations (section 6.1.3)	88
Exercice 6.5 ★★ — Plafonner vaut mieux que grossir (section 6.1.1)	88
Exercice 6.6 ★ — <i>Burning cost</i> à la main (section 6.2.1)	88
Exercice 6.7 ★★ — Le levier de l'inflation (section 6.2.1)	89
Exercice 6.8 ★★ — Pareto et prix de tranche (section 6.2.2)	89
Exercice 6.9 ★★ — La formule GPD (section 6.2.3)	89
Exercice 6.10 ★★ — Les intervalles bootstrap couvrent-ils la vérité ? (section 6.2.4)	89
Exercice 6.11 ★★ — Quote-part ou excédent de sinistre ? (section 6.3.2)	89
Exercice 6.12 ★★ — Dimensionner un stop-loss (section 6.3.4)	89
Corrigés	89
Corrigé 6.1	89

Corrigé 6.2	90
Corrigé 6.3	90
Corrigé 6.4	90
Corrigé 6.5	90
Corrigé 6.6	90
Corrigé 6.7	91
Corrigé 6.8	91
Corrigé 6.9	91
Corrigé 6.10	91
Corrigé 6.11	92
Corrigé 6.12	92
Chapitre 7 : Gestion actif-passif et théorie du portefeuille — exercices et applications	93
Préparation	93
Applications	94
Application 7.1 — Prix, duration et convexité d'une obligation	94
Application 7.2 — Construire le passif d'un portefeuille d'assurance vie	95
Application 7.3 — Apparier l'actif au passif	96
Application 7.4 — Chocs de courbe et durations par maturité	97
Application 7.5 — La frontière efficiente sur cinq actifs	98
Application 7.6 — Contributions au risque et parité des risques	99
Application 7.7 — Estimation, instabilité et rétrécissement	100
Application 7.8 — Surplus, VaR et simulation sur dix ans	101
Exercices	103
Exercice 7.1 ★ — Prix et duration à la main (section 7.1.2 du livre)	103
Exercice 7.2 ★ — Variation du prix : duration et convexité (7.1.2)	103
Exercice 7.3 ★★ — Durations remarquables (7.1.2, 7.1.3)	103
Exercice 7.4 ★★ — L'écart de duration en euros (7.1.4)	103
Exercice 7.5 ★★ — Les conditions de Redington en chiffres (7.1.4)	103
Exercice 7.6 ★ — Rendement et risque d'un portefeuille (7.2.1)	103
Exercice 7.7 ★★ — Variance minimale à deux actifs (7.2.2)	103
Exercice 7.8 ★★ — Contributions au risque à la main (7.2.4)	104
Exercice 7.9 ★★★ — Le portefeuille tangent (7.2.3)	104
Exercice 7.10 ★★ — Combien d'années pour distinguer deux actifs ? (7.2.6)	104
Exercice 7.11 ★★★ — Précision d'une VaR à 99,5 % (7.3.3)	104
Exercice 7.12 ★★ — Échéancier de refixation d'une banque (7.1.5, 7.1.6)	104
Corrigés	104
Corrigé 7.1	104
Corrigé 7.2	105
Corrigé 7.3	105
Corrigé 7.4	105
Corrigé 7.5	105
Corrigé 7.6	106
Corrigé 7.7	106
Corrigé 7.8	106
Corrigé 7.9	107
Corrigé 7.10	107
Corrigé 7.11	107
Corrigé 7.12	108
Projet du volume et auto-évaluation — exercices et applications	109
Projet du volume	109

P.1 Le cahier des charges	109
P.2 Étape 1 : auditer les données et découper hors période	110
P.3 Étape 2 : la fréquence	111
P.4 Étape 3 : la sévérité et les gros sinistres	112
P.5 Étape 4 : la prime pure et sa validation hors période	113
P.6 Étape 5 : du modèle au tarif	115
P.7 Étape 6 : robustesse et capital	116
P.8 Étape 7 : les notes réglementaires	118
P.9 Étape 8 : le rapport	119
P.10 Les limites de l'étude	120
P.11 Variante : un score de crédit	120
Auto-évaluation	121
Risque de crédit (chapitre 1)	121
Modélisation actuarielle (chapitre 2)	121
Mesures de risque et stress tests (chapitre 3)	122
Cadre réglementaire (chapitre 4)	122
Assurance vie (chapitre 5)	122
Réassurance (chapitre 6)	122
Actif-passif et portefeuille (chapitre 7)	123
Corrigés des questions	123
Votre grille d'auto-évaluation	127

Mode d'emploi

« On comprend en lisant, on retient en calculant. »

Ce cahier est le **compagnon du livre** du volume V (*Risque et assurance*). Le livre explique les idées ; le cahier les fait travailler. Il contient, chapitre par chapitre, des **applications guidées**, des **exercices** et leurs **corrigés**, puis le **projet du volume** (construire un modèle de tarification avec validation et notes réglementaires) et l'auto-évaluation.

Comment utiliser ce cahier

1. **Lisez d'abord la section du livre.** Chaque section qui a un prolongement ici se termine par une ligne de ce genre : > ■ **Pour s'entraîner.** Cahier, chapitre 1 : application 1.1, exercices 1.1 à 1.4.
2. **Essayez avant de regarder le corrigé.** Les énoncés sont regroupés dans la partie *Exercices*, les corrigés dans la partie *Corrigés* du même chapitre : on peut chercher sans voir la réponse.
3. **Calculez d'abord à la main.** Dans ce volume, presque chaque formule se vérifie sur trois ou quatre contrats : faites-le avec un crayon avant d'écrire du code. Le code confirme, il ne remplace pas la compréhension.
4. **Faites les applications dans l'ordre** : chacune raconte une petite étude, avec une question, des étapes, du code et une lecture des résultats.
5. **Comparez à la vérité programmée.** Les données du volume sont simulées : quand le cahier le propose, relisez l'estimation à la lumière du mécanisme qui a produit les données. En pratique, vous n'aurez jamais cette chance : c'est précisément ce qui rend la validation indispensable.
6. **Dites l'incertitude.** Un chiffre de risque sans intervalle, sans hypothèse ni période n'est pas un résultat.

Numérotation et niveaux

Chaque chapitre du cahier correspond au chapitre du livre de même numéro.

Élément	Numérotation	Exemple
Application	Application N.k	Application 2.1 : première application du chapitre 2
Exercice	Exercice N.k	Exercice 2.3 : troisième exercice du chapitre 2

Élément	Numérotation	Exemple
Corrigé	Corrigé N.k	Corrigé 2.3 : correction de l'exercice 2.3

La difficulté des exercices est indiquée par des étoiles :

Niveau	Signification
★	application directe d'une formule ou d'une idée vue dans la section
★★	demande de combiner deux idées, ou un petit raisonnement
★★★	demande de la réflexion, une démonstration ou une petite expérience

Un exercice cite la section du livre qu'il exerce, par exemple « (section 1.2) ».

Données et environnement

Les données sont celles du livre, dans le dossier `donnees/`. Tous les jeux sont **simulés** avec des graines fixes (script `build/donnees5.py`), sauf `credit_default.csv` (jeu réel, UCI, licence CC0) : vos résultats seront identiques à ceux du livre. Le catalogue complet, avec les colonnes de chaque fichier, figure dans la section « Carte du volume, données et environnement » du livre ; en voici l'essentiel.

Famille	Fichiers	Chapitres
Crédit	<code>credit_default</code> , <code>credits_conso</code> , <code>recouvrements</code> , <code>revolving_defaults</code> , <code>portefeuille_ifrs9</code> , <code>notations_panel</code> , <code>taux_default_macro</code>	1, 3, 4
Assurance dommages et santé	<code>polices_auto</code> , <code>sinistres_auto</code> , <code>triangle_rc</code> , <code>triangle_dommages</code> , <code>triangle_choc</code> , <code>sante_assures</code>	2, projet
Marchés et pertes	<code>rendements_marche</code> , <code>courbe_taux</code> , <code>pertes_operationnelles</code>	3, 7
Vie et réassurance	<code>mortalite_population</code> , <code>portefeuille_vie</code> , <code>sinistres_gros</code> , <code>cat_annuel</code>	5, 6, 7
Fraude et Takaful	<code>transactions_lab</code> , <code>comptes_lab</code> , <code>takaful_fonds</code>	4

Les fichiers de vérité ne sont pas des entrées. `triangle_*_verite`, `marche_verite`, `mortalite_verite`, `mortalite_kt_vrai` et `verite_lab` contiennent la bonne réponse que le modèle est censé retrouver ou la réalité à venir. Ils servent à **juger** un résultat après coup, jamais à

l'obtenir. Les utiliser comme variables d'entrée serait une fuite d'information (volume III, section 1.1).

Chaque chapitre du cahier est **autonome** : il commence par ses imports et recharge ses données. Les applications sont dimensionnées pour tourner en **quelques dizaines de secondes** sur un ordinateur ordinaire, sans carte graphique ni réseau.

Lancer le code

Placez-vous dans le dossier du volume (celui qui contient `donnees/`), car les chemins sont relatifs (`donnees/credits_conso.csv`). Vous pouvez travailler dans un notebook Jupyter, dans un éditeur avec la commande `python`, ou dans un terminal interactif.

Les chemins. Si Python répond `FileNotFoundError`, c'est presque toujours que vous n'êtes pas dans le bon dossier : vérifiez avec `import os; print(os.getcwd())`.

Les graines. Les simulations du cahier fixent leur graine (`np.random.default_rng(42)`, par exemple). Si vous changez la graine, vos chiffres changeront un peu : c'est une excellente façon de **voir l'incertitude** d'une estimation de queue, mais ne comparez pas alors vos résultats à ceux du corrigé à la décimale.

Vérifier son installation

Deux courts blocs vérifient que les bibliothèques sont installées et que les données se chargent. Les numéros de version peuvent différer des nôtres (voir le livre), à condition que tout s'exécute.

1. Les bibliothèques.

```
import sys
from importlib import import_module
from importlib.metadata import version
print("python", sys.version.split()[0])
for p in ["numpy", "pandas", "scipy", "sklearn", "statsmodels", "arch", "lifelines", "matplotlib"]:
    import_module(p)
    print(f"{p:12s}", version("scikit-learn" if p == "sklearn" else p))
```

```
python 3.13.3
numpy      2.5.3
pandas     3.0.6
scipy      1.18.1
sklearn    1.9.1
statsmodels 0.15.0
arch       8.0.0
lifelines  0.30.3
matplotlib 3.11.2
```

2. Les données et un premier modèle. Nous chargeons le portefeuille de prêts, vérifions le taux de défaut, puis ajustons le modèle de fréquence le plus simple de l'assurance (un seul coefficient, avec l'exposition comme décalage) :

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

prets = pd.read_csv("donnees/credits_conso.csv")
auto = pd.read_csv("donnees/polices_auto.csv")
print("prêts :", len(prets), "| taux de défaut à 12 mois :", round(prets["defaut_12m"].mean(), 4))

X = np.ones((len(auto), 1))
modele = sm.GLM(auto["nb_sinistres"], X, family=sm.families.Poisson(),
    ↪ offset=np.log(auto["exposition"])).fit()
print("fréquence annuelle estimée :", round(float(np.exp(modele.params.iloc[0])), 4))

prêts : 40000 | taux de défaut à 12 mois : 0.0597
fréquence annuelle estimée : 0.0661
```

Vous devez lire une version pour chaque bibliothèque, **40 000 prêts avec un taux de défaut d'environ 6 %**, et une **fréquence annuelle d'environ 6,6 %** (c'est le nombre de sinistres divisé par l'exposition totale, ce que le GLM retrouve).

Mini-diagnostic de départ

Huit questions pour vérifier que les outils des volumes précédents sont en place. Répondez par écrit, puis comparez avec les corrigés plus bas ; chaque corrigé indique le volume à relire en cas d'hésitation. Il n'y a pas de note : l'objectif est de savoir **où revenir** avant de commencer.

1. Un prêt a une probabilité de défaut de 2 %. Sur 100 prêts **indépendants**, quelle est la probabilité qu'au moins un fasse défaut ?
2. Sur 10 000 prêts indépendants de probabilité de défaut 2 %, quels sont l'espérance et l'écart-type du nombre de défauts ?
3. Un contrat a une fréquence annuelle de sinistres de 0,065 et a été en vigueur pendant une demi-année. Dans le modèle de Poisson, quelle est la probabilité qu'il n'ait eu aucun sinistre ?
4. Dans une régression logistique, $\text{logit}(p) = -3 + 0,5x$. Quelle est la probabilité de défaut pour $x = 2$, et de quel facteur l'**odds** est-il multiplié quand x augmente d'une unité ?
5. Dans un GLM de Poisson à lien logarithmique, le coefficient d'une zone vaut 0,20 par rapport à la zone de référence. De quel facteur la fréquence est-elle multipliée ?
6. Pourquoi un découpage **aléatoire** entre apprentissage et test est-il trompeur quand les contrats sont observés sur trois années successives ?
7. Un modèle annonce « aucun défaut » pour tous les prêts d'un portefeuille où 6 % des prêts font défaut. Quelle est son exactitude ? Que vaut l'AUC d'un modèle qui classe au hasard ?
8. Les sinistres matériels ont un écart-type d'environ 1 800 €. Quel est l'écart-type de la **moyenne** de 5 000 sinistres indépendants ?

```
from math import exp, sqrt

print("Q1 :", round(1 - 0.98**100, 4))
print("Q2 :", 10000 * 0.02, "et", round(sqrt(10000 * 0.02 * 0.98), 2))
print("Q3 :", round(exp(-0.065 * 0.5), 4))
p = 1 / (1 + exp(-(-3 + 0.5 * 2)))
print("Q4 :", round(p, 4), "et", round(exp(0.5), 3))
print("Q5 :", round(exp(0.20), 3))
print("Q7 :", 1 - 0.06)
print("Q8 :", round(1800 / sqrt(5000), 1))
```

Q1 : 0.8674
 Q2 : 200.0 et 14.0
 Q3 : 0.968
 Q4 : 0.1192 et 1.649
 Q5 : 1.221
 Q7 : 0.94
 Q8 : 25.5

Corrigés du diagnostic.

1. $1 - 0,98^{100} \approx 0,8674$: presque sûr malgré une probabilité individuelle faible (volume I, section 2.1).
2. Binomiale : espérance $10\,000 \times 0,02 = 200$, écart-type $\sqrt{10\,000 \times 0,02 \times 0,98} \approx 14,0$ (volume I, section 2.2).
3. $e^{-0,065 \times 0,5} \approx 0,968$: le nombre de sinistres suit une loi de Poisson de moyenne $\lambda \times$ exposition (volume II, section 2.3).
4. $\text{logit}(p) = -2$, donc $p = 1/(1 + e^2) \approx 0,119$; l'odds est multiplié par $e^{0,5} \approx 1,649$ (volume II, section 2.2).
5. $e^{0,20} \approx 1,221$: un coefficient dans un GLM à lien logarithmique est un **logarithme de facteur multiplicatif** (volume II, section 2.3).
6. Un découpage aléatoire place dans l'apprentissage des contrats **postérieurs** à ceux du test : le modèle « voit le futur », et la performance est surestimée. On valide **hors période** (apprendre sur 2022-2023, tester sur 2024) (volume III, section 1.1).
7. L'exactitude est de 94 %, et pourtant le modèle ne détecte rien : l'exactitude est trompeuse quand l'événement est rare. Un classement au hasard a une **AUC de 0,5** (volume III, section 5.1).
8. $1800/\sqrt{5000} \approx 25,5$ € : l'écart-type de la moyenne décroît en $1/\sqrt{n}$ (volume I, section 2.4).

Si plus de deux questions vous ont gêné, relisez d'abord les sections de volume I et II indiquées : elles sont le socle de ce volume.

Plan du cahier

Chapitre	Contenu
1. Risque de crédit et scoring	construire et évaluer une grille de score, comparer modèles de défaut, mesurer Gini et KS ; ⊕ WOE et IV, perte attendue IFRS 9, matrices de transition
2. Modélisation actuarielle	fréquence et sévérité, construction d'un tarif, provisionnement par triangles ; ⊕ GLM tarifaires et crédibilité, <i>chain ladder</i> et Mack, assurance santé
3. Mesures de risque et stress tests	VaR et ES par trois méthodes, scénarios de crise ; ⊕ risque opérationnel, rétro-test
4. Cadre réglementaire	capital de Bâle, SCR de Solvabilité, répartition de l'excédent d'un fonds de Takaful ; ⊕ fraude et lutte contre le blanchiment
⊕ 5. Assurance vie	table de mortalité, primes et réserves, modèle de Lee-Carter
⊕ 6. Réassurance	tarifier un excédent de sinistre, comparer des couvertures
⊕ 7. Actif-passif et portefeuille	duration et adossement, frontière efficiente

Chapitre	Contenu
Projet et auto-évaluation	un modèle de tarification avec validation hors période et notes réglementaires, puis des questions pour vérifier ses acquis

Chapitre 1 : Risque de crédit et scoring — exercices et applications

Ce chapitre du cahier accompagne le chapitre 1 du livre. Il contient **huit applications guidées** (de petites études que vous refaites sur les données du volume) et **quatorze exercices** de difficulté croissante (★ calcul à la main, ★★ calcul puis code, ★★★ étude plus ouverte), tous **corrigés** en fin de chapitre. Les données sont **simulées** (générateur `build/donnees5.py`, vérité programmée en docstring), sauf le jeu réel `credit_default.csv` (UCI, CC0). Le fichier est **autonome** : une seule cellule recharge tout.

```
import sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import outils_ch01 as O
import donnees5
from sklearn.linear_model import LogisticRegression
import statsmodels.api as sm

tr, te = O.charger_credits()                                # 28 000 prêts de développement, 12 000 de test
↳ (graine fixe)
y_tr, y_te = tr["defaut_12m"].values, te["defaut_12m"].values
print(len(tr), len(te), round(y_tr.mean(), 4), round(y_te.mean(), 4))

28000 12000 0.0597 0.0597
```

Applications

Application 1.1 — Construire la grille complète (section 1.1)

Objectif. Refaire, étape par étape, la grille du livre : tables WOE, régression, points, carte, score de chaque dossier du test, et vérifier la propriété du PDO.

Étape 1 — les tables WOE et l'IV. `O.tables_woe` applique les classes de `O.BORNES` (bornes choisies à la main) et calcule, pour chaque variable, effectifs, taux de défaut, WOE (lissé d'une demi-observation) et IV.

```
tables = O.tables_woe(tr)
iv = pd.Series({v: t["iv"].sum() for v, t in tables.items()}).sort_values(ascending=False)
print(iv.round(3).to_string())
```

```
print(tables["revenu_annuel"][["effectif", "taux_defaut", "woe", "iv"]].round(3).to_string())
```

```
taux_endettement      0.403
anciennete_emploi     0.258
nb_incidents_12m      0.240
revenu_annuel         0.236
age                   0.145
montant               0.071
logement              0.054
anciennete_relation   0.051
duree_mois            0.029
objet                 0.015
```

	effectif	taux_defaut	woe	iv
classe				
1: <20000	3963	0.117	-0.731	0.105
2: 20000-26000	4594	0.072	-0.208	0.008
3: 26000-32000	4800	0.058	0.036	0.000
4: 32000-40000	5041	0.051	0.163	0.004
5: 40000-50000	4004	0.034	0.574	0.037
6: 50000+	4200	0.025	0.884	0.081
manquant	1398	0.069	-0.153	0.001

Étape 2 — la régression sur les WOE. On prédit « bon » (1 – défaut) à partir des WOE ; les coefficients proches de 1 correspondent à des variables presque indépendantes.

```
W_tr = 0.vers_woe(tr, tables)
lr = LogisticRegression(max_iter=2000).fit(W_tr, 1 - y_tr)
print(pd.Series(lr.coef_[0], index=W_tr.columns).round(2).to_string())
print("intercept :", round(float(lr.intercept_[0]), 3))
```

```
age                0.85
taux_endettement   0.72
anciennete_emploi  0.80
revenu_annuel      0.65
montant            0.25
duree_mois         0.35
nb_incidents_12m   0.85
anciennete_relation 1.04
logement           1.01
objet              0.98
intercept : 2.746
```

Étape 3 — la carte de score. La classe 0. Grille refait les deux étapes et convertit en points (base 600 pour une cote de 50 contre 1, PDO de 20).

```
G = 0.Grille(tr, base=600, cotes_base=50, pdo=20)
carte = G.points_classes()
print("facteur", round(G.facteur, 2), " décalage", round(G.decalage, 2), " lignes de la carte :",
      ↪ len(carte))
print(carte[carte.variable.isin(["revenu_annuel", "logement",
      ↪ "objet"])].drop(columns="variable").to_string(index=False))
```

```
facteur 28.85  décalage 487.12  lignes de la carte : 50
```

	classe	effectif	taux_defaut	woe	points
1: <20000	3963	0.1166	-0.731	42.9	
2: 20000-26000	4594	0.0725	-0.208	52.7	
3: 26000-32000	4800	0.0577	0.036	57.3	
4: 32000-40000	5041	0.0512	0.163	59.7	
5: 40000-50000	4004	0.0345	0.574	67.4	
6: 50000+	4200	0.0255	0.884	73.3	
manquant	1398	0.0687	-0.153	53.8	
heberge	5700	0.0744	-0.236	49.7	
locataire	11747	0.0668	-0.120	53.1	

proprietaire	10553	0.0438	0.326	66.2
auto	11071	0.0557	0.073	58.7
conso	9900	0.0690	-0.155	52.3
travaux	7029	0.0528	0.130	60.3

Étape 4 — scores et performance sur le test. Un score élevé signale un dossier sûr : pour l'AUC, on passe au risque en changeant le signe.

```
s_te = G.score(te)
print("AUC", round(0.auc(y_te, -s_te), 4), " Gini", round(0.gini(y_te, -s_te), 4), " KS",
      ↪ round(0.ks(y_te, -s_te), 4))
print("score : min", round(s_te.min()), " médian", round(np.median(s_te)), " max", round(s_te.max()))
```

```
AUC 0.7712 Gini 0.5425 KS 0.4058
score : min 442 médian 583 max 659
```

Étape 5 — vérifier le PDO. Si la mise à l'échelle est juste, la log-cote observée est une fonction affine du score de pente 1/facteur : regroupons le test en vingt tranches de score, calculons la log-cote observée (bons contre mauvais) et régressons.

```
g = pd.DataFrame({"s": s_te, "y": y_te}); g["q"] = pd.qcut(g["s"], 20, labels=False)
t = g.groupby("q").agg(s=("s", "mean"), mauvais=("y", "sum"), n=("y", "size"))
t["logcote"] = np.log((t["n"] - t["mauvais"] + 0.5) / (t["mauvais"] + 0.5))
pente = np.polyfit(t["s"], t["logcote"], 1)[0]
print("pente observée", round(pente, 4), " pente attendue 1/facteur =", round(1 / G.facteur, 4))
print("points pour doubler la cote, observés :", round(np.log(2) / pente, 1), "(PDO visé : 20)")
```

```
pente observée 0.0336 pente attendue 1/facteur = 0.0347
points pour doubler la cote, observés : 20.7 (PDO visé : 20)
```

À vous. (a) Refaites la grille avec une base de 650 points pour une cote de 30 contre 1 et un PDO de 40 : la performance change-t-elle ? (b) Retirez les trois variables d'IV le plus faible : que perd l'AUC ?

Application 1.2 — L'inférence des rejets (section 1.1.7)

Objectif. Mesurer le biais d'une grille construite sur les seuls acceptés, puis tester une correction par **parcelling** (étiquettes imputées aux refusés), en comparant à ce qu'aurait donné l'observation complète (que nous avons ici, par construction).

Étape 1 — la politique d'acceptation. On refuse les endettements supérieurs à 50 % et les clients de plus d'un incident.

```
def acceptes(d):
    return (d["taux_endettement"] <= 0.5) & (d["nb_incidents_12m"] <= 1)
ok = acceptes(tr)
print("part acceptée :", round(ok.mean(), 3), " défaut des acceptés :", round(tr.loc[ok,
      ↪ "defaut_12m"].mean(), 4),
      " défaut des refusés (inconnu en pratique) :", round(tr.loc[-ok, "defaut_12m"].mean(), 4))
```

```
part acceptée : 0.885 défaut des acceptés : 0.0449 défaut des refusés (inconnu en pratique) : 0.1738
```

Étape 2 — grille sur les acceptés seuls, mesurée sur tous les demandeurs du test.

```
Ga = 0.Grille(tr[ok])
def bilan(G_, nom):
    return {"grille": nom, "AUC (tous)": round(0.auc(y_te, -G_.score(te)), 4), "PD annoncée":
      ↪ round(float(G_.pd_predite(te).mean()), 4)}
res = [bilan(0.Grille(tr), "tous les demandeurs (oracle)"), bilan(Ga, "acceptés seuls")]
```

Étape 3 — parcelling. On score les refusés avec la grille des acceptés, on leur impute une étiquette tirée au sort avec la probabilité $\min(1, m \times \text{PD annoncée})$, où m est un coefficient de prudence (on teste 1 et 2), puis l'on reconstruit la grille sur acceptés + refusés étiquetés.

```
rng = np.random.default_rng(0)
p_ref = Ga.pd_predite(tr[~ok])
for m in (1, 2):
    aug = tr.copy()
    aug.loc[~ok, "defaut_12m"] = (rng.random((~ok).sum()) < np.minimum(1, m * p_ref)).astype(int)
    res.append(bilan(0.Grille(aug), f"parcelling, coefficient {m}"))
print(pd.DataFrame(res).to_string(index=False)); print("défaut réel du test :", round(y_te.mean(), 4))
```

	grille	AUC (tous)	PD annoncée
tous les demandeurs (oracle)	0.7712	0.0593	
acceptés seuls	0.7156	0.0439	
parcelling, coefficient 1	0.7198	0.0439	
parcelling, coefficient 2	0.7634	0.0494	
défaut réel du test :	0.0597		

À vous. Quelle valeur de m rapproche le plus la PD annoncée du défaut réel ? Pourquoi ne pouvez-vous pas la choisir ainsi en pratique ?

Application 1.3 — Trois modèles, un verdict (section 1.2)

Objectif. Comparer la grille, la logistique brute et un boosting monotone, avec incertitude, calibration et motifs de refus.

Étape 1 — les trois modèles (mêmes données, aucun réglage sur le test).

```
import lightgbm as lgb
def brut(d):
    X = d[["age", "revenu_annuel", "anciennete_emploi", "montant", "duree_mois", "taux_endettement",
    ↪ "nb_incidents_12m", "anciennete_relation"]].copy()
    for k in ["revenu_annuel", "anciennete_emploi"]:
        X[k + "_manq"] = X[k].isna().astype(int)
    return pd.concat([X, pd.get_dummies(d[["logement", "objet"]]).astype(int)], axis=1)
Xb_tr, Xb_te = brut(tr), brut(te)
med = Xb_tr.median(); mu, sd = Xb_tr.fillna(med).mean(), Xb_tr.fillna(med).std()
p_lr = LogisticRegression(max_iter=3000).fit((Xb_tr.fillna(med) - mu) / sd,
    ↪ y_tr).predict_proba((Xb_te.fillna(med) - mu) / sd)[: , 1]
signes = {"taux_endettement": 1, "nb_incidents_12m": 1, "revenu_annuel": -1, "anciennete_emploi": -1,
    ↪ "anciennete_relation": -1}
gbm = lgb.LGBMClassifier(n_estimators=300, learning_rate=0.03, num_leaves=8, min_child_samples=200,
    ↪ subsample=0.8, subsample_freq=1,
    ↪ colsample_bytree=0.8, monotone_constraints=[signes.get(c, 0) for c in
    ↪ Xb_tr.columns], random_state=0, verbose=-1).fit(Xb_tr, y_tr)
p_gbm = gbm.predict_proba(Xb_te)[: , 1]
r_grille = -0.Grille(tr).score(te)
print({k: round(0.auc(y_te, v), 4) for k, v in {"logistique": p_lr, "grille": r_grille, "boosting
    ↪ monotone": p_gbm}.items()})
```

```
{'logistique': 0.7618, 'grille': 0.7712, 'boosting monotone': 0.7737}
```

Étape 2 — intervalles apparié et isolé.

```
for nom, v in {"logistique": p_lr, "grille": r_grille, "boosting monotone": p_gbm}.items():
    print(nom, np.round(0.bootstrap_auc(y_te, v, 300), 3))
for a, b, na, nb in [(r_grille, p_lr, "grille", "logistique"), (r_grille, p_gbm, "grille",
    ↪ "boosting")]:
    m, ic = 0.bootstrap_diff_auc(y_te, a, b, 300, 1)
    print(f"AUC {na} - AUC {nb} : {m_:.4f} [{ic[0]:+.4f} ; {ic[1]:+.4f}]")
```

```
logistique [0.743 0.78 ]
grille [0.751 0.791]
boosting monotone [0.754 0.791]
AUC grille - AUC logistique : +0.0097 [+0.0009 ; +0.0182]
AUC grille - AUC boosting : -0.0022 [-0.0089 ; +0.0040]
```

Étape 3 — la calibration par dixième (probabilités annoncées par le boosting et par la logistique, contre le défaut observé), et le score de Brier.

```
from sklearn.metrics import brier_score_loss
cal = pd.DataFrame({"logistique": p_lr, "boosting": p_gbm, "défaut": y_te})
cal["groupe"] = pd.qcut(cal["boosting"], 10, labels=False) + 1
print(cal.groupby("groupe").mean().round(4).to_string())
print({k: round(brier_score_loss(y_te, v), 5) for k, v in {"logistique": p_lr, "boosting":
↳ p_gbm}.items()})
```

	logistique	boosting	défaut
groupe			
1	0.0082	0.0075	0.0092
2	0.0146	0.0130	0.0192
3	0.0205	0.0177	0.0175
4	0.0271	0.0231	0.0225
5	0.0346	0.0296	0.0258
6	0.0431	0.0379	0.0450
7	0.0554	0.0493	0.0508
8	0.0716	0.0665	0.0675
9	0.1036	0.1004	0.1000
10	0.2185	0.2523	0.2392

```
{'logistique': 0.05041, 'boosting': 0.04997}
```

À vous. (a) Recalibrez le boosting par une régression logistique sur sa sortie (volume III, section 5.2) sur un échantillon à part, et comparez les Brier. (b) Imposez au boosting aussi la monotonie de l'âge : que se passe-t-il, et pourquoi est-ce une erreur ?

Application 1.4 — Performance, seuil et stabilité (section 1.3)

Objectif. Choisir un seuil d'acceptation **par le profit**, calculer un PSI et constater que le test binomial rejette une PD correcte quand les défauts sont corrélés.

Étape 1 — le profit d'un seuil. Hypothèses de l'exercice (illustratives) : un bon prêt rapporte 450 € de marge, un défaut coûte 4 000 € de perte nette. Pour chaque seuil de score, le profit attendu est la somme, sur les dossiers acceptés, de la marge des bons moins la perte des mauvais.

```
G = 0.Grille(tr); s_te = G.score(te)
seuils = np.arange(500, 640, 10)
profit = [(450 * ((y_te == 0) & (s_te >= c)).sum() - 4000 * ((y_te == 1) & (s_te >= c)).sum()) / 1e3
↳ for c in seuils]
tab = pd.DataFrame({"seuil": seuils, "acceptés (%)": [100 * (s_te >= c).mean() for c in seuils],
↳ "profit (k€)": profit}).round(1)
print(tab.to_string(index=False))
print("meilleur seuil :", int(seuils[int(np.argmax(profit))]), "points")
```

seuil	acceptés (%)	profit (k€)
500	98.9	2457.9
510	98.0	2570.4
520	96.8	2716.0
530	94.7	2810.9
540	91.2	2893.4
550	85.7	2920.8
560	78.3	2923.4
570	67.6	2703.9

580	53.9	2328.6
590	39.0	1745.6
600	24.3	1112.4
610	12.5	599.4
620	5.2	256.4
630	1.7	86.4

meilleur seuil : 560 points

Étape 2 — le PSI. Simulons une nouvelle population plus jeune et plus endettée (tirage pondéré dans le test) et comparons.

```
w = np.exp(1.2 * (te["age"] < 30) + 0.8 * (te["taux_endettement"] > 0.4)); w = w / w.sum()
nv = te.sample(6000, replace=True, weights=w, random_state=3)
print("PSI score :", round(0.psi(G.score(tr), G.score(nv), 10), 3), " PSI âge :",
    ↪ round(0.psi(tr["age"], nv["age"], 10), 3),
    " défaut observé :", round(nv["defaut_12m"].mean(), 4), " PD annoncée :",
    ↪ round(float(G.pd_predite(nv).mean()), 4))
```

PSI score : 0.129 PSI âge : 0.246 défaut observé : 0.0893 PD annoncée : 0.0873

Étape 3 — le test binomial et la corrélation. On simule un portefeuille de 20 000 prêts dont la vraie PD est 3 %, avec un facteur commun (corrélation d'actifs ρ , modèle de Vasicek) : la PD conditionnelle est $\Phi((\Phi^{-1}(0,03) - \sqrt{\rho}Z)/\sqrt{1-\rho})$ avec Z gaussien. On compte les rejets du test binomial à 5 %.

```
from scipy.stats import binomtest, norm
rng = np.random.default_rng(0)
for rho in (0.0, 0.01, 0.03):
    rej = 0
    for _ in range(1000):
        pc = norm.cdf((norm.ppf(0.03) - np.sqrt(rho) * rng.normal()) / np.sqrt(1 - rho))
        rej += binomtest(int(rng.binomial(20000, pc)), 20000, 0.03).pvalue < 0.05
    print(f"corrélation {rho:.2f} : le test rejette à tort {rej / 1000:.1%} des portefeuilles")
```

```
corrélation 0.00 : le test rejette à tort 5.1% des portefeuilles
corrélation 0.01 : le test rejette à tort 73.4% des portefeuilles
corrélation 0.03 : le test rejette à tort 84.4% des portefeuilles
```

À vous. Tracez le profit en fonction du seuil (avec un pas de 2 points) si la perte d'un défaut passe de 4 000 € à 6 000 € : le seuil optimal monte-t-il ou descend-il ?

Application 1.5 — WOE, IV et fusion monotone (section 1.4)

Objectif. Apprendre à regarder un découpage : fusion monotone, IV de développement contre IV de test, effet d'un découpage forcé sur la performance.

Étape 1 — fusion monotone sur quatre variables (sens attendu : le risque décroît avec la variable).

```
for nom in ("revenu_annuel", "anciennete_relation", "anciennete_emploi", "age"):
    b, n, m = 0.fusion_monotone(tr[nom], y_tr, 10, -1)
    print(f"{nom:20s} {len(b):2d} classes, IV {0.iv_partition(n, m):.3f}, taux (%) {np.round(100 * m /
    ↪ n, 1)}")
```

```
revenu_annuel      9 classes, IV 0.267, taux (%) [13.2  8.1  7.6  5.7  5.7  4.4  3.7  3.1  2.2]
anciennete_relation 9 classes, IV 0.054, taux (%) [8.2 7.  6.7 6.3 6.  5.5 5.3 4.6 3.5]
anciennete_emploi  7 classes, IV 0.262, taux (%) [10.2  8.6  6.6  5.3  4.6  3.2  2.8]
age                 5 classes, IV 0.127, taux (%) [13.4  6.3  5.8  5.1  5. ]
```

Étape 2 — le bruit. L'IV d'une variable sans aucun lien avec le défaut grandit avec le nombre de

classes ; sur le test, il ne devrait pas.

```
rng = np.random.default_rng(0)
bruit_tr, bruit_te = rng.normal(size=len(tr)), rng.normal(size=len(te))
def iv_dev_test(x, y, x2, y2, k):
    q = np.unique(np.quantile(x, np.linspace(0, 1, k + 1)[1:-1])); i = np.searchsorted(q, x,
    ↪ side="right")
    n = np.bincount(i, minlength=len(q) + 1); m = np.bincount(i, weights=y, minlength=len(q) + 1)
    b = n - m; pb = (b + .5) / (b.sum() + .5 * len(n)); pm = (m + .5) / (m.sum() + .5 * len(n)); woe =
    ↪ np.log(pb / pm)
    j = np.searchsorted(q, x2, side="right"); n2 = np.bincount(j, minlength=len(q) + 1); m2 =
    ↪ np.bincount(j, weights=y2, minlength=len(q) + 1); b2 = n2 - m2
    ↪ return round(0.iv_partition(n, m), 4), round(float(((b2 / b2.sum() - m2 / m2.sum()) * woe).sum()),
    ↪ 4)
for k in (3, 10, 30, 100):
    print(k, "classes (bruit) : IV développement, IV test =", iv_dev_test(bruit_tr, y_tr, bruit_te,
    ↪ y_te, k))
```

```
3 classes (bruit) : IV développement, IV test = (0.0001, 0.0008)
10 classes (bruit) : IV développement, IV test = (0.0022, 0.0008)
30 classes (bruit) : IV développement, IV test = (0.0194, -0.0025)
100 classes (bruit) : IV développement, IV test = (0.0553, -0.0155)
```

Étape 3 — forcer la monotonie de l'âge coûte-t-il de la performance ? On remplace les classes de l'âge par celles de la fusion monotone (bornes inférieures renvoyées par `fusion_monotone`) et l'on recalcule l'AUC de test. Les bornes du module sont **restaurées** ensuite.

```
import copy
B0 = copy.deepcopy(0.BORNES)
b_age, _, _ = 0.fusion_monotone(tr["age"], y_tr, 10, -1)
base = 0.auc(y_te, -0.Grille(tr).score(te))
0.BORNES["age"] = list(b_age) + [np.inf]
forcee = 0.auc(y_te, -0.Grille(tr).score(te))
0.BORNES.clear(); 0.BORNES.update(B0)
print("bornes de l'âge forcées monotones :", np.round(b_age[1:], 0), "\nAUC test, classes de la main
    ↪ :", round(base, 4), " classes monotones :", round(forcee, 4))
```

```
bornes de l'âge forcées monotones : [26. 32. 36. 39.]
AUC test, classes de la main : 0.7712 classes monotones : 0.7668
```

À vous. Faites la même comparaison pour le revenu (la monotonie y est justifiée) et pour le taux d'endettement.

Application 1.6 — LGD et CCF (section 1.5)

Objectif. Modéliser la LGD (quatre modèles) et le CCF, puis calculer une exposition.

Étape 1 — la distribution et les segments.

```
lg = pd.read_csv("donnees/recouvrements.csv")
print(lg["lgd_realisee"].describe().round(3).to_string())
print(lg.groupby("garantie")["lgd_realisee"].agg(["mean", "count"]).round(3).to_string())
print("pertes nulles :", round((lg["lgd_realisee"] == 0).mean(), 3), " pertes > 95 % :",
    ↪ round((lg["lgd_realisee"] > 0.95).mean(), 3))
```

```
count    6000.000
mean      0.459
std       0.342
min       0.000
25%      0.124
50%      0.450
75%      0.772
```

```

max          1.000
              mean count
garantie
aucune       0.609   2993
caution     0.269   1852
nantissement 0.373   1155
pertes nulles : 0.12 pertes > 95 % : 0.091

```

Étape 2 — quatre modèles (moyenne, linéaire, fractionnel, deux étapes), sur 70 % des prêts, jugés sur 30 %.

```

from sklearn.model_selection import train_test_split
ltr, lte = train_test_split(lg, test_size=0.3, random_state=1)
def conc(d): return pd.get_dummies(d[["garantie", "objet"]],
    ↪ drop_first=True).astype(float).assign(ead=np.log(d["ead"]) - 9)
Xl = sm.add_constant(conc(ltr)); Xt = sm.add_constant(conc(lte)).reindex(columns=Xl.columns); yl =
    ↪ lte["lgd_realisee"].values
p_ols = np.clip(sm.OLS(ltr["lgd_realisee"], Xl).fit().predict(Xt), 0, 1)
p_fr = sm.GLM(ltr["lgd_realisee"], Xl, family=sm.families.Binomial()).fit().predict(Xt)
pz = sm.Logit((ltr["lgd_realisee"] == 0).astype(int), Xl).fit(dis=0).predict(Xt)
pos = ltr[ltr["lgd_realisee"] > 0]
p_2 = (1 - pz) * sm.GLM(pos["lgd_realisee"], Xl.loc[pos.index],
    ↪ family=sm.families.Binomial()).fit().predict(Xt)
r2 = lambda p: 1 - np.sum((p - yl) ** 2) / np.sum((yl - yl.mean()) ** 2)
for nom, p in [("moyenne", np.full(len(yl), ltr["lgd_realisee"].mean())), ("linéaire", p_ols),
    ↪ ("fractionnel", p_fr), ("deux étapes", p_2)]:
    print(f"{nom:12s} erreur absolue {np.mean(np.abs(p - yl)):.4f} R² {r2(np.asarray(p)):.4f}")

```

```

moyenne      erreur absolue 0.3063   R² -0.0003
linéaire     erreur absolue 0.2591   R² 0.2165
fractionnel  erreur absolue 0.2591   R² 0.2163
deux étapes  erreur absolue 0.2591   R² 0.2163

```

Étape 3 — le CCF et l'exposition.

```

rv = pd.read_csv("donnees/revolving_defaults.csv"); rv["util"] = rv["tirage_12m_avant"] / rv["limite"]
print(rv.groupby(pd.qcut(rv["util"], 5))["ccf_observe"].mean().round(3).to_string())
ccf = rv["ccf_observe"].mean()
ead = rv["tirage_12m_avant"] + ccf * (rv["limite"] - rv["tirage_12m_avant"])
print("CCF moyen", round(ccf, 3), "| EAD réelle", round(rv["ead"].sum() / 1e6, 1), "M€ | EAD par CCF",
    ↪ round(ead.sum() / 1e6, 1), "M€ | tirages seuls", round(rv["tirage_12m_avant"].sum() / 1e6, 1),
    ↪ "M€")

```

```

util
(0.00441, 0.214]    0.495
(0.214, 0.328]     0.443
(0.328, 0.441]     0.414
(0.441, 0.577]     0.357
(0.577, 0.966]     0.299
CCF moyen 0.402 | EAD réelle 23.8 M€ | EAD par CCF 23.4 M€ | tirages seuls 14.5 M€

```

À vous. Estimez une LGD par **segment** (garantie × objet) et comparez son erreur à celle des modèles : que concluez-vous sur l'intérêt de modèles plus compliqués ?

Application 1.7 — Perte attendue IFRS 9 (section 1.5)

Objectif. Calculer les étapes et la perte attendue d'un portefeuille, vérifier à la main un prêt, puis mesurer la sensibilité aux règles de basculement et aux scénarios.

Étape 1 — étapes et perte attendue par étape.

```
pf = pd.read_csv("donnees/portefeuille_ifrs9.csv")
pf["etape"] = 0.etape_ifrs9(pf); pf["ecl"] = 0.ecl_ifrs9(pf, pf["etape"].values)
t = pf.groupby("etape").agg(prets=("ead", "size"), ead_M=("ead", lambda s: s.sum() / 1e6),
    ↪ ecl_M=("ecl", lambda s: s.sum() / 1e6))
t["couverture_%"] = 100 * t["ecl_M"] / t["ead_M"]
print(t.round(2).to_string()); print("ECL totale (M€) :", round(t["ecl_M"].sum(), 2))
```

	prets	ead_M	ecl_M	couverture_%
etape				
1	16607	251.65	2.92	1.16
2	2923	43.43	1.78	4.09
3	470	7.32	2.89	39.48
ECL totale (M€) :			7.59	

Étape 2 — vérifier un prêt à la main. Prenons un prêt d'étape 2 et recalculons sa perte sur la durée de vie : somme sur les années t de $(1 - h)^{t-1}h \times \text{LGD} \times \text{EAD}_t / (1 + r)^{t-0,5}$, avec $\text{EAD}_t = \text{EAD} (1 - (t - 0,5)/T)$ et un prorata pour la dernière année.

```
p1 = pf[pf["etape"] == 2].iloc[0]
h, lgd, E, r, T = p1["pd_actuelle"], p1["lgd_estimee"], p1["ead"], p1["taux_effectif"],
    ↪ p1["maturite_residuelle"]
tot = 0.0
for t_ in range(1, int(np.ceil(T)) + 1):
    frac = min(1, T - (t_ - 1)); e_t = E * max(0, 1 - (t_ - 0.5) / T)
    tot += (1 - h) ** (t_ - 1) * h * frac * lgd * e_t / (1 + r) ** (t_ - 0.5)
print("à la main :", round(tot, 2), "| fonction :", round(float(p1["ecl"]), 2), "| (h, LGD, EAD, r, T)
    ↪ =", (h, lgd, E, r, T))
```

```
à la main : 49.18 | fonction : 49.18 | (h, LGD, EAD, r, T) = (np.float64(0.04207), np.float64(0.243),
    ↪ np.float64(7610.0), np.float64(0.0342), np.float64(1.4))
```

Étape 3 — sensibilité au seuil de hausse sensible du risque.

```
lignes = []
for seuil in (2.0, 2.5, 3.0, 4.0):
    et = 0.etape_ifrs9(pf, seuil_ratio=seuil)
    lignes.append((seuil, int((et == 2).sum()), 0.ecl_ifrs9(pf, et).sum() / 1e6))
print(pd.DataFrame(lignes, columns=["seuil du ratio de PD", "prêts en étape 2", "ECL
    ↪ (M€)"]).round(2).to_string(index=False))
```

seuil du ratio de PD	prêts en étape 2	ECL (M€)
2.0	4431	7.90
2.5	2923	7.59
3.0	2174	7.40
4.0	1489	7.23

Étape 4 — scénarios. On traduit trois scénarios macroéconomiques en multiplicateurs de PD par une régression du logit du taux de défaut sur la conjoncture (historique de 80 trimestres).

```
mac = pd.read_csv("donnees/taux_defaut_macro.csv"); mac["logit"] = np.log(mac["taux_defaut"] / (1 -
    ↪ mac["taux_defaut"]))
mod = sm.OLS(mac["logit"], sm.add_constant(mac[["croissance_pib", "chomage",
    ↪ "variation_immo"]])).fit()
sc = {"central": (1.4, 8.2, 0.6), "défavorable": (-1.0, 9.5, -2.5), "favorable": (2.4, 7.4, 2.5)};
    ↪ poids = {"central": .5, "défavorable": .3, "favorable": .2}
taux = {k: 1 / (1 + np.exp(-(mod.params["const"] + v[0] * mod.params["croissance_pib"] + v[1] *
    ↪ mod.params["chomage"] + v[2] * mod.params["variation_immo"]))) for k, v in sc.items()}
```

```
ecl = {k: 0.ecl_ifrs9(pf, pf["etape"].values, mult=taux[k] / taux["central"]).sum() / 1e6 for k in sc}
print({k: round(100 * v, 2) for k, v in taux.items()}, {k: round(v, 2) for k, v in ecl.items()})
print("ECL pondérée :", round(sum(poids[k] * ecl[k] for k in sc), 2), "M€")
```

```
{'central': np.float64(2.61), 'défavorable': np.float64(10.51), 'favorable': np.float64(1.29)}
↪ {'central': np.float64(7.59), 'défavorable': np.float64(19.89), 'favorable': np.float64(5.27)}
ECL pondérée : 10.82 M€
```

À vous. Que devient l'ECL pondérée avec des poids 40 / 40 / 20 ? Et si le scénario défavorable fait basculer en étape 2 tous les prêts dont le retard dépasse 15 jours (utilisez `retard_2=15`) ?

Application 1.8 — Matrice de migration (section 1.6)

Objectif. Estimer la matrice de transition, mesurer son incertitude, voir l'effet de la conjoncture, calculer des PD à plusieurs années et vérifier par simulation.

Étape 1 — les effectifs et la matrice par cohortes.

```
panel = pd.read_csv("donnees/notations_panel.csv"); P_vrai = donnees5.matrice_vraie()
N = 0.compter_transitions(panel); M = 0.matrice_cohortes(N)
print(pd.DataFrame(N, index=range(1, 8), columns=["sortie"] + list(range(1, 8)) +
↪ ["défaut"]).to_string())
print("PD estimée (%) :", np.round(100 * M[:, 7], 2), "\nPD vraie (%) :", np.round(100 * P_vrai[:,
↪ 7], 2))
```

	sortie	1	2	3	4	5	6	7	défaut
1	96	2150	205	29	1	2	0	0	1
2	280	254	5669	447	91	31	16	6	6
3	404	55	605	8401	625	174	46	26	23
4	390	26	91	653	7770	650	163	75	55
5	215	2	8	56	416	4179	439	141	109
6	115	0	5	3	44	174	1977	279	186
7	37	0	0	0	5	20	87	702	387
PD estimée (%) :	[	0.04	0.09	0.23	0.58	2.04	6.97	32.22]	
PD vraie (%) :	[	0.1	0.1	0.2	0.5	1.7	5.9	28.3]	

Étape 2 — un intervalle de confiance par bootstrap d'emprunteurs.

```
C = np.zeros((5000, 7, 9), dtype=np.int32)
np.add.at(C, (panel["id_emprunteur"].values - 1, panel["note_debut"].values - 1,
↪ panel["note_fin"].values), 1)
rng = np.random.default_rng(0)
sim = []
for _ in range(200):
    Nb = C[rng.integers(0, 5000, 5000)].sum(axis=0); sim.append((Nb[:, 8] / Nb[:, 1:].sum(axis=1)))
lo, hi = np.percentile(sim, [2.5, 97.5], axis=0)
print(pd.DataFrame({"PD estimée": M[:, 7], "bas": lo, "haut": hi}, index=range(1,
↪ 8)).round(4).to_string())
```

	PD estimée	bas	haut
1	0.0004	0.0000	0.0013
2	0.0009	0.0003	0.0020
3	0.0023	0.0015	0.0033
4	0.0058	0.0044	0.0074
5	0.0204	0.0167	0.0245
6	0.0697	0.0614	0.0800
7	0.3222	0.2941	0.3462

Étape 3 — calme contre récession.

```
calme = 0.matrice_cohortes(0.compter_transitions(panel, [0, 1, 2, 3, 8, 9])); rec =
↳ 0.matrice_cohortes(0.compter_transitions(panel, [5, 6]))
print(pd.DataFrame({"PD calme (%)": 100 * calme[:, 7], "PD récession (%)": 100 * rec[:, 7]},
↳ index=range(1, 8)).round(2).to_string())
```

	PD calme (%)	PD récession (%)
1	0.07	0.00
2	0.08	0.08
3	0.13	0.49
4	0.45	1.11
5	1.39	4.71
6	4.95	14.07
7	24.49	51.55

Étape 4 — PD à 5 ans, par puissance de matrice et par simulation. On simule 20 000 trajectoires par note de départ avec la matrice estimée et l'on compte les défauts avant cinq ans.

```
M_abs = np.vstack([M, np.r_[np.zeros(7), 1.0]]); PD_5 = np.linalg.matrix_power(M_abs, 5)[:7, 7]
rng = np.random.default_rng(1); cum = np.cumsum(M_abs, axis=1); sim5 = []
for note in range(7):
    etat = np.full(20000, note)
    for _ in range(5):
        etat = (rng.random(20000)[: , None] > cum[etat]).sum(axis=1).clip(max=7)
    sim5.append((etat == 7).mean())
print(pd.DataFrame({"puissance de matrice": PD_5, "simulation": sim5}, index=range(1,
↳ 8)).round(4).to_string())
```

	puissance de matrice	simulation
1	0.0037	0.0028
2	0.0120	0.0118
3	0.0270	0.0282
4	0.0649	0.0663
5	0.1696	0.1699
6	0.3926	0.3944
7	0.7648	0.7607

À vous. Estimez la matrice sur les **cinq premières années** seulement, puis prédisez les défauts observés sur les cinq dernières : à quel point la prédiction est-elle trompée par la conjoncture ?

Exercices

Exercice 1.1 ★ — Échelle de points (section 1.1)

Un établissement fixe un score de **650 points pour une cote de 30 contre 1** et un **PDO de 40**.

(a) Calculez le facteur et le décalage. (b) Quel score correspond à une cote de 120 contre 1 ? (c) Quelle probabilité de défaut correspond à un score de 600 ?

Exercice 1.2 ★ — WOE et IV à la main (section 1.1.4)

Une variable a trois classes. Classe A : 4 000 bons, 40 mauvais ; classe B : 5 000 bons, 150 mauvais ; classe C : 1 000 bons, 110 mauvais. Calculez le WOE de chaque classe et l'IV de la variable (sans lissage). Que concluez-vous sur la lecture usuelle de l'IV ?

Exercice 1.3 ★★ — Fenêtres et fuite d'information (section 1.1.2)

- (a) Classez en « autorisée » ou « interdite » pour un score d'octroi : l'âge à la demande, le nombre de retards du prêt dans ses six premiers mois, le revenu déclaré, le nombre d'incidents de paiement des douze mois précédant la demande, le solde du compte six mois après l'octroi, l'objet du prêt. (b) Un prêt est observé six mois seulement au lieu de douze : si le risque de défaut est réparti uniformément sur les douze mois, quel taux de défaut mesure-t-on ? Vérifiez par simulation.

Exercice 1.4 ★ — Perte attendue et taux minimal (section 1.5)

Un prêt de 8 000 € a une PD de 4 % à un an et une LGD de 50 %. (a) Quelle est sa perte attendue sur l'année ? (b) De combien de points de taux doit-on majorer le coût de refinancement pour la couvrir, à exposition constante ? (c) Si la LGD réelle est de 61 % (prêt sans garantie), que devient la majoration ?

Exercice 1.5 ★★ — Contraintes de monotonie (section 1.2.4)

Entraînez un boosting libre et un boosting monotone (voir l'application 1.3) puis calculez, sur une grille de taux d'endettement de 0,05 à 0,95, la probabilité prédite pour un dossier « médian » en ne faisant varier que ce taux. Comptez les **inversions** (endroits où la probabilité baisse quand le taux monte) pour chaque modèle.

Exercice 1.6 ★ — AUC par comptage (section 1.3.1)

Huit dossiers, avec leur score de risque et leur issue : (0,95 ; M), (0,85 ; B), (0,70 ; M), (0,65 ; B), (0,55 ; M), (0,40 ; B), (0,30 ; B), (0,10 ; B) (M = mauvais, B = bon). Calculez l'AUC par comptage des paires, le Gini, puis le KS à la main.

Exercice 1.7 ★★ — Le Gini ne dépend pas de la proportion de mauvais (section 1.3.2)

Simulez des scores gaussiens (moyenne 0 pour les bons, 1 pour les mauvais, écart-type 1) avec une proportion de mauvais $\pi = 2\%$ puis 30 %. Calculez l'AUC, puis l'accuracy ratio par l'aire sous la CAP, et comparez avec $2\text{AUC} - 1$.

Exercice 1.8 ★★ — PSI à la main (section 1.3.6)

La répartition du score en quatre classes était (30 %, 30 %, 25 %, 15 %) au développement et est de (20 %, 25 %, 30 %, 25 %) aujourd'hui. Calculez le PSI et interprétez-le avec les repères usuels. Quelle classe contribue le plus ?

Exercice 1.9 ★ — IV avec une classe sans mauvais (section 1.4.4)

Une classe contient 300 bons et 0 mauvais, l'échantillon compte 20 000 bons et 400 mauvais. Que vaut son WOE sans lissage ? Avec un lissage d'une demi-observation par classe (la variable a cinq classes) ? Quelle règle adopter ?

Exercice 1.10 ★★ — IV d'une variable de bruit (section 1.4.2)

Répétez 30 fois l'expérience d'une variable de pur bruit découpée en 50 classes d'effectifs égaux. Donnez la moyenne et le maximum de l'IV de développement. Proposez, à partir de ce résultat, un **seuil** en dessous duquel l'IV d'une variable à 50 classes ne prouve rien.

Exercice 1.11 ★ — Perte attendue d'un prêt (section 1.5.4)

Un prêt de 20 000 € a une PD annuelle de 2 %, une LGD de 40 %, une maturité résiduelle de 3 ans, un taux effectif de 5 %. (a) Perte attendue à 12 mois (étape 1) avec actualisation à mi-année. (b) Perte sur la durée de vie (étape 2) avec une exposition qui s'amortit linéairement ($EAD_t = EAD(1 - (t - 0,5)/T)$) et une actualisation à mi-année. (c) Rapport des deux.

Exercice 1.12 ★★ — L'effet de falaise (section 1.5.4)

Sur `portefeuille_ifrs9.csv`, faites varier le **seuil de retard de l'étape 2** (15, 30, 45, 60 jours, avec le ratio de PD à 2,5) et mesurez le nombre de prêts en étape 2 et la perte attendue totale. Quelle est la sensibilité de la provision à cette convention ?

Exercice 1.13 ★ — Une matrice à trois états (section 1.6.4)

Trois états : A (sain), B (surveillé), D (défaut absorbant). Annuellement : $A \rightarrow A$ 0,90, $A \rightarrow B$ 0,08, $A \rightarrow D$ 0,02 ; $B \rightarrow A$ 0,10, $B \rightarrow B$ 0,75, $B \rightarrow D$ 0,15. Calculez à la main la PD à 2 ans puis à 3 ans de A et de B, et vérifiez avec numpy.

Exercice 1.14 ★★★ — Chapman–Kolmogorov et conjoncture (section 1.6.5)

Sur `notations_panel.csv`, comparez, pour chaque note de départ, la probabilité observée de défaut à **deux ans** (note en début d'année t , défaut avant la fin de $t + 1$, observé directement sur les emprunteurs suivis deux années) à celle que donne P^2 de la matrice estimée. Pourquoi l'écart n'est-il pas nul ? Recommencez en calculant P^2 avec les matrices *calme* et *récession* séparées et en pondérant par la fréquence de ces années.

Corrigés

Corrigé 1.1

- (a) $\text{facteur} = 40 / \ln 2 = 57,71$; $\text{décalage} = 650 - 57,71 \times \ln 30 = 650 - 196,29 = 453,71$.
 (b) $453,71 + 57,71 \times \ln 120 = 453,71 + 276,28 = 730$: on le vérifie sans calcul, puisque $120 = 30 \times 4$ est deux doublements de la cote, soit $650 + 2 \times 40 = 730$. (c) $\text{cote} = \exp((600 - 453,71)/57,71) = \exp(2,535) = 12,6$; $\text{PD} = 1/(1 + 12,6) = 7,3 \%$.

```
f = 40 / np.log(2); d = 650 - f * np.log(30)
print(round(f, 2), round(d, 2), round(d + f * np.log(120), 1), round(1 / (1 + np.exp((600 - d) / f)),
    4))
```

```
57.71 453.72 730.0 0.0735
```

Corrigé 1.2

Totaux : bons 10 000, mauvais 300. Classe A : $g = 0,40$, $b = 40/300 = 0,1333$, $WOE = \ln(0,40/0,1333) = \ln 3 = 1,099$. Classe B : $g = 0,50$, $b = 0,50$, $WOE = 0$. Classe C : $g = 0,10$, $b = 110/300 = 0,3667$, $WOE = \ln(0,2727) = -1,299$. IV = $(0,40 - 0,1333) \times 1,099 + 0 + (0,10 - 0,3667) \times (-1,299) = 0,293 + 0,346 = 0,639$. Au-dessus de 0,5 : lecture « suspect » ou, ici, simplement une variable **très** discriminante ; on cherche d'abord si elle est connue à la date de la demande.

```
n = np.array([[4000, 40], [5000, 150], [1000, 110]]); g = n[:, 0] / n[:, 0].sum(); b = n[:, 1] / n[:, 1].sum()
print(np.round(np.log(g / b), 3), round(float((g - b) * np.log(g / b)).sum()), 3))
```

```
[ 1.099  0.    -1.299] 0.639
```

Corrigé 1.3

- (a) **Autorisées** : âge à la demande, revenu déclaré, incidents des douze mois **précédant** la demande, objet. **Interdites** : retards du prêt dans ses six premiers mois et solde du compte six mois après l'octroi (postérieurs à la demande). (b) Si le défaut est réparti uniformément sur les douze mois, observer six mois ne révèle que **la moitié** des défauts : on mesure environ 0,5× le taux vrai ; avec un taux de 6 %, on lit 3 %, et l'on prend pour « bons » des prêts qui feront défaut ensuite.

```
rng = np.random.default_rng(0); n = 200000
instant = np.where(rng.random(n) < 0.06, rng.uniform(0, 12, n), np.inf) # date du défaut
↳ (mois), ∞ = pas de défaut
print("taux sur 12 mois :", round(float((instant <= 12).mean()), 4), "taux sur 6 mois :",
↳ round(float((instant <= 6).mean()), 4))
```

```
taux sur 12 mois : 0.0598  taux sur 6 mois : 0.0299
```

Corrigé 1.4

- (a) $0,04 \times 0,5 \times 8\,000 = 160$ €. (b) $160/8\,000 = 2$ % de l'exposition : une majoration de **2 points** de taux (à exposition constante sur l'année). (c) Avec une LGD de 61 % : $0,04 \times 0,61 = 2,44$ %, soit 0,44 point de plus : utiliser la LGD moyenne de 50 % sous-tarifie les prêts sans garantie.

Corrigé 1.5

```
d_med = {c: (te[c].median() if pd.api.types.is_numeric_dtype(te[c]) else te[c].mode()[0]) for c in
↳ te.columns if c not in ("id_credit", "defaut_12m")}
grille_dti = np.arange(0.05, 0.96, 0.01)
ref = pd.DataFrame([d_med] * len(grille_dti)); ref["taux_endettement"] = grille_dti
signes = {"taux_endettement": 1, "nb_incidents_12m": 1, "revenu_annuel": -1, "anciennete_emploi": -1,
↳ "anciennete_relation": -1}
def brut(d_):
    X = d_[["age", "revenu_annuel", "anciennete_emploi", "montant", "duree_mois", "taux_endettement",
↳ "nb_incidents_12m", "anciennete_relation"]].copy()
    for k in ["revenu_annuel", "anciennete_emploi"]: X[k + "_manq"] = X[k].isna().astype(int)
    return pd.concat([X, pd.get_dummies(d_[["logement", "objet"]]).astype(int)], axis=1)
Xa = brut(tr); mc = [signes.get(c, 0) for c in Xa.columns]
par = dict(n_estimators=300, learning_rate=0.03, num_leaves=8, min_child_samples=200, subsample=0.8,
↳ subsample_freq=1, colsample_bytree=0.8, random_state=0, verbose=-1)
libre = lgb.LGBMClassifier(**par).fit(Xa, y_tr); mono = lgb.LGBMClassifier(monotone_constraints=mc,
↳ **par).fit(Xa, y_tr)
Xr = brut(ref).reindex(columns=Xa.columns, fill_value=0)
```

```
for nom, m in [("libre", libre), ("monotone", mono)]:
    p = m.predict_proba(Xr)[: , 1]
    print(nom, "inversions :", int((np.diff(p) < -1e-12).sum()), "sur", len(p) - 1, "pas ; plus forte
    ↪ baisse :", round(float(np.diff(p).min()), 5))
```

```
libre inversions : 7 sur 90 pas ; plus forte baisse : -0.00166
monotone inversions : 0 sur 90 pas ; plus forte baisse : 0.0
```

Le boosting monotone n'a **aucune inversion** par construction ; le libre en a généralement quelques-unes : de petites baisses locales, accidents d'échantillon que l'on ne saurait pas justifier devant un client ou un auditeur.

Corrigé 1.6

Mauvais : 0,95 ; 0,70 ; 0,55. Bons : 0,85 ; 0,65 ; 0,40 ; 0,30 ; 0,10. Paires ($3 \times 5 = 15$) : 0,95 bat les 5 bons ; 0,70 bat 0,65, 0,40, 0,30, 0,10, soit 4 (pas 0,85) ; 0,55 bat 0,40, 0,30, 0,10, soit 3. Total $5 + 4 + 3 = 12$ sur 15 : AUC = 0,8, Gini = 0,6. KS : on classe par score décroissant ; TPR et FPR cumulés : après 0,95 : (1/3 ; 0) ; 0,85 : (1/3 ; 1/5) ; 0,70 : (2/3 ; 1/5) ; 0,65 : (2/3 ; 2/5) ; 0,55 : (1 ; 2/5) ; 0,40 : (1 ; 3/5) ; ... La plus grande différence TPR – FPR est $1 - \frac{2}{5} = 0,6$ (après 0,55). KS = 0,6.

```
y = np.array([1, 0, 1, 0, 1, 0, 0, 0]); s = np.array([.95, .85, .70, .65, .55, .40, .30, .10])
print(0. auc(y, s), 0. gini(y, s), 0. ks(y, s))
```

```
0.8 0.6000000000000001 0.6
```

Corrigé 1.7

```
from sklearn.metrics import roc_auc_score
rng = np.random.default_rng(0)
for pi in (0.02, 0.30):
    n = 400000; y = (rng.random(n) < pi).astype(int); s = rng.normal(y, 1.0)
    o = np.argsort(-s); cap = np.cumsum(y[o]) / y.sum(); x = np.arange(1, n + 1) / n
    aire = np.trapezoid(cap, x); ar = (aire - 0.5) / ((1 - y.mean() / 2) - 0.5)
    print(f"π = {y.mean():.3f} AUC {roc_auc_score(y, s):.4f} 2·AUC-1 {2 * roc_auc_score(y, s) -
    ↪ 1:.4f} accuracy ratio {ar:.4f}")
```

```
π = 0.020 AUC 0.7600 2·AUC-1 0.5201 accuracy ratio 0.5201
π = 0.300 AUC 0.7587 2·AUC-1 0.5174 accuracy ratio 0.5174
```

Les deux accuracy ratios coïncident avec $2 \text{AUC} - 1$ ($\approx 0,52$, car l'AUC théorique vaut $\Phi(1/\sqrt{2}) = 0,760$), **quelle que soit la proportion de mauvais** : c'est la démonstration du livre, vérifiée.

Corrigé 1.8

Contributions $(p_a - p_d) \ln(p_a/p_d)$: classe 1 : $(0,20 - 0,30) \ln(0,20/0,30) = (-0,10)(-0,405) = 0,0405$; classe 2 : $(-0,05) \ln(0,25/0,30) = (-0,05)(-0,182) = 0,0091$; classe 3 : $(0,05) \ln(0,30/0,25) = 0,05 \times 0,182 = 0,0091$; classe 4 : $(0,10) \ln(0,25/0,15) = 0,10 \times 0,511 = 0,0511$. PSI = 0,1098 : au-dessus de 0,10, un changement à **examiner**. La classe 4 contribue le plus (0,051, près de la moitié) : sa part est passée de 15 % à 25 %, dix points de population qui ont changé de classe.

```
pa, pd_ = np.array([.20, .25, .30, .25]), np.array([.30, .30, .25, .15])
c = (pa - pd_) * np.log(pa / pd_); print(np.round(c, 4), round(float(c.sum()), 4))
```

```
[0.0405 0.0091 0.0091 0.0511] 0.1099
```

Corrigé 1.9

Sans lissage : $b_j = 0$, $WOE = \ln(g_j/0) = +\infty$: inutilisable (et l'IV est infini). Avec lissage : $g = (300 + 0,5)/(20\,000 + 0,5 \times 5) = 0,015$; $b = 0,5/(400 + 2,5) = 0,00124$; $WOE = \ln(0,015/0,00124) = 2,49$: fini, mais **très grand** et fondé sur 300 observations sans aucun mauvais. Règle : lisser, mais surtout **fusionner** une classe sans mauvais avec sa voisine (ou la plafonner), parce que l'absence de mauvais dans un échantillon fini ne prouve pas qu'il n'y en aura jamais (règle des « trois sur n » : zéro défaut sur 300 est compatible avec un taux vrai jusqu'à 1 %).

```
g, b = 300.5 / 20002.5, 0.5 / 402.5; print(round(g, 4), round(b, 5), round(float(np.log(g / b)), 2))
```

```
0.015 0.00124 2.49
```

Corrigé 1.10

```
rng = np.random.default_rng(0); ivs = []
for _ in range(30):
    x = rng.normal(size=len(tr)); q = np.unique(np.quantile(x, np.linspace(0, 1, 51)[1:-1])); i =
    ↪ np.searchsorted(q, x, side="right")
    n = np.bincount(i, minlength=len(q) + 1); m = np.bincount(i, weights=y_tr, minlength=len(q) + 1);
    ↪ ivs.append(0.iv_partition(n, m))
print("IV du bruit à 50 classes : moyenne", round(float(np.mean(ivs)), 4), " maximum",
    ↪ round(float(np.max(ivs)), 4))
```

```
IV du bruit à 50 classes : moyenne 0.0299 maximum 0.0417
```

L'IV de développement d'un pur bruit à 50 classes est en moyenne de **0,030**, avec un maximum de 0,042 sur 30 répétitions : une variable à 50 classes dont l'IV est de 0,03 ne prouve **rien**. Un seuil raisonnable est de l'ordre du maximum observé sur du bruit ($\approx 0,04$ à $0,05$), ce que justifie un ordre de grandeur : l'IV d'un bruit à k classes vaut environ $(k - 1) (1/M + 1/B)$, soit ici $49 \times (1/1\,671 + 1/26\,329) \approx 0,03$.

Corrigé 1.11

- (a) $0,02 \times 0,40 \times 20\,000/1,05^{0,5} = 160/1,0247 = 156,15$ €. (b) $h = 0,02$; $EAD_1 = 20\,000(1 - 0,5/3) = 16\,667$, $EAD_2 = 10\,000$, $EAD_3 = 3\,333$; probabilités marginales 0,02, 0,0196, 0,019208 ; facteurs d'actualisation $1,05^{-0,5} = 0,97590$, $1,05^{-1,5} = 0,92943$, $1,05^{-2,5} = 0,88517$. Termes : $0,02 \times 0,4 \times 16\,667 \times 0,9759 = 130,12$; $0,0196 \times 0,4 \times 10\,000 \times 0,92943 = 72,86$; $0,019208 \times 0,4 \times 3\,333 \times 0,88517 = 22,67$. Somme $\approx 225,65$ €. (c) Rapport $225,65/156,15 = 1,45$.

```
d1 = pd.DataFrame({"pd_actuelle": [0.02], "lgd_estimee": [0.40], "ead": [20000.0], "taux_effectif":
    ↪ [0.05], "maturite_residuelle": [3.0]})
print(0.ecl_ifrs9(d1, np.array([1])), 0.ecl_ifrs9(d1, np.array([2])))
```

```
[156.14401167] [225.65701242]
```

Corrigé 1.12

```
pf = pd.read_csv("donnees/portefeuille_ifrs9.csv"); lignes = []
for jours in (15, 30, 45, 60):
    et = 0.etape_ifrs9(pf, retard_2=jours); lignes.append((jours, int((et == 2).sum()),
    ↪ 0.ecl_ifrs9(pf, et).sum() / 1e6))
print(pd.DataFrame(lignes, columns=["retard (jours)", "prêts en étape 2", "ECL
    ↪ (M€)"]).round(2).to_string(index=False))
```

retard (jours)	prêts en étape 2	ECL (M€)
15	3195	7.62
30	2923	7.59
45	2768	7.57
60	2618	7.55

Passer de 30 à 15 jours fait entrer davantage de prêts en étape 2, donc en provision sur la vie : la perte attendue **monte** de 0,4 % (7,62 contre 7,59 M€), et elle baisse de 0,5 % en passant de 30 à 60 jours (7,55 M€). La sensibilité est **faible** (moins de 1 %) parce que les retards de 15 à 60 jours concernent peu de prêts, et parce que d'autres critères (ratio de PD, restructuration) alimentent déjà l'étape 2 ; elle est bien plus forte pour le **seuil du ratio de PD** (application 1.7, étape 3 : de 7,23 à 7,90 M€ entre les seuils de 4 et 2). La conclusion générale reste : la provision dépend de **conventions** de basculement qu'il faut documenter et surveiller.

Corrigé 1.13

Matrice P (A, B, D) : $\begin{bmatrix} 0,90 & 0,08 & 0,02 \\ 0,10 & 0,75 & 0,15 \\ 0 & 0 & 1 \end{bmatrix}$. À 2 ans : $P_{A \rightarrow D}^2 = 0,90 \times 0,02 + 0,08 \times 0,15 + 0,02 \times 1 = 0,018 + 0,012 + 0,02 = 0,050$; $P_{B \rightarrow D}^2 = 0,10 \times 0,02 + 0,75 \times 0,15 + 0,15 = 0,002 + 0,1125 + 0,15 = 0,2645$. $P_{A \rightarrow A}^2 = 0,90^2 + 0,08 \times 0,10 = 0,818$; $P_{A \rightarrow B}^2 = 0,90 \times 0,08 + 0,08 \times 0,75 = 0,132$. À 3 ans : $P_{A \rightarrow D}^3 = 0,818 \times 0,02 + 0,132 \times 0,15 + 0,050 = 0,01636 + 0,0198 + 0,050 = 0,0862$; $P_{B \rightarrow A}^2 = 0,10 \times 0,90 + 0,75 \times 0,10 = 0,165$, $P_{B \rightarrow B}^2 = 0,10 \times 0,08 + 0,75^2 = 0,5705$; $P_{B \rightarrow D}^3 = 0,165 \times 0,02 + 0,5705 \times 0,15 + 0,2645 = 0,0033 + 0,0856 + 0,2645 = 0,3534$.

```
P = np.array([[.90, .08, .02], [.10, .75, .15], [0, 0, 1.0]])
for n in (2, 3): print(n, np.round(np.linalg.matrix_power(P, n)[:2, 2], 4))
```

```
2 [0.05  0.2645]
3 [0.0862 0.3534]
```

Corrigé 1.14

```
panel = pd.read_csv("donnees/notations_panel.csv"); N = 0.compter_transitions(panel); M =
↳ 0.matrice_cohortes(N)
absorbe = lambda M_: np.vstack([M_, np.r_[np.zeros(7), 1.0]])
P2 = np.linalg.matrix_power(absorbe(M), 2)[:7, 7]
calme = absorbe(0.matrice_cohortes(0.compter_transitions(panel, [0, 1, 2, 3, 4, 7, 8, 9]))); rec =
↳ absorbe(0.matrice_cohortes(0.compter_transitions(panel, [5, 6])))
regime = lambda t: rec if t in (5, 6) else calme # matrice de
↳ l'année t selon le régime
a = panel.set_index(["id_emprunteur", "annee"]); lignes = []
for k in range(1, 8):
    dep = panel[(panel["note_debut"] == k) & (panel["annee"] <= 8) & (panel["note_fin"] > 0)] #
    ↳ notés k en t, non sortis en fin de t
    suiv = a.reindex(list(zip(dep["id_emprunteur"], dep["annee"] + 1)))
    obs = ((dep["note_fin"].values == 8) | (suiv["note_fin"].values == 8)).mean() #
    ↳ défaut en t ou en t+1
    w = dep.groupby("annee").size()
    p_reg = sum(w[t] * (regime(t) @ regime(t + 1))[k - 1, 7] for t in w.index) / w.sum()
    lignes.append((k, len(dep), obs, P2[k - 1], p_reg))
print(pd.DataFrame(lignes, columns=["note", "n", "observé", "P² (matrice moyenne)", "P² par
↳ régime"]).round(4).to_string(index=False))
```

note	n	observé	P² (matrice moyenne)	P² par régime
1	2163	0.0014	0.0009	0.0009
2	6022	0.0033	0.0025	0.0026
3	9304	0.0060	0.0062	0.0065
4	8860	0.0157	0.0159	0.0164
5	4960	0.0512	0.0510	0.0528

6	2426	0.1645	0.1565	0.1612
7	1084	0.5148	0.5160	0.5162

La probabilité de défaut à deux ans **observée** est très proche de P^2 pour toutes les notes (par exemple 0,0512 contre 0,0510 pour la note 5) : la relation de **Chapman-Kolmogorov** $P^{(2)} = P \cdot P$ est vérifiée, ce qui est cohérent avec des données simulées par une chaîne de Markov. Les écarts qui subsistent sont petits et viennent de deux sources : le bruit d'échantillonnage (surtout pour les notes extrêmes, 1 084 observations pour la note 7) et le fait que la matrice moyenne ne tient pas compte de **l'ordre des années** : la colonne « par régime », qui utilise la matrice de récession pour les années 5 et 6 et pondère par les effectifs réellement observés, rapproche la prédiction de l'observation pour la note 6 (0,1612 contre 0,1645 observé, au lieu de 0,1565). Sur des données réelles, ce test est un bon moyen de repérer un effet d'élan ou une dépendance à la durée dans la note : un P^2 nettement inférieur à l'observé signalerait une persistance que le modèle ignore.

Pistes pour les « À vous » des applications

- **1.1** Un score de base différent change tous les points, mais **pas le rang** : l'AUC est identique (c'est une transformation affine de la log-cote). Retirer les trois variables d'IV le plus faible (objet, durée, ancienneté de la relation) coûte 0,006 d'AUC (0,765 contre 0,771) : la carte est plus courte pour une perte modeste, à comparer à l'intervalle de l'AUC ($\pm 0,02$).
- **1.2** Le coefficient 2 rapproche la PD annoncée du défaut réel (0,049 contre 0,060), le coefficient 1 ne corrige presque rien : mais on ne peut le choisir ainsi qu'**ici**, parce que nous connaissons l'issue des refusés. En pratique, le coefficient est un jugement, validé par des données externes ou une expérience d'acceptation.
- **1.3** (a) Une régression logistique sur la sortie du boosting, ajustée sur 30 % du développement mis de côté, fait baisser le Brier très légèrement (0,0504 contre 0,0502) : ce boosting est déjà presque calibré. (b) Une contrainte de monotonie sur l'âge est une **erreur** : la vérité est en U ; le modèle ne pourrait plus représenter le sur-risque des plus de 65 ans.
- **1.4** Avec une perte de 6 000 € par défaut, le seuil optimal **monte** (on devient plus sélectif) : de 556 points à 560 (et à 570 pour une perte de 8 000 €) sur une grille de pas 2 ; avec le pas de 10 de l'étape 1, la différence est invisible. Le profit au seuil optimal baisse de 2 958 à 2 337 k€.
- **1.5** Pour le revenu, la fusion monotone ne coûte presque rien (AUC 0,7709 contre 0,7712, neuf classes) ; pour le taux d'endettement, elle coûte 0,004 (0,767, sept classes) : la monotonie y est vraie, mais la fusion regroupe des classes de risques voisins et perd un peu de finesse.
- **1.6** Les LGD par segment (garantie $\times$ objet) font aussi bien que les modèles : l'erreur ne baisse pas, ce qui confirme que la moyenne par segment est ce que l'on peut prédire.
- **1.7** Avec les poids 40 / 40 / 20, l'ECL pondérée passe de 10,8 à 12,0 M€ (le scénario défavorable pèse davantage). Un seuil de retard de 15 jours ajoute 272 prêts en étape 2 (3 195 contre 2 923) mais l'ECL du scénario défavorable ne bouge que de 0,07 M€ (19,96 contre 19,89).
- **1.8** Estimée sur les cinq premières années (0 à 4, sans récession), la matrice prédit 1,8 % de défauts pour les années 5 et 6, contre 3,9 % observés : elle **sous-estime de plus de moitié** : la conjoncture ne se laisse pas prévoir par une matrice moyenne.

Chapitre 2 : Modélisation actuarielle — exercices et applications

Ce chapitre du cahier accompagne le chapitre 2 du livre. Les **applications** refont, étape par étape et avec le code, les études du livre (comptage, queue, modèle collectif, GLM de tarification, validation, provisionnement, crédibilité, santé) ; les **exercices** se travaillent d'abord à la main, puis se vérifient par le calcul. Données : `polices_auto.csv`, `sinistres_auto.csv`, les triangles et `sante_assures.csv`, toutes **simulées**. Le cahier est autonome : il recharge ses données.

Applications

Préparation commune

À exécuter une fois : elle charge les données, revalorise les montants en euros de 2024 et sépare **estimation (2022-2023)** et **test (2024)**.

```
import sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
from outils_ch02 import *

pol, sin = charger_polices()
sin = sin.merge(pol[["id_police", "puissance", "zone", "classe_age"]], on="id_police")
sin["rev"] = sin["montant"] * 1.04 ** (2024 - sin["annee"]) # euros de 2024
tr, te = pol[pol["annee"] <= 2023].copy(), pol[pol["annee"] == 2024].copy()
print(len(pol), "lignes police-année ;", len(sin), "sinistres ; exposition 2024 :",
      round(te["exposition"].sum()))
```

```
100000 lignes police-année ; 5722 sinistres ; exposition 2024 : 32081
```

Application 2.1 — Compter les sinistres : exposition et Poisson (section 2.1.2)

Objectif. Estimer la fréquence annuelle par année d'exposition, mesurer son incertitude, et mesurer le biais d'une estimation « par contrat ».

Étape 1 — le taux et son erreur-type. Le taux estimé est $\hat{\lambda} = \sum N_i / \sum e_i$ et son erreur-type $\sqrt{\hat{\lambda} / \sum e_i}$.

```
N, E = pol["nb_sinistres"].sum(), pol["exposition"].sum()
lam = N / E
print(f"taux = {lam:.4f} ± {np.sqrt(lam / E):.4f} (par contrat : {N / len(pol):.4f})")
```

```
taux = 0.0661 ± 0.0009 (par contrat : 0.0572)
```

Étape 2 — par année et par zone. On calcule le taux de chaque année et de chaque zone, avec l'intervalle de confiance exact à 95 % de Garwood pour une moyenne de Poisson, déduit de la loi du χ^2 : de $\chi^2_{0,025}(2n)/2$ à $\chi^2_{0,975}(2n+2)/2$ pour n sinistres, divisé par l'exposition.

```
g = pol.groupby("zone").agg(n=("nb_sinistres", "sum"), e=("exposition", "sum"))
bas, haut = stats.chi2.ppf(0.025, 2 * g["n"]) / 2, stats.chi2.ppf(0.975, 2 * g["n"] + 2) / 2
g["taux %"] = 100 * g["n"] / g["e"]
g["bas %"], g["haut %"] = 100 * bas / g["e"], 100 * haut / g["e"]
print(g[["e", "n", "taux %", "bas %", "haut %"]].round(2))
```

	e	n	taux %	bas %	haut %
zone					
Zone A	10358.32	559	5.40	4.96	5.86
Zone B	17463.34	1066	6.10	5.74	6.48
Zone C	21637.76	1311	6.06	5.74	6.40
Zone D	17471.88	1168	6.69	6.31	7.08
Zone E	11917.69	953	8.00	7.50	8.52
Zone F	7752.55	665	8.58	7.94	9.26

Lecture. Le taux global vaut 6,61 %, avec une erreur-type de 0,09 point ; l'estimation « par contrat » (5,72 %) le sous-estime de 13 %. Les intervalles par zone se recouvrent beaucoup (la zone A va de 5,0 % à 5,9 %, la zone F de 7,9 % à 9,3 %) : les écarts de fréquence entre zones **existent**, mais une comparaison zone par zone, sans modèle, manque de précision.

Pour aller plus loin. Refaites le calcul sur 2022 seul puis sur 2024 seul : comparez l'erreur-type à l'écart entre les deux taux (est-il statistiquement significatif ?).

Application 2.2 — Sur-dispersion et binomiale négative (section 2.1.3)

Objectif. Montrer, par simulation puis sur les données, que l'hétérogénéité non observée produit une variance supérieure à la moyenne.

Étape 1 — Poisson–Gamma par simulation. On tire $\Theta \sim \text{Gamma}$ de moyenne 1 et de variance α , puis $N | \Theta \sim \text{Poisson}(\mu\Theta)$.

```
rng = np.random.default_rng(0)
mu, alpha = 0.2, 0.8
theta = rng.gamma(1 / alpha, alpha, 1_000_000) # forme 1/alpha, échelle alpha : moyenne 1,
↳ variance alpha
n_sim = rng.poisson(mu * theta)
print("moyenne", n_sim.mean().round(4), "; variance", n_sim.var().round(4), "; théorie :", mu + alpha
↳ * mu ** 2)
```

```
moyenne 0.1997 ; variance 0.2318 ; théorie : 0.232
```

Étape 2 — la même chose sur les données. On ajuste un modèle de Poisson puis un modèle

binomial négatif avec les mêmes variables et l'exposition en décalage.

```
F = "nb_sinistres ~ C(classe_age) + puissance + C(zone) + bonus_malus + C(usage) + C(carburant) +
↪ age_vehicule"
poi = smf.glm(F, pol, family=sm.families.Poisson(), offset=np.log(pol["exposition"])).fit()
nbm = smf.negativebinomial(F, pol, offset=np.log(pol["exposition"])).fit(dis=0)
print("dispersion de Pearson :", round(poi.pearson_chi2 / poi.df_resid, 3))
print("alpha =", round(nbm.params["alpha"], 3), "± ", round(nbm.bse["alpha"], 3))
```

```
dispersion de Pearson : 1.027
alpha = 0.473 ± 0.09
```

Lecture. La simulation donne une variance de 0,232 pour une valeur théorique de 0,232. Sur les données, la dispersion de Pearson est de 1,027 et $\hat{\alpha} = 0,47$ (0,09) : la sur-dispersion est discrète (le taux est faible) mais **significative** (5,3 erreurs-types de zéro). Comparez avec la vérité programmée, 0,4.

Application 2.3 — La queue et la charge annuelle (sections 2.1.5 et 2.1.6)

Objectif. Ajuster une loi de Pareto généralisée, écrêter, ajouter une charge pour gros sinistres, puis simuler la charge d'une année.

Étape 1 — la GPD à 100 000 €. Les excès au-delà du seuil sont ajustés par `genpareto` avec une position fixée à zéro.

```
u = 100_000
exces = sin.loc[sin["rev"] > u, "rev"] - u
xi, _, echelle = stats.genpareto.fit(exces, floc=0)
print(len(exces), "excès ; xi =", round(xi, 3), "; échelle =", round(echelle))
print("part du coût au-delà du seuil :", round((sin["rev"] - u).clip(lower=0).sum() /
↪ sin["rev"].sum(), 3))
```

```
91 excès ; xi = 0.664 ; échelle = 72867
part du coût au-delà du seuil : 0.337
```

Étape 2 — écrêtement et charge. On plafonne chaque sinistre à 100 000 € et l'on calcule la charge commune des gros sinistres par année d'exposition.

```
ecrete = sin["rev"].clip(upper=u)
charge = (sin["rev"] - u).clip(lower=0).sum() / pol["exposition"].sum()
print("moyenne écrêtée :", round(ecrete.mean(), 1), "; moyenne brute :", round(sin["rev"].mean(), 1), ";
↪ charge :", round(charge, 1), "€ par année d'exposition")
```

```
moyenne écrêtée : 5328 ; moyenne brute : 8033 ; charge : 178.7 € par année d'exposition
```

Étape 3 — la charge annuelle d'un portefeuille de 2024. Le nombre de sinistres est de Poisson de moyenne $\hat{\lambda} \times$ exposition 2024 ; les montants sont rééchantillonnés dans l'historique (revalorisé).

```
rng = np.random.default_rng(1)
lam24 = pol["nb_sinistres"].sum() / pol["exposition"].sum() * te["exposition"].sum()
x = sin["rev"].values
S = np.array([rng.choice(x, n).sum() for n in rng.poisson(lam24, 2000)])
print("moyenne", round(S.mean() / 1e6, 2), "M€ ; écart-type", round(S.std() / 1e6, 2), "M€ ; quantile
↪ 99,5 %", round(np.quantile(S, 0.995) / 1e6, 2), "M€")
```

```
moyenne 17.0 M€ ; écart-type 2.41 M€ ; quantile 99,5 % 23.9 M€
```

Lecture. À 100 000 €, $\hat{\xi} = 0,66$ sur 91 excès : une valeur supérieure à 0,5 (variance infinie), mais très incertaine. La part du coût située **au-delà** de 100 000 € (l'excès) représente 34 % du coût total. L'écrêtement ramène la moyenne de 8 033 € à 5 328 € et laisse une charge de 179 € par année

d'exposition. La charge annuelle simulée a pour moyenne 17,0 M€, pour écart-type 2,41 M€ et pour quantile à 99,5 % 23,9 M€.

Pour aller plus loin. Refaites l'étape 3 en remplaçant le rééchantillonnage par une simulation où les montants dépassant 100 000 € sont tirés dans la GPD ajustée : le quantile à 99,5 % change-t-il ? De combien ?

Application 2.4 — Un tarif par deux GLM (section 2.2.2)

Objectif. Estimer un GLM de fréquence (2022-2023), un GLM de sévérité des sinistres matériels, puis calculer une prime pure pour trois profils.

Étape 1 — la fréquence.

```
F = ("nb_sinistres ~ C(classe_age, Treatment('40-49')) + puissance + C(zone, Treatment('Zone C'))"
    + " + bonus_malus + C(usage) + C(carburant) + age_vehicule")
freq = smf.glm(F, tr, family=sm.families.Poisson(), offset=np.log(tr["exposition"])).fit()
rel = np.exp(freq.params).round(3)
print(rel[["puissance", "bonus_malus", "C(usage)[T.professionnel]"]])
```

```
puissance          1.058
bonus_malus        1.006
C(usage)[T.professionnel]  1.153
dtype: float64
```

Étape 2 — la sévérité, par type. Matériel : GLM Gamma ; corporel : moyenne unique après écrêtement à 100 000 € ; plus la charge pour gros sinistres.

```
st = sin[sin["annee"] <= 2023]
sev = smf.glm("rev ~ puissance + C(zone, Treatment('Zone C'))", st[st["type"] == "materiel"],
             family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
q = (st["type"] == "corporel").mean(); m_cor = st.loc[st["type"] == "corporel",
    ↪ "rev"].clip(upper=100_000).mean()
charge = (st["rev"] - 100_000).clip(lower=0).sum() / tr["exposition"].sum()
print("part corporelle", round(q, 3), "; moyenne écrêtée", round(m_cor), "; charge", round(charge, 1))
```

```
part corporelle 0.101 ; moyenne écrêtée 29686 ; charge 234.0
```

Étape 3 — trois profils. Prime pure = $\hat{\lambda}(x) [(1 - q)\hat{\mu}_{\text{mat}}(x) + q m_{\text{cor}}] + \text{charge}$.

```
prof = pd.DataFrame({"classe_age": ["18-24", "40-49", "70+"], "puissance": [7, 5, 3], "zone": ["Zone
    ↪ F", "Zone C", "Zone A"],
                    "bonus_malus": [100, 70, 60], "usage": ["prive"] * 3, "carburant": ["essence",
    ↪ "diesel", "hybride_electrique"],
                    "age_vehicule": [3.0, 6.0, 2.0]})
lam_p = freq.predict(prof, offset=np.zeros(3)); sev_p = (1 - q) * sev.predict(prof) + q * m_cor
prof["prime pure"] = (lam_p * sev_p + charge).round(0)
print(prof[["classe_age", "zone", "prime pure"]])
```

```
classe_age  zone  prime pure
0      18-24  Zone F      1319.0
1      40-49  Zone C       508.0
2       70+  Zone A       460.0
```

Lecture. Les relativités estimées pour la puissance (1,058 par niveau), le bonus-malus (1,006 par point) et l'usage professionnel (1,153) se comparent à leurs valeurs programmées (1,041 ; 1,006 ; 1,105). Les trois profils sont tarifés 1 319 €, 508 € et 460 € : le premier coûte 2,9 fois le troisième.

Application 2.5 — Valider hors période et mesurer l'antisélection (sections 2.2.5 et 2.2.6)

Objectif. Calculer le Gini de tarification de 2024 pour un tarif plat et le tarif de l'application 2.4, puis simuler un concurrent mieux segmenté.

Étape 1 — les primes de 2024 et les coûts observés (écrêtés).

```
te["lam"] = freq.predict(te, offset=np.zeros(len(te)))
te["pp"] = te["lam"] * ((1 - q) * sev.predict(te) + q * m_cor) + charge
s24 = sin[sin["annee"] == 2024]
c24 = cout_par_police(te, s24[["id_police"]].assign(montant=s24["rev"].clip(upper=100_000).values))
e24, y24 = te["exposition"].values, c24["cout"].values
base = st["rev"].clip(upper=100_000).sum() / tr["exposition"].sum() + charge
print("prime moyenne du tarif :", round((te["pp"] * e24).sum() / e24.sum()), "; tarif plat :",
      ↪ round(base))
```

```
prime moyenne du tarif : 591 ; tarif plat : 592
```

Étape 2 — le Gini de chaque tarif, avec un intervalle. Le tarif plat n'a aucun pouvoir de classement ; un tarif aléatoire sert de repère du bruit.

```
tarifs = {"plat": np.full(len(te), base) * e24, "complet": te["pp"].values * e24}
gini = {k: lorenz_gini(v, y24, e24)[2] for k, v in tarifs.items()}
rng = np.random.default_rng(0)
bruit = np.std([lorenz_gini(rng.random(len(y24)) * e24, y24, e24)[2] for _ in range(50)])
print({k: round(v, 3) for k, v in gini.items()}, "; bruit d'un tarif aléatoire :", round(bruit, 3))
```

```
{'plat': 0.018, 'complet': 0.178} ; bruit d'un tarif aléatoire : 0.037
```

Étape 3 — l'antisélection. Un concurrent applique le tarif complet ; nous appliquons un tarif plat. Nos assurés restent s'ils sont moins chers chez nous ; la charge des gros sinistres est fixée à son niveau prévu.

```
reste = tarifs["plat"] <= tarifs["complet"]
sp0 = (y24.sum() + charge * e24.sum()) / tarifs["plat"].sum()
sp1 = (y24[reste].sum() + charge * e24[reste].sum()) / tarifs["plat"][reste].sum()
print("part conservée", round(reste.mean(), 3), "; sinistres/primes :", round(sp0, 3), "->",
      ↪ round(sp1, 3))
```

```
part conservée 0.375 ; sinistres/primes : 0.975 -> 1.145
```

Lecture. Le Gini est de 0,02 pour le tarif plat (le bruit d'un tarif aléatoire est de 0,037) et de 0,18 pour le tarif complet. Après antisélection, la mutuelle ne garde que 38 % de son exposition et son ratio sinistres sur primes passe de 97 % à 114 %.

Pour aller plus loin. Remplacez la règle « on part si le concurrent est moins cher » par une probabilité de départ qui croît avec l'écart de prix (par exemple une logistique de pente 10 sur l'écart relatif) : l'antisélection est-elle aussi forte ?

Application 2.6 — Chain ladder, de la main au code (section 2.3)

Objectif. Refaire le chain ladder du livre sur le triangle de responsabilité civile, estimer une queue et comparer aux paiements réels.

```
tri = pd.read_csv("donnees/triangle_rc.csv"); ver = pd.read_csv("donnees/triangle_rc_verite.csv")
Cum = triangle_cumule(tri).values
V = ver.pivot(index="annee_survenance", columns="delai", values="paiement_cumule").values
f = facteurs_chain_ladder(Cum)
paye = dernier_cumul(Cum)
print("facteurs :", f.round(3))
```

```
facteurs : [2.52  1.612 1.304 1.206 1.129 1.087 1.043 1.038 1.017]
```

```
queue, taux = facteur_queue(f, depuis=4)
res = {}
for nom, fq in (("sans queue", 1.0), ("queue extrapolée", queue)):
    _, ult = projeter(Cum, f, fq); res[nom] = (ult - paye).sum() / 1e6
res["réel"] = (V[:, -1] - paye).sum() / 1e6
print(round(queue, 4), {k: round(float(v), 1) for k, v in res.items()})
```

```
1.0293 {'sans queue': 230.2, 'queue extrapolée': 249.3, 'réel': 242.1}
```

Lecture. Les facteurs vont de 2,52 à 1,017. Le facteur de queue extrapolé vaut 1,029 (la décroissance des $f_j - 1$ est de 0,61 par délai). Les provisions sont de 230,2 M€ sans queue, 249,3 M€ avec la queue extrapolée, pour 242,1 M€ réellement payés ensuite.

Pour aller plus loin. Recommencez avec `facteur_queue(f, depuis=6)` : de combien la provision bouge-t-elle ? Que conclure sur le choix de la queue ?

Application 2.7 — Déviance et crédibilité (section 2.4)

Objectif. Tester l'apport d'une variable par le rapport de vraisemblance, puis estimer une crédibilité de Bühlmann–Straub sur des cellules tarifaires.

Étape 1 — le test de la zone.

```
F0 = "nb_sinistres ~ C(classe_age, Treatment('40-49')) + puissance + bonus_malus + C(usage) +
    C(carburant) + age_vehicule"
m0 = smf.glm(F0, tr, family=sm.families.Poisson(), offset=np.log(tr["exposition"])).fit()
dd = m0.deviance - freq.deviance
print("baisse de déviance :", round(dd, 1), "pour 5 paramètres ; p =", stats.chi2.sf(dd, 5))
```

```
baisse de déviance : 80.0 pour 5 paramètres ; p = 8.567383518416896e-16
```

Étape 2 — la crédibilité sur 108 cellules (zone × puissance × usage), estimée sur 2022-2023 et jugée sur 2024.

```
pol["cellule"] = pol["zone"] + "|" + pol["puissance"].astype(str) + "|" + pol["usage"]
g = pol[pol["annee"] <= 2023].groupby(["cellule", "annee"]).agg(n=("nb_sinistres", "sum"),
    e=("exposition", "sum")).reset_index()
fr = g.pivot(index="cellule", columns="annee", values="n") / g.pivot(index="cellule",
    columns="annee", values="e")
w = g.pivot(index="cellule", columns="annee", values="e").fillna(0)
cs = buhlmann_straub(fr.values, w.values)
print({k: round(cs[k], 4) for k in ("mu", "s2", "a", "k")}, "; Z médian", round(np.median(cs["Z"]),
    3))
```

```
{'mu': 0.0658, 's2': 0.074, 'a': 0.0001, 'k': 502.6088} ; Z médian 0.325
```

Étape 3 — le jugement sur 2024.

```
t24 = pol[pol["annee"] == 2024].groupby("cellule").agg(n=("nb_sinistres", "sum"), e=("exposition",
    "sum"))
d = pd.DataFrame({"brute": cs["xi"], "crédibilité": cs["Z"] * cs["xi"] + (1 - cs["Z"]) * cs["mu"],
    "moyenne": cs["mu"]}, index=fr.index).join(t24, how="inner")
print({k: round(deviance_poisson(d["n"], (d[k] * d["e"]).clip(lower=1e-6)), 1) for k in ("brute",
    "crédibilité", "moyenne")})
```

```
{'brute': 276.9, 'crédibilité': 140.7, 'moyenne': 195.0}
```

Lecture. Ajouter la zone fait baisser la déviance de 80,0 pour 5 paramètres : le test la retient sans hésitation ($p = 8,6 \times 10^{-16}$ environ). Le point de crédibilité est de 503 années d'exposition ; sur 2024, la déviance des cellules est de 276,9 pour l'expérience brute, 195,0 pour la moyenne et 140,7 pour la crédibilité : la crédibilité est la meilleure des trois.

Application 2.8 — Bornhuetter–Ferguson, Mack et bootstrap (section 2.5)

Objectif. Appliquer BF et Cape Cod, calculer l'erreur de Mack et simuler le bootstrap sur le triangle de responsabilité civile.

Étape 1 — BF et Cape Cod.

```
prime = tri.groupby("annee_survenance")["prime_acquise"].first().values
ders = [int(np.max(np.where(~np.isnan(Cum[i]))[0])) for i in range(Cum.shape[0])]
pct = np.array([1 / (np.prod(f[d:]) * queue) for d in ders]) # part déjà payée attendue
rho_cc = paye.sum() / (prime * pct).sum() # Cape Cod : a priori appris
↳ dans les données
bf = bornhuetter_ferguson(Cum, f, prime, 0.80, queue); cc = bornhuetter_ferguson(Cum, f, prime,
↳ rho_cc, queue)
print("a priori Cape Cod :", round(rho_cc, 3), "; provisions BF / CC (M€) :", round((bf - paye).sum()
↳ / 1e6, 1), round((cc - paye).sum() / 1e6, 1))
```

a priori Cape Cod : 0.919 ; provisions BF / CC (M€) : 207.4 238.3

Étape 2 — l'erreur de Mack.

```
mk, se, sig2 = mack(Cum)
print("provision CL :", round(mk["reserve"].sum() / 1e6, 1), "M€ ; erreur-type de Mack :", round(se /
↳ 1e6, 2), "M€")
```

provision CL : 230.2 M€ ; erreur-type de Mack : 5.36 M€

Étape 3 — le bootstrap ODP (500 simulations).

```
inc = triangle_cumule(tri, "paiement_incremental").values
sims, centrale, phi = bootstrap_odp(inc, n_boot=500, graine=1)
print("centrale :", round(centrale / 1e6, 1), "; écart-type :", round(sims.std() / 1e6, 2), ";
↳ quantile 99,5 % :", round(np.quantile(sims, 0.995) / 1e6, 1))
```

centrale : 230.2 ; écart-type : 5.68 ; quantile 99,5 % : 246.3

Lecture. L'a priori Cape Cod est de 92 % (le ratio programmé moyen est de l'ordre de 92 %) ; BF avec 80 % donne 207,4 M€, Cape Cod 238,3 M€, pour une provision réelle de 242,1 M€. L'erreur de Mack est de 5,4 M€ ; le bootstrap donne 5,7 M€ et une provision centrale de 230,2 M€, **identique** à celle du chain ladder (230,2 M€).

Application 2.9 — Un portefeuille de santé (section 2.6)

Objectif. Mesurer la concentration du coût, séparer sélection adverse et aléa moral, et estimer l'effet d'une prime unique.

Étape 1 — concentration et facteurs.

```
sante = pd.read_csv("donnees/sante_assures.csv")
sante["cpe"] = sante["cout_total"] / sante["exposition"]
cs = np.sort(sante["cout_total"].values[::-1])
print("5 % les plus coûteux :", round(cs[: int(0.05 * len(cs))].sum() / cs.sum(), 3), "du coût")
print(sante.groupby("ald").apply(lambda g: g["cout_total"].sum() /
↳ g["exposition"].sum()).round(0).to_dict())
```

```
5 % les plus coûteux : 0.456 du coût
{0: 982.0, 1: 3464.0}
```

Étape 2 — ajuster l'effet du niveau.

```
fo = "cpe ~ age + I(age ** 2) + ald + C(niveau, Treatment('basique')) + C(sexe)"
mg = smf.glm(fo, sante, family=sm.families.Gamma(sm.families.links.Log()),
  <- freq_weights=sante["exposition"]).fit()
brut = sante.groupby("niveau").apply(lambda g: g["cout_total"].sum() / g["exposition"].sum())
print("brut premium/basique :", round(brut["premium"] / brut["basique"], 2), "; ajusté :",
  round(float(np.exp(mg.params["C(niveau, Treatment('basique'))[T.premium]")), 2))
```

```
brut premium/basique : 2.14 ; ajusté : 1.48
```

Lecture. Les 5 % les plus coûteux portent 46 % du coût ; l'ALD multiplie le coût par 3,5. Le rapport brut entre « premium » et « basique » est de 2,14, le rapport ajusté de 1,48 : une partie seulement de l'écart brut vient du comportement ; le reste est de la **sélection**.

Exercices

Exercice 2.1 ★ — Moyenne et variance de la charge (section 2.1.6)

Le nombre annuel de sinistres d'un portefeuille suit une loi de Poisson de moyenne 0,5 par contrat, et un sinistre coûte 1 000 € avec la probabilité 0,7 ou 3 000 € avec la probabilité 0,3. Calculez $E[S]$, $\text{Var}(S)$ et l'écart-type de la charge d'un contrat, puis vérifiez par simulation.

Exercice 2.2 ★★ — Estimer une fréquence avec exposition (section 2.1.2)

Quatre contrats ont pour expositions (1;0,5;0,25;1) et pour nombres de sinistres (0;1;0;1). Calculez l'estimateur $\hat{\lambda}$ du taux annuel, son erreur-type approchée, et comparez-le à l'estimation « par contrat ». Quel est l'intervalle de confiance exact à 95 % (Garwood) pour le nombre moyen de sinistres, puis pour le taux annuel ?

Exercice 2.3 ★★ — Poisson contre binomiale négative (section 2.1.3)

Pour un contrat de moyenne $\mu = 0,1$ et une sur-dispersion $\alpha = 0,5$, calculez, sous une loi de Poisson puis sous une binomiale négative de même moyenne, $P(N = 0)$ et $P(N = 2)$, et le rapport variance sur moyenne. Que concluez-vous sur la prévision des contrats à plusieurs sinistres ?

Exercice 2.4 ★ — De la prime pure au prix (section 2.2.1 et 2.2.4)

Un segment a une fréquence de 8 % et une sévérité moyenne de 2 500 €. Les frais fixes sont de 40 € par contrat, les frais proportionnels (commissions, taxes) de 14 % du prix, la marge de 5 %. Calculez la prime pure, le prix commercial, et le ratio sinistres sur primes attendu. De combien le prix doit-il augmenter si la sévérité augmente de 4 % ?

Exercice 2.5 ★★ — Lire des relativités (section 2.2.2)

Un GLM de fréquence a pour coefficients (référence : zone C, 40-49 ans) : constante $-3,00$; zone F $+0,30$; 18-24 ans $+0,60$; usage professionnel $+0,10$. Calculez la fréquence d'un conducteur de 20 ans, en zone F, à usage professionnel, puis la même avec une exposition de 0,5 an. Quelle est la relativité de la zone F par rapport à la zone C ? Que devient-elle si l'on prend la zone F comme référence ?

Exercice 2.6 ★★★ — Un Gini à la main (section 2.2.5)

Six contrats d'exposition 1 ont pour primes prédites (100, 200, 150, 300, 250, 120) et pour coûts observés (0, 500, 0, 400, 100, 0). Classez-les, tracez la courbe de Lorenz ordonnée, calculez le Gini par la méthode des trapèzes. Comparez à la courbe de la diagonale et commentez.

Exercice 2.7 ★ — Chain ladder sur un petit triangle (section 2.3.3)

Soit le triangle cumulé (en k€) : année 1 : 120, 180, 198 ; année 2 : 130, 195 ; année 3 : 140. Calculez les facteurs $\hat{f}_0, \hat{f}_1$ puis les ultimes et la provision totale. Refaites le calcul avec la **moyenne simple** des rapports individuels au lieu de la moyenne pondérée. Les résultats diffèrent-ils ?

Exercice 2.8 ★★ — Le facteur de queue (section 2.3.4)

Les derniers facteurs d'un triangle sont $\hat{f}_5 = 1,120, \hat{f}_6 = 1,060, \hat{f}_7 = 1,030$. En supposant que $\hat{f}_j - 1$ décroît géométriquement de raison r estimée sur ces trois valeurs, calculez r , puis le facteur de queue (produit infini). Que devient-il si r passe de 0,5 à 0,6 ? Commentez la sensibilité.

Exercice 2.9 ★★ — Test du rapport de vraisemblance (section 2.4.1)

Deux modèles emboîtés ont pour déviances 1 250,0 (8 paramètres) et 1 238,5 (11 paramètres) sur le même jeu. Calculez la baisse de déviance, la p-valeur du test, et dites si les trois paramètres supplémentaires sont justifiés. Comparez avec le critère AIC (qui ajoute $2 \times$ le nombre de paramètres à la déviance, à une constante près).

Exercice 2.10 ★★★ — Crédibilité de Bühlmann–Straub (section 2.4.4)

La variance de processus est $s^2 = 0,07$ (pour une fréquence par année d'exposition) et la variance entre groupes $a = 0,00014$. Calculez k . Trois groupes ont pour expositions 100, 600 et 4 000 années, et pour fréquences observées 9 %, 5 % et 6,4 %, la moyenne générale étant 6,6 %. Calculez les facteurs de crédibilité et les estimations. Vérifiez ensuite, numériquement, l'exactitude de la crédibilité dans le modèle Poisson–Gamma ($\alpha = 0,4, \lambda = 0,066$).

Exercice 2.11 ★ — Bornhuetter–Ferguson et chain ladder (section 2.5.2)

Une année de survenance a déjà payé 20 M€, la part payée attendue est de 40 %, la prime acquise est de 100 M€ et le ratio a priori est de 80 %. Calculez l'ultime par chain ladder, l'ultime par BF, et vérifiez la relation de moyenne pondérée entre les deux.

Exercice 2.12 ★★ — La variance de développement de Mack (section 2.5.3)

Trois années ont, au délai 0, des cumuls (100, 120, 110) et au délai 1 des cumuls (160, 170, 180). Calculez $\hat{f}_0$, les rapports individuels, puis $\hat{\sigma}_0^2 = \frac{1}{n-1} \sum_i C_{i,0} (r_i - \hat{f}_0)^2$. Comment ce paramètre intervient-il dans l'erreur d'une provision ?

Exercice 2.13 ★★★ — Diagnostiquer un choc calendaire (section 2.5.5)

Reprenez le diagnostic des résidus de Pearson par année calendaire **sur le triangle triangle_dommages.csv** (qui n'a pas de choc) puis sur triangle_choc.csv. Comparez la plus grande moyenne de diagonale. Le diagnostic est-il utilisable sur un triangle court de six délais ?

Exercice 2.14 ★★ — Sélection et comportement en santé (section 2.6.3)

Dans le niveau « basique », 10 % des assurés ont une ALD (coût annuel moyen 3 000 €) et 90 % n'en ont pas (900 €). Dans le niveau « premium », 36 % ont une ALD et 64 % n'en ont pas, et la garantie premium augmente de 40 % la consommation de tous. Calculez le coût moyen de chaque niveau, le rapport brut, puis décomposez-le en un effet de **sélection** (composition) et un effet de **comportement** (+40 %).

Corrigés**Corrigé 2.1**

$E[X] = 0,7 \times 1\,000 + 0,3 \times 3\,000 = 1\,600$ €, $E[X^2] = 0,7 \times 10^6 + 0,3 \times 9 \times 10^6 = 3,4 \times 10^6$. Pour un Poisson de moyenne $\lambda = 0,5$, $E[S] = \lambda E[X] = 800$ € et $\text{Var}(S) = \lambda E[X^2] = 1,7 \times 10^6$, soit un écart-type de 1 304 €. La simulation de 1 000 000 de contrats le confirme.

```
rng = np.random.default_rng(0)
n = rng.poisson(0.5, 1_000_000)
tot = np.zeros(len(n)); k = n.max()
for j in range(1, k + 1):
    tot += np.where(n >= j, rng.choice([1000, 3000], len(n), p=[0.7, 0.3]), 0)
print(tot.mean().round(1), tot.std().round(1))
```

799.9 1302.7

La moyenne simulée est de 799,9 € et l'écart-type de 1302,7 €, en accord avec la théorie.

Corrigé 2.2

$\hat{\lambda} = \sum N_i / \sum e_i = 2/2,75 = 0,727$ par an. L'erreur-type est $\sqrt{\hat{\lambda} / \sum e_i} = \sqrt{0,727/2,75} = 0,514$: l'incertitude est énorme avec deux sinistres seulement. L'estimation « par contrat » donne $2/4 = 0,5$: elle sous-estime de 31 %, parce que deux contrats ne sont pas des années entières. L'intervalle exact de Garwood à 95 % pour 2 sinistres observés va de 0,24 à 7,22 sinistres, soit, par année d'exposition, de 0,09 à 2,63 : un facteur de près de 30 entre les deux bornes.

```
N, E = 2, 2.75
bas, haut = stats.chi2.ppf(0.025, 2 * N) / 2, stats.chi2.ppf(0.975, 2 * N + 2) / 2
print(round(N / E, 3), round(np.sqrt(N / E / E), 3), round(bas, 2), round(haut, 2))
```

0.727 0.514 0.24 7.22

Corrigé 2.3

Avec $r = 1/\alpha = 2$ et $p = r/(r + \mu) = 2/2,1$, la binomiale négative donne $P(N = 0) = p^r = (2/2,1)^2 = 0,9070$; la loi de Poisson, $e^{-0,1} = 0,9048$: les zéros sont presque identiques. Pour $N = 2$: binomiale négative $\frac{\Gamma(4)}{2!\Gamma(2)}p^2(1-p)^2 = 0,00617$; Poisson $\frac{0,1^2}{2}e^{-0,1} = 0,00452$, soit 1,4 fois moins. Le rapport variance sur moyenne est $1 + \alpha\mu = 1,05$. **Poisson sous-estime nettement les contrats à plusieurs sinistres**, bien que les zéros soient presque identiques : la différence est dans la queue.

```
mu, alpha = 0.1, 0.5; r = 1 / alpha
print(stats.nbinom.pmf([0, 2], r, r / (r + mu)).round(5), stats.poisson.pmf([0, 2], mu).round(5))
```

```
[0.90703 0.00617] [0.90484 0.00452]
```

Corrigé 2.4

Prime pure : $\pi = 0,08 \times 2\,500 = 200$ €. Prix : $P = (\pi + F)/(1 - \tau - m) = (200 + 40)/(1 - 0,14 - 0,05) = 240/0,81 = 296,3$ €. Ratio sinistres sur primes attendu : $200/296,3 = 67,5$ %. Si la sévérité augmente de 4 %, $\pi' = 208$ et $P' = 248/0,81 = 306,2$ €, soit 3,3 % de hausse : **moins de 4 %**, parce que les frais fixes (40 €) n'augmentent pas avec la sévérité.

Corrigé 2.5

Conducteur de 20 ans, zone F, professionnel : $\ln \lambda = -3,00 + 0,30 + 0,60 + 0,10 = -2,00$, soit $\lambda = e^{-2} = 0,1353$ sinistre par an ; avec une exposition de 0,5 an, $\mu = 0,5\lambda = 0,0677$ (on ajoute $\ln 0,5$ au prédicteur). La relativité de la zone F par rapport à C est $e^{0,30} = 1,350$. Avec F pour référence, la relativité de C devient $e^{-0,30} = 0,741$ ($= 1/1,350$) : **changer de référence inverse la relativité** mais ne change aucune prime. Les valeurs de la constante changent aussi : la constante est la fréquence de la modalité de référence.

Corrigé 2.6

On classe par prime croissante : 100 (coût 0), 120 (0), 150 (0), 200 (500), 250 (100), 300 (400). Le coût total est de 1 000 ; les parts cumulées de coût après chaque contrat sont 0, 0, 0, 0,5, 0,6, 1,0 pour des parts cumulées d'exposition de 1/6, 2/6, ..., 1. L'aire sous la courbe par trapèzes vaut $\frac{1}{6}[\frac{0+0}{2} + \frac{0+0}{2} + \frac{0+0}{2} + \frac{0+0,5}{2} + \frac{0,5+0,6}{2} + \frac{0,6+1}{2}] = \frac{1}{6}(0,25 + 0,55 + 0,8) = 0,2667$. Le Gini est $1 - 2 \times 0,2667 = 0,467$: la courbe est nettement sous la diagonale (aire 0,5 pour un tarif aveugle), car les trois contrats les moins chers n'ont rien coûté. Avec six contrats le chiffre est très bruité.

```
prime = np.array([100, 200, 150, 300, 250, 120.]); cout = np.array([0, 500, 0, 400, 100, 0.])
print(round(lorenz_gini(prime, cout, np.ones(6))[2], 3))
```

```
0.467
```

Le code redonne 0,467.

Corrigé 2.7

Moyenne pondérée : $\hat{f}_0 = (180 + 195)/(120 + 130) = 1,5$; $\hat{f}_1 = 198/180 = 1,1$. Ultimes : année 1 = 198 ; année 2 = $195 \times 1,1 = 214,5$; année 3 = $140 \times 1,5 \times 1,1 = 231$. Provision : $0 + 19,5 + 91 = 110,5$ k€. Avec la **moyenne simple** : rapports individuels du délai 0 : $180/120 = 1,5$ et $195/130 = 1,5$; leur moyenne vaut 1,5 aussi ; pour le délai 1, un seul rapport, 1,1. Les résultats sont **identiques** parce que les deux rapports individuels du délai 0 sont égaux (les deux moyennes ne diffèrent

que si les rapports diffèrent). Sur le triangle du livre, où ils diffèrent, la moyenne pondérée donne plus de poids aux grandes années.

```
Ct = np.array([[120, 180, 198], [130, 195, np.nan], [140, np.nan, np.nan]])
f = facteurs_chain_ladder(Ct); _, u = projeter(Ct, f); print(f.round(3), (u -
↪ dernier_cumul(Ct)).sum())
```

```
[1.5 1.1] 110.50000000000006
```

Corrigé 2.8

Les écarts à 1 sont 0,120, 0,060, 0,030 : les rapports successifs valent 0,5 et 0,5, donc $r = 0,5$. Le facteur de queue est $\prod_{k \geq 1} (1 + 0,030 r^k) \approx \exp(\sum_k 0,030 r^k) = \exp(0,030 r / (1 - r))$; pour $r = 0,5$, $\exp(0,030) = 1,0305$ (valeur exacte du produit : 1,0303). Pour $r = 0,6$, $\exp(0,030 \times 1,5) = 1,0460$ (exact : 1,0458). **Passer de 0,5 à 0,6 déplace le facteur de queue de 1,5 point, soit environ 10 M€ sur une provision d'ultimes de 650 M€** : la queue est à la fois petite et déterminante.

Corrigé 2.9

Baisse de déviance : $1\,250,0 - 1\,238,5 = 11,5$ pour 3 paramètres. $P(\chi_3^2 > 11,5) = 9,3 \times 10^{-3}$: **on rejette** le modèle simple au seuil de 1 %, les trois paramètres sont justifiés. L'AIC augmente de $2 \times 3 = 6$ pour la pénalité et baisse de 11,5 pour la déviance : le gain net est de $11,5 - 6 = 5,5$ en faveur du grand modèle. Les deux critères concluent dans le même sens ; si la baisse avait été de 5 par exemple, l'AIC aurait préféré le petit modèle (gain net -1) et le test, avec $p \approx 0,17$, n'aurait pas rejeté non plus : les deux critères se rejoignent presque toujours, mais pas par construction.

Corrigé 2.10

$k = s^2/a = 0,07/0,00014 = 500$ années. Facteurs : $Z_1 = 100/600 = 0,167$, $Z_2 = 600/1\,100 = 0,545$, $Z_3 = 4\,000/4\,500 = 0,889$. Estimations : $0,167 \times 9 + 0,833 \times 6,6 = 7,00$ %, $0,545 \times 5 + 0,455 \times 6,6 = 5,73$ %, $0,889 \times 6,4 + 0,111 \times 6,6 = 6,42$ %. Le petit groupe est ramené presque au niveau de la moyenne (de 9 % à 7,0 %), le gros est quasiment cru. Pour l'exactitude Poisson–Gamma, $k = 1/(\alpha\lambda) = 37,9$; vérifions que $Z(N/e) + (1 - Z)\lambda = \lambda(r + N)/(r + \lambda e)$ pour $e = 100$ et $N = 9$ ($r = 1/\alpha = 2,5$) : les deux membres valent 0,0834 (écart de $1\{, \}0 \times 10^{-15}$).

```
r, lam, e, N = 2.5, 0.066, 100, 9
k = r / lam; Z = e / (e + k)
print(round(Z * N / e + (1 - Z) * lam, 5), round(lam * (r + N) / (r + lam * e), 5))
```

```
0.08341 0.08341
```

Corrigé 2.11

Chain ladder : ultime = $20/0,4 = 50,0$ M€. BF : $20 + 100 \times 0,8 \times (1 - 0,4) = 68,0$ M€. Relation : $\hat{p} \hat{U}^{\text{CL}} + (1 - \hat{p})P\rho = 0,4 \times 50 + 0,6 \times 80 = 68,0$ M€, identique à BF. BF est **plus proche de l'a priori** (80) que de l'extrapolation (50) parce que l'année n'est payée qu'à 40 % : on fait confiance à l'information a priori pour les 60 % restants.

Corrigé 2.12

$\hat{f}_0 = (160 + 170 + 180)/(100 + 120 + 110) = 510/330 = 1,5455$. Rapports individuels : 1,6 ; 1,4167 ; 1,6364. Écarts à $\hat{f}_0$: 0,0545, -0,1288, 0,0909. $\hat{\sigma}_0^2 = \frac{1}{2}[100 e_1^2 + 120 e_2^2 + 110 e_3^2] = 1,598$. Ce paramètre mesure la **variabilité résiduelle** des rapports de développement autour de $\hat{f}_0$: il entre dans la variance de processus ($\hat{\sigma}_0^2/(\hat{f}_0^2 \hat{C}_{i,0})$) et dans la variance d'estimation ($\hat{\sigma}_0^2/(\hat{f}_0^2 \sum_j C_{j,0})$) de la formule de Mack, pour toutes les années qui doivent encore traverser ce délai.

Corrigé 2.13

Le diagnostic se programme en quelques lignes : on ajuste le GLM de Poisson à effets ligne et colonne, on calcule les résidus de Pearson standardisés, on les moyenne par année calendaire $i + j$.

```
def residus_diag(I):
    n, m = I.shape; ii, jj = np.indices((n, m)); obs = ~np.isnan(I)
    X = np.zeros((obs.sum(), n + m - 1)); X[np.arange(obs.sum()), ii[obs]] = 1
    mc = jj[obs] > 0; X[np.arange(obs.sum())[mc], n + jj[obs][mc] - 1] = 1
    y = I[obs]; mod = sm.GLM(y, X, family=sm.families.Poisson()).fit(); mu = mod.fittedvalues
    phi = np.sum((y - mu) ** 2 / mu) / (len(y) - len(mod.params))
    r = (y - mu) / np.sqrt(phi * mu); d = (ii + jj)[obs]
    return pd.Series(r).groupby(d).mean()
for nom in ("dommages", "choc"):
    t_ = pd.read_csv(f"donnees/triangle_{nom}.csv")
    rd = residus_diag(triangle_cumule(t_, "paiement_incremental").values)
    print(nom, rd.round(2).to_dict())
```

```
dommages {0: 0.33, 1: -0.46, 2: 0.5, 3: -0.33, 4: 0.52, 5: -0.62, 6: -0.08, 7: 0.04, 8: -0.03, 9: 0.34}
choc {0: 0.22, 1: 0.19, 2: -0.35, 3: 0.02, 4: -0.07, 5: -0.17, 6: -0.38, 7: 1.41, 8: -0.6, 9: -0.13}
```

Sur le triangle de dommages, la plus grande moyenne de diagonale en valeur absolue est de 0,62 ; sur le triangle « choc », elle atteint 1,41 (diagonale 7, celle du choc). Le diagnostic **fonctionne**, mais sur un triangle de six délais les diagonales les plus courtes ne contiennent que quelques cellules et leurs moyennes sont bruitées : on ne regarde que les diagonales qui ont assez de cellules, et l'on s'appuie aussi sur des graphiques de résidus (livre, section 2.5.5).

Corrigé 2.14

Coût moyen en basique : $0,10 \times 3\,000 + 0,90 \times 900 = 1\,110$ €. En premium : les coûts de consommation sont majorés de 40 % : $1,4 \times (0,36 \times 3\,000 + 0,64 \times 900) = 1,4 \times 1\,656 = 2\,318,4$ €. Rapport brut : $2\,318,4/1\,110 = 2,09$. **Sélection** : le coût des assurés premium s'ils avaient le comportement basique, comparé au coût des assurés basique : $1\,656/1\,110 = 1,492$. **Comportement** : 1,4. Et $1,492 \times 1,4 = 2,09$, la décomposition multiplicative du livre. Environ 54 % du log de l'écart brut vient de la sélection et 46 % du comportement : **tarifier sur le rapport brut reviendrait à faire payer à tous un comportement qui, en partie, est une différence de santé.**

Chapitre 3 : Mesures de risque et stress tests — exercices et applications

Comment utiliser ce chapitre. Les **applications** sont de petites études guidées sur les données du chapitre (rendements de marché, macroéconomie et défauts, courbe de taux, incidents opérationnels) : lisez l'énoncé, essayez, puis exécutez les blocs. Les **exercices** sont notés ★ (direct), ★★ (demande un raisonnement) ou ★★★ (étude plus longue) ; chacun renvoie à la section du livre qui l'éclaire, et tous sont **corrigés** en fin de chapitre. Les données sont **simulées** (build/donnees5.py) ; les montants sont en M€ et les pertes comptées positivement.

Une seule cellule charge ce dont les applications ont besoin.

```
import sys
sys.path.insert(0, "build")
import numpy as np
import pandas as pd
from scipy import stats
import outils_ch03 as O

r, regime = O.charger_marche()          # rendements journaliers de cinq actifs, régime vrai (calme /
↳ stress)
L = O.pertes_portefeuille(r)            # perte du portefeuille de 100 M€ (35 % A, 15 % B, 30 %
↳ obligations, 10 % immo, 10 % matières)
rg = regime.values
print(len(L), "jours ;", round((rg == "stress").mean() * 100, 1), "% de jours de stress")

4000 jours ; 7.0 % de jours de stress
```

Applications

Application 3.1 — La VaR et l'ES du portefeuille, trois façons (sections 3.1.3 à 3.1.7)

Objectif. Calculer la VaR et l'ES à 95 %, 99 % et 99,9 % par la méthode historique, la loi normale et une simulation de Student, et comparer.

Étape 1 — Historique. On lit le quantile empirique et la moyenne des pertes au-delà.

```
for a in (0.95, 0.99, 0.999):
    print(a, "VaR", round(O.var_hist(L, a), 2), "ES", round(O.es_hist(L, a), 2))
```

```
0.95 VaR 0.92 ES 1.6
0.99 VaR 2.04 ES 2.94
0.999 VaR 3.87 ES 4.67
```

Étape 2 — Loi normale. Avec la moyenne et l'écart-type observés, on applique $\mu + \sigma z_\alpha$ et $\mu + \sigma \varphi(z_\alpha)/(1 - \alpha)$.

```
mu, sd = L.mean(), L.std()
for a in (0.95, 0.99, 0.999):
    print(a, "VaR", round(0.var_normale(mu, sd, a), 2), "ES", round(0.es_normale(mu, sd, a), 2))
```

```
0.95 VaR 1.12 ES 1.41
0.99 VaR 1.59 ES 1.82
0.999 VaR 2.12 ES 2.31
```

Étape 3 — Student. On ajuste une loi de Student à la perte du portefeuille (`stats.t.fit`) et l'on en déduit les mêmes quantités.

```
nu, loc, echelle = stats.t.fit(L)
print("degrés de liberté ajustés :", round(nu, 1))
for a in (0.95, 0.99, 0.999):
    q = stats.t.ppf(a, nu, loc, echelle)
    print(a, "VaR", round(q, 2))
```

```
degrés de liberté ajustés : 3.1
0.95 VaR 0.95
0.99 VaR 1.83
0.999 VaR 4.06
```

Lecture. La loi de Student ajustée a 3,1 degrés de liberté, c'est-à-dire des queues bien plus lourdes que la normale ; à 99,9 %, elle donne 4,06 M€ contre 3,87 M€ pour l'historique et 2,12 M€ pour la normale. **Pour aller plus loin :** quelle méthode vous paraît la plus fiable à 99,9 % avec 4 000 jours ? Relisez la section 3.1.9 avant de répondre.

Application 3.2 — EWMA et GARCH filtré : la couverture selon le régime (section 3.1.5)

Objectif. Reproduire la comparaison du livre et la prolonger à d'autres niveaux de confiance.

Étape 1 — Découper. On calibre sur les 2 000 premiers jours, on évalue sur les 2 000 suivants.

```
apprentissage, test = L[:2000], L[2000:]
rt = rg[2000:]
v_fixe = 0.var_hist(apprentissage, 0.99)
sig, nu_g, _ = 0.garch_filtre(apprentissage, test)      # paramètres figés, volatilité filtrée jour
↳ après jour
v_garch = np.array([0.var_student(0, s, nu_g, 0.99) for s in sig])
print("VaR fixe :", round(v_fixe, 2), "| VaR GARCH min / médiane / max :",
      ↳ np.round(np.quantile(v_garch, [0, .5, 1]), 2))
```

```
VaR fixe : 1.75 | VaR GARCH min / médiane / max : [0.91 1.42 8.86]
```

Étape 2 — Fréquences de dépassement. On compte les jours où la perte dépasse la VaR, par régime.

```
for nom, v in (("fixe", v_fixe), ("GARCH", v_garch)):
    e = test > v
    print(nom, "| total", round(100 * e.mean(), 2), "% | calme", round(100 * e[rt == "calme"].mean(),
      ↳ 2), "% | stress", round(100 * e[rt == "stress"].mean(), 2), "%")
```

```
fixe | total 1.65 % | calme 0.61 % | stress 10.43 %
GARCH | total 1.15 % | calme 0.89 % | stress 3.32 %
```

Étape 3 — EWMA. La variance du jour est $\lambda\sigma_{t-1}^2 + (1 - \lambda)L_{t-1}^2$ avec $\lambda = 0,94$; la fonction `0.var_ewma` renvoie l'écart-type prévu pour chaque lendemain.

```
sig_ew = 0.var_ewma(L)[1999:-1] # prévisions pour les jours 2000 à 3999
v_ew = 2.326 * sig_ew
e = test > v_ew
print("EWMA | total", round(100 * e.mean(), 2), "% | stress", round(100 * e[rt == "stress"].mean(),
↪ 2), "%")
```

```
EWMA | total 1.9 % | stress 3.79 %
```

Pour aller plus loin. Refaites l'étape 2 avec la VaR à 95 % ($z = 1,645$) : la couverture est-elle meilleure ou moins bonne qu'à 99 % ? Que dit-on de la queue quand l'écart est plus grand à 99 % qu'à 95 % ?

Application 3.3 — Simulation de Monte-Carlo : la loi change la queue (section 3.1.4)

Objectif. Mesurer à quel point la VaR simulée dépend de la loi supposée, à matrice de covariance donnée.

Étape 1 — Paramètres. Les moyennes et la covariance des cinq rendements.

```
Sigma = np.cov(r[0.ACTIFS].values.T)
mu_v = r[0.ACTIFS].values.mean(axis=0)
rng = np.random.default_rng(1)
N = 200000
Z = rng.multivariate_normal(np.zeros(5), Sigma, N) # chocs normaux de covariance Sigma
```

Étape 2 — Simuler pour plusieurs lois. Pour une Student à ν degrés de liberté de même covariance, on divise les chocs par $\sqrt{W/\nu}$ avec $W \sim \chi_\nu^2$ et l'on multiplie par $\sqrt{(\nu - 2)/\nu}$.

```
def var_simulee(nu, a=0.99):
    X = mu_v + (Z if nu is None else Z * np.sqrt((nu - 2) / nu) / np.sqrt(rng.chisquare(nu, N) /
↪ nu))[:, None]
    return 0.var_hist(-(X @ 0.POIDS) * 0.VALEUR, a)

for nu in (None, 30, 10, 6, 4):
    print("normale" if nu is None else f"Student {nu}>2)", "| VaR 99 % :", round(var_simulee(nu), 2),
↪ "| VaR 99,9 % :", round(var_simulee(nu, 0.999), 2))
```

```
normale | VaR 99 % : 1.59 | VaR 99,9 % : 2.12
Student 30 | VaR 99 % : 1.62 | VaR 99,9 % : 2.23
Student 10 | VaR 99 % : 1.69 | VaR 99,9 % : 2.53
Student 6 | VaR 99 % : 1.75 | VaR 99,9 % : 2.95
Student 4 | VaR 99 % : 1.8 | VaR 99,9 % : 3.48
```

Lecture. À covariance identique, plus les degrés de liberté sont faibles, plus la VaR à 99,9 % grimpe, et c'est à ce niveau que la loi compte le plus. À 99 %, l'écart est beaucoup plus modeste. **Pour aller plus loin :** pourquoi la normale et la Student à 30 degrés de liberté donnent-elles presque le même résultat ?

Application 3.4 — Bootstrap d'un chiffre de queue (section 3.1.9)

Objectif. Voir comment l'incertitude sur la VaR dépend de la taille de l'échantillon et du niveau.

Étape 1 — Une fonction qui rééchantillonne. On tire avec remise, on recalcule la VaR, on prend l'intervalle à 95 %.

```
rng = np.random.default_rng(5)

def intervalle(x, a, B=600):
    est = [0.var_hist(rng.choice(x, len(x)), a) for _ in range(B)]
    return np.quantile(est, [0.025, 0.975])
```

Étape 2 — Faire varier la taille. On répète sur les 500, 1 000, 2 000 et 4 000 derniers jours.

```
for n in (500, 1000, 2000, 4000):
    x = L[-n:]
    lo, hi = intervalle(x, 0.99)
    print(f"n = {n:>4} | VaR 99 % = {0.var_hist(x, 0.99):.2f} | intervalle [{lo:.2f} ; {hi:.2f}] |
    ↳ largeur relative {100 * (hi - lo) / 0.var_hist(x, 0.99):.0f} %")
```

```
n = 500 | VaR 99 % = 2.23 | intervalle [1.69 ; 3.31] | largeur relative 73 %
n = 1000 | VaR 99 % = 1.84 | intervalle [1.42 ; 2.26] | largeur relative 46 %
n = 2000 | VaR 99 % = 2.21 | intervalle [1.84 ; 2.61] | largeur relative 35 %
n = 4000 | VaR 99 % = 2.04 | intervalle [1.74 ; 2.36] | largeur relative 30 %
```

Lecture. L'intervalle se resserre avec la taille de l'échantillon, mais lentement (comme la racine de n). **Pour aller plus loin :** refaites l'étape 2 pour la VaR à 99,9 % ; combien de jours faudrait-il pour que la largeur relative soit de 10 % ?

Application 3.5 — Un stress macroéconomique de crédit (sections 3.2.4 à 3.2.7)

Objectif. Estimer le modèle macro-défaut, construire un scénario et chiffrer la perte.

Étape 1 — Estimer. Régression du logit du taux de défaut sur la croissance, le chômage et la variation de l'immobilier.

```
import statsmodels.api as sm
t = pd.read_csv("donnees/taux_defaut_macro.csv")
y = np.log(t["taux_defaut"] / (1 - t["taux_defaut"]))
X = sm.add_constant(t[["croissance_pib", "chomage", "variation_immo"]])
macro = sm.OLS(y, X).fit()
print(macro.params.round(3).to_dict())
print("R2 =", round(macro.rsquared, 3), "| écart-type résiduel du logit :", round(macro.resid.std(),
↳ 3))
```

```
{'const': -5.03, 'croissance_pib': -0.401, 'chomage': 0.245, 'variation_immo': -0.063}
R2 = 0.989 | écart-type résiduel du logit : 0.057
```

Étape 2 — Un scénario et sa perte. On suppose, pendant un an, une croissance de -1% , un chômage de $9,5\%$ et une variation de l'immobilier de -3% par trimestre. La perte est le taux de défaut multiplié par l'encours (300 M€) et la LGD (45 %).

```
x_scen = np.array([1.0, -1.0, 9.5, -3.0]) # constante, croissance, chômage, immobilier
taux = 1 / (1 + np.exp(-x_scen @ macro.params.values))
print("taux de défaut :", round(100 * taux, 1), "% | perte :", round(taux * 300 * 0.45, 1), "M€")
```

```
taux de défaut : 10.8 % | perte : 14.6 M€
```

Étape 3 — Tester la robustesse. Réestimez le modèle sur les quarts 1 à 40 seulement et comparez la prédiction du même scénario.

```
tr = t["trimestre"] <= 40
m40 = sm.OLS(y[tr], X[tr]).fit()
taux40 = 1 / (1 + np.exp(-x_scen @ m40.params.values))
print("prédiction du scénario avec les 40 premiers trimestres :", round(100 * taux40, 1), "%")
```

```
prédiction du scénario avec les 40 premiers trimestres : 10.5 %
```

Lecture. Les deux estimations du même scénario sont proches, parce que la relation simulée est stable. Imaginez qu'elle ne le soit pas : que feriez-vous pour mesurer cette fragilité en pratique ? (Indice : estimer sur des sous-périodes décalées, et comparer les coefficients.)

Application 3.6 — Un choc de taux sur un petit portefeuille d'obligations (section 3.2.6)

Objectif. Comparer un choc parallèle, un choc de pentification et le scénario historique de 3.2.6 sur trois obligations de 10 M€.

Étape 1 — Les obligations et la courbe. Le mois 120 est la courbe « actuelle ».

```
cb = pd.read_csv("donnees/courbe_taux.csv")
f = 0.courbe_a(120, cb)
bonds = [(2, 0.02), (5, 0.03), (10, 0.035)]
P0 = [0.prix_obligation(c, m, f) for m, c in bonds]
print("prix initiaux :", np.round(P0, 2))
```

```
prix initiaux : [ 99.25 102.57 109.16]
```

Étape 2 — Trois chocs. Parallèle de +100 pb ; pentification (+0 pb à 1 an, +150 pb à 30 ans, interpolés) ; historique (variation observée entre les mois 60 et 90).

```
chg_hist = cb[cb["mois"] == 90].iloc[0, 1:].values.astype(float) - cb[cb["mois"] == 60].iloc[0,
↳ 1:].values.astype(float)
chocs = {"parallèle +100 pb": lambda tt: 0.01,
        "pentification": lambda tt: np.interp(tt, [1, 30], [0.0, 0.015]),
        "historique 60-90": lambda tt: np.interp(tt, 0.MATS, chg_hist)}
for nom, choc in chocs.items():
    pertes = [10 * (1 - 0.prix_obligation(c, m, f, choc=choc) / p0) for (m, c), p0 in zip(bonds, P0)]
    print(f"{nom:20s} perte totale : {sum(pertes):.2f} M€ | par obligation :", np.round(pertes, 2))
```

```
parallèle +100 pb    perte totale : 1.44 M€ | par obligation : [0.19 0.45 0.81]
pentification        perte totale : 0.46 M€ | par obligation : [0.01 0.09 0.36]
historique 60-90    perte totale : 2.19 M€ | par obligation : [0.25 0.66 1.27]
```

Lecture. Quelle obligation concentre la perte dans chaque scénario ? Une pentification coûte-t-elle plus ou moins qu'un choc parallèle ? **Pour aller plus loin :** refaites l'étape 2 avec un aplatissement (+150 pb à 1 an, 0 à 30 ans).

Application 3.7 — Perte opérationnelle agrégée : avec et sans queue lourde (sections 3.3.1 à 3.3.3)

Objectif. Mesurer l'effet du choix de la loi de sévérité sur la VaR à 99 % de la perte annuelle.

Étape 1 — Les données.

```
op = pd.read_csv("donnees/pertes_operationnelles.csv")
net = op["perte_nette"].values
lam = len(op) / 10
print("incidents par an :", round(lam, 1), "| perte nette moyenne :", round(net.mean()), "€")
```

incidents par an : 180.2 | perte nette moyenne : 14238 €

Étape 2 — Sévérité log-normale seule. On ajuste $\ln X$ par une loi normale sur toutes les pertes et l'on simule 20 000 années.

```
mu_l, sg_l = np.log(net).mean(), np.log(net).std()
sev_ln = lambda rng, k: np.exp(rng.normal(mu_l, sg_l, k))
tot_ln = 0.perte_agreee_mc(lam, sev_ln, n_sim=20000, seed=1)
print("log-normale seule | moyenne", round(tot_ln.mean() / 1e6, 2), "M€ | VaR 99 %",
      ↪ round(np.quantile(tot_ln, 0.99) / 1e6, 2), "M€")
```

log-normale seule | moyenne 1.51 M€ | VaR 99 % 2.7 M€

Étape 3 — Corps log-normal et queue GPD. On utilise l'ajustement du livre (seuil de 100 000 €).

```
par = 0.ajuster_lda(net)
tot_gpd = 0.perte_agreee_mc(lam, lambda rng, k: 0.tirer_lda(rng, k, par), n_sim=20000, seed=1)
print("corps + GPD | moyenne", round(tot_gpd.mean() / 1e6, 2), "M€ | VaR 99 %",
      ↪ round(np.quantile(tot_gpd, 0.99) / 1e6, 2), "M€")
print("pire année observée :",
      ↪ round(op.groupby(op["date_evenement"].str[:4])["perte_nette"].sum().max() / 1e6, 2), "M€")
```

corps + GPD | moyenne 10.56 M€ | VaR 99 % 31.89 M€
pire année observée : 6.72 M€

Lecture. La log-normale seule donne une VaR bien plus basse : elle ne voit pas la queue. La GPD la relève fortement, mais au prix d'une forte incertitude (section 3.3.3). **Pour aller plus loin :** plafonnez chaque perte à 20 M€ (plafond=20e6 dans 0.tirer_lda) et observez l'effet sur la VaR à 99 %.

Application 3.8 — Un backtest complet (sections 3.4.1 à 3.4.6)

Objectif. Dérouler tout le contrôle d'une VaR : dépassements, Kupiec, Christoffersen, feu tricolore.

Étape 1 — Les prévisions glissantes. 0.var_glissantes calcule, à partir du 1 001^e jour, la VaR du lendemain pour quatre méthodes.

```
res = 0.var_glissantes(L)
te = L[1000:]
for nom, (v, es) in res.items():
    print(f"{nom:14s} VaR moyenne : {v.mean():.2f} M€ | dépassements : {int((te > v).sum())} sur
          ↪ {len(te)}")
```

historique	VaR moyenne : 1.97 M€ dépassements : 44 sur 3000
normale	VaR moyenne : 1.59 M€ dépassements : 62 sur 3000
EWMA	VaR moyenne : 1.47 M€ dépassements : 56 sur 3000
GARCH-Student	VaR moyenne : 1.66 M€ dépassements : 36 sur 3000

Étape 2 — Les tests. Kupiec sur le nombre, Christoffersen sur l'enchaînement.

```
n = len(te)
for nom, (v, es) in res.items():
    e = (te > v).astype(int)
    print(f"{nom:14s} Kupiec p = {0.kupiec(int(e.sum()), n, 0.01)[1]:.4f} | Christoffersen p =
          ↪ {0.christoffersen_ind(e)[1]:.4f}")
```

historique	Kupiec p = 0.0163 Christoffersen p = 0.0289
normale	Kupiec p = 0.0000 Christoffersen p = 0.1840

EWMA	Kupiec p = 0.0000 Christoffersen p = 0.1443
GARCH-Student	Kupiec p = 0.2858 Christoffersen p = 0.3496

Étape 3 — Le feu tricolore sur la dernière année. On compte les dépassements sur les 250 derniers jours et on lit la zone.

```
zones = 0.zones_tricolores()
for nom, (v, es) in res.items():
    k = int((te[-250:] > v[-250:]).sum())
    print(f"{nom:14s} {k} dépassements sur 250 jours → zone {zones.loc[k, 'zone']}")
```

historique	2 dépassements sur 250 jours → zone verte
normale	6 dépassements sur 250 jours → zone orange
EWMA	5 dépassements sur 250 jours → zone orange
GARCH-Student	4 dépassements sur 250 jours → zone verte

Lecture. Les tests sur 3 000 jours et le feu tricolore sur 250 jours concluent-ils pareil ? Pourquoi la fenêtre d'un an est-elle moins discriminante (section 3.4.5) ? **Pour aller plus loin :** calculez le backtest d'une VaR à 95 % ($z = 1,645$, EWMA) ; combien de dépassements attend-on ?

Exercices

Exercice 3.1 ★ — VaR et ES sur vingt pertes (sections 3.1.2 et 3.1.7)

On observe vingt pertes journalières (en M€) : 0,3 ; -0,5 ; 1,2 ; 0,8 ; -0,2 ; 2,9 ; 0,1 ; 0,6 ; -0,9 ; 1,7 ; 0,4 ; 3,8 ; -0,3 ; 0,9 ; 1,1 ; 0,2 ; -0,6 ; 0,7 ; 5,2 ; 1,5. Calculez à la main la VaR à 90 %, à 95 % et à 99 %, puis l'ES à 90 % et à 95 %.

Exercice 3.2 ★ — VaR normale d'un portefeuille de deux actifs (section 3.1.3)

Un portefeuille de 50 M€ est investi à 60 % dans l'actif 1 (volatilité journalière de 1,2 %) et à 40 % dans l'actif 2 (0,8 %), avec une corrélation de 0,3. En supposant des pertes normales de moyenne nulle, calculez la VaR à 99 % sur un jour, puis la somme des VaR de chaque actif pris seul : quel est le gain de diversification ?

Exercice 3.3 ★★ — Vérifier numériquement l'ES normale (section 3.1.7)

Pour une loi normale centrée réduite, calculez $\varphi(z_\alpha)/(1 - \alpha)$ pour $\alpha = 97,5\%$ et 99% , vérifiez-le par simulation (deux millions de tirages) et comparez l'ES à 97,5 % avec la VaR à 99 %.

Exercice 3.4 ★★ — Sous-additivité avec trois prêts (section 3.1.8)

Deux prêts indépendants de 1 M€ ont une probabilité de défaut de 3 % chacun, avec une perte de 1 M€ en cas de défaut. Calculez à la main, au niveau de 95 %, la VaR et l'ES de chaque prêt, puis de leur somme. La VaR est-elle sous-additive ? L'ES l'est-elle ?

Exercice 3.5 ★★ — La règle de la racine avec de la mémoire (section 3.1.6)

Des pertes journalières suivent $L_t = \varphi L_{t-1} + \varepsilon_t$ avec $\varphi = 0,2$ et ε_t normal d'écart-type 1. Calculez l'écart-type de la perte sur dix jours par la formule exacte, comparez-le à $\sqrt{10}$ fois l'écart-type quotidien, et vérifiez par simulation. Que devient la VaR à dix jours par la règle de la racine ?

Exercice 3.6 ★ — Taux de défaut et perte d'un scénario (section 3.2.5)

Le modèle $\ln \frac{p}{1-p} = -5,03 - 0,401 \text{ croissance} + 0,245 \text{ chômage} - 0,063 \text{ immo}$ est estimé sur nos données. Pour une croissance de -1% , un chômage de 9% et une variation de l'immobilier de -2% , calculez le taux de défaut, puis la perte annuelle sur un encours de 200 M€ avec une LGD de 45% .

Exercice 3.7 ★★ — Duration et convexité d'une obligation (section 3.2.6)

Une obligation de 5 ans, de coupon annuel de 4% et de nominal 100, est actualisée sur une courbe plate à 3% . Calculez son prix, sa duration modifiée et sa convexité (par dérivation numérique), puis comparez, pour un choc parallèle de $+100$ pb et de $+300$ pb, la variation exacte du prix aux approximations par la duration seule et par la duration avec la convexité.

Exercice 3.8 ★★★ — Stress inversé et sensibilité à la LGD (section 3.2.7)

En reprenant la famille de scénarios de 3.2.7 (de la base $s = 0$ au scénario sévère $s = 1$), trouvez l'intensité s^* qui épuise un coussin de 30 M€ sur un encours de 500 M€, pour des LGD de 40% , 46% et 55% . Comment varient la croissance et le chômage du scénario fatal ? Commentez le rôle de la LGD, qui n'est pas observée au moment du stress.

Exercice 3.9 ★★ — Contributions au risque (section 3.3.4)

Trois actifs de poids 50% , 30% et 20% ont pour volatilités journalières $1,0\%$, $1,5\%$ et $0,5\%$, avec des corrélations de $0,6$ (actifs 1–2), $0,1$ (1–3) et 0 (2–3). Calculez la volatilité du portefeuille et la contribution de chaque actif (en part du total). Quel actif contribue le plus, et comment cela se compare-t-il à son poids ?

Exercice 3.10 ★★ — Perte agrégée d'un modèle fréquence-sévérité (sections 3.3.1 et 3.3.3)

Les incidents d'une ligne métier arrivent selon une loi de Poisson de paramètre 20 par an, avec des pertes log-normales de paramètres $\mu = 9$ et $\sigma = 1,5$ (en €). Calculez l'espérance et l'écart-type de la perte annuelle par les formules exactes, puis simulez 200 000 années pour estimer la VaR à $99,9\%$. Comparez avec l'approximation normale $\mathbb{E} + z_{99,9\%} \sigma$.

Exercice 3.11 ★ — Kupiec et feu tricolore (sections 3.4.2 et 3.4.6)

Un modèle de VaR à 99% a produit 10 dépassements en 500 jours. Calculez la statistique de Kupiec et sa probabilité critique. Sur une autre fenêtre de 250 jours, il a produit 7 dépassements : dans quelle zone du feu tricolore se trouve-t-il, et avec quelle probabilité cumulée ?

Exercice 3.12 ★★★ — Contrôler une expected shortfall (section 3.4.7)

Avec les prévisions glissantes de la section 3.4.4, calculez, pour la méthode historique et pour le GARCH-Student, l'écart moyen entre la perte réalisée et l'ES prévue les jours de dépassement. Donnez un intervalle de confiance par bootstrap et dites si l'ES est compatible avec les pertes réalisées.

Corrigés

Corrigé 3.1

Triées : -0,9 ; -0,6 ; -0,5 ; -0,3 ; -0,2 ; 0,1 ; 0,2 ; 0,3 ; 0,4 ; 0,6 ; 0,7 ; 0,8 ; 0,9 ; 1,1 ; 1,2 ; 1,5 ; 1,7 ; 2,9 ; 3,8 ; 5,2. Le rang de la VaR est $\lceil an \rceil$ avec $n = 20$: rang 18 à 90 %, 19 à 95 %, 20 à 99 %.

```
x = np.array([0.3, -0.5, 1.2, 0.8, -0.2, 2.9, 0.1, 0.6, -0.9, 1.7, 0.4, 3.8, -0.3, 0.9, 1.1, 0.2,
             -0.6, 0.7, 5.2, 1.5])
for a in (0.90, 0.95, 0.99):
    print(a, "rang", int(np.ceil(a * 20)), "VaR", 0.var_hist(x, a), "ES", round(0.es_hist(x, a), 3))
```

```
0.9 rang 18 VaR 2.9 ES 3.967
0.95 rang 19 VaR 3.8 ES 4.5
0.99 rang 20 VaR 5.2 ES 5.2
```

La VaR vaut 2,9 (90 %), 3,8 (95 %) et 5,2 (99 %). L'ES à 90 % est la moyenne des trois plus grandes pertes, $(2,9 + 3,8 + 5,2)/3 = 3,967$; à 95 %, la moyenne des deux plus grandes, $(3,8 + 5,2)/2 = 4,5$. À 99 %, la VaR et l'ES coïncident (5,2) : avec vingt observations, la queue à 1 % ne contient qu'une valeur. Moralité : on ne peut pas parler de quantile à 99 % sans assez de données.

Corrigé 3.2

Avec $w = (0,6; 0,4)$ et $V = 50$: $\sigma_p^2 = 0,36 \times 1,44 \times 10^{-4} + 0,16 \times 0,64 \times 10^{-4} + 2 \times 0,6 \times 0,4 \times 0,3 \times (0,012 \times 0,008) = 7,59 \times 10^{-5}$, donc $\sigma_p = 0,871$ % et la perte journalière a pour écart-type $0,871$ % $\times 50 = 0,436$ M€. La VaR à 99 % est $2,326 \times 0,436 \approx 1,01$ M€.

```
w, vol, rho, V = np.array([0.6, 0.4]), np.array([0.012, 0.008]), 0.3, 50.0
S = np.array([[vol[0]**2, rho * vol[0] * vol[1]], [rho * vol[0] * vol[1], vol[1]**2]])
sp = np.sqrt(w @ S @ w) * V
z = stats.norm.ppf(0.99)
somme = z * (w * vol).sum() * V
print("écart-type", round(sp, 4), "| VaR 99 %", round(z * sp, 3), "| somme des VaR", round(somme, 3),
      "\n| gain", round(somme - z * sp, 3))
```

```
écart-type 0.4356 | VaR 99 % 1.013 | somme des VaR 1.21 | gain 0.196
```

La VaR du portefeuille (1,01 M€) est inférieure à la somme des VaR individuelles (1,21 M€) : le gain de diversification est d'environ 0,2 M€, soit 16 % de la somme. Il disparaîtrait si la corrélation valait 1.

Corrigé 3.3

```
for a in (0.975, 0.99):
    z = stats.norm.ppf(a)
    print(a, "z =", round(z, 4), "| phi(z)/(1-a) =", round(stats.norm.pdf(z) / (1 - a), 4))
rng = np.random.default_rng(0)
Z = rng.standard_normal(2_000_000)
for a in (0.975, 0.99):
    q = np.quantile(Z, a)
    print(a, "ES simulée :", round(Z[Z >= q].mean(), 4))
```

```
0.975 z = 1.96 | phi(z)/(1-a) = 2.3378
0.99 z = 2.3263 | phi(z)/(1-a) = 2.6652
0.975 ES simulée : 2.341
0.99 ES simulée : 2.6663
```

La formule et la simulation concordent (2,338 à 97,5 % ; 2,665 à 99 %). La VaR à 99 % de la loi normale réduite est $z = 2,326$: l'ES à 97,5 % (2,338) est presque identique. **Interprétation.** Pour des pertes normales, passer de la VaR 99 % à l'ES 97,5 % ne change presque rien ; pour des queues lourdes, l'ES est plus grande, parce qu'elle moyenne les pertes extrêmes que la VaR ignore.

Corrigé 3.4

À 95 %, pour un prêt : $P(L = 0) = 0,97 \geq 0,95$ donc $\text{VaR} = 0$; la somme des VaR vaut 0. Pour les deux prêts : $P(L = 0) = 0,97^2 = 0,9409 < 0,95$ donc $\text{VaR} = 1$: la VaR du total (1) **dépasse** la somme (0) : elle n'est **pas** sous-additive. Pour l'ES : un prêt, $\text{ES} = (0,03 \times 1)/0,05 = 0,6$, soit 1,2 pour deux ; le total, $P(L = 1) = 0,0582$ et $P(L = 2) = 0,0009$, donne $\text{ES} = [(0,9991 - 0,95) \times 1 + 0,0009 \times 2]/0,05 = 1,018 \leq 1,2$: l'ES **est** sous-additive ici.

```
p = 0.03
pr = {0: (1 - p) ** 2, 1: 2 * p * (1 - p), 2: p ** 2}
print("P(L=0), P(L=1), P(L=2) :", {k: round(v, 4) for k, v in pr.items()})
es_somme = 2 * (p * 1) / 0.05
es_total = ((0.9991 - 0.95) * 1 + 0.0009 * 2) / 0.05
print("ES chacun", round(p / 0.05, 3), "| somme des ES", es_somme, "| ES du total", round(es_total, 3))
```

```
P(L=0), P(L=1), P(L=2) : {0: 0.9409, 1: 0.0582, 2: 0.0009}
ES chacun 0.6 | somme des ES 1.2 | ES du total 1.018
```

Corrigé 3.5

Pour un AR(1), $\text{Var}(L_t) = \sigma_\varepsilon^2 / (1 - \varphi^2)$ et $\text{Var}(S_h) = \text{Var}(L) \left[h + 2 \sum_{k=1}^{h-1} (h - k) \varphi^k \right]$.

```
phi, h = 0.2, 10
var_L = 1 / (1 - phi ** 2)
facteur = h + 2 * sum((h - k) * phi ** k for k in range(1, h))
print("écart-type à 10 jours (exact) :", round(np.sqrt(var_L * facteur), 4), "| racine de 10 fois  
↳ l'écart-type quotidien :", round(np.sqrt(h * var_L), 4))
rng = np.random.default_rng(2)
eps = rng.standard_normal((100000, 60))
x = np.zeros_like(eps)
for t_ in range(1, 60):
    x[:, t_] = phi * x[:, t_ - 1] + eps[:, t_]
S = x[:, 49:59].sum(axis=1)
print("écart-type simulé :", round(S.std(), 4))
```

```
écart-type à 10 jours (exact) : 3.8696 | racine de 10 fois l'écart-type quotidien : 3.2275
écart-type simulé : 3.8623
```

L'écart-type exact à dix jours est 3,87, contre 3,23 par la règle de la racine : la règle **sous-estime** le risque de 20 % et la VaR à dix jours (normale) est donc, elle aussi, sous-estimée dans la même proportion. L'autocorrélation positive fait que les pertes se cumulent. Avec $\varphi < 0$ (retour à la moyenne), la règle surestimerait au contraire.

Corrigé 3.6

Logit = $-5,03 + 0,401 + 0,245 \times 9 + 0,063 \times 2 = -2,298$; le taux vaut $1/(1 + e^{2,298}) \approx 9,1$ % ; la perte annuelle est $0,091 \times 200 \times 0,45 \approx 8,2$ M€.

```
logit = -5.03 - 0.401 * (-1) + 0.245 * 9 - 0.063 * (-2)
p = 1 / (1 + np.exp(-logit))
print("logit", round(logit, 3), "| taux", round(100 * p, 2), "% | perte", round(p * 200 * 0.45, 2),  
↳ "M€")
```

```
logit -2.298 | taux 9.13 % | perte 8.22 M€
```

Corrigé 3.7

```
y0 = 0.03
flat = lambda tt: y0
prix = lambda ch=0.0: 0.prix_obligation(0.04, 5, flat, choc=lambda tt: ch)
P, h = prix(), 1e-4
D = -(prix(h) - prix(-h)) / (2 * h * P)
C = (prix(h) + prix(-h) - 2 * P) / (h * h * P)
print("prix", round(P, 3), "| duration modifiée", round(D, 3), "| convexité", round(C, 2))
for ch in (0.01, 0.03):
    print(f"+{int(ch * 1e4)} pb | exact {100 * (prix(ch) / P - 1):.2f} % | duration {100 * (-D *
        ↵ ch):.2f} % | duration + convexité {100 * (-D * ch + 0.5 * C * ch ** 2):.2f} %")

prix 104.58 | duration modifiée 4.504 | convexité 25.57
+100 pb | exact -4.38 % | duration -4.50 % | duration + convexité -4.38 %
+300 pb | exact -12.43 % | duration -13.51 % | duration + convexité -12.36 %
```

La duration seule surestime la baisse ; l'ajout de la convexité réduit nettement l'écart, qui reste petit pour +100 pb et plus visible pour +300 pb (le développement de Taylor est un développement local). Avec un nominal de 100, un coupon de 4 % et un taux de 3 %, le prix est au-dessus du pair, ce qui est cohérent avec un coupon supérieur au rendement.

Corrigé 3.8

```
from scipy import optimize
import statsmodels.api as sm
t = pd.read_csv("donnees/taux_defaut_macro.csv")
y = np.log(t["taux_defaut"] / (1 - t["taux_defaut"]))
macro = sm.OLS(y, sm.add_constant(t[["croissance_pib", "chomage", "variation_immo"]])).fit()
base = t.iloc[-4:][["croissance_pib", "chomage", "variation_immo"]].mean().values
sev = np.array([-3.0, 11.0, -6.0])

def perte(s, lgd, ead=500.0):
    pib, cho_fin, immo = base + s * (sev - base)
    cho = np.linspace(base[1], cho_fin, 5)[1:]
    Xs = np.column_stack([np.ones(4), np.full(4, pib), cho, np.full(4, immo)])
    return (1 / (1 + np.exp(-Xs @ macro.params.values))).mean() * ead * lgd

for lgd in (0.40, 0.46, 0.55):
    s = optimize.brentq(lambda s: perte(s, lgd) - 30.0, 0, 1.5)
    pib, cho_fin, immo = base + s * (sev - base)
    print(f"LGD {lgd:.2f} | s* = {s:.2f} | croissance {pib:.2f} % | chômage {cho_fin:.2f} %")

LGD 0.40 | s* = 0.71 | croissance -1.74 % | chômage 10.20 %
LGD 0.46 | s* = 0.65 | croissance -1.46 % | chômage 10.02 %
LGD 0.55 | s* = 0.57 | croissance -1.11 % | chômage 9.80 %
```

Plus la LGD est élevée, moins il faut de défaut pour épuiser le coussin : le scénario fatal se rapproche de la base, c'est-à-dire qu'une récession **plus douce** suffit. La LGD, elle, n'est pas observée au moment d'une crise (les recouvrements s'étalent sur plusieurs années) et **elle augmente en récession** (les garanties perdent de la valeur) : un stress qui fixe la LGD à sa moyenne de période calme sous-estime la perte. C'est pourquoi on teste aussi des LGD « de crise ».

Corrigé 3.9

```
w = np.array([0.5, 0.3, 0.2])
vol = np.array([0.010, 0.015, 0.005])
corr = np.array([[1, 0.6, 0.1], [0.6, 1, 0.0], [0.1, 0.0, 1]])
S = corr * np.outer(vol, vol)
sp = np.sqrt(w @ S @ w)
contrib = w * (S @ w) / sp ** 2
print("volatilité du portefeuille :", round(100 * sp, 4), "% | contributions :", np.round(100 *
↳ contrib, 1), "| somme :", round(contrib.sum(), 6))
```

```
volatilité du portefeuille : 0.8617 % | contributions : [52.5 45.5 2. ] | somme : 1.0
```

L'actif 1 pèse 50 % et fournit environ 52 % du risque ; l'actif 2, qui ne pèse que 30 %, en fournit environ 45 % parce qu'il est le plus volatil et corrélé à l'actif 1 ; l'actif 3 pèse 20 % mais ne contribue que pour 2 % (très faible volatilité, corrélations quasi nulles) : **le poids n'est pas la contribution**. La somme des contributions vaut 1 par construction (théorème d'Euler).

Corrigé 3.10

Pour une loi de Poisson composée, $E[S] = \lambda E[X]$ et $\text{Var}(S) = \lambda E[X^2]$, avec $E[X] = e^{\mu + \sigma^2/2}$ et $E[X^2] = e^{2\mu + 2\sigma^2}$.

```
lam_, mu_, sg_ = 20, 9.0, 1.5
esp = lam_ * np.exp(mu_ + sg_ ** 2 / 2)
ecart = np.sqrt(lam_ * np.exp(2 * mu_ + 2 * sg_ ** 2))
tot = 0.perte_agregee_mc(lam_, lambda rng, k: np.exp(rng.normal(mu_, sg_, k)), n_sim=200000, seed=4)
print("espérance exacte", round(esp), "| simulée", round(tot.mean()), "| écart-type exact",
↳ round(ecart), "| simulé", round(tot.std()))
print("VaR 99,9 % simulée :", round(np.quantile(tot, 0.999)), "| approximation normale :", round(esp
↳ + 3.0902 * ecart))
```

```
espérance exacte 499185 | simulée 498919 | écart-type exact 343817 | simulé 345670
VaR 99,9 % simulée : 3418811 | approximation normale : 1561650
```

La simulation retrouve les valeurs exactes de l'espérance et de l'écart-type. La VaR à 99,9 % simulée vaut 2,19 fois l'approximation normale : la perte annuelle est très asymétrique, et une cloche de Gauss de même moyenne et de même écart-type ne la décrit pas.

Corrigé 3.11

```
lr, p = 0.kupiec(10, 500, 0.01)
print("Kupiec : LR =", round(lr, 3), "| probabilité critique =", round(p, 4))
zt = 0.zones_tricolores()
print("7 dépassements sur 250 jours :", zt.loc[7, "zone"], "| probabilité cumulée", round(zt.loc[7,
↳ "proba_cumulee"], 4))
```

```
Kupiec : LR = 3.914 | probabilité critique = 0.0479
7 dépassements sur 250 jours : orange | probabilité cumulée 0.996
```

Avec 10 dépassements pour 5 attendus, la statistique dépasse 3,84 et le test rejette l'hypothèse d'une fréquence de 1 % au seuil de 5 % (la probabilité critique est inférieure à 0,05). Sur 250 jours, 7 dépassements placent le modèle en **zone orange** (la probabilité cumulée de 7 ou moins sous le bon modèle est d'environ 99,6 %, au-dessus de 95 % mais sous 99,99 %).

Corrigé 3.12

```

res = 0.var_glissantes(L)
te = L[1000:]
rng = np.random.default_rng(8)
for nom in ("historique", "GARCH-Student"):
    v, es = res[nom]
    exc = te > v
    ecart = te[exc] - es[exc]
    moy = ecart.mean()
    b = [rng.choice(ecart, len(ecart)).mean() for _ in range(2000)]
    lo, hi = np.quantile(b, [0.025, 0.975])
    print(f"{nom:14s} | {exc.sum()} dépassements | écart moyen {moy:.2f} M€ | intervalle à 95 %
    ↳ [{lo:.2f} ; {hi:.2f}]")

```

```

historique      | 44 dépassements | écart moyen 0.27 M€ | intervalle à 95 % [0.02 ; 0.55]
GARCH-Student  | 36 dépassements | écart moyen 0.16 M€ | intervalle à 95 % [-0.05 ; 0.40]

```

Si l'ES était parfaite, l'écart moyen entre la perte réalisée et l'ES prévue les jours de dépassement serait voisin de zéro. Un intervalle qui contient zéro ne permet pas de rejeter la prévision, un intervalle strictement positif indique que l'ES **sous-estime** la perte. Avec une trentaine de dépassements, la précision est faible : on ne peut conclure que sur des écarts importants, ce qui rappelle pourquoi le contrôle d'une ES est plus délicat que celui d'une VaR (section 3.4.7).

Chapitre 4 : Cadre réglementaire — exercices et applications

Ce chapitre du cahier accompagne le chapitre 4 du livre. Les **applications** refont pas à pas ce que le livre a résumé : le capital IRB d'un portefeuille de prêts, la simulation de la perte au quantile 99,9 %, la procyclicité du capital, l'agrégation du capital de solvabilité d'un assureur, sa marge de risque et sa résistance aux chocs, la marge sur services contractuels d'IFRS 17, la comparaison de modèles de Takaful, et la détection d'opérations suspectes. Les **exercices** (★ à ★★★) s'appuient sur les sections du livre ; chacun a son corrigé en fin de fichier. Tous les paramètres (coefficients, matrice de corrélation, taux de frais, seuils d'alerte) sont des **valeurs d'illustration** ; rien ici n'est un conseil juridique, comptable ou financier, et les textes réglementaires en vigueur dans votre juridiction priment.

Préparation

Une seule cellule recharge les données et refait le modèle de probabilité de défaut du livre (régression logistique estimée sur une moitié de l'échantillon, appliquée à l'autre moitié) ainsi que les fonctions du chapitre, rangées dans `build/outils_ch04.py` : formule IRB, agrégation du capital de solvabilité, simulation d'un fonds de Takaful, variables par compte pour la lutte contre le blanchiment. Toutes les données sont **simulées** (graines fixes).

```
import os, sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np
import pandas as pd
from scipy.stats import norm
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from outils_ch04 import *

credits = charger("credits_conso.csv")
recouv = charger("recouvrements.csv")
Xc = pd.get_dummies(credits.drop(columns=["id_credit", "default_12m"]), drop_first=True).astype(float)
Xc = Xc.fillna(Xc.median())
Xa, Xb, ya, yb = train_test_split(Xc, credits["default_12m"], test_size=0.5, random_state=0)
mu_c, sd_c = Xa.mean(), Xa.std()
pd_hat = LogisticRegression(max_iter=3000).fit((Xa - mu_c) / sd_c, ya).predict_proba((Xb - mu_c) /
↳ sd_c)[: , 1]
LGD = round(recouv["lgd_realisee"].mean(), 2)
pf = pd.DataFrame({"pd": np.clip(pd_hat, 0.0003, 0.60), "ead": credits.loc[Xb.index,
↳ "montant"].values * 0.7, "default": yb.values})
```

```
EAD = pf["ead"].sum()
print("prêts :", len(pf), "| exposition (M€) :", round(EAD / 1e6, 1), "| LGD :", LGD, "| PD moyenne  
↪ :", round(pf["pd"].mean(), 4))
```

```
prêts : 20000 | exposition (M€) : 135.5 | LGD : 0.46 | PD moyenne : 0.0582
```

Applications

Application 4.1 — Le capital IRB d'un portefeuille, de la PD au poids de risque

Sections du livre : 4.1.4, 4.1.5. **Objectif** : calculer la perte attendue, le capital et les actifs pondérés d'un portefeuille de prêts, puis regarder où se concentre le capital.

Étape 1 — Le capital de chaque prêt. `k_detail` applique la formule « autres expositions de détail » : corrélation d'actifs fonction de la PD, pas d'ajustement d'échéance, plancher de PD à 0,03 %.

```
pf["k"] = k_detail(pf["pd"], LGD) # capital par euro d'exposition
pf["rwa"] = 12.5 * pf["k"] * pf["ead"] # actifs pondérés
EL = (pf["pd"] * LGD * pf["ead"]).sum()
print("perte attendue : %.2f M€ (%.2f %%)" % (EL / 1e6, 100 * EL / EAD))
print("capital IRB : %.2f M€ (%.2f %%)" % ((pf["k"] * pf["ead"]).sum() / 1e6, 100 * (pf["k"] *  
↪ pf["ead"]).sum() / EAD))
print("RWA IRB : %.1f M€ (poids moyen %.1f %%) contre %.1f M€ en standard à 75 %%" % (  
pf["rwa"].sum() / 1e6, 100 * pf["rwa"].sum() / EAD, 0.75 * EAD / 1e6))
```

```
perte attendue : 4.47 M€ (3.30 %)
capital IRB : 7.44 M€ (5.49 %)
RWA IRB : 93.0 M€ (poids moyen 68.6 %) contre 101.6 M€ en standard à 75 %
```

Étape 2 — Où est le capital ? On découpe le portefeuille en cinq classes de PD de même effectif (quintiles) et on regarde, par classe, la PD moyenne, le poids de risque IRB et la part du capital total.

```
pf["classe"] = pd.qcut(pf["pd"], 5, labels=["C1 (sûrs)", "C2", "C3", "C4", "C5 (risqués)"])
g = pf.groupby("classe", observed=True)
tab = pd.DataFrame({"PD moyenne (%)": 100 * g["pd"].mean(), "exposition (M€)": g["ead"].sum() / 1e6,  
"poids IRB (%)": 100 * g["rwa"].sum() / g["ead"].sum(),  
"part du capital (%)": 100 * g["rwa"].sum() / pf["rwa"].sum(),  
"défaits observés (%)": 100 * g["default"].mean()})
print(tab.round(1).to_string())
```

	PD moyenne (%)	exposition (M€)	poids IRB (%)	part du capital (%)	défaits observés (%)
classe					
C1 (sûrs)	1.0	22.7	44.3	10.8	1.0
C2	2.2	24.2	60.3	15.7	2.7
C3	3.6	25.5	65.7	18.0	4.0
C4	6.0	27.6	69.4	20.6	6.2
C5 (risqués)	16.3	35.5	91.3	34.9	16.2

Étape 3 — IRB contre standard, classe par classe. Le poids standard (75 %) ne dépend pas de la classe : en quelles classes le modèle interne est-il plus favorable, et dans lesquelles plus sévère ?

```
tab["gain IRB vs standard (points de poids)"] = 75 - tab["poids IRB (%)"]
print(tab[["poids IRB (%)", "gain IRB vs standard (points de poids)"]].round(1).to_string())
```

	poids IRB (%)	gain IRB vs standard (points de poids)
classe		
C1 (sûrs)	44.3	30.7

C2	60.3	14.7
C3	65.7	9.3
C4	69.4	5.6
C5 (risqués)	91.3	-16.3

Lecture. La perte attendue est de 4,47 M€ (3,3 % de l'exposition) et le capital de 7,44 M€ (5,5 %), soit un poids de risque moyen de 68,6 % contre 75 % en standard. **Le capital se concentre sur les mauvais emprunteurs** : la classe la plus risquée (C5, PD moyenne 16 %) ne pèse que 26 % de l'exposition mais porte **35 % du capital** ; son poids de risque (91 %) est le seul à dépasser les 75 % du standard (16 points de plus), alors que la classe la plus sûre (C1, PD 1 %) est à 44 %, soit 31 points de moins. Le modèle interne est donc **plus favorable sur les bons emprunteurs et plus sévère sur les mauvais** : c'est ce que fait un cadre sensible au risque. Les défauts observés par classe (1,0 % ; 2,7 % ; 4,0 % ; 6,2 % ; 16,2 %) suivent la PD moyenne : le modèle de PD est bien calibré, ce que le capital suppose.

Pour aller plus loin. (1) Remplacez la LGD moyenne par une LGD par garantie (recouvrements.csv) : que devient le capital ? (2) Appliquez la formule « entreprises » (k_entreprise, échéance 2,5 ans) au même portefeuille : de combien le capital augmente-t-il ? Est-ce pertinent pour des prêts à des particuliers ? (3) Les PD dépassent 0,5 pour quelques prêts : plafonnez-les à 0,30 et mesurez l'effet.

Application 4.2 — Simuler la perte au quantile 99,9 % : granularité et corrélation

Sections du livre : 4.1.4, 4.1.6. **Objectif** : retrouver la formule IRB par simulation, mesurer l'effet de la taille du portefeuille et de la corrélation d'actifs.

Étape 1 — Une simulation. On tire 200 000 « années » pour un portefeuille de 20 000 prêts de PD 6 % et de corrélation d'actifs 4,6 %.

```
p0, rho0 = 0.06, float(rho_detail_autre(0.06))
pertes = perte_portefeuille_vasicek(p0, rho0, 20000, 200000, seed=3)
q_sim = np.quantile(pertes, 0.999)
q_for = float(k_vasicek(p0, 1.0, rho0)) + p0
print("rho = %.4f | perte moyenne simulée = %.4f | quantile simulé = %.4f | formule = %.4f" % (rho0,
↪ pertes.mean(), q_sim, q_for))
```

```
rho = 0.0459 | perte moyenne simulée = 0.0600 | quantile simulé = 0.1811 | formule = 0.1804
```

Étape 2 — La granularité. La formule suppose une infinité de petits prêts. Que devient le quantile avec 50, 200, 1 000 ou 20 000 prêts ?

```
res = {}
for n in (50, 200, 1000, 20000):
    L = perte_portefeuille_vasicek(p0, rho0, n, 200000, seed=4)
    res[n] = round(float(np.quantile(L, 0.999)), 4)
print("quantile 99,9 % selon le nombre de prêts :", res, "| formule :", round(q_for, 4))
```

```
quantile 99,9 % selon le nombre de prêts : {50: 0.24, 200: 0.2, 1000: 0.183, 20000: 0.1795} | formule
↪ : 0.1804
```

Étape 3 — La corrélation. La corrélation est imposée par le texte ; que se passe-t-il si la vraie vaut le double ?

```
for r in (0.02, rho0, 0.09, 0.18):
    L = perte_portefeuille_vasicek(p0, r, 20000, 200000, seed=5)
    print("rho = %.3f : quantile 99,9 % simulé = %.4f | par la formule = %.4f" % (r, np.quantile(L,
↪ 0.999), float(k_vasicek(p0, 1.0, r)) + p0))
```

```
rho = 0.020 : quantile 99,9 % simulé = 0.1287 | par la formule = 0.1294
rho = 0.046 : quantile 99,9 % simulé = 0.1793 | par la formule = 0.1804
rho = 0.090 : quantile 99,9 % simulé = 0.2529 | par la formule = 0.2553
rho = 0.180 : quantile 99,9 % simulé = 0.3897 | par la formule = 0.3939
```

Lecture. Pour 20 000 prêts, le quantile simulé (0,1811) est à 0,4 % de la formule (0,1804) : l'écart est celui de la simulation. La **granularité** pèse bien davantage : avec 50 prêts le quantile à 99,9 % vaut 0,24, avec 200 prêts 0,20, avec 1 000 prêts 0,183 ; il ne rejoint la formule (0,1795 pour 20 000 prêts, 0,1804 par la formule) qu'avec beaucoup de prêts. Un portefeuille de quelques dizaines de gros prêts ne peut donc pas être traité par la formule IRB sans ajustement de concentration : c'est l'objet du pilier 2. Quant à la **corrélation**, elle est le paramètre le plus influent : le quantile passe de 0,129 pour $\rho = 0,02$ à 0,390 pour $\rho = 0,18$, soit un facteur 3. La simulation confirme la formule pour chacune des valeurs.

Pour aller plus loin. Tracez l'histogramme de pertes et placez-y la perte attendue et le quantile ; calculez le **quantile à 99 %** et à **99,99 %** : de combien le capital changerait-il si le régulateur changeait le niveau de confiance ?

Application 4.3 — La procyclicité du capital

Section du livre : 4.1.6. Objectif : mesurer ce qui se passe quand la PD utilisée est celle de l'instant (*point in time*) plutôt que la moyenne du cycle (*through the cycle*).

Étape 1 — Les données. `taux_defaut_macro.csv` donne, pour 80 trimestres, le taux de défaut d'un portefeuille. On le prend comme PD de l'instant.

```
macro = charger("taux_defaut_macro.csv")
macro["pd_pit"] = macro["taux_defaut"]
macro["pd_ttc"] = macro["pd_pit"].mean()
print("taux de défaut : moyenne %.2f %% | minimum %.2f %% | maximum %.2f %% (trimestre %d)" % (
    100 * macro["pd_pit"].mean(), 100 * macro["pd_pit"].min(), 100 * macro["pd_pit"].max(),
    macro.loc[macro["pd_pit"].idxmax(), "trimestre"]))
```

```
taux de défaut : moyenne 3.06 %% | minimum 0.99 %% | maximum 12.83 %% (trimestre 52)
```

Étape 2 — Le capital selon la PD. Pour une LGD de 46 %, on calcule le capital par euro d'exposition avec la PD de l'instant, avec la PD moyenne, et avec une PD lissée (moyenne mobile sur 16 trimestres).

```
macro["k_pit"] = k_detail(macro["pd_pit"], LGD)
macro["k_ttc"] = k_detail(macro["pd_ttc"], LGD)
macro["pd_lisse"] = macro["pd_pit"].rolling(16, min_periods=1).mean()
macro["k_lisse"] = k_detail(macro["pd_lisse"], LGD)
pic = macro["pd_pit"].idxmax()
print("capital au pic de la crise : instant %.2f %% | moyenne du cycle %.2f %% | lissé %.2f %%" % (
    100 * macro.loc[pic, "k_pit"], 100 * macro.loc[pic, "k_ttc"], 100 * macro.loc[pic, "k_lisse"]))
print("capital minimum / maximum (instant) : %.2f %% / %.2f %%" % (100 * macro["k_pit"].min(), 100 *
    macro["k_pit"].max()))
```

```
capital au pic de la crise : instant 6.78 %% | moyenne du cycle 5.15 %% | lissé 5.04 %%
capital minimum / maximum (instant) : 3.73 %% / 6.78 %%
```

Étape 3 — Le ratio de la banque. Supposons que la banque détienne des fonds propres égaux à 1,3 fois le capital IRB **moyen** (calculé avec la PD moyenne) et ne les augmente pas. Quel ratio « fonds propres / capital exigé » obtient-elle trimestre après trimestre ?

```
fonds = 1.3 * macro["k_ttc"].iloc[0]
for nom in ("k_pit", "k_lisse"):
    ratio = fonds / macro[nom]
```

```
print("%-8s : ratio minimum %.2f (trimestre %d) | trimestres sous 1,00 : %d" % (nom, ratio.min(),
↳ macro.loc[ratio.idxmin(), "trimestre"], int((ratio < 1).sum())))
```

```
k_pit      : ratio minimum 0.99 (trimestre 52) | trimestres sous 1,00 : 1
k_lisse    : ratio minimum 1.23 (trimestre 65) | trimestres sous 1,00 : 0
```

Lecture. Au pic de la crise (trimestre 52, taux de défaut de 12,8 %), le capital calculé avec la PD de l'instant est de 6,78 % de l'exposition, contre 5,15 % avec la PD moyenne du cycle : **+32 %** en pleine crise ; du creux (3,73 %) au sommet (6,78 %), il augmente de **82 %**. Une banque qui détiendrait des fonds propres égaux à 1,3 fois le capital moyen verrait son ratio tomber à 0,99 : elle passe, pour un trimestre, **sous l'exigence**, alors que ses fonds propres n'ont pas bougé. Avec une PD lissée sur 16 trimestres, le ratio ne descend pas sous 1,23, mais l'exigence **réagit tardivement** (le minimum se produit au trimestre 65, après la crise) : le lissage supprime la procyclicité au prix d'un capital sous-estimé quand la crise éclate. Aucune des deux solutions n'est gratuite : d'où les coussins contracycliques, qui demandent plus de capital **avant** la crise.

Pour aller plus loin. Un **coussin contracyclique** consiste à exiger plus de capital quand le crédit croît vite. Définissez un coussin proportionnel à l'écart de la PD lissée à sa moyenne et regardez s'il aplatit le ratio.

Application 4.4 — Agréger les modules du capital de solvabilité

Sections du livre : 4.2.4, 4.2.5. **Objectif :** calculer le capital de solvabilité requis d'un assureur par agrégation de modules, mesurer la diversification et tester la sensibilité à la matrice de corrélation.

Étape 1 — Le calcul de référence. Les modules sont ceux du livre (en M€) : marché 110, défaut des contreparties 20, vie 30, santé 45, non-vie 140.

```
modules = ["marché", "défaut", "vie", "santé", "non-vie"]
charges = np.array([110, 20, 30, 45, 140.])
corr = np.array([[1, .25, .25, .25, .25], [.25, 1, .25, .25, .5], [.25, .25, 1, .25, 0],
↳ [.25, .25, .25, 1, .5], [.25, .5, 0, .5, 1]])
bscr = agreger(charges, corr)
print("somme %.0f | SCR de base %.1f | diversification %.1f (%.1f %)" % (charges.sum(), bscr,
↳ charges.sum() - bscr, 100 * (charges.sum() - bscr) / charges.sum()))
```

```
somme 345 | SCR de base 241.8 | diversification 103.2 (29.9 %)
```

Étape 2 — Trois cas extrêmes et une variante. Corrélation 1 partout (aucune diversification), 0 partout (diversification maximale), la matrice de référence, et la même matrice avec une corrélation de 0,5 entre marché et non-vie (une grande tempête qui fait aussi chuter les marchés).

```
R1 = np.ones((5, 5))
R0 = np.eye(5)
Rv = corr.copy(); Rv[0, 4] = Rv[4, 0] = 0.5
for nom, R in (("corrélation 1", R1), ("référence", corr), ("marché-non-vie à 0,5", Rv), ("corrélation
↳ 0", R0)):
    print("%-22s SCR de base %.1f" % (nom, agreger(charges, R)))
```

```
corrélation 1      SCR de base 345.0
référence          SCR de base 241.8
marché-non-vie à 0,5  SCR de base 257.2
corrélation 0      SCR de base 187.1
```

Étape 3 — La contribution de chaque module. La dérivée du SCR de base par rapport à une charge s'appelle sa **contribution marginale** ; la somme des contributions (charge × dérivée) redonne le SCR de base (théorème d'Euler).

```

contrib = charges * (corr @ charges) / bscr
print(pd.DataFrame({"module": modules, "charge": charges, "contribution": contrib.round(1), "part
↳ (%)": (100 * contrib / bscr).round(1)}).to_string(index=False))
print("somme des contributions :", round(contrib.sum(), 1), "= SCR de base", round(bscr, 1))

```

module	charge	contribution	part (%)
marché	110.0	76.8	31.7
défaut	20.0	11.3	4.7
vie	30.0	9.1	3.8
santé	45.0	28.8	11.9
non-vie	140.0	115.8	47.9
somme des contributions : 241.8 = SCR de base 241.8			

Lecture. La diversification est considérable mais **dépend de la matrice** : le SCR de base vaut 345 M€ si tout est parfaitement corrélé, 241,8 M€ avec la matrice de référence, 187,1 M€ si les modules étaient indépendants. Une **seule** corrélation changée (marché–non-vie de 0,25 à 0,5) le fait passer à 257,2 M€ (+15,4 M€, +6 %). Les contributions montrent que **les deux plus gros modules portent 80 % du SCR de base** (non-vie 47,9 %, marché 31,7 %) alors qu'ils représentent 72,5 % de la somme des charges : les petits modules profitent davantage de la diversification, les gros en profitent moins. La somme des contributions redonne exactement le SCR de base (propriété d'homogénéité de degré 1).

Pour aller plus loin. Le module non-vie lui-même agrège des sous-modules (primes et réserves, catastrophes) : décomposez les 140 M€ en 120 et 60 avec une corrélation de 0,25, et vérifiez que le total d'ensemble reste cohérent.

Application 4.5 — Marge de risque et résistance aux chocs

Sections du livre : 4.2.3, 4.2.6. **Objectif :** calculer la marge de risque par la méthode du coût du capital, voir comment elle dépend du rythme d'écoulement, puis tester le ratio de solvabilité après des chocs de marché.

Étape 1 — La marge de risque. On reprend le SCR du livre (253,8 M€), un coût du capital de 6 % et une actualisation à 2 %. Trois profils d'écoulement des engagements : rapide, central, lent.

```

scr = bscr + 12.0 # avec 12 M€ de risque opérationnel
def marge_risque(scr0, motif, taux=0.02, c=0.06):
    return c * sum(scr0 * m / (1 + taux) ** (t + 1) for t, m in enumerate(motif))
profils = {"rapide": [1, .5, .2, .05], "central": [1, .7, .45, .25, .1], "lent": [1, .85, .7, .55, .4,
↳ .28, .18, .1, .05]}
RM = {nom: marge_risque(scr, m) for nom, m in profils.items()}
print({nom: round(v, 1) for nom, v in RM.items()})

```

```
{'rapide': 25.8, 'central': 36.5, 'lent': 58.8}
```

Étape 2 — Le ratio après un choc. Avec 1 270 M€ d'actifs et 900 M€ de meilleure estimation, les fonds propres valent actifs – meilleure estimation – marge de risque. Un choc de marché fait baisser les actifs ; on suppose aussi (hypothèse simple) que le SCR monte de 10 % pour chaque tranche de 50 M€ de choc, car le portefeuille de placements devient plus concentré.

```

FP0 = 1270 - 900 - RM["central"]
lignes = []
for choc in (0, 50, 100, 150, 200):
    fp = FP0 - choc
    scr_c = scr * (1 + 0.10 * choc / 50)
    lignes.append({"choc (M€)": choc, "fonds propres": round(fp, 1), "ratio, SCR fixe (%)": round(100
↳ * fp / scr, 1),
    "ratio, SCR en hausse (%)": round(100 * fp / scr_c, 1)})
print(pd.DataFrame(lignes).to_string(index=False))

```

choc (M€)	fonds propres	ratio, SCR fixe (%)	ratio, SCR en hausse (%)
0	333.5	131.4	131.4
50	283.5	111.7	101.5
100	233.5	92.0	76.7
150	183.5	72.3	55.6
200	133.5	52.6	37.6

Lecture. La marge de risque dépend du **rythme d'écoulement** : 25,8 M€ pour un portefeuille qui s'éteint en quatre ans, 36,5 M€ pour le profil central (cinq ans), 58,8 M€ pour un profil lent (neuf ans), soit **2,3 fois** celle du profil rapide : les branches à queue longue immobilisent plus longtemps du capital. Pour le ratio, avec un SCR constant, la mutuelle supporte un choc de 79,7 M€ avant de passer sous 100 % ; si le SCR monte aussi (ici +10 % par tranche de 50 M€), la limite tombe à **environ 53 M€**, et à 100 M€ de choc le ratio est de 76,7 % au lieu de 92 %. **Le ratio baisse par le haut et par le bas à la fois** : les fonds propres diminuent et le capital requis augmente.

Pour aller plus loin. Quel choc maximal la mutuelle supporte-t-elle sans passer sous 100 % ? Trouvez-le par une recherche (dichotomie) dans les deux hypothèses.

Application 4.6 — La marge sur services contractuels, année par année

Section du livre : 4.4.1. Objectif : programmer le déroulement de la CSM d'un groupe de contrats sur trois ans, avec un écart d'expérience et une révision d'hypothèses.

Étape 1 — La fonction. Elle prend la prime, la valeur actuelle des sinistres attendus, l'ajustement pour risque, la durée de couverture, les sinistres réellement observés chaque année et les révisions de la valeur actuelle des **sinistres futurs** (positives = plus de sinistres). Taux d'actualisation nul, comme dans le livre.

```
def ifrs17(prime, vp_sin, ra, annees, sin_obs=None, revisions=None):
    csm = prime - (vp_sin + ra)
    if csm < 0:
        # contrat déficitaire : perte immédiate
        return pd.DataFrame([{"année": 0, "CSM ouverture": 0.0, "libération": 0.0, "produit": 0.0,
                               "charge de sinistres": 0.0, "résultat des services": csm}])
    sin_att, ra_ann, lignes = vp_sin / annees, ra / annees, []
    for a in range(1, annees + 1):
        csm += -(revisions or {}).get(a, 0.0)
        lib = csm / (annees - a + 1)
        obs = (sin_obs or {}).get(a, sin_att)
        lignes.append({"année": a, "CSM ouverture": round(csm, 2), "libération": round(lib, 2),
                       "produit": round(sin_att + ra_ann + lib, 2),
                       "charge de sinistres": round(obs, 2), "résultat des services": round(sin_att +
                                                ra_ann + lib - obs, 2)})
        csm -= lib
    return pd.DataFrame(lignes)
print(ifrs17(100, 70, 8, 3).to_string(index=False))
```

année	CSM ouverture	libération	produit	charge de sinistres	résultat des services
1	22.00	7.33	33.33	23.33	10.0
2	14.67	7.33	33.33	23.33	10.0
3	7.33	7.33	33.33	23.33	10.0

Étape 2 — Une mauvaise surprise. Les sinistres de l'année 1 sont de 28 (au lieu de 23,33) et, au début de l'année 2, on révisé de +6 la valeur actuelle des sinistres futurs.

```
print(ifrs17(100, 70, 8, 3, sin_obs={1: 28.0}, revisions={2: 6.0}).to_string(index=False))
```

année	CSM ouverture	libération	produit	charge de sinistres	résultat des services
1	22.00	7.33	33.33	28.00	5.33
2	8.67	4.33	30.33	23.33	7.00
3	4.33	4.33	30.33	23.33	7.00

Étape 3 — Contrat déficitaire. Sinistres attendus de 95.

```
print(ifs17(100, 95, 8, 3).to_string(index=False))
```

année	CSM ouverture	libération	produit	charge de sinistres	résultat des services
0	0.0	0.0	0.0	0.0	-3

Lecture. Sans surprise, le résultat des services est de 10 par an et le produit de 33,33 (le tiers de la prime). Avec la mauvaise surprise de l'année 1, le résultat de cette année tombe à 5,33 (l'écart d'expérience de 4,67 passe **immédiatement** en résultat) ; la révision de +6 des sinistres futurs, elle, **réduit la CSM** et fait baisser la libération des années 2 et 3 de 7,33 à 4,33 : le résultat de ces deux années est de 7 (10 moins 3) au lieu de 10. Sur trois ans, le résultat cumulé est de 19,33 = 30 – 4,67 – 6 : **l'écart passé et l'écart futur sont tous deux payés, mais à des dates différentes**. Le contrat déficitaire, enfin, déclenche une perte de 3 **dès la souscription** et n'a pas de CSM à libérer.

Pour aller plus loin. Ajoutez un taux d'actualisation de 3 % sur les flux d'exécution et la **désactualisation** de la CSM au taux fixé à la souscription : de combien la CSM libérée change-t-elle ?

Application 4.7 — Comparer des modèles de Takaful : où les deux parties s'y retrouvent

Sections du livre : 4.3.2, 4.5.2 à 4.5.5. **Objectif :** explorer les paramètres du modèle hybride pour le fonds auto, et trouver ceux pour lesquels ni l'opérateur ni les participants ne sont systématiquement perdants.

Étape 1 — Un fonds, un modèle. `simuler_fonds` fait tourner les quinze années d'un fonds et rend une table annuelle : excédent, prêt sans intérêt, remboursement, distribution, réserve, résultat de l'opérateur.

```
tk = charger("takaful_fonds.csv")
auto = tk[tk["fonds"] == "auto"]
r = simuler_fonds(auto, "hybride", wakala=0.12, part_mudaraba=0.30)
print((r.set_index("annee")["excedent", "qard_nouveau", "distribue", "reserve_fin",
↪ "resultat_operateur"]].tail(5) / 1e6).round(2).to_string())
```

annee	excedent	qard_nouveau	distribue	reserve_fin	resultat_operateur
2020	4.68	0.0	3.28	14.91	1.40
2021	6.07	0.0	4.25	16.73	1.30
2022	7.28	0.0	5.10	18.91	1.43
2023	11.51	0.0	8.06	22.36	1.62
2024	4.96	0.0	3.48	23.85	1.76

Étape 2 — Une grille de paramètres. On fait varier le taux de wakala (de 10 % à 22 %) et la part de moudaraba (0, 30 %, 60 %) et on retient, pour chaque couple, le résultat cumulé de l'opérateur, les années en déficit et les sommes distribuées.

```
lignes = []
for w in (0.10, 0.12, 0.14, 0.16, 0.18, 0.20, 0.22):
    for m in (0.0, 0.30, 0.60):
        r = simuler_fonds(auto, "hybride", wakala=w, part_mudaraba=m)
        lignes.append({"wakala (%)": round(100 * w), "moudaraba (%)": round(100 * m), "opérateur
↪ (M€)": r["resultat_operateur"].sum() / 1e6,
                        "années en déficit": int((r["excedent"] < 0).sum()), "distribué (M€)":
↪ r["distribue"].sum() / 1e6})
```

```
grille = pd.DataFrame(lignes)
print(grille.pivot(index="wakala (%)", columns="moudaraba (%)", values="opérateur
↳ (M€)").round(1).to_string())
```

moudaraba (%)	0	30	60
wakala (%)			
10	-0.0	2.3	4.5
12	15.1	17.0	18.9
14	30.2	31.7	33.2
16	45.3	46.4	47.5
18	60.4	61.0	61.6
20	75.5	75.7	75.9
22	90.6	90.6	90.7

Étape 3 — La région viable. On cherche les couples où l'opérateur gagne au moins 5 M€ sur quinze ans, où le fonds a au plus 2 années en déficit et où les participants reçoivent au moins 40 M€.

```
viable = grille[(grille["opérateur (M€)"] >= 5) & (grille["années en déficit"] <= 2) &
↳ (grille["distribué (M€)"] >= 40)]
print(viable.round(1).to_string(index=False) if len(viable) else "aucun couple viable")
print(grille[grille["moudaraba (%)"] == 30][["wakala (%)", "années en déficit", "distribué
↳ (M€)"]].round(1).to_string(index=False))
```

wakala (%)	moudaraba (%)	opérateur (M€)	années en déficit	distribué (M€)
12	0	15.1	0	57.1
12	30	17.0	0	55.7
12	60	18.9	0	54.3
14	0	30.2	0	45.6
14	30	31.7	0	44.5
14	60	33.2	0	43.4
wakala (%)	années en déficit	distribué (M€)		
10	0	66.9		
12	0	55.7		
14	0	44.5		
16	1	33.4		
18	2	23.1		
20	6	13.9		
22	8	2.4		

Lecture. Le résultat de l'opérateur est **presque linéaire** dans le taux de wakala : 15 M€ de plus tous les deux points (de 0 à 10 % de frais, l'opérateur ne fait que couvrir ses frais réels), alors que la part de moudaraba n'ajoute guère (moins de 4,5 M€ pour 60 points). Du côté des participants, la distribution baisse d'environ 11 M€ à chaque hausse de 2 points du wakala (de 67 M€ à 10 % à 2 M€ à 22 %, pour une moudaraba de 30 %), et les années déficitaires apparaissent à partir de 16 % (une année), 18 % (deux), 20 % (six), 22 % (huit). La **région viable** (opérateur ≥ 5 M€, au plus 2 années en déficit, participants ≥ 40 M€) est étroite : elle se réduit à des taux de wakala de 12 % à 14 % (avec n'importe quelle part de moudaraba testée). Au-delà, la condition sur les participants est violée ; en deçà, l'opérateur ne couvre pas ses frais. **Le taux d'un contrat de Takaful est donc une négociation entre deux contraintes, et le ratio sinistres/cotisations du fonds en fixe la fenêtre.**

Pour aller plus loin. Ajoutez une commission de performance de 10 % de l'excédent positif pour l'opérateur et regardez si elle élargit la région viable. Ne tirez aucune conclusion d'un seul tirage de quinze années : simulez les sinistres (loi gamma de moyenne 78 % des cotisations et d'écart-type 4,5 points) et regardez la **distribution** du résultat de l'opérateur.

Application 4.8 — Détecter des opérations suspectes, de bout en bout

Sections du livre : 4.6.2 à 4.6.7. **Objectif** : enchaîner variables par compte, règles, graphe, anomalies et apprentissage supervisé, puis choisir le nombre d'alertes à examiner par un calcul de coût.

Étape 1 — Variables par compte et règles.

```
tx, comptes, verite = charger("transactions_lab.csv"), charger("comptes_lab.csv"),
    ↪ charger("verite_lab.csv")
F = variables_comptes(tx, comptes).merge(verite, on="id_compte")
y = F["suspect"].values
regle = (F["max_depots_sous_seuil_14j"] >= 5) | ((F["max_virements_entrants_3j"] >= 8) &
    ↪ (F["part_sortie_risque"] >= 0.5)) | (F["n_sorties_pays_risque"] >= 4)
print("comptes :", len(F), "| suspects :", int(y.sum()), "| alertes par règles :", int(regle.sum()),
    ↪ "| vrais cas trouvés :", int((regle & (y == 1)).sum()))
```

comptes : 3000 | suspects : 57 | alertes par règles : 42 | vrais cas trouvés : 42

Étape 2 — Le graphe. Virements d'au moins 2 000 €, cycles de longueur au plus 4.

```
import networkx as nx
gros = tx[(tx["type"] == "virement") & (tx["sens"] == "debit") & (tx["contrepartie"] > 0) &
    ↪ (tx["montant"] >= 2000)]
G = nx.DiGraph(); G.add_edges_from(zip(gros["id_compte"], gros["contrepartie"]))
F["dans_cycle"] = F["id_compte"].isin(set(c for cyc in nx.simple_cycles(G, length_bound=4) for c in
    ↪ cyc)).astype(int)
print("comptes dans un cycle :", int(F["dans_cycle"].sum()), "| dont suspects :",
    ↪ int(F.loc[F["dans_cycle"] == 1, "suspect"].sum()))
```

comptes dans un cycle : 12 | dont suspects : 12

Étape 3 — Anomalies et supervisé. Forêt d'isolement (sans étiquettes) et boosting (étiquettes, validation croisée stratifiée), avec la variable de graphe.

```
from sklearn.ensemble import IsolationForest
from sklearn.model_selection import StratifiedKFold, cross_val_predict
import lightgbm as lgb
cols = ["n_tx", "montant_total", "n_depots_sous_seuil", "max_depots_sous_seuil_14j",
    ↪ "max_virements_entrants_3j", "n_sorties_pays_risque", "montant_pays_risque",
    ↪ "part_sortie_risque"]
Xl = np.log1p(F[cols].values)
score_if = IsolationForest(n_estimators=200, random_state=0,
    ↪ contamination=0.02).fit(Xl).score_samples(Xl)
gbm = lgb.LGBMClassifier(n_estimators=150, learning_rate=0.05, num_leaves=8, min_child_samples=5,
    ↪ verbose=-1, random_state=0, n_jobs=1)
score_gbm = cross_val_predict(gbm, np.column_stack([Xl, F["dans_cycle"]]), y, cv=StratifiedKFold(5,
    ↪ shuffle=True, random_state=0), method="predict_proba")[:, 1]
```

Étape 4 — Choisir le nombre d'alertes. Un examen coûte 200 € ; un cas manqué coûte 15 000 € (valeurs d'illustration). Le coût total de k alertes examinées est $200k + 15\,000 \times (\text{cas non trouvés})$.

```
def cout(score, k):
    trouvees = y[np.argsort(-score)[:k]].sum()
    return 200 * k + 15000 * (y.sum() - trouvees)
for nom, s in (("forêt d'isolement", score_if), ("boosting + cycle", score_gbm)):
    ks = np.arange(0, 601)
    c = np.array([cout(s, k) for k in ks])
    print("%-20s k optimal = %3d | coût minimal = %6.0f € | coût sans aucune alerte = %6.0f €" % (nom,
        ↪ ks[c.argmin()], c.min(), c[0]))
```

forêt d'isolement	k optimal = 292 coût minimal = 58400 € coût sans aucune alerte = 855000 €
boosting + cycle	k optimal = 61 coût minimal = 12200 € coût sans aucune alerte = 855000 €

Lecture. Les règles lèvent 42 alertes, toutes exactes (rappel de 74 %), et le graphe désigne 12 comptes dans des cycles, tous suspects : les deux méthodes se complètent. Pour le choix du nombre d'alertes, le **boosting enrichi du graphe** a un coût minimal de 12 200 € en examinant 61 comptes (à peine plus que les 57 cas réels), contre 855 000 € sans aucune alerte (57 cas manqués $\times$ 15 000 €). La **forêt d'isolement** a besoin de 292 alertes pour son optimum, et son coût minimal est de 58 400 €, près de cinq fois plus : ses alertes sont moins précises, il faut en examiner davantage pour trouver les mêmes cas. Le **seuil optimal** dépend directement du rapport entre le coût d'un cas manqué et celui d'un examen (ici 75 contre 1) : plus le cas manqué coûte cher, plus on accepte d'alertes inutiles.

Pour aller plus loin. Faites varier le coût d'un cas manqué (1 000 €, 15 000 €, 150 000 €) : comment le nombre optimal d'alertes évolue-t-il ? Que vous apprend la comparaison avec la capacité réelle de l'équipe ?

Exercices

Exercice 4.1 ★ — La formule IRB à la main (section 4.1.4)

Une exposition de détail a une PD de 1 %, une LGD de 45 % et une corrélation d'actifs de 12 %. Calculez le capital par euro d'exposition K et le poids de risque. On donne $\Phi^{-1}(0,01) = -2,3263$, $\Phi^{-1}(0,999) = 3,0902$ et $\Phi(-1,34) \approx 0,0901$.

Exercice 4.2 ★ — Ratios et coussins (section 4.1.2)

Une banque a 200 M€ d'actifs pondérés, 12 M€ de fonds propres de base (CET1), 2 M€ d'instruments hybrides de catégorie 1 et 4 M€ de catégorie 2. Avec les minimums de 4,5 %, 6 % et 8 % et un coussin de conservation de 2,5 %, quelles sont les exigences et le déficit éventuel ? Combien de CET1 faut-il lever pour tout satisfaire ?

Exercice 4.3 ★★ — La perte attendue du modèle de Vasicek (section 4.1.4)

Vérifiez numériquement que l'espérance de la PD conditionnelle $p(Z)$ vaut la PD, pour deux couples (PD, ρ) de votre choix. Pourquoi cette propriété justifie-t-elle de retrancher la perte attendue pour obtenir le capital ?

Exercice 4.4 ★★ — À partir de quelle PD l'IRB devient-il plus sévère que le standard ? (section 4.1.5)

Avec une LGD de 46 %, pour quelle PD le poids de risque de la formule « autres expositions de détail » dépasse-t-il 75 % ? Comparez avec le poids de la formule des entreprises (échéance 2,5 ans) à cette PD.

Exercice 4.5 ★★ — Sensibilités du capital (sections 4.1.5, 4.1.6)

Pour un portefeuille homogène (PD 6 %, LGD 46 %), calculez le capital par euro d'exposition : (a) avec la corrélation de la formule ; (b) avec une corrélation doublée ; (c) avec une LGD de ralentissement de 56 % ; (d) avec les deux changements. Quel paramètre pèse le plus ?

Exercice 4.6 ★★ — Un SCR à trois modules, à la main (section 4.2.4)

Modules : marché 100, non-vie 60, vie 40 (M€). Corrélations : marché–non-vie 0,25, marché–vie 0,25, non-vie–vie 0. Calculez le SCR de base et la diversification.

Exercice 4.7 ★★ — Une marge de risque, à la main (section 4.2.3)

Le capital requis futur d'un portefeuille en extinction est de 60, 30 et 10 M€ aux trois prochaines dates ; le taux d'actualisation est de 3 % et le coût du capital de 6 %. Calculez la marge de risque. Que devient-elle si l'écoulement est deux fois plus lent (60, 60, 30, 30, 10, 10) ?

Exercice 4.8 ★★★ — Queues et agrégation (section 4.2.5)

Reprenez les cinq modules de l'application 4.4. Simulez le quantile à 99,5 % de la perte totale avec une copule de Student à 3, 4, 8 et 30 degrés de liberté, marginales normales calibrées sur les mêmes charges isolées. Que constatez-vous quand le nombre de degrés de liberté augmente ?

Exercice 4.9 ★ — Une CSM à la main (section 4.4.1)

Un groupe de contrats de quatre ans encaisse 200 de primes ; les sinistres attendus valent 130 en valeur actuelle, l'ajustement pour risque 20. Taux d'actualisation nul, couverture constante. Calculez la CSM initiale, sa libération annuelle et le produit annuel. Puis : à la fin de l'année 2, la valeur actuelle des sinistres futurs est révisée de +10 ; que devient la libération des années 3 et 4 ?

Exercice 4.10 ★★ — Le taux de wakala qui convient aux deux parties (section 4.5.2)

Pour chacun des trois fonds, calculez l'intervalle de taux de wakala compris entre les frais réels de l'opérateur (10 % des cotisations) et le taux qui annule l'excédent moyen des participants. Que dire du fonds dont l'intervalle est le plus étroit ?

Exercice 4.11 ★★ — Régler une règle de fractionnement (section 4.6.3)

Pour la règle « au moins n dépôts d'espèces entre 9 000 et 10 000 € dans une fenêtre de w jours », faites varier n (3 à 8) et w (7, 14, 30). Quel couple choisiriez-vous si l'équipe ne peut pas examiner plus de 30 comptes ?

Exercice 4.12 ★★★ — Le graphe complet ne désigne personne (section 4.6.4)

Construisez le graphe de **tous** les virements entre comptes et mesurez la taille de sa plus grande composante fortement connexe. Puis ne gardez que les virements d'au moins 500, 1 000, 2 000 et 3 000 € et indiquez pour chaque seuil le nombre de cycles de longueur au plus 4 et la part des comptes trouvés qui sont de vrais allers-retours.

Corrigé 4.1

On calcule d'abord $\sqrt{\rho} = \sqrt{0,12} = 0,3464$ et $\sqrt{1-\rho} = \sqrt{0,88} = 0,9381$. Le numérateur de l'argument de Φ est $-2,3263 + 0,3464 \times 3,0902 = -1,2558$; divisé par 0,9381, il vaut $-1,3386$, et $\Phi(-1,3386) \approx 0,0904$. Donc $K = 0,45 \times (0,0904 - 0,01) = 0,45 \times 0,0804 \approx 0,0362$: environ **3,6 % de l'exposition** (le calcul exact donne 3,61 %, l'écart venant des arrondis de Φ), soit un poids de risque d'environ **45 %** ($12,5 \times 0,0361 \approx 45,2$ %). Vérification :

```
print(round(float(k_vasicek(0.01, 0.45, 0.12)), 4), "| poids de risque :", round(12.5 *
↳ float(k_vasicek(0.01, 0.45, 0.12)), 3))
```

```
0.0361 | poids de risque : 0.452
```

Corrigé 4.2

Exigences en M€ pour 200 M€ de RWA : CET1 = $(4,5 + 2,5) \% = 7 \% \rightarrow 14$; catégorie 1 = $(6 + 2,5) \% = 8,5 \% \rightarrow 17$; total = $(8 + 2,5) \% = 10,5 \% \rightarrow 21$. Disponible : CET1 12, catégorie 1 12 + 2 = 14, total 14 + 4 = 18. Déficits : $14 - 12 = 2$, $17 - 14 = 3$, $21 - 18 = 3$. Lever **3 M€ de CET1** résout tout : CET1 15 (≥ 14), catégorie 1 17 (≥ 17), total 21 (≥ 21). Un déficit en CET1 de 2 M€ ne suffirait pas, car la catégorie 1 et le total restent à 1 M€ du but : une levée de CET1 compte **dans les trois ratios à la fois**.

Corrigé 4.3

```
rng = np.random.default_rng(0)
Z = rng.standard_normal(2_000_000)
for p_, r_ in ((0.02, 0.10), (0.15, 0.04)):
    pc = norm.cdf((norm.ppf(p_) - np.sqrt(r_) * Z) / np.sqrt(1 - r_))
    print("PD = %.2f, rho = %.2f : moyenne de p(Z) = %.4f" % (p_, r_, pc.mean()))
```

```
PD = 0.02, rho = 0.10 : moyenne de p(Z) = 0.0200
PD = 0.15, rho = 0.04 : moyenne de p(Z) = 0.1500
```

L'espérance de $p(Z)$ vaut la PD : c'est l'**identité** $E[\Phi((c - \sqrt{\rho}Z)/\sqrt{1-\rho})] = \Phi(c)$, qui exprime que la probabilité inconditionnelle de défaut est la PD. Comme le taux de perte moyen vaut donc $\text{LGD} \times \text{PD}$, c'est la **perte attendue** ; le quantile de $p(Z)$ moins cette moyenne est l'excédent de perte, la **perte inattendue**.

Corrigé 4.4

```
from scipy.optimize import brentq
pd_crit = brentq(lambda p_: 12.5 * float(k_detail(p_, 0.46)) - 0.75, 0.001, 0.5)
print("PD critique (détail) : %.4f | poids des entreprises à cette PD : %.3f" % (pd_crit, 12.5 *
↳ float(k_entreprise(pd_crit, 0.46))))
```

```
PD critique (détail) : 0.0908 | poids des entreprises à cette PD : 1.904
```

La formule de détail devient plus sévère que les 75 % du standard pour une PD d'environ **9 %** ; en dessous, elle est plus favorable. Pour la même PD, la formule des entreprises donne un poids bien plus élevé (corrélacion plus forte, ajustement d'échéance) : **la catégorie de l'exposition compte autant que sa PD**, d'où l'importance de classer correctement les expositions.

Corrigé 4.5

```
rho_h = float(rho_detail_autre(0.06))
for nom, r_, l_ in (("référence", rho_h, 0.46), ("corrélation doublée", 2 * rho_h, 0.46), ("LGD de
↳ ralentissement 56 %", rho_h, 0.56), ("les deux", 2 * rho_h, 0.56)):
    print("%-28s K = %.4f" % (nom, float(k_vasicek(0.06, l_, r_))))
```

```
référence          K = 0.0554
corrélation doublée K = 0.0912
LGD de ralentissement 56 % K = 0.0674
les deux           K = 0.1110
```

Une LGD relevée de 10 points (+22 %) augmente le capital **dans la même proportion** (+22 %) ; doubler la corrélation l'augmente d'environ **65 %** (de 5,54 % à 9,12 %). Le capital est donc plus sensible à la corrélation qu'à la LGD dans cette plage, ce qui explique que le régulateur **impose** la corrélation : laissée au choix des banques, elle serait la variable d'ajustement la plus tentante. Les deux effets se multiplient (le dernier cas).

Corrigé 4.6

$BSCR^2 = 100^2 + 60^2 + 40^2 + 2(0,25)(100)(60) + 2(0,25)(100)(40) + 2(0)(60)(40) = 10\,000 + 3\,600 + 1\,600 + 3\,000 + 2\,000 = 20\,200$; $BSCR = \sqrt{20\,200} \approx 142,1$ M€. La somme des charges est de 200 : la **diversification** retire $200 - 142,1 \approx 57,9$ M€ (29 %).

```
print(round(agreger([100, 60, 40], [[1, .25, .25], [.25, 1, 0], [.25, 0, 1]]), 1))
```

```
142.1
```

Corrigé 4.7

$RM = 0,06 (60/1,03 + 30/1,03^2 + 10/1,03^3) = 0,06 (58,25 + 28,28 + 9,15) = 0,06 \times 95,68 \approx 5,74$ M€. Avec un écoulement deux fois plus lent, la somme actualisée vaut $60/1,03 + 60/1,03^2 + 30/1,03^3 + 30/1,03^4 + 10/1,03^5 + 10/1,03^6$ et la marge est de **11,16 M€**, presque le double ($\times 1,9$).

```
def rm_(srs, taux=0.03, c=0.06):
    return c * sum(s / (1 + taux) ** (t + 1) for t, s in enumerate(srs))
print(round(rm_([60, 30, 10]), 2), round(rm_([60, 60, 30, 30, 10, 10]), 2))
```

```
5.74 11.16
```

Corrigé 4.8

```
from scipy.stats import t as loi_t
corr5 = np.array([[1, .25, .25, .25, .25], [.25, 1, .25, .25, .5], [.25, .25, 1, .25, 0], [.25, .25,
↳ .25, 1, .5], [.25, .5, 0, .5, 1]])
c5 = np.array([110, 20, 30, 45, 140.])
rng = np.random.default_rng(0)
Z = rng.standard_normal((300000, 5)) @ np.linalg.cholesky(corr5).T
for ddl in (3, 4, 8, 30):
    W = rng.chisquare(ddl, 300000) / ddl
    T = Z / np.sqrt(W[:, None])
    tot = (norm.ppf(loi_t.cdf(T, ddl)) * c5 / norm.ppf(0.995)).sum(axis=1)
    print("copule de Student, %2d ddl : quantile 99,5 %% = %.1f" % (ddl, np.quantile(tot, 0.995)))
print("formule standard :", round(agreger(c5, corr5), 1))
```

```

copule de Student, 3 ddl : quantile 99,5 % = 266.5
copule de Student, 4 ddl : quantile 99,5 % = 258.5
copule de Student, 8 ddl : quantile 99,5 % = 252.0
copule de Student, 30 ddl : quantile 99,5 % = 243.9
formule standard : 241.8

```

Plus le nombre de degrés de liberté est **faible**, plus la dépendance de queue est forte et plus la perte au quantile 99,5 % dépasse la formule standard. À 30 degrés de liberté la copule de Student est presque gaussienne et le quantile redevient proche de la formule : **c'est la dépendance dans les queues, non la corrélation linéaire, qui fait l'écart** (section 4.2.5).

Corrigé 4.9

CSM initiale : $200 - (130 + 20) = 50$. Libération annuelle : $50/4 = 12,5$. Produit annuel : sinistres attendus $130/4 = 32,5$, plus ajustement $20/4 = 5$, plus CSM $12,5$: **50** par an, le quart de la prime. Après l'année 2, la CSM restante est $50 - 2 \times 12,5 = 25$; la révision de +10 des sinistres futurs la ramène à 15, libérée par moitié sur les années 3 et 4 : **7,5 par an** au lieu de 12,5. Vérification :

```
print(ifs17(200, 130, 20, 4, revisions={3: 10.0}).to_string(index=False))
```

année	CSM ouverture	libération	produit	charge de sinistres	résultat des services
1	50.0	12.5	50.0	32.5	17.5
2	37.5	12.5	50.0	32.5	17.5
3	15.0	7.5	45.0	32.5	12.5
4	7.5	7.5	45.0	32.5	12.5

Corrigé 4.10

```

for fonds in ("famille", "auto", "sante"):
    d = tk[tk["fonds"] == fonds]
    lr = (d["sinistres"] / d["cotisations"]).mean()
    fr = (d["frais_gestion"] / d["cotisations"]).mean()
    print("%-8s frais réels %.1f %% | taux qui annule l'excédent moyen %.1f %% | largeur de
    ↪ l'intervalle %.1f points" % (fonds, 100 * fr, 100 * (1 - lr), 100 * (1 - lr - fr)))

```

```

famille frais réels 10.0 % | taux qui annule l'excédent moyen 26.4 % | largeur de l'intervalle 16.4
↪ points
auto    frais réels 10.0 % | taux qui annule l'excédent moyen 21.9 % | largeur de l'intervalle 11.9
↪ points
sante   frais réels 10.0 % | taux qui annule l'excédent moyen 24.3 % | largeur de l'intervalle 14.3
↪ points

```

L'intervalle est le plus étroit pour le **fonds auto** (de 10 % à 21,9 %, soit 11,9 points) : son ratio sinistres/cotisations est le plus élevé. Un taux de wakala dans cet intervalle fait gagner l'opérateur mais laisse peu de marge pour les écarts annuels de sinistralité (écart-type de 4,5 points) : **plus l'intervalle est étroit, plus la fixation du taux est sensible**, et plus le fonds dépendra d'un prêt sans intérêt de l'opérateur.

Corrigé 4.11

```

tx, comptes, verite = charger("transactions_lab.csv"), charger("comptes_lab.csv"),
    ↪ charger("verite_lab.csv")
dep = tx[(tx["type"] == "especes") & (tx["sens"] == "credit") & (tx["montant"] >= 9000) &
    ↪ (tx["montant"] < 10000)].copy()
dep["jour"] = (pd.to_datetime(dep["date"]) - pd.to_datetime("2024-01-01")).dt.days
susp = set(verite.loc[verite["suspect"] == 1, "id_compte"])

```

```
lignes = []
for w in (7, 14, 30):
    mx = dep.groupby("id_compte")["jour"].apply(lambda s: max_fenetre(np.sort(s.values), w))
    for n in range(3, 9):
        al = set(mx[mx >= n].index)
        lignes.append({"fenêtre (j)": w, "n": n, "alertes": len(al), "vrais cas": len(al & susp),
            ↪ "précision": round(len(al & susp) / max(len(al), 1), 2)})
res = pd.DataFrame(lignes)
print(res[res["n"].isin([3, 5, 8])].to_string(index=False))
```

fenêtre (j)	n	alertes	vrais cas	précision
7	3	23	15	0.65
7	5	15	15	1.00
7	8	9	9	1.00
14	3	40	15	0.38
14	5	15	15	1.00
14	8	15	15	1.00
30	3	46	15	0.33
30	5	19	15	0.79
30	8	15	15	1.00

Pour une capacité de 30 comptes, la règle **(14 jours, au moins 5 dépôts)** lève 15 alertes, toutes de vrais cas (les 15 fractionneurs) : précision et rappel de 100 % **sur ce schéma**. Un seuil plus bas (3 dépôts) lève 40 alertes à précision de 38 % (au-delà de la capacité), une fenêtre plus large (30 jours) fait entrer les commerces légitimes (19 alertes, précision de 79 % pour $n = 5$), une fenêtre de 7 jours convient pour $n = 5$ (15 cas sur 15) mais perd 6 cas sur 15 pour $n = 8$. Le meilleur couple est donc celui qui est **le plus sélectif sans perdre de cas**.

Corrigé 4.12

```
import networkx as nx
vir = tx[(tx["type"] == "virement") & (tx["sens"] == "debit") & (tx["contrepartie"] > 0)]
Gtot = nx.DiGraph(); Gtot.add_edges_from(zip(vir["id_compte"], vir["contrepartie"]))
cc = max(nx.strongly_connected_components(Gtot), key=len)
print("graphe complet : %d virements distincts, plus grande composante fortement connexe : %d comptes"
    ↪ % (Gtot.number_of_edges(), len(cc)))
anneaux = set(verite.loc[verite["schema"] == "aller_retour", "id_compte"])
for seuil in (500, 1000, 2000, 3000):
    e = vir[vir["montant"] >= seuil]
    Gs = nx.DiGraph(); Gs.add_edges_from(zip(e["id_compte"], e["contrepartie"]))
    cyc = list(nx.simple_cycles(Gs, length_bound=4))
    cs = set(c for cy in cyc for c in cy)
    print("seuil %4d € : %4d virements | %2d cycles | %3d comptes | part de vrais anneaux %.2f" %
        ↪ (seuil, Gs.number_of_edges(), len(cyc), len(cs), len(cs & anneaux) / max(len(cs), 1)))
```

```
graphe complet : 17652 virements distincts, plus grande composante fortement connexe : 2969 comptes
seuil 500 € : 2669 virements | 4 cycles | 13 comptes | part de vrais anneaux 0.92
seuil 1000 € : 904 virements | 3 cycles | 12 comptes | part de vrais anneaux 1.00
seuil 2000 € : 226 virements | 3 cycles | 12 comptes | part de vrais anneaux 1.00
seuil 3000 € : 80 virements | 3 cycles | 12 comptes | part de vrais anneaux 1.00
```

Le graphe complet compte 17 652 virements distincts et une composante fortement connexe de **2 969 comptes sur 3 000** : presque chaque compte est atteignable depuis tous les autres, et chercher des cycles dans ce graphe n'identifie personne. Avec un seuil de 500 €, on trouve 4 cycles et 13 comptes dont 92 % sont de vrais allers-retours (un cycle fortuit s'ajoute) ; dès 1 000 €, les 12 comptes trouvés sont tous des allers-retours. **Restreindre le graphe** (au montant, à la période, à la longueur des cycles) est la condition pour que la détection de motifs veuille dire quelque chose.

Chapitre 5 : ⊕ Assurance vie — exercices et applications

Orientation. Ce fichier accompagne le chapitre 5 du livre (tables de mortalité, mathématiques actuarielles de la vie, modèle de Lee–Carter). Il contient **huit applications guidées** (petites études sur les données du volume) puis **douze exercices** gradués (★ direct, ★★ demande un raisonnement, ★★★ plus délicat), tous corrigés à la fin. Les données sont **simulées** (build/donnees5.py) : la vérité programmée est connue et sert de juge. Le fichier est **autonome** : il recharge ses données et ses outils (build/outils_ch05.py). Essayez chaque énoncé avant de lire le corrigé.

```
import os, sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
from outils_ch05 import *

def NUM(cle, valeur):
    print("NUM", cle, valeur)          # nombre cité dans la prose, relu lors de la vérification

pop = charger()                        # décès et expositions (sexe, âge, année)
M = {s: surface(pop, s) for s in "FM"}; E = {s: surface(pop, s, "exposition") for s in "FM"}; D = {s:
    ↪ surface(pop, s, "deces") for s in "FM"}
AX, BX, KT = verite()                 # vérité programmée (a_x, b_x, k_t)
pv = pd.read_csv(os.path.join(DONNEES, "portefeuille_vie.csv"))
ages = np.arange(100)
I = 0.02; V_, D_ = 1 / (1 + I), I / (1 + I)          # taux technique d'illustration
a30 = np.arange(30, 100)
P_GM = {s: gm_ajuste(a30, D[s][2019].loc[a30].to_numpy(), E[s][2019].loc[a30].to_numpy()) for s in
    ↪ "FM"} # Gompertz–Makeham 2019
Q = {s: prolonge(q_depuis_m(M[s][2019].to_numpy()), P_GM[s]) for s in "FM"}
    ↪ # q_x 2019 fermés à 120 ans
T = {s: table_vie(Q[s]) for s in "FM"}
print({s: (round(float(T[s].e[0]), 2), round(float(T[s].e[65]), 2)) for s in "FM"})

{'F': (74.39, 14.45), 'M': (71.54, 12.68)}
```

Applications

Application 5.1 — Table de période des hommes et comparaison à la vérité (section 5.1)

Objectif. Construire la table 2019 des hommes, comparer espérances de vie des deux sexes, et juger l'erreur d'estimation face à la vérité programmée.

Étape 1 — la table. La cellule d'amorçage a déjà construit $T["M"]$ (et $T["F"]$) : taux observés, $q = 1 - e^{-m}$, fermeture à 120 ans par la loi de Gompertz–Makeham ajustée sur les âges 30 à 99.

Étape 2 — comparer à la table vraie. Calculez la table obtenue avec les taux vrais $m_x = \exp(a_x + b_x k_t)$ de 2019 (fonction `m_vrai`) et comparez e_0 et e_{65} pour les deux sexes.

```
vrai = {s: table_vie(prolonge(q_depuis_m(m_vrai(s, 2019)), P_GM[s])) for s in "FM"}
for s in "FM":
    print(s, "e0 estimé", round(T[s].e[0], 2), "vrai", round(vrai[s].e[0], 2), "| e65 estimé",
          round(T[s].e[65], 2), "vrai", round(vrai[s].e[65], 2))
```

```
F e0 estimé 74.39 vrai 74.37 | e65 estimé 14.45 vrai 14.5
M e0 estimé 71.54 vrai 71.53 | e65 estimé 12.68 vrai 12.66
```

Étape 3 — la surmortalité masculine. Le rapport des taux hommes/femmes varie-t-il avec l'âge ? Un rapport calculé âge par âge est très bruité (quelques dizaines de décès par cellule) : regroupez par dizaines d'âges, en divisant les décès totaux par les expositions totales de la dizaine.

```
def taux_dizaine(s, a0):
    cols = slice(a0, a0 + 9)
    return D[s][2019].loc[cols].sum() / E[s][2019].loc[cols].sum()
rapports = {f"{a0}-{a0 + 9}": round(float(taux_dizaine("M", a0) / taux_dizaine("F", a0)), 2) for a0 in
            range(30, 90, 10)}
print(rapports)
print("écart d'espérance de vie à 65 ans (F - H) :", round(T["F"].e[65] - T["M"].e[65], 2), "ans")
```

```
{'30-39': 1.47, '40-49': 1.23, '50-59': 1.32, '60-69': 1.32, '70-79': 1.3, '80-89': 1.33}
écart d'espérance de vie à 65 ans (F - H) : 1.77 ans
```

À conclure. Les écarts entre estimé et vrai sont de quelques centièmes d'année (population de centaines de milliers de personnes). Le rapport hommes/femmes est **à peu près constant** autour de 1,3 : c'est une propriété de la simulation (mortalité masculine programmée à 1,35 fois la féminine, atténuée par une tendance plus favorable). Sur des données réelles, il varie nettement avec l'âge, avec un maximum aux âges de jeune adulte.

Application 5.2 — Lisser les taux d'un portefeuille : bruts, Gompertz–Makeham, Whittaker–Henderson (section 5.1.4)

Objectif. Comparer trois estimations de la mortalité des assurés par âge et juger laquelle s'approche le mieux de la vérité programmée ($0,75 \times$ la population).

Étape 1 — les taux bruts. On reconstruit les lignes contrat-année, puis décès et expositions par âge.

```
L = lignes_police_annee(pv)
L["m_ref"] = [M[s].loc[a, t] for s, a, t in zip(L["sexe"], L["age"], L["annee"])]
L["attendu"] = L["expo"] * L["m_ref"]
g = L.groupby("age").agg(D=("deces", "sum"), E=("expo", "sum"), att=("attendu", "sum")).reset_index()
g = g[(g["age"] >= 30) & (g["age"] <= 79) & (g["E"] > 0)].reset_index(drop=True)
g["brut"] = g["D"] / g["E"]
g["vrai"] = 0.75 * g["att"] / g["E"]          # taux attendu d'après la table de population × 0,75
↳ (par âge, avec la composition réelle)
```

```
print(g[["age", "D", "E", "brut", "vrai"]].head(4).round(4).to_string(index=False))
```

```
age D      E    brut   vrai
30 0 1042.0 0.0000 0.0007
31 1 1176.5 0.0008 0.0008
32 2 1301.0 0.0015 0.0009
33 2 1499.0 0.0013 0.0009
```

Étape 2 — Gompertz–Makeham. Ajustez la loi par maximum de vraisemblance poissonien sur ces âges et évaluez-la.

```
p_pf = gm_ajuste(g["age"], g["D"], g["E"])
g["gm"] = gm_mu(g["age"] + 0.5, p_pf)
print("doublement tous les", round(np.log(2) / np.exp(p_pf[2]), 2), "ans")
```

```
doublement tous les 6.41 ans
```

Étape 3 — Whittaker–Henderson. On cherche le vecteur $\hat{g} = \ln \hat{m}$ qui minimise la **vraisemblance de Poisson pénalisée** $\sum_x (E_x e^{g_x} - D_x g_x) + \frac{\lambda}{2} \sum_x (\Delta^3 g_x)^2$: fidélité aux données, plus une pénalité sur les différences troisièmes (une parabole n'est pas pénalisée). Le critère est convexe : on le résout par des itérations de Newton, avec pour gradient $Ee^g - D + \lambda \Delta^{3\top} \Delta^3 g$ et pour hessien $\text{diag}(Ee^g) + \lambda \Delta^{3\top} \Delta^3$.

```
def whittaker(D, E, lam, ordre=3, it=40):
    n = len(D)
    Dm = np.diff(np.eye(n), ordre, axis=0)                # matrice des différences d'ordre 3
    P = lam * Dm.T @ Dm
    g = np.full(n, np.log(D.sum() / E.sum()))             # départ : taux constant
    for _ in range(it):
        mu = E * np.exp(g)
        g = g - np.linalg.solve(np.diag(mu) + P, mu - D + P @ g)
    return np.exp(g)
g["wh"] = whittaker(g["D"].to_numpy(float), g["E"].to_numpy(float), lam=100.0)
```

Étape 4 — juger. Pour chaque méthode, calculez l'erreur relative moyenne (valeur absolue) par rapport à vrai. Essayez ensuite $\lambda = 1, 10, 100, 10^4$.

```
for col in ("brut", "gm", "wh"):
    print(col, round(float(np.mean(np.abs(g[col] / g["vrai"] - 1))), 3))
for lam in (1, 10, 100, 1e4):
    h = whittaker(g["D"].to_numpy(float), g["E"].to_numpy(float), lam)
    print("lambda", lam, round(float(np.mean(np.abs(h / g["vrai"] - 1))), 3))
```

```
brut 0.332
gm 0.069
wh 0.134
lambda 1 0.206
lambda 10 0.143
lambda 100 0.134
lambda 10000.0 0.095
```

À conclure. L'erreur relative moyenne par rapport à la vérité est de 33 % pour les taux bruts, 7 % pour Gompertz–Makeham et 13 % pour Whittaker–Henderson ($\lambda = 100$). Les deux lissages rapprochent nettement de la vérité. Whittaker–Henderson dépend de λ (21 % pour $\lambda = 1$, 10 % pour $\lambda = 10^4$: trop petit, il colle au bruit ; très grand, il tend vers une parabole du logarithme) alors que Gompertz–Makeham n'a aucun réglage mais **impose une forme** : ici cette forme est exactement celle qui a servi à simuler, ce qui la favorise.

Application 5.3 — Rapport réel/attendu par sexe, contrat et capital (section 5.1.5)

Objectif. Décomposer le rapport réel/attendu et le pondérer par le capital.

Étape 1 — par sexe et par contrat. Calculez D , l'attendu et le rapport avec son intervalle de Poisson pour chaque sexe puis chaque type de contrat.

```
L["attendu"] = L["expo"] * L["m_ref"]
def tableau(par):
    lignes = []
    for k, h in L.groupby(par):
        r, b, hi = ae_ic(int(h["deces"].sum()), h["attendu"].sum())
        lignes.append((k, int(h["deces"].sum()), round(h["attendu"].sum(), 1), round(r, 3), round(b, 3), round(hi, 3)))
    return pd.DataFrame(lignes, columns=[par, "décès", "attendus", "A/E", "bas", "haut"])
print(tableau("sexe").to_string(index=False)); print(tableau("contrat").to_string(index=False))
```

sexe	décès	attendus	A/E	bas	haut
F	326	424.2	0.768	0.687	0.857
M	506	664.0	0.762	0.697	0.831

contrat	décès	attendus	A/E	bas	haut
temporaire_10	255	328.8	0.776	0.683	0.877
temporaire_20	303	379.1	0.799	0.712	0.895
vie_entiere	274	380.3	0.720	0.638	0.811

Étape 2 — pondérer par le capital. Le coût d'un décès est le capital assuré : le rapport pertinent pour le résultat est $\sum C_i \delta_i / \sum C_i E_i m_i^{\text{réf}}$. Calculez-le et comparez au rapport en nombre.

```
ae_n = L["deces"].sum() / L["attendu"].sum()
ae_c = (L["capital"] * L["deces"]).sum() / (L["capital"] * L["attendu"]).sum()
print(f"A/E en nombre {ae_n:.3f}    A/E pondéré par le capital {ae_c:.3f}")
```

A/E en nombre 0.765 A/E pondéré par le capital 0.768

Étape 3 — précision. L'intervalle de Poisson ne s'applique plus tel quel au rapport pondéré. Proposez une méthode (indice : *bootstrap* sur les contrats) et appliquez-la avec 500 rééchantillonnages.

```
rng = np.random.default_rng(1)
par_contrat = L.groupby("id_contrat").agg(dec=("deces", "sum"), att=("attendu", "sum"),
    cap=("capital", "first"))
n = len(par_contrat); vals = []
for _ in range(500):
    h = par_contrat.iloc[rng.integers(0, n, n)]
    vals.append((h["cap"] * h["dec"]).sum() / (h["cap"] * h["att"]).sum())
print("A/E pondéré : IC bootstrap à 95 % =", np.percentile(vals, [2.5, 97.5]).round(3))
```

A/E pondéré : IC bootstrap à 95 % = [0.704 0.831]

À conclure. Par sexe ou par contrat, **aucun sous-groupe ne se détache** de 0,75 (la sélection programmée est la même partout) ; le rapport pondéré par le capital est très proche du rapport en nombre mais son intervalle est **plus large**, parce qu'un décès de gros capital pèse beaucoup.

Application 5.4 — Grille de tarifs et coût d'une tarification unisexe (section 5.2)

Objectif. Construire une grille de primes pures et mesurer l'effet d'une prime unique pour les deux sexes.

Étape 1 — la grille. Pour des temporaires de 10, 20 et 30 ans souscrites à 30, 40, 50 ans, calculez la prime annuelle pure pour 1 000 € de capital, pour chaque sexe (table 2019, 2 %).

```
def prime_mille(q, x, n):
    At, at = temporaire(q, I, x, n)
    return 1000 * At / at
lignes = [(x, n, round(prime_mille(Q["F"], x, n), 2), round(prime_mille(Q["M"], x, n), 2)) for x in
    ↪ (30, 40, 50) for n in (10, 20, 30)]
print(pd.DataFrame(lignes, columns=["âge", "durée", "femmes", "hommes"]).to_string(index=False))
```

âge	durée	femmes	hommes
30	10	0.98	1.43
30	20	1.74	2.26
30	30	3.11	4.06
40	10	2.68	3.30
40	20	4.59	5.90
40	30	8.23	10.55
50	10	7.03	9.26
50	20	12.31	16.04
50	30	20.66	25.58

Étape 2 — la prime unisexe. Si l'assureur propose la **même prime** aux deux sexes, l'équivalence se fait sur l'ensemble de la population assurée : la prime unisexe est $P_u = \frac{w_H A^H + w_F A^F}{w_H \ddot{a}^H + w_F \ddot{a}^F}$ où w_H, w_F sont les parts de chaque sexe (on prend 55 % d'hommes comme dans le portefeuille). Calculez-la pour une temporaire de 20 ans à 40 ans, et le transfert implicite entre sexes.

```
x, n, wH = 40, 20, 0.55
AF_, aF_ = temporaire(Q["F"], I, x, n); AM_, aM_ = temporaire(Q["M"], I, x, n)
Pu = 1000 * (wH * AM_ + (1 - wH) * AF_) / (wH * aM_ + (1 - wH) * aF_)
Pf, Pm = prime_mille(Q["F"], x, n), prime_mille(Q["M"], x, n)
print(f"F {Pf:.2f} H {Pm:.2f} unisexe {Pu:.2f} → les femmes paient {100 * (Pu / Pf - 1):.0f} % de
    ↪ plus, les hommes {100 * (1 - Pu / Pm):.0f} % de moins")
```

F 4.59 H 5.90 unisexe 5.31 → les femmes paient 16 % de plus, les hommes 10 % de moins

À conclure. Une prime unique **redistribue** : les femmes subventionnent les hommes. Si la composition du portefeuille change (davantage de femmes, car les hommes trouvent l'offre moins chère...), la prime unisexe devient insuffisante : c'est une **anti-sélection**.

Application 5.5 — Provisions : prospectif, rétrospectif, changement de base (section 5.2.4)

Objectif. Calculer la provision d'une vie entière et mesurer ce qui arrive quand la table change en cours de contrat.

Étape 1 — vie entière à 40 ans. Capital 100 000 €, prime annuelle pure payable à vie. Calculez la prime puis la provision aux durées 10, 20 et 30 ans.

```
A_, a_ = valeurs(Q["F"], I)
x = 40; C = 100000.0
P = C * A_[x] / a_[x]
for t in (10, 20, 30):
    print(f"t = {t:2d} provision = {C * A_[x + t] - P * a_[x + t]:9.0f} € ({100 * (C * A_[x + t] -
    ↪ P * a_[x + t]) / C:.1f} % du capital)")
print("prime annuelle :", round(P, 1))
```

t = 10	provision =	19253 €	(19.3 % du capital)
t = 20	provision =	39997 €	(40.0 % du capital)
t = 30	provision =	60144 €	(60.1 % du capital)
prime annuelle : 1961.7			

Étape 2 — changement de base. La prime a été fixée avec la table de 1980 (2 %). Dix ans plus

tard, on évalue la provision avec la table de **2019** : c'est le **boni ou mali de changement de base**. Calculez-le pour une temporaire de 20 ans à 40 ans, capital 100 000 €, au bout de 10 ans.

```
q80 = prolonge(q_depuis_m(M["F"][1980].to_numpy(), P_GM["F"]))
At, at = temporaire(q80, I, 40, 20); P80 = 100000 * At / at
V80 = reserve_prospective(q80, I, 40, 20, 100000, P80, 10) # provision selon l'ancienne base
V19 = reserve_prospective(Q["F"], I, 40, 20, 100000, P80, 10) # même contrat, nouvelle base
print(f"prime {P80:.1f} provision ancienne base {V80:.0f} nouvelle base {V19:.0f} écart {V19 -
↪ V80:.0f} €")
```

```
prime 815.6 provision ancienne base 3730 nouvelle base -1004 écart -4734 €
```

À conclure. La provision d'une vie entière grimpe jusqu'à 19,3 %, 40,0 % puis 60,1 % du capital aux durées 10, 20 et 30 ans. Pour la temporaire, la mortalité a baissé depuis 1980 : la provision de la même police est de 3730 € avec l'ancienne base et de -1004 € avec la nouvelle (un montant **néгатif** : les primes futures valent plus que les sinistres futurs, la prime de 1980 surpaye le risque de 2019). En pratique, on ne comptabilise pas de provision négative (on plafonne à zéro) et l'on libère un boni ; pour une rente, ce serait l'inverse : un **mali** à financer.

Application 5.6 — Lee–Carter pour les hommes : ajustement, vérité et projection (section 5.3)

Objectif. Refaire pour les hommes l'analyse du livre, de bout en bout.

Étape 1 — ajustement. Ajustez le modèle (SVD puis recalage) et mesurez l'écart à la vérité.

```
M_, E_, D_m = M["M"], E["M"], D["M"]
ax_, bx_, kt_, sv = lc_ajuste(M_)
kt_r = lc_recale(ax_, bx_, kt_, D_m.to_numpy(), E_.to_numpy())
ktv = KT["kt_M"].to_numpy()
print(f"première composante : {100 * sv[0] ** 2 / (sv ** 2).sum():.1f} % rmse k : SVD
↪ {np.sqrt(np.mean((kt_ - ktv) ** 2)):.2f} recalé {np.sqrt(np.mean((kt_r - ktv) ** 2)):.2f}")
print("corrélacion b estimé / vrai :", round(np.corrcoef(bx_, BX)[0, 1], 3), " max |a_x - a_x vrai|
↪ :", round(np.abs(ax_ - AX["M"]).max(), 3))
```

```
première composante : 72.3 % rmse k : SVD 1.16 recalé 0.50
corrélacion b estimé / vrai : 0.961 max |a_x - a_x vrai| : 0.054
```

Étape 2 — dérive et écart-type. Estimez la dérive et σ , avec et sans l'année 2018 traitée par interpolation.

```
i18 = list(ANNEES).index(2018)
k_i = kt_r.copy(); k_i[i18] = (k_i[i18 - 1] + k_i[i18 + 1]) / 2
print("brut :", np.round(derive_sigma(kt_r), 3), " 2018 interpolée :", np.round(derive_sigma(k_i),
↪ 3))
```

```
brut : [-1.335 1.876] 2018 interpolée : [-1.335 1.497]
```

Étape 3 — projeter. Simulez 1 000 trajectoires sur 30 ans et donnez l'espérance de vie à 65 ans en 2049 (médiane et intervalle à 90 %). L'espérance de vie se calcule à partir de k avec $a_x + b_x k$ et la fermeture de Gompertz–Makeham ajustée sur les hommes.

```
delta, sigma = derive_sigma(k_i)
sim = lc_projetter(ax_, bx_, kt_r, 30, 1000, delta, sigma, np.random.default_rng(5))
e65 = lambda k: table_vie(prolonge(q_depuis_m(np.exp(ax_ + bx_ * k)), P_GM["M"])).e[65]
k49 = np.percentile(sim[:, -1], [5, 50, 95])
print("e65 2019 :", round(e65(kt_r[-1]), 2), " e65 2049 (5 %, médiane, 95 %) :",
↪ [round(float(e65(k)), 2) for k in k49[:-1]])
```

```
e65 2019 : 12.68      e65 2049 (5 %, médiane, 95 %) : [13.86, 14.48, 15.02]
```

À conclure. Pour les hommes comme pour les femmes, la dérive est bien estimée (vraie dérive : $-1,2 \times 1,1 = -1,32$ pour les hommes, puisque $k^H = 1,1 k^F$ dans le jeu) et σ est gonflé par le choc de 2018 et par l'erreur d'estimation.

Application 5.7 — Rentes : risque de tendance contre risque individuel (section 5.3.4)

Objectif. Chiffrer le risque de longévité d'un portefeuille de rentiers de 65 ans (hommes), avec et sans incertitude de paramètre.

Étape 1 — le coût d'une rente par trajectoire. On calcule, pour chaque trajectoire de k , le coût d'une rente de 1 € par an payée d'avance à 65 ans (taux 2 %), en suivant la cohorte : la mortalité de l'année 2019 + j s'applique à l'âge 65 + j .

```
def cout_rente(chemins):
    n = chemins.shape[0]; pv = np.zeros(n); surv = np.ones(n)
    kk = np.column_stack([np.full(n, kt_r[-1]), chemins[:, :54]])
    for j in range(55):
        age = 65 + j; pv += V_ ** j * surv
        m = np.exp(ax_[age] + bx_[age] * kk[:, j]) if age < 100 else gm_mu(age + 0.5, P_GM["M"])
        surv = surv * np.exp(-m)
    return pv
rng = np.random.default_rng(3)
sim55 = lc_projetter(ax_, bx_, kt_r, 55, 2000, delta, sigma, rng)
pv_sans = cout_rente(sim55)
print("sans incertitude de paramètre : moyenne", round(pv_sans.mean(), 2), " quantile 99,5 %",
      ↪ round(np.quantile(pv_sans, 0.995), 2))
```

```
sans incertitude de paramètre : moyenne 11.7  quantile 99,5 % 11.99
```

Étape 2 — avec incertitude de paramètre. Tirez la dérive de chaque trajectoire dans $\mathcal{N}(\hat{\delta}, \sigma/\sqrt{T-1})$.

```
sd_d = sigma / np.sqrt(len(kt_r) - 1)
paths = kt_r[-1] + np.cumsum(rng.normal(delta, sd_d, 2000)[: , None] + rng.normal(0, sigma, (2000,
      ↪ 55)), axis=1)
pv_avec = cout_rente(paths)
print("avec incertitude de paramètre : moyenne", round(pv_avec.mean(), 2), " quantile 99,5 %",
      ↪ round(np.quantile(pv_avec, 0.995), 2), " écart-type relatif", round(100 * pv_avec.std() /
      ↪ pv_avec.mean(), 2), "%")
```

```
avec incertitude de paramètre : moyenne 11.71  quantile 99,5 % 12.03  écart-type relatif 1.13 %
```

Étape 3 — le seuil de diversification. Un rentier seul a un coefficient de variation de son coût (loi des durées de vie) d'environ 45 %. À partir de combien de rentiers le risque de tendance domine-t-il ? Utilisez $N^* = (cv_{\text{indiv}}/cv_{\text{tendance}})^2$.

```
cv_t = pv_avec.std() / pv_avec.mean()
print("N* ≈", round((0.45 / cv_t) ** 2, -2))
```

```
N* ≈ 1600.0
```

À conclure. L'incertitude de paramètre élargit un peu la distribution, mais **ne change pas l'ordre de grandeur** du risque de tendance (de l'ordre de 1,13 %), très inférieur à un choc forfaitaire de 20 % sur les taux de décès (qui augmente le coût de la rente de 9,9 %). Au-delà de 1 600 rentiers environ, le risque restant est celui de la tendance : **il ne se mutualise pas**.

Application 5.8 — Validation hors période : plusieurs dates de coupure (section 5.3.4)

Objectif. Répéter l'épreuve du livre pour plusieurs dates de coupure, et juger si la projection de Lee-Carter bat de façon régulière la table figée.

Étape 1 — la boucle. Pour chaque date de coupure $c \in \{2004, 2009, 2014\}$, ajustez le modèle sur les années jusqu'à c , projetez la médiane jusqu'en 2019 et comparez l'espérance de vie à 65 ans prévue à celle de la table **observée** de 2019 ; comparez avec la table de l'année c figée.

```
e65_obs = T["F"].e[65]
e65_de = lambda a_, b_, k: table_vie(prolonge(q_depuis_m(np.exp(a_ + b_ * k)), P_GM["F"])).e[65]
lignes = []
for c in (2004, 2009, 2014):
    cols = [t for t in ANNEES if t <= c]
    Mc = M["F"].loc[:, cols]
    a_, b_, k_, _ = lc_ajuste(Mc)
    k_ = lc_recale(a_, b_, k_, D["F"].loc[:, cols].to_numpy(), E["F"].loc[:, cols].to_numpy())
    dl, sg = derive_sigma(k_)
    h = 2019 - c
    prev = e65_de(a_, b_, k_[-1] + h * dl); fige = e65_de(a_, b_, k_[-1])
    lignes.append((c, h, round(prev, 2), round(fige, 2), round(abs(prev - e65_obs), 2), round(abs(fige - e65_obs), 2)))
print(pd.DataFrame(lignes, columns=["coupure", "horizon", "e65 projeté", "e65 figé", "erreur projet.", "erreur figé", "projet.", "erreur figé"]).to_string(index=False))
```

coupure	horizon	e65 projeté	e65 figé	erreur projet.	erreur figé
2004	15	14.51	13.63	0.06	0.82
2009	10	14.33	13.79	0.12	0.66
2014	5	14.36	14.09	0.09	0.36

Étape 2 — conclure. Calculez le rapport moyen des erreurs. Une seule date de coupure ne suffit pas à conclure : pourquoi ?

À conclure. Pour les trois dates, la projection a une erreur plus petite que la table figée (en moyenne 7,7 fois plus petite), et l'avantage **croît avec l'horizon** : l'erreur de la table figée augmente avec l'horizon alors que celle de la projection reste faible. Mais les trois ajustements partagent les mêmes données de 2019 pour juger (et l'erreur de projection est elle-même incertaine, de l'ordre de la précision de l'espérance de vie observée) : la **robustesse** d'une conclusion se juge sur plusieurs dates.

Exercices

Exercice 5.1 ★ — Du taux à la probabilité (section 5.1.1)

Dans une cellule d'âge 70 ans, on observe 1 850 décès pour 63 000 années-personnes. (1) Calculez le taux central m_{70} et la probabilité de décès q_{70} avec les deux conventions (force constante, décès uniformes). (2) Donnez l'erreur relative de $\hat{m}$ et un intervalle approximatif à 95 % pour m . (3) À partir de quelle valeur de m les deux conventions diffèrent-elles de plus de 1 % (en valeur relative) ? À quel âge cela correspond-il dans la table des femmes de 2019 ?

Exercice 5.2 ★ — Une table à la main (section 5.1.2)

Soient $q_{60} = 0,010$, $q_{61} = 0,012$, $q_{62} = 0,014$, $q_{63} = 0,017$. À partir de $\ell_{60} = 100\,000$, calculez $\ell_{61}, \dots, \ell_{64}$, les décès d_x , la probabilité de survie à 4 ans ${}_4p_{60}$ et la probabilité de décéder **entre 62 et 64 ans** (c'est-à-dire ${}_{2|2}q_{60}$).

Exercice 5.3 ★★ — Gompertz et doublement (section 5.1.4)

Avec la loi $\mu(x) = B e^{c x}$ (on néglige A), $c = 0,10$ et $\mu(60) = 0,012$. (1) Calculez $\mu(70)$ et $\mu(80)$. (2) Montrez que la mortalité double tous les $\ln 2/c$ ans et donnez ce nombre. (3) Calculez la probabilité de survivre de 60 à 70 ans, $\exp(-\int_{60}^{70} \mu)$, en forme close puis numériquement.

Exercice 5.4 ★★ — Combien de décès pour un A/E précis ? (section 5.1.5)

Un portefeuille observe 120 décès pour 160 attendus. (1) Donnez le rapport réel/attendu et un intervalle approché à 95 % par la loi normale. (2) Calculez l'intervalle exact de Poisson avec `ae_ic`. (3) Combien de décès faudrait-il, à rapport égal, pour que la demi-largeur de l'intervalle soit de 5 % du rapport ? (4) Que concluez-vous sur la possibilité de produire des rapports par tranche d'âge et de capital pour ce portefeuille ?

Exercice 5.5 ★ — Prime d'une temporaire de 3 ans (section 5.2.1)

Refaites à la main le calcul du livre avec $q_{60} = 0,012$, $q_{61} = 0,013$, $q_{62} = 0,015$, $i = 3\%$ et un capital de 50 000 €. Donnez la prime unique pure puis la prime annuelle nivelée payée d'avance.

Exercice 5.6 ★★ — Capital et rente avec une mortalité constante (section 5.2.2)

On suppose $q_x = q$ constant pour tous les âges. (1) Montrez que $\ddot{a} = \frac{1}{1 - v(1 - q)}$ et $A = \frac{vq}{1 - v(1 - q)}$. (2) Vérifiez $A = 1 - d\ddot{a}$. (3) Application numérique : $q = 0,02$ et $i = 2\%$.

Exercice 5.7 ★★ — Sensibilité de la rente au taux technique (section 5.2.5)

- (1) Calculez le coût d'une rente viagère de 1 000 € par an à 65 ans (femmes, table 2019) pour les taux techniques 0 %, 1 %, 2 %, 3 %, 4 %. (2) Quel est le taux d'intérêt pour lequel la rente coûte exactement la durée moyenne restante de vie actualisée à 0 %, et que vaut cette durée ? (3) De combien baisse le coût quand le taux passe de 1 % à 2 % ? Comparez au changement de table de 2019 vers 1980.

Exercice 5.8 ★★ — Provision après un an, deux méthodes (section 5.2.4)

Temporaire de 3 ans, 60 ans, $q = (0,013; 0,014; 0,015)$, $i = 2\%$, capital 100 000 € et prime nivelée du livre (1 370,4 €). Calculez ${}_1V$ par la méthode prospective puis par la méthode rétrospective, et vérifiez la récurrence $({}_0V + P)(1 + i) = q_{60}C + p_{60}{}_1V$.

Exercice 5.9 ★★★ — Identifiabilité et contraintes de Lee-Carter (section 5.3.1)

- (1) Montrez que les deux transformations $(a_x, b_x, k_t) \mapsto (a_x + c b_x, b_x, k_t - c)$ et $(a_x, b_x, k_t) \mapsto (a_x, \lambda b_x, k_t/\lambda)$ laissent inchangés tous les taux. (2) Les contraintes $\sum_t k_t = 0$ et $\sum_x b_x = 1$ suffisent-elles à fixer une solution unique ? (3) Numériquement : prenez le k_t estimé (femmes) du jeu, appliquez la transformation avec $c = 5$ puis $\lambda = 2$ et vérifiez que les taux ajustés ne changent pas ; que valent alors $\sum b_x$ et $\sum k_t$?

Exercice 5.10 ★★ — Une SVD de rang 1 sur une petite matrice (section 5.3.2)

Soit la matrice des $\ln m$ centrés par ligne, pour 3 âges et 4 années : $Z = \begin{pmatrix} 0,30 & 0,10 & -0,10 & -0,30 \\ 0,15 & 0,05 & -0,05 & -0,15 \\ 0,33 & 0,08 & -0,12 & -0,29 \end{pmatrix}$.

(1) Calculez la SVD avec numpy, donnez la part de variation de la première composante. (2) Déduisez b_x (somme 1) et k_t et vérifiez que $b_x k_t$ reproduit Z à quelques centièmes. (3) Pourquoi cette matrice se factorise-t-elle presque parfaitement, contrairement aux données du chapitre ?

Exercice 5.11 ★★ — Incertitude sur la dérive (section 5.3.3)

On estime $\hat{\delta} = (k_T - k_1)/(T - 1)$ avec $T = 40$ années et un écart-type d'accroissements $\sigma = 1,57$. (1) Quel est l'écart-type de $\hat{\delta}$ si les k_t étaient observés sans erreur ? (2) Donnez un intervalle à 95 % pour δ autour de $-1,29$. (3) Combien d'années faudrait-il pour réduire l'écart-type de $\hat{\delta}$ à 0,10 ? Qu'en pensez-vous ?

Exercice 5.12 ★★★ — Diversification et tendance (section 5.3.4)

Un rentier isolé a un coefficient de variation $cv_1 = 0,45$ de son coût actualisé ; le risque de tendance a un coefficient de variation $cv_t = 0,010$ pour tout portefeuille. (1) Écrivez le coefficient de variation du coût **moyen** d'un portefeuille de N rentiers, en supposant les deux risques indépendants et en additionnant les variances. (2) Calculez-le pour $N = 100, 1\,000, 10\,000, 100\,000$. (3) À partir de quel N le risque de tendance fournit-il plus de la moitié de la variance totale ? (4) Quelle décision de gestion cela inspire-t-il ?

Corrigés**Corrigé 5.1**

```
from scipy.optimize import brentq
D1, E1 = 1850, 63000
m1 = D1 / E1
print("m70 =", round(m1, 5), " q (force constante) =", round(1 - np.exp(-m1), 5), " q (uniforme) =",
      round(m1 / (1 + m1 / 2), 5))
print("erreur relative =", round(100 / np.sqrt(D1), 2), "% IC 95 % pour m :", np.round([m1 * (1 -
      1.96 / np.sqrt(D1)), m1 * (1 + 1.96 / np.sqrt(D1))], 5))
ecart = lambda m: (m / (1 + m / 2)) / (1 - np.exp(-m)) - 1
m_crit = brentq(lambda m: ecart(m) - 0.01, 0.01, 2.0)
age_crit = int(np.argmax(M["F"][2019].to_numpy() > m_crit))
print("écart relatif à m = 0,1 :", round(100 * ecart(0.1), 2), "% à m = 0,2 :", round(100 *
      ecart(0.2), 2), "% seuil de 1 % : m =", round(m_crit, 3), " atteint à", age_crit, "ans")
```

```
m70 = 0.02937    q (force constante) = 0.02894    q (uniforme) = 0.02894
erreur relative = 2.32 %    IC 95 % pour m : [0.02803 0.0307 ]
écart relatif à m = 0,1 : 0.08 % à m = 0,2 : 0.3 % seuil de 1 % : m = 0.378 atteint à 93 ans
```

(1) $m_{70} = 0,02937$; $q^{\text{const}} = 0,02894$ et $q^{\text{unif}} = 0,02894$, quasi identiques. (2) L'erreur relative est $1/\sqrt{1850} \approx 2,32\%$, soit un intervalle à 95 % de $[0,02803 ; 0,0307]$. (3) L'écart relatif entre les deux conventions est de 0,08 % à $m = 0,1$ et de 0,3 % à $m = 0,2$; il atteint 1 % pour $m \approx 0,38$, valeur que la table des femmes de 2019 franchit à 93 ans. Les deux conventions ne se distinguent donc qu'aux âges très élevés.

Corrigé 5.2

$\ell_{61} = 100\,000 \times 0,990 = 99\,000$; $\ell_{62} = 99\,000 \times 0,988 = 97\,812$; $\ell_{63} = 97\,812 \times 0,986 = 96\,442,6$; $\ell_{64} = 96\,442,6 \times 0,983 = 94\,803,1$. Décès : $d_{60} = 1\,000$, $d_{61} = 1\,188$, $d_{62} = 1\,369,4$, $d_{63} = 1\,639,5$.
 ${}_4p_{60} = 0,99 \times 0,988 \times 0,986 \times 0,983 = 0,94803$. ${}_{2|2}q_{60} = ({}_2p_{60})(1 - {}_2p_{62}) = \frac{\ell_{62} - \ell_{64}}{\ell_{60}} = 0,97812 - 0,94803 = 0,03009$: il y a environ 3 % de chances de décéder entre 62 et 64 ans pour une personne de 60 ans.

```
q = np.array([0.010, 0.012, 0.014, 0.017]); l = 100000 * np.concatenate([[1], np.cumprod(1 - q)])
print(np.round(l, 1), np.round(l[:-1] * q, 1), round(l[4] / l[0], 5), round((l[2] - l[4]) / l[0], 5))
assert abs(l[3] - 96442.6) < 0.1 and abs(l[4] - 94803.1) < 0.1 and abs((l[2] - l[4]) / l[0] - 0.03009)
↳ < 1e-5
```

```
[100000. 99000. 97812. 96442.6 94803.1] [1000. 1188. 1369.4 1639.5] 0.94803 0.03009
```

Corrigé 5.3

(1) $\mu(70) = 0,012e^1 = 0,03262$ et $\mu(80) = 0,012e^2 = 0,08867$. (2) $\mu(x+h)/\mu(x) = e^{ch} = 2 \iff h = \ln 2/c = 6,93$ ans. (3) $\int_{60}^{70} \mu = \frac{\mu(60)}{c}(e^{c \cdot 10} - 1) = 0,12(e - 1) \approx 0,2062$, donc la survie est $e^{-0,2062} = 0,8137$.

```
c, mu60 = 0.10, 0.012
from scipy.integrate import quad
I_ = quad(lambda x: mu60 * np.exp(c * (x - 60)), 60, 70)[0]
print(round(mu60 * np.exp(c * 10), 5), round(mu60 * np.exp(c * 20), 5), round(np.log(2) / c, 2),
↳ round(I_, 5), round(np.exp(-I_), 4))
```

```
0.03262 0.08867 6.93 0.20619 0.8137
```

Corrigé 5.4

```
ae0, b0, h0 = ae_ic(120, 160)
print("A/E =", round(ae0, 3), " normal :", round(ae0 * (1 - 1.96 / np.sqrt(120)), 3), round(ae0 * (1 +
↳ 1.96 / np.sqrt(120)), 3), " exact :", round(b0, 3), round(h0, 3))
print("décès nécessaires pour ±5 % :", int(np.ceil((1.96 / 0.05) ** 2)))
```

```
A/E = 0.75 normal : 0.616 0.884 exact : 0.622 0.897
décès nécessaires pour ±5 % : 1537
```

(1) Le rapport est $120/160 = 0,75$; l'intervalle normal est $[0,616 ; 0,884]$. (2) L'intervalle exact est $[0,622 ; 0,897]$, un peu **dissymétrique** et plus large du côté supérieur (la loi de Poisson est asymétrique pour un petit nombre de décès). (3) $1,96/\sqrt{D} \leq 0,05 \iff D \geq 1537$. (4) Avec 120 décès, **aucune analyse par sous-groupe n'est possible** : en divisant en trois tranches d'âge, chacune aurait environ 40 décès, soit ± 31 % de précision. Il faut grouper, ou renoncer à l'analyse fine.

Corrigé 5.5

Avec $i = 3\%$, $v = 0,970874$; $v^2 = 0,942596$; $v^3 = 0,915142$. $p_{60} = 0,988$, $p_{61} = 0,987$. Décès : $q_{60} = 0,012$; $p_{60}q_{61} = 0,988 \times 0,013 = 0,012844$; $p_{60}p_{61}q_{62} = 0,988 \times 0,987 \times 0,015 = 0,014627$. Donc $A^1 = 0,970874 \times 0,012 + 0,942596 \times 0,012844 + 0,915142 \times 0,014627 = 0,011650 + 0,012107 + 0,013386 = 0,037143$; **prime unique** = $50\,000 \times 0,037143 = 1\,857,2$ €. Rente temporaire : $\ddot{a} = 1 + 0,970874 \times 0,988 + 0,942596 \times 0,988 \times 0,987 = 1 + 0,959223 + 0,919178 = 2,878401$; **prime annuelle** = $1\,857,2/2,8784 = 645,2$ €.

```

qq = np.array([0.012, 0.013, 0.015]); v3 = 1 / 1.03
pp = np.cumprod(np.r_[1, 1 - qq[:-1]])
A1 = sum(v3 ** (k + 1) * pp[k] * qq[k] for k in range(3)); a1 = sum(v3 ** k * pp[k] for k in range(3))
print(round(A1, 6), round(50000 * A1, 1), round(a1, 6), round(50000 * A1 / a1, 1))
assert abs(50000 * A1 - 1857.2) < 0.1 and abs(a1 - 2.878401) < 1e-5 and abs(50000 * A1 / a1 - 645.2) <
↪ 0.1

```

```
0.037143 1857.2 2.878401 645.2
```

Corrigé 5.6

- (1) Avec q constant, ${}_kp = (1 - q)^k$ et $\ddot{a} = \sum_{k \geq 0} (v(1 - q))^k = \frac{1}{1 - v(1 - q)}$; $A = \sum_{k \geq 0} v^{k+1} (1 - q)^k q = vq \ddot{a}$. (2) $1 - d\ddot{a} = \frac{1 - v(1 - q) - (1 - v)}{1 - v(1 - q)} = \frac{vq}{1 - v(1 - q)} = A$ (car $d = 1 - v$). (3) Avec $q = 0,02$, $v = 1/1,02$: $1 - v(1 - q) = 1 - 0,960784 = 0,039216$, donc $\ddot{a} = 25,5$ et $A = 0,5$.

```

q0, v0 = 0.02, 1 / 1.02
a0 = 1 / (1 - v0 * (1 - q0)); A0 = v0 * q0 * a0
print(round(a0, 4), round(A0, 4), round(1 - (1 - v0) * a0, 4))

```

```
25.5 0.5 0.5
```

Corrigé 5.7

```

for ii in (0.0, 0.01, 0.02, 0.03, 0.04):
    print(ii, round(1000 * valeurs(Q["F"], ii)[1][65]))
a0_ = valeurs(Q["F"], 0.0)[1][65]
q80 = prolonge(q_depuis_m(M["F"][1980].to_numpy(), P_GM["F"]))
print("durée moyenne (ä à 0 %) :", round(a0_, 2), " 1980 vs 2019 à 2 % :", round(valeurs(q80,
↪ 0.02)[1][65], 3), round(valeurs(Q["F"], 0.02)[1][65], 3))

```

```

0.0 14951
0.01 13720
0.02 12653
0.03 11723
0.04 10907
durée moyenne (ä à 0 %) : 14.95   1980 vs 2019 à 2 % : 10.948 12.653

```

- (1) Les coûts sont de 14951, 13720, 12653, 11723 et 10907 € pour 0, 1, 2, 3 et 4 %. (2) À 0 %, la rente coûte simplement le **nombre moyen de versements** (le premier étant payé d'avance) : 14,95, soit $e_{65} + \frac{1}{2}$ environ ; c'est la durée de vie résiduelle moyenne, à une demi-année près. (3) De 1 % à 2 %, le coût baisse de 7,8 % ; passer de la table de 2019 à celle de 1980 à 2 % ferait baisser le coût de 13,5 % : **une table ancienne d'une quarantaine d'années pèse davantage qu'un point de taux technique.**

Corrigé 5.8

Prospective : ${}_1V = C A_{61:\bar{2}}^1 - P \ddot{a}_{61:\bar{2}}$ avec $A_{61:\bar{2}}^1 = vq_{61} + v^2 p_{61} q_{62} = 0,013725 + 0,014215 = 0,027941$ et $\ddot{a}_{61:\bar{2}} = 1 + v p_{61} = 1,966667$, donc ${}_1V = 2794,1 - 1370,4 \times 1,966667 \approx 99$ €. Rétrospective : ${}_1V = \frac{P(1+i) - C q_{60}}{p_{60}}$ (une seule année écoulée). Récurrence de Thiele : $({}_0V + P)(1+i) = q_{60}C + p_{60}{}_1V$ avec ${}_0V = 0$: c'est exactement la formule rétrospective.

```
qq3 = [0.013, 0.014, 0.015]; C3, P3 = 100000, 1370.4
Q3 = np.zeros(65); Q3[60:63] = qq3; Q3[63] = 1.0
V1_pro = reserve_prospective(Q3, I, 60, 3, C3, P3, 1)
V1_retro = (P3 * (1 + I) - C3 * qq3[0]) / (1 - qq3[0])
print(round(V1_pro, 2), round(V1_retro, 2), round((0 + P3) * (1 + I) - (qq3[0] * C3 + (1 - qq3[0]) *
↪ V1_pro), 3))
```

```
99.0 99.1 0.097
```

Les deux méthodes donnent ${}_1V = 99,0$ € (méthode prospective) et $99,1$ € (méthode rétrospective) ; l'écart de quelques centimes tient à l'arrondi de la prime à $1\,370,4$ € (la prime exacte est de l'ordre de $1\,370,36$ €).

Corrigé 5.9

- (1) Pour la première transformation : $a_x + c b_x + b_x(k_t - c) = a_x + b_x k_t$; pour la seconde : $a_x + (\lambda b_x)(k_t/\lambda) = a_x + b_x k_t$. (2) La première liberté est supprimée par $\sum k_t = 0$ (on ne peut plus déplacer une constante), la seconde par $\sum b_x = 1$ (on ne peut plus échanger d'échelle) : **la solution est unique à l'exception d'un signe commun** ($b \rightarrow -b, k \rightarrow -k$, même taux), que l'on fixe en imposant que b_x soit positif. (3) Vérification :

```
ax_f, bx_f, kt_f, _ = lc_ajuste(M["F"])
base = np.exp(ax_f[:, None] + bx_f[:, None] * kt_f[None, :])
c_, lam = 5.0, 2.0
a2, b2, k2 = ax_f + c_ * bx_f, bx_f, kt_f - c_
a3, b3, k3 = a2, lam * b2, k2 / lam
print(np.abs(np.exp(a3[:, None] + b3[:, None] * k3[None, :]) / base - 1).max(), round(b3.sum(), 2),
↪ round(k3.sum(), 1))
```

```
1.887379141862766e-15 2.0 -100.0
```

Les taux sont identiques (écart relatif maximal de l'ordre de 10^{-15}), mais $\sum b_x = 2,0$ et $\sum k_t = -100,0$: on a quitté la normalisation. C'est pourquoi on **compare toujours des estimations à des valeurs normalisées de la même façon**.

Corrigé 5.10

```
Z = np.array([[0.30, 0.10, -0.10, -0.30], [0.15, 0.05, -0.05, -0.15], [0.33, 0.08, -0.12, -0.29]])
U, S, Vt = np.linalg.svd(Z, full_matrices=False)
b = U[:, 0] / U[:, 0].sum(); k = S[0] * Vt[0] * U[:, 0].sum()
print("part de la 1re composante :", round(S[0] ** 2 / (S ** 2).sum(), 4), " b =", b.round(3), " k =",
↪ k.round(3))
print("écart maximal |Z - b k| :", np.abs(Z - np.outer(b, k)).max().round(3))
```

```
part de la 1re composante : 0.9981 b = [0.395 0.197 0.408] k = [ 0.783 0.227 -0.272 -0.737]
écart maximal |Z - b k| : 0.013
```

- (1) La première composante explique 99,81 % de la variation. (2) On obtient $b = (0,395; 0,197; 0,408)$ et $k = (0,783; 0,227; -0,272; -0,737)$, et le produit $b_x k_t$ reproduit

Z à 0,013 près. (3) Cette matrice a été **construite** (presque exactement de rang 1, à de petits écarts près) : il n'y a pas de bruit de Poisson. Les données du chapitre, elles, contiennent des décès aléatoires : la première composante n'y explique que 64 %.

Corrigé 5.11

- (1) Les accroissements sont indépendants de variance σ^2 , donc $\text{Var}(\hat{\delta}) = \sigma^2(T-1)/(T-1)^2 = \sigma^2/(T-1)$ et l'écart-type est $\sigma/\sqrt{T-1} = 0,25$. (2) Intervalle à 95 % : $-1,29 \pm 1,96 \times 0,25$, soit $[-1,78; -0,8]$: la dérive est connue à environ ± 38 %. (3) Il faudrait $T-1 = \sigma^2/0,10^2 \approx 246$ années, soit près de deux siècles et demi de données annuelles : **on ne peut pas réduire cette incertitude par la collecte**, seulement la reconnaître et la propager dans les projections.

```
sg, T_ = 1.57, 40
sd = sg / np.sqrt(T_ - 1)
print(round(sd, 3), np.round([-1.29 - 1.96 * sd, -1.29 + 1.96 * sd], 2), round(100 * 1.96 * sd /
↳ 1.29), "%", round(sg ** 2 / 0.10 ** 2))
```

```
0.251 [-1.78 -0.8 ] 38 % 246
```

Corrigé 5.12

- (1) Variances additives : $cv^2(N) = cv_1^2/N + cv_t^2$. (2) Le code donne les valeurs ci-dessous. (3) Le risque de tendance fournit plus de la moitié de la variance totale quand $cv_t^2 > cv_1^2/N$, c'est-à-dire $N > (cv_1/cv_t)^2 = 2025$. (4) Passé quelques milliers de rentiers, **grossir ne réduit plus le risque** : il faut le transférer (réassurance de longévité, chapitre 6) ou le couvrir par une marge ou un capital, et non attendre que la mutualisation le fasse.

```
cv1, cvt = 0.45, 0.010
for N in (100, 1000, 10000, 100000):
    print(N, round(100 * np.sqrt(cv1 ** 2 / N + cvt ** 2), 2), "% part de la tendance :", round(100
↳ * cvt ** 2 / (cv1 ** 2 / N + cvt ** 2)), "%")
print("N* =", round((cv1 / cvt) ** 2))
```

```
100 4.61 % part de la tendance : 5 %
1000 1.74 % part de la tendance : 33 %
10000 1.1 % part de la tendance : 83 %
100000 1.01 % part de la tendance : 98 %
N* = 2025
```

Chapitre 6 : ⊕ Réassurance — exercices et applications

Orientation. Ce chapitre du cahier accompagne le chapitre 6 du livre (réassurance et tarification). Les **applications** reprennent, en petites étapes commentées, les études du livre sur les données `sinistres_gros.csv` et `cat_annuel.csv` (simulées : nous connaissons la vérité). Les **exercices** (★ direct, ★★ demande un raisonnement, ★★★ petite étude) sont corrigés à la fin, avec le calcul à la main quand il est faisable. Une seule cellule charge les bibliothèques et les données ; chaque bloc suivant s'appuie dessus.

```
import sys
import numpy as np, pandas as pd
from scipy import stats, integrate, optimize
sys.path.insert(0, "build")
import outils_ch06 as O          # tranche(), burning_cost(), prix_gpd(), simuler()...
sg, cat = O.charger()
x = sg["montant"].values         # 2 898 sinistres, 2010-2024
c = cat["perte_cat"].values      # 40 années de pertes de catastrophe
```

Applications

Application 6.1 — Répartir huit sinistres entre trois traités (sections 6.1.2 et 6.1.3)

Objectif. Reproduire le tableau du livre, puis chercher quelle plénitude cède autant que l'excédent de sinistre.

Étape 1 — Les trois partages. On reprend huit sinistres (k€) et leurs capitaux assurés.

```
X = np.array([30, 45, 80, 120, 250, 600, 900, 2000.]) # sinistres
V = np.array([200, 300, 500, 800, 1000, 2500, 3000, 5000.]) # capitaux assurés
t = pd.DataFrame({"sinistre": X, "capital": V})
t["quote_part_30"] = 0.3 * X                               # 30 % de chaque sinistre
t["plenitude_1000"] = np.maximum(0, 1 - 1000 / V) * X      # cession proportionnelle au dépassement
↳ de 1 000
t["xl_500_xs_500"] = O.tranche(X, 500, 500)               # tranche par risque
print(t.sum().round(1).to_string())
```

sinistre	4025.0
capital	13300.0
quote_part_30	1207.5
plenitude_1000	2560.0
xl_500_xs_500	1000.0

Étape 2 — Une plénitude qui cède autant que l'excédent de sinistre. On cherche R tel que la plénitude cède 1 000.

```
cede = lambda R: (np.maximum(0, 1 - R / V) * X).sum()
R_eq = optimize.brentq(lambda R: cede(R) - 1000, 1000, 5000)
print(f"plénitude équivalente : {R_eq:,.0f} (cède {cede(R_eq):.0f})")
```

```
plénitude équivalente : 2,714 (cède 1000)
```

Lecture. La plénitude qui cède autant que la tranche « 500 xs 500 » est de 2 714 k€, c'est-à-dire que la cédante devrait **garder** près de 2 714 k€ par risque pour ne céder que 1 000 : la tranche, qui n'intervient qu'au-delà de 500 sur chaque sinistre, est une façon **bien plus précise** de couper la queue. À cession égale, les deux traités ne protègent pas des mêmes années : la plénitude cède aussi des parts de petits sinistres de gros risques.

Application 6.2 — Le coefficient de variation selon la rétention (section 6.1.1)

Objectif. Mesurer ce que gagne la cédante en plafonnant chaque sinistre.

Étape 1 — CV d'un sinistre plafonné. On plafonne chaque sinistre à une rétention r et l'on calcule le CV du sinistre net, puis celui de la charge annuelle $\sqrt{(1 + CV^2)/\lambda}$.

```
lam = 0.LAMBDA_2025
lignes = []
for r in [1e5, 2.5e5, 5e5, 1e6, 2e6, np.inf]:
    xn = np.minimum(x, r)
    cv = xn.std() / xn.mean()
    lignes.append({"retention": r, "cv_sinistre": cv, "cv_annuel": np.sqrt((1 + cv ** 2) / lam)})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

retention	cv_sinistre	cv_annuel
100000.0	0.851	0.083
250000.0	1.202	0.099
500000.0	1.547	0.117
1000000.0	1.902	0.136
2000000.0	2.258	0.156
inf	2.508	0.171

Étape 2 — Taille de portefeuille équivalente. Quel portefeuille (nombre de sinistres) donnerait, sans réassurance, le CV annuel obtenu avec une rétention de 500 000 € ?

```
cv0 = x.std() / x.mean(); cv5 = np.minimum(x, 5e5).std() / np.minimum(x, 5e5).mean()
lam_equiv = lam * (1 + cv0 ** 2) / (1 + cv5 ** 2)
print(f"{lam_equiv:,.0f} sinistres par an, soit {lam_equiv / lam:.1f} fois le portefeuille")
```

```
535 sinistres par an, soit 2.1 fois le portefeuille
```

Lecture. Plafonner à 500 000 € ramène le CV d'un sinistre de 2,51 à 1,55 et celui de la charge annuelle à 11,7 % ; obtenir la même régularité sans réassurance demanderait un portefeuille **2,1 fois plus gros**. Aller plus bas (100 000 €) abaisse encore le CV, mais au prix d'une prime cédée bien plus élevée : c'est le même arbitrage qu'en section 6.3.

Application 6.3 — Le *burning cost* d'une tranche (section 6.2.1)

Objectif. Calculer à la main (avec pandas) le *burning cost* de la tranche « 1 M€ xs 1 M€ », puis mesurer l'effet de chaque correction.

Étape 1 — Charge annuelle de la tranche. On applique la tranche à chaque sinistre, puis on somme par année.

```
a, L = le6, le6
par_an = sg.assign(charge=0.tranche(sg["montant"], a, L)).groupby("annee_survenance")["charge"].sum()
t = pd.DataFrame({"brut": par_an, "facteur": 0.facteur_2025(par_an.index.values)})
t["ajuste"] = t["brut"] * t["facteur"]
print((t / le6).round(2).head(6).to_string())
```

annee_survenance	brut	facteur	ajuste
2010	1.56	0.0	2.43
2011	3.18	0.0	4.81
2012	2.25	0.0	3.31
2013	2.10	0.0	2.99
2014	3.00	0.0	4.15
2015	0.84	0.0	1.13

Étape 2 — Moyennes et comparaison à la vérité.

```
vrai = 0.prix_vrai(a, L)
for nom in ["brut", "ajuste"]:
    print(f"{nom:7s} {t[nom].mean()/le6:.2f} M€ ({t[nom].mean()/vrai - 1:+.0%} par rapport à la
    ↪ vérité {vrai/le6:.2f} M€)")
```

```
brut    1.95 M€ (-26% par rapport à la vérité 2.63 M€)
ajuste  2.50 M€ (-5% par rapport à la vérité 2.63 M€)
```

Étape 3 — Une indexation à 4 % par habitude. On recommence en gonflant chaque sinistre de $1,04^{2025-t}$ avant la tranche.

```
infl = np.array([0.tranche(sg.loc[sg["annee_survenance"] == y, "montant"].values * 1.04 ** (2025 -
    ↪ y), a, L).sum() for y in t.index])
print(f"avec inflation 4 % : {(infl * t['facteur']).mean()/le6:.2f} M€")
```

```
avec inflation 4 % : 4.55 M€
```

Lecture. Le brut donne 1,95 M€, l'ajusté 2,50 M€, la vérité 2,63 M€ : l'ajustement d'exposition corrige un biais de 26 %. L'indexation inutile à 4 % donne 4,55 M€, 73 % de trop : la tranche amplifie les hypothèses d'inflation.

Application 6.4 — La courbe d'exposition (section 6.2.2)

Objectif. Estimer $G(d) = E[\min(X, d)]/E[X]$ et en déduire le prix de tranches.

Étape 1 — La courbe.

```
G = lambda d: 0.lev_empirique(x, d) / x.mean()
for d in [1e5, 2.5e5, 5e5, 1e6, 2e6, 3e6]:
    print(f"G({d/le6:g} M€) = {G(d):.3f}")
```

```
G(0.1 M€) = 0.378
G(0.25 M€) = 0.569
G(0.5 M€) = 0.735
G(1 M€) = 0.877
G(2 M€) = 0.964
G(3 M€) = 0.993
```

Étape 2 — Prix par fréquence × sévérité. La charge de « L xs a » vaut $\lambda E[X] (G(a + L) - G(a))$.

```
for a, L in 0.COUCHES:
    prix = 0.LAMBDA_2025 * x.mean() * (G(a + L) - G(a))
    print(f"{L/1e6:g} xs {a/1e6:g} : {prix/1e6:.2f} M€ (vérité {0.prix_vrai(a, L)/1e6:.2f} M€)")
```

```
0.5 xs 0.5 : 4.11 M€ (vérité 4.47 M€)
1 xs 1 : 2.52 M€ (vérité 2.63 M€)
2 xs 2 : 1.00 M€ (vérité 1.42 M€)
```

Lecture. L'approche par la courbe donne 4,11, 2,52 et 1,00 M€ pour les trois tranches, contre 4,47, 2,63 et 1,42 M€ en vérité : les écarts sont du même ordre que ceux du *burning cost* ajusté, car les deux utilisent les mêmes sinistres. 3,6 % de la charge attendue se situe au-delà de 2 M€ de sinistre, mais ces sinistres sont **15 sur 2 898** : la précision de G aux grandes valeurs est faible.

Application 6.5 — Ajuster une GPD : stabilité selon le seuil (section 6.2.3)

Objectif. Voir comment l'indice de queue et le prix d'une tranche varient avec le seuil.

Étape 1 — Ajustements.

```
lignes = []
for u in [2e5, 3e5, 5e5, 7.5e5, 1e6]:
    xi, beta, n = 0.ajuster_gpd(sg, u)
    lam_u = 0.taux_depassement(sg, u)
    lignes.append({"seuil": u, "n": n, "xi": xi, "beta": beta, "prix_lxs1": 0.prix_gpd(1e6, 1e6, u,
    ↪ xi, beta, lam_u)})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

seuil	n	xi	beta	prix_lxs1
200000.0	366	0.318	306656.347	2329996.465
300000.0	259	0.217	397435.465	2403833.599
500000.0	177	0.416	312974.734	2208253.719
750000.0	87	0.205	539176.465	2440414.159
1000000.0	49	-0.153	973149.529	2529452.430

Étape 2 — Intervalle de confiance de $\hat{\xi}$. L'écart-type approché de l'estimateur est $(1 + \hat{\xi})/\sqrt{n}$.

```
for r in lignes:
    se = (1 + r["xi"]) / np.sqrt(r["n"])
    print(f"u = {r['seuil']}>9,.0f} xi = {r['xi']:+.2f} [{r['xi'] - 1.96*se:+.2f} ; {r['xi'] +
    ↪ 1.96*se:+.2f}]")
```

```
u = 200,000 xi = +0.32 [+0.18 ; +0.45]
u = 300,000 xi = +0.22 [+0.07 ; +0.37]
u = 500,000 xi = +0.42 [+0.21 ; +0.62]
u = 750,000 xi = +0.20 [-0.05 ; +0.46]
u = 1,000,000 xi = -0.15 [-0.39 ; +0.08]
```

Lecture. $\hat{\xi}$ varie de -0,15 à 0,42 selon le seuil, et le prix de « 1 M€ xs 1 M€ » de 2,21 à 2,53 M€ (vérité 2,63 M€). L'intervalle de $\hat{\xi}$ au seuil de 1 M€ (49 dépassements) contient **zéro** : on ne sait même pas si la queue est lourde ou légère. Aucun seuil ne donne un résultat « stable » ; il faut le dire dans le rapport.

Application 6.6 — Bootstrap : par années ou par sinistres ? (section 6.2.4)

Objectif. Comparer deux façons de rééchantillonner et comprendre laquelle sous-estime l'incertitude.

Étape 1 — Par années (celle du livre).

```
a, L = 2e6, 2e6
b_ans = 0.bootstrap_annees(sg, a, L, B=1000)
print(f"par années : [{np.percentile(b_ans, 2.5)/1e6:.2f} ; {np.percentile(b_ans, 97.5)/1e6:.2f}] M€")
```

```
par années : [0.54 ; 1.53] M€
```

Étape 2 — Par sinistres. On tire 2 898 sinistres avec remise, indépendamment de leur année, et l'on calcule la charge moyenne ajustée de la tranche en gardant la répartition des années.

```
rng = np.random.default_rng(3)
an = sg["annee_survenance"].values; w = 0.facteur_2025(an)
b_sin = []
for _ in range(1000):
    i = rng.integers(0, len(x), len(x))
    b_sin.append((0.tranche(x[i], a, L) * w[i]).sum() / 15)
print(f"par sinistres : [{np.percentile(b_sin, 2.5)/1e6:.2f} ; {np.percentile(b_sin, 97.5)/1e6:.2f}] M€")
```

```
par sinistres : [0.45 ; 1.66] M€
```

Lecture. Les deux intervalles ont des largeurs de 0,99 et 1,21 M€. Le bootstrap par sinistres fixe le nombre total de sinistres et **ignore la variabilité du nombre de sinistres d'une année à l'autre** ; il traite comme interchangeables des sinistres d'années différentes. Ses hypothèses diffèrent de celles du bootstrap par années, et son résultat aussi : un intervalle égal à 123 % de celui par années. Aucun des deux n'est « le bon » : on choisit celui dont les hypothèses correspondent à la façon dont les données sont produites (ici, des années d'exposition différentes). Dans les deux cas, la vérité (1,42 M€) tombe dans l'intervalle.

Application 6.7 — Une tranche de catastrophe (section 6.2.5)

Objectif. Estimer une tranche de catastrophe avec 40 ans de données, et mesurer ce que l'on ignore.

Étape 1 — Loi de Pareto ajustée.

```
alpha, p = 0.pareto_cat(cat)
print(f"alpha = {alpha:.2f} fréquence des années à événement = {p:.1%} ({(c > 0).sum()} années)")
```

```
alpha = 1.33 fréquence des années à événement = 42.5% (17 années)
```

Étape 2 — La tranche « 10 M€ xs 10 M€ ».

```
a, L = 1e7, 1e7
emp = 0.tranche(c, a, L).mean()
par = 0.prix_pareto(a, L, p, alpha)
print(f"empirique {emp/1e6:.2f} M€ | Pareto {par/1e6:.2f} M€ | vérité {0.prix_vrai_cat(a, L)/1e6:.2f} M€")
print(f"années touchées dans l'historique : {(c > a).sum()} sur {len(c)}")
```

```
empirique 0.26 M€ | Pareto 0.31 M€ | vérité 0.50 M€
années touchées dans l'historique : 2 sur 40
```

Lecture. Dans l'historique, la tranche n'est touchée que 2 années sur 40 ; l'espérance empirique

est de 0,26 M€ et ne repose que sur ces années-là. Le prix vrai de 0,50 M€ correspond à un ROL de 5,0 % et à une période de retour de 20 ans : **40 ans d'historique ne permettent pas de la vérifier.**

Application 6.8 — La prime de risque (section 6.2.6)

Objectif. Calculer la prime technique d'une tranche selon deux principes, et mesurer la sensibilité à leurs paramètres.

Étape 1 — La distribution de la charge annuelle de la tranche « 1 M€ xs 1 M€ ».

```
ch = 0.charge_annuelle_vraie(1e6, 1e6, N=20000, seed=5)
E, sd, var995 = ch.mean(), ch.std(), np.percentile(ch, 99.5)
K = var995 - E
print(f"E = {E/1e6:.2f}   écart-type = {sd/1e6:.2f}   VaR 99,5 % = {var995/1e6:.2f}   K = {K/1e6:.2f}
      ↪ (M€)")
```

E = 2.61 écart-type = 1.43 VaR 99,5 % = 7.04 K = 4.43 (M€)

Étape 2 — Grilles de paramètres.

```
print("Principe de l'écart-type : chargement relatif selon k")
print({k: f"{k*sd/E:.0%}" for k in [0.05, 0.10, 0.15, 0.20, 0.30]})
print("Coût du capital : chargement relatif selon i")
print({i: f"{i*K/E:.0%}" for i in [0.04, 0.06, 0.08, 0.10, 0.12]})
```

Principe de l'écart-type : chargement relatif selon k
 {0.05: '3%', 0.1: '5%', 0.15: '8%', 0.2: '11%', 0.3: '16%'}
 Coût du capital : chargement relatif selon i
 {0.04: '7%', 0.06: '10%', 0.08: '14%', 0.1: '17%', 0.12: '20%'}

Lecture. L'espérance vaut 2,61 M€, l'écart-type 1,43 M€, la perte inattendue $K = 4,43$ M€. Le chargement relatif va de 3 % à 16 % avec le principe de l'écart-type (k de 0,05 à 0,30), de 7 % à 20 % avec le coût du capital (de 4 % à 12 %) : **les paramètres comptent autant que le principe.**

Application 6.9 — Comparer des programmes (section 6.3.2)

Objectif. Refaire la comparaison du livre avec vos propres paramètres.

Étape 1 — Le résultat brut.

```
simu = 0.simuler(N=10000, seed=11)
brute = simu["brute"]
prime = brute.mean() / 0.70
resultat = 0.75 * prime - brute
q = lambda r: np.percentile(r, 0.5)
K0 = resultat.mean() - q(resultat)
print(f"K sans réassurance : {K0/1e6:.2f} M€   ruine à 8 M€ : {(resultat < -8e6).mean():.2%}")
```

K sans réassurance : 13.94 M€ ruine à 8 M€ : 3.03%

Étape 2 — Une fonction qui évalue un programme.

```
def bilan(recup, prime_cedee):
    r = resultat + recup - prime_cedee
    K = r.mean() - q(r)
    return {"cout": prime_cedee - recup.mean(), "K": K, "economie": K0 - K, "ruine": (r < -8e6).mean()}
```

Étape 3 — Quote-part : quelle part céder ? Avec une commission de 25 %, on fait varier α .

```
for alpha in [0.1, 0.2, 0.3, 0.4]:
    b = bilan(alpha * brute + 0.25 * alpha * prime, alpha * prime)
    print(f"alpha {alpha:.0%} : coût {b['cout']/1e6:.2f} M€ K {b['K']/1e6:.1f} M€ coût par €
    ↳ économisé {b['cout']/b['economie']:.3f}")
```

```
alpha 10% : coût 0.21 M€ K 12.5 M€ coût par € économisé 0.148
alpha 20% : coût 0.41 M€ K 11.2 M€ coût par € économisé 0.148
alpha 30% : coût 0.62 M€ K 9.8 M€ coût par € économisé 0.148
alpha 40% : coût 0.82 M€ K 8.4 M€ coût par € économisé 0.148
```

Étape 4 — Excédent de sinistre : quelle priorité ? Même chargement de 35 % pour une tranche de portée 1 M€.

```
for a in [2.5e5, 5e5, 1e6, 2e6]:
    rec = 0.agreger(simu, 0.tranche(simu["sinistres"], a, 1e6))
    b = bilan(rec, 1.35 * rec.mean())
    print(f"priorité {a/1e6:g} M€ : coût {b['cout']/1e6:.2f} économie de capital
    ↳ {b['economie']/1e6:.2f} par € {b['cout']/b['economie']:.2f}")
```

```
priorité 0.25 M€ : coût 3.43 économie de capital 5.49 par € 0.63
priorité 0.5 M€ : coût 1.99 économie de capital 4.53 par € 0.44
priorité 1 M€ : coût 0.88 économie de capital 2.96 par € 0.30
priorité 2 M€ : coût 0.29 économie de capital 1.38 par € 0.21
```

Lecture. La quote-part coûte 0,41 M€ pour 20 % et 0,82 M€ pour 40 % : son coût et son effet sur K sont **proportionnels** à α , donc le coût par euro reste constant (0,148). Pour l'excédent de sinistre, la priorité change tout : à 250 000 € de priorité la cédante paie une prime de 13,24 M€ pour récupérer 34 % de sa sinistralité moyenne ; à 2 M€ de priorité elle paie 1,10 M€ pour une protection qui n'intervient presque jamais.

Application 6.10 — Contrepartie et épuisement (section 6.3.4)

Objectif. Faire varier la probabilité de défaut du réassureur, puis le nombre de réintégrations.

Étape 1 — Défaut du réassureur sur un stop-loss.

```
rec = 0.tranche(brute, 38e6, 10e6); cout_sl = 1.4 * rec.mean()
rng = np.random.default_rng(5)
rus = {}
for pd_def in [0.0, 0.005, 0.02, 0.05]:
    d = rng.random(len(brute)) < pd_def
    r = resultat + np.where(d, 0.5 * rec, rec) - cout_sl
    rus[pd_def] = (r < -8e6).mean()
    print(f"défaut {pd_def:.1%} (indépendant) : ruine {rus[pd_def]:.3%}")
```

```
défaut 0.0% (indépendant) : ruine 0.020%
défaut 0.5% (indépendant) : ruine 0.020%
défaut 2.0% (indépendant) : ruine 0.040%
défaut 5.0% (indépendant) : ruine 0.140%
```

Étape 2 — Plafond annuel d'une tranche « 2 M€ xs 2 M€ ». La charge annuelle ch_2 est celle de la vérité ; on fait varier le nombre de réintégrations k (plafond $(1 + k) \times 2$ M€).

```
ch2 = 0.charge_annuelle_vraie(2e6, 2e6)
for k in [0, 1, 2, 3]:
    plafond = (1 + k) * 2e6
    print(f"{k} réintégration(s) : plafond atteint dans {(ch2 >= plafond).mean():.1%} des années ;
    ↳ couvre {np.minimum(ch2, plafond).mean() / ch2.mean():.1%} de l'espérance")
```

0 réintégration(s) : plafond atteint dans 36.6% des années ; couvre 73.7% de l'espérance
 1 réintégration(s) : plafond atteint dans 7.7% des années ; couvre 95.3% de l'espérance
 2 réintégration(s) : plafond atteint dans 1.1% des années ; couvre 99.4% de l'espérance
 3 réintégration(s) : plafond atteint dans 0.1% des années ; couvre 99.9% de l'espérance

Lecture. Un défaut **indépendant** à 5 % par an fait passer la probabilité de ruine de 0,02 % à 0,14 % : peu en points de pourcentage, mais bien plus en proportion (ces probabilités reposent sur quelques années simulées seulement : lisez-les comme des ordres de grandeur). Le plafond annuel est plus décisif : sans réintégration, la garantie n'**honore que 74 % de l'espérance**, avec une réintégration 95 %, avec trois 100 %.

Exercices

Exercice 6.1 ★ — Quote-part avec commission (section 6.1.2)

Une cédante encaisse 10 M€ de primes, a 3 M€ de frais (30 %) et subit 7 M€ de sinistres. Elle cède 40 % en quote-part avec une commission de cession de 30 % de la prime cédée. Calculez la prime cédée, les sinistres cédés, la commission, puis le résultat de la cédante et du réassureur avant et après. Que se passe-t-il si les sinistres valent 8 M€ ?

Exercice 6.2 ★ — Excédent de plénitude (section 6.1.2)

Une plénitude de 500 000 € est fixée. Un risque de capital assuré 2 M€ subit un sinistre de 1,2 M€ ; sa prime est de 20 000 €. Un second risque de 400 000 € subit un sinistre de 300 000 €. Calculez les parts cédées de sinistre et de prime.

Exercice 6.3 ★ — Une tranche par risque (section 6.1.3)

La tranche « 300 xs 200 » (en k€) s'applique aux sinistres 150, 250, 400 et 700. Calculez les récupérations et le net. Même question avec un plafond annuel de 400.

Exercice 6.4 ★★ — Réintégrations (section 6.1.3)

Une tranche « 1 000 xs 1 000 » (k€) coûte 400 par an. Elle a **deux** réintégrations : la première à 100 % pro rata du montant, la seconde à 50 %. Les sinistres de l'année touchent la tranche pour 600, 1 000, 1 000 puis 500. Calculez les paiements du réassureur (plafond annuel $3 \times 1\,000$), les primes de réintégration et le coût net.

Exercice 6.5 ★★ — Plafonner vaut mieux que grossir (section 6.1.1)

Un portefeuille compte $\lambda = 100$ sinistres par an, de coefficient de variation $CV(X) = 3$. Calculez le CV de la charge annuelle. Quel serait-il avec un portefeuille quatre fois plus grand ? Avec un plafonnement qui ramène le CV d'un sinistre à 1 ? Conclusion ?

Exercice 6.6 ★ — Burning cost à la main (section 6.2.1)

Les charges annuelles (M€) d'une tranche sur cinq ans sont 0,8 ; 0 ; 2,4 ; 0,5 ; 1,2, pour des expositions respectives de 100, 110, 120, 130 et 140. Quelle est la prime pure pour une exposition de 150 ? Quel est l'écart avec la moyenne brute ?

Exercice 6.7 ★★ — Le levier de l'inflation (section 6.2.1)

Quatre sinistres de 0,9 ; 1,2 ; 1,5 et 2,5 M€ traversent la tranche « 1 M€ xs 1 M€ ». Avec 5 % d'inflation par an pendant trois ans, de combien augmente le coût de la tranche ? Comparez à l'augmentation des sinistres.

Exercice 6.8 ★★ — Pareto et prix de tranche (section 6.2.2)

Les dépassements de 1 M€ sont au nombre de 10 par an et suivent une loi de Pareto de paramètre α : $P(X > x \mid X > 1 \text{ M€}) = (10^6/x)^\alpha$. Calculez la charge annuelle de « 2 M€ xs 1 M€ » pour $\alpha = 2,5$ puis $\alpha = 1,5$. Commentez.

Exercice 6.9 ★★ — La formule GPD (section 6.2.3)

Avec $u = 500\,000$, $\xi = 0,3$, $\beta = 400\,000$ et $\lambda_u = 12$ par an, calculez la charge annuelle de « 1 M€ xs 1 M€ » par la formule fermée, puis par intégration numérique de la survie. Que devient le prix si ξ passe à 0,5 ?

Exercice 6.10 ★★★ — Les intervalles bootstrap couvrent-ils la vérité ? (section 6.2.4)

Rejouez 200 fois « quinze années d'observation » sous la vérité, calculez pour la tranche « 2 M€ xs 2 M€ » l'intervalle bootstrap par années, et mesurez la **part des intervalles qui contiennent la vérité**. Est-ce 95 % ?

Exercice 6.11 ★★★ — Quote-part ou excédent de sinistre ? (section 6.3.2)

Dans l'étude de l'application 6.9 (commission de 25 %), quelle part α de quote-part économise autant de capital que l'excédent « 1 M€ xs 1 M€ » à 35 % de chargement ? Que coûte-t-elle ? À partir de quelle commission la quote-part coûterait-elle autant que l'excédent ?

Exercice 6.12 ★★★ — Dimensionner un stop-loss (section 6.3.4)

Avec la portée de 10 M€ et le chargement de 40 % du livre, quelle est la **priorité la plus haute** (donc la moins chère) qui maintient la probabilité de ruine à 8 M€ sous 1 % ? Donnez le coût correspondant.

Corrigés

Corrigé 6.1

Prime cédée $0,4 \times 10 = 4$; sinistres cédés $0,4 \times 7 = 2,8$; commission $0,3 \times 4 = 1,2$. **Avant** : $10 - 3 - 7 = 0$. **Après**, la cédante garde une prime de 6, des sinistres de 4,2 et des frais de $3 - 1,2 = 1,8$: $6 - 4,2 - 1,8 = 0$. Le réassureur : $4 - 2,8 - 1,2 = 0$. La commission couvre exactement les frais : à 70 % de sinistralité, tout le monde est à l'équilibre. Avec 8 M€ de sinistres : avant, $10 - 3 - 8 = -1$; après, la cédante $6 - 4,8 - 1,8 = -0,6$ et le réassureur $4 - 3,2 - 1,2 = -0,4$: la perte se **partage 60/40**, comme la prime.

Corrigé 6.2

Premier risque : taux de cession $\tau = 1 - 500\,000/2\,000\,000 = 75\%$; sinistre cédé $0,75 \times 1,2 = 0,9$ M€, gardé 0,3 M€ ; prime cédée $0,75 \times 20\,000 = 15\,000$ €. Second risque : le capital (400 000) est inférieur à la plénitude : $\tau = 0$; rien n'est cédé, le sinistre de 300 000 € reste chez la cédante.

Corrigé 6.3

Paielements : 150 $\rightarrow$ 0 ; 250 $\rightarrow$ 50 ; 400 $\rightarrow$ 200 (le sinistre dépasse 200 de 200, portée 300) ; 700 $\rightarrow$ 300 (dépasse de 500, plafonné à 300). Total **550**, net $1\,500 - 550 = 950$. Avec un plafond annuel de 400, le réassureur paie $\min(550, 400) = 400$ et le net vaut 1 100.

Corrigé 6.4

Plafond annuel 3 000. Premier sinistre sur la tranche : 600 payés ; reconstitution de 600 à 100 % : prime $400 \times 600/1000 = 240$ (il reste 400 de capacité à 100 %). Deuxième : 1 000 payés (cumul 1 600) ; 400 reconstitués à 100 % ($400 \times 400/1000 = 160$) puis 600 à 50 % ($0,5 \times 400 \times 600/1000 = 120$). Troisième : 1 000 payés (cumul 2 600) ; la capacité restante de la seconde réintégration est de 400 à 50 % : $0,5 \times 400 \times 400/1000 = 80$. Quatrième : 500 demandés, mais il ne reste que $3\,000 - 2\,600 = 400$ de plafond : 400 payés, aucune reconstitution. **Paielements** : $600 + 1\,000 + 1\,000 + 400 = 3\,000$; **primes de réintégration** : $240 + 160 + 120 + 80 = 600$; coût net $400 + 600 - 3\,000 = -2\,000$, soit un gain de 2 000.

```
def reintegrations(pertes, L, prime, taux_reint, plafond):
    """taux_reint : liste des taux (un par réintégration) ; la capacité de chacune vaut L"""
    reste = [L] * len(taux_reint); paye = prime_reint = 0.0
    for p in pertes:
        p = min(p, L, plafond - paye)
        paye += p; a_reconstituer = p
        for j, t in enumerate(taux_reint):
            r = min(a_reconstituer, reste[j]); prime_reint += t * prime * r / L; reste[j] -= r;
    a_reconstituer -= r
    return paye, prime_reint
print(reintegrations([600, 1000, 1000, 500], 1000, 400, [1.0, 0.5], 3000))
```

```
(3000.0, 600.0)
```

Corrigé 6.5

$CV(S) = \sqrt{(1+9)/100} = 0,316$. Avec quatre fois plus de sinistres : $\sqrt{10/400} = 0,158$, la moitié. Avec un plafonnement qui ramène le CV d'un sinistre à 1 : $\sqrt{(1+1)/100} = 0,141$, **mieux** que le portefeuille quadruplé. Le rapport des tailles équivalentes est $(1+9)/(1+1) = 5$: plafonner équivaut à multiplier le portefeuille par 5. Conclusion : la réassurance non proportionnelle apporte une régularité que la mutualisation seule ne peut pas donner.

Corrigé 6.6

Facteurs $150/E_t$: 1,5 ; 1,364 ; 1,25 ; 1,154 ; 1,071. Charges ajustées : $0,8 \times 1,5 = 1,2$; $0 ; 2,4 \times 1,25 = 3,0$; $0,5 \times 1,154 = 0,577$; $1,2 \times 1,071 = 1,286$. Somme 6,063, **moyenne** 1,213 M€. La moyenne brute vaut $4,9/5 = 0,98$ M€ : l'écart (+24 %) vient de l'exposition qui croît.

Corrigé 6.7

Coefficient d'inflation : $1,05^3 = 1,1576$. Sinistres : $0,9 \rightarrow 1,042$; $1,2 \rightarrow 1,389$; $1,5 \rightarrow 1,736$; $2,5 \rightarrow 2,894$. Charges de la tranche (portée 1 M€) : avant $0 + 0,2 + 0,5 + 1 = 1,7$ M€ ; après $0,042 + 0,389 + 0,736 + 1 = 2,167$ M€. Le coût de la tranche monte de **27,5 %**, alors que les sinistres montent de 15,8 % : c'est l'effet de levier (le dernier sinistre était déjà plafonné, et le premier entre dans la tranche).

Corrigé 6.8

Avec $\lambda_u = 10$, $u = 10^6$: charge = $10 \int_{10^6}^{3 \times 10^6} (10^6/x)^\alpha dx = 10 \cdot \frac{10^{6\alpha}}{\alpha - 1} [(10^6)^{1-\alpha} - (3 \cdot 10^6)^{1-\alpha}]$. Pour $\alpha = 2,5$: 5,38 M€. Pour $\alpha = 1,5$: 8,45 M€, soit **1,6 fois plus** : pour une queue un peu plus lourde, le prix de la même tranche augmente de plus de moitié, d'où l'importance de α .

Corrigé 6.9

La formule donne $\lambda_u \frac{\beta}{1-\xi} [(1 + \xi \frac{a-u}{\beta})^{1-1/\xi} - (1 + \xi \frac{a+L-u}{\beta})^{1-1/\xi}]$ avec $a = 10^6$, $L = 10^6$.

```
u, beta, lam_u = 5e5, 4e5, 12
for xi in [0.3, 0.5]:
    formule = 0.prix_gpd(1e6, 1e6, u, xi, beta, lam_u)
    numerique = integrate.quad(lambda t: lam_u * (1 + xi * (t - u) / beta) ** (-1 / xi), 1e6, 2e6)[0]
    print(f"xi = {xi} : formule {formule/1e6:.4f} M€ intégration {numerique/1e6:.4f} M€")
```

```
xi = 0.3 : formule 2.0805 M€ intégration 2.0805 M€
xi = 0.5 : formule 2.5686 M€ intégration 2.5686 M€
```

Les deux calculs coïncident (2,080 M€ pour $\xi = 0,3$) ; en passant à $\xi = 0,5$, le prix devient 2,569 M€, soit **23 % de plus** pour un changement de 0,2 sur l'indice de queue.

Corrigé 6.10

```
rng = np.random.default_rng(2026)
a, L, B_out, vrai = 2e6, 2e6, 200, 0.prix_vrai(2e6, 2e6)
dedans = 0
for _ in range(B_out):
    par_an = np.array([0.tranche(0.tirer_sinistres_vrais(rng, rng.poisson(160 * 1.03 ** i))), a,
    ↪ L).sum() * 1.03 ** (14 - i) * 1.03 for i in range(15)])
    boot = par_an[rng.integers(0, 15, (300, 15))].mean(axis=1)
    dedans += np.percentile(boot, 2.5) <= vrai <= np.percentile(boot, 97.5)
print(f"couverture empirique : {dedans / B_out:.1%}")
```

```
couverture empirique : 91.0%
```

La couverture est de 91 %, **inférieure aux 95 % annoncés** : avec quinze observations d'une variable à queue lourde, le bootstrap par années sous-estime l'incertitude (un rééchantillonnage ne peut pas inventer une année pire que celles observées). Un intervalle de bootstrap est un **minimum** d'incertitude.

Corrigé 6.11

Sans réassurance, K est proportionnel à la part gardée : une quote-part α économise αK_0 . L'excédent « 1 M€ xs 1 M€ » économise 2,96 M€, d'où $\alpha^* = 21,2\%$. Le coût de la quote-part vaut $\alpha(0,75P - E[S])$ avec $P = E[S]/0,7$, soit 0,44 M€, contre 0,88 M€ pour l'excédent. La commission qui rendrait le coût de la quote-part égal à celui de l'excédent est de 20,0 % (au lieu de 25 %) : **la quote-part n'est moins chère que parce que la commission rembourse les frais.**

Corrigé 6.12

On balaie la priorité d'un stop-loss de portée 10 M€ et on garde la plus haute pour laquelle la ruine reste sous 1 %.

```
res = []
for a in np.arange(34, 42, 0.5):
    rec = 0.tranche(brute, a * 1e6, 10e6)
    b = bilan(rec, 1.4 * rec.mean()); res.append((a, b["cout"], b["ruine"]))
ok = [r for r in res if r[2] < 0.01]
print(ok[-1] if ok else "aucune priorité ne suffit")
```

```
(np.float64(38.5), np.float64(31785.67775999999), np.float64(0.0002))
```

La priorité maximale est de 38,5 M€, pour un coût de 0,032 M€ par an et une ruine de 0,02 %. Au-dessus, une partie des années où le capital de 8 M€ est consommé (charge brute au-delà de 38,9 M€) n'est plus couverte : **la priorité doit rester inférieure au point où le capital est épuisé.**

Chapitre 7 : Gestion actif-passif et théorie du portefeuille — exercices et applications

Ce chapitre du cahier accompagne le chapitre 7 du livre (actif-passif, théorie du portefeuille, surplus). Il est, comme le chapitre, **facultatif**. Les **applications** reprennent en code, par petites étapes, les calculs du livre ; les **exercices** (★ direct, ★★ demande de réfléchir, ★★★ démonstration ou petite étude) se terminent par des **corrigés** : cherchez d'abord, regardez ensuite.

Données simulées, modèle jouet. Les courbes de taux et les rendements de marché sont simulés et **indépendants** ; le passif est un échéancier fixe sans rachats ni options. Rien ici n'est un conseil en placement.

Préparation

Tout le cahier repose sur un module écrit à la main, `build/outils_ch07.py` (table de mortalité, flux d'un passif vie, valeur actuelle, duration, frontière efficiente, scénarios de surplus). Les applications **n'en dépendent que pour charger les données** et pour les parties longues ; les calculs de base sont réécrits dans le cahier.

```
import sys
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.optimize import minimize

sys.path.insert(0, "build")
import outils_ch07 as O

pd.set_option("display.width", 200)
b = O.Bilan(surplus=0.10)          # bilan jouet : passif d'un portefeuille vie, actif = 110 % du passif
c, rend, regimes, pv, mo = O.charger()
print(f"passif : {b.L0/1e6:.1f} M€ ; actif : {b.A0/1e6:.1f} M€ ; {b.nb_contrats} contrats")

passif : 460.2 M€ ; actif : 506.2 M€ ; 16908 contrats
```

Applications

Application 7.1 — Prix, duration et convexité d'une obligation

Sections du livre : 7.1.2. Objectif : écrire soi-même le prix, la duration et la convexité, les vérifier par différences finies et mesurer l'erreur des approximations.

Étape 1 — Le prix et la duration, à la main. Une obligation à 10 ans, coupon de 5 %, nominal 100, taux 3 % (composition annuelle).

```
def prix(flux, y):
    t = np.arange(1, len(flux) + 1)
    return float((flux / (1 + y) ** t).sum())

def duration_mac(flux, y):
    t = np.arange(1, len(flux) + 1)
    va = flux / (1 + y) ** t
    return float((t * va).sum() / va.sum())

flux = np.r_[np.full(9, 5.0), 105.0]
P0 = prix(flux, 0.03)
print(round(P0, 3), round(duration_mac(flux, 0.03), 3), round(duration_mac(flux, 0.03) / 1.03, 3))
```

117.06 8.272 8.031

Étape 2 — Vérification par différences finies. La duration modifiée est $-(1/P) dP/dy$, la convexité $(1/P) d^2P/dy^2$.

```
h = 1e-4
Pm, Pp = prix(flux, 0.03 - h), prix(flux, 0.03 + h)
dmod_num = -(Pp - Pm) / (2 * h) / P0
conv_num = (Pp - 2 * P0 + Pm) / h**2 / P0
print(round(dmod_num, 4), round(conv_num, 3))
```

8.0308 80.023

Étape 3 — Erreur des approximations. On compare le prix exact après un choc Δy aux approximations « duration seule » et « duration + convexité ».

```
dm, cv = duration_mac(flux, 0.03) / 1.03, conv_num
lignes = []
for dy in (-0.03, -0.01, -0.005, 0.005, 0.01, 0.03):
    exact = prix(flux, 0.03 + dy) / P0 - 1
    lignes.append([100 * dy, 100 * exact, 100 * (-dm * dy), 100 * (-dm * dy + 0.5 * cv * dy**2)])
print(pd.DataFrame(lignes, columns=["choc (pt)", "exact (%)", "duration (%)", "avec convexité (%)"]).round(3).to_string(index=False))
```

choc (pt)	exact (%)	duration (%)	avec convexité (%)
-3.0	28.139	24.092	27.693
-1.0	8.446	8.031	8.431
-0.5	4.117	4.015	4.115
0.5	-3.917	-4.015	-3.915
1.0	-7.645	-8.031	-7.631
3.0	-20.861	-24.092	-20.491

Lecture. Pour $\pm 0,5$ point, la duration seule est presque exacte ; à 3 points, elle se trompe de plusieurs points de pourcentage, et la convexité corrige l'essentiel. L'erreur de la duration seule est **toujours dans le même sens** (elle sous-estime le prix) : c'est le signe de la convexité positive.

Ici $P = 117,06$, $D_{\text{mod}} = 8,03$ et $C = 80,0$; à +3 points, l'écart entre le prix exact et l'approximation de la duration seule est de 3,23 point de pourcentage, ramené à -0,37 avec la convexité.

Pour aller plus loin. Refaites le calcul pour un zéro-coupon à 10 ans ($D_{\text{mod}} = 10/1,03$) : sa convexité est plus grande que celle de l'obligation à coupons de même échéance, pourquoi ?

Application 7.2 — Construire le passif d'un portefeuille d'assurance vie

Sections du livre : 7.1.3. Objectif : de la table de mortalité à la duration du passif, et mesurer la sensibilité du résultat aux hypothèses.

Étape 1 — La table de mortalité. Décès cumulés sur 2015–2019 divisés par les expositions, par sexe et par âge, puis $q = 1 - \exp(-m)$.

```
d = mo[mo["annee"].between(2015, 2019)].groupby(["sexe", "age"])[["deces", "exposition"]].sum()
q = {s: 1 - np.exp(-(d.loc[s, "deces"] / d.loc[s, "exposition"]).clip(upper=0.9)) for s in ("F", "M")}
tab = pd.DataFrame(q)
print(tab.loc[[0, 30, 50, 70, 90]].round(5))
```

	F	M
age		
0	0.00337	0.00439
30	0.00080	0.00105
50	0.00497	0.00610
70	0.03620	0.04804
90	0.25820	0.32794

Étape 2 — Le rapport décès observés / décès attendus du portefeuille. On applique la table à chaque contrat (âge atteint en 2017) et on compare au nombre de décès observés.

```
age_mid = np.clip(pv["age_emission"] + (2017 - pv["annee_emission"]).clip(lower=0), 0, 99)
att = np.array([tab.loc[a, s] for a, s in zip(age_mid, pv["sexe"])]) * pv["exposition_2015_2019"]
ae = pv["deces"].sum() / att.sum()
print(pv["deces"].sum(), "décès observés ;", round(att.sum(), 1), "attendus ; A/E =", round(ae, 3))
```

832 décès observés ; 1061.9 attendus ; A/E = 0.783

Étape 3 — Les flux, la valeur actuelle et la duration. `0.flux_passif` construit l'échéancier des contrats en vigueur au 1er janvier 2020 (sans décès en 2015–2019, non échus).

```
flux, n = 0.flux_passif(pv, tab, ae)
y = 0.taux_annuels(b.c0)
print(n, "contrats ; flux total", round(flux.sum() / 1e6, 1), "M€ ; VA", round(0.vp(flux, y) / 1e6,
↳ 1), "M€")
print("duration de Macaulay", round(0.duration_macaulay(flux, y), 2), "; modifiée",
↳ round(0.duration_modifiee(flux, y), 2))
```

16908 contrats ; flux total 716.4 M€ ; VA 460.2 M€
duration de Macaulay 16.57 ; modifiée 16.18

Étape 4 — Sensibilité aux hypothèses. On fait varier le rapport A/E (mortalité plus faible ou plus forte) et on ajoute des **rachats** (chaque année, 3 % des contrats encore en vigueur disparaissent).

```
def passif(ratio, rachat=0.0):
    f, _ = 0.flux_passif(pv, tab, ratio)
    f = f * (1 - rachat) ** np.arange(len(f)) # survie du contrat au rachat
    return f, 0.vp(f, y), 0.duration_modifiee(f, y)
lignes = {f"A/E = {r:.2f}, rachat {100*k:.0f} %": (passif(r, k)[1] / 1e6, passif(r, k)[2]) for r, k in
↳ ((ae, 0), (0.6, 0), (1.0, 0), (ae, 0.03))}
print(pd.DataFrame(lignes, index=["VA (M€)", "duration modifiée"]).T.round(2).to_string())
```

	VA (M€)	duration modifiée
A/E = 0.78, rachat 0 %	460.23	16.18
A/E = 0.60, rachat 0 %	412.18	17.51

A/E = 1.00, rachat 0 %	507.14	14.96
A/E = 0.78, rachat 3 %	303.70	12.58

Lecture. Une mortalité plus forte avance les décès : la valeur actuelle monte et la duration baisse. Les rachats retirent les flux lointains : la duration baisse beaucoup. Une duration calculée sur un passif sans rachats **surestime** donc la sensibilité aux taux.

Ici, $A/E = 0,783$ et $D_{\text{mod}} = 16,2$; avec $A/E = 0,6$ elle vaut 17,5, avec $A/E = 1$ 15,0, et avec des rachats de 3 % par an 12,6.

Application 7.3 — Apparier l'actif au passif

Sections du livre : 7.1.4. **Objectif :** calculer l'écart de duration en euros, bâtir un portefeuille de zéro-coupons apparié, tester les conditions de Redington par revalorisation complète.

Étape 1 — Fonctions de revalorisation. On revalorise le passif et un portefeuille de zéro-coupons (poids par maturité, valeur initiale A_0) après un déplacement parallèle.

```
y0 = b.y0
def val_actif(w, ys):
    return sum(b.A0 * wk * (1 + ys[m - 1]) ** (-m) / (1 + y0[m - 1]) ** (-m) for m, wk in w.items())
def dS(w, dy):
    ys = y0 + dy
    return (val_actif(w, ys) - b.A0 - (0.vp(b.flux, ys) - b.L0)) / 1e6
D_L = 0.duration_modifiee(b.flux, y0)
D_cible = D_L * b.L0 / b.A0
print("D_L =", round(D_L, 2), "; durée cible de l'actif =", round(D_cible, 2))
```

$D_L = 16.18$; durée cible de l'actif = 14.71

Étape 2 — Deux zéros-coupons à 10 et 20 ans. On cherche les poids ($w, 1 - w$) qui donnent la duration cible.

```
d10, d20 = 10 / (1 + y0[9]), 20 / (1 + y0[19])
w20 = (D_cible - d10) / (d20 - d10)
w_app = {10: 1 - w20, 20: w20}
print({k: round(float(v), 3) for k, v in w_app.items()})
print(pd.Series({f"{100*dy:+.0f} pt": round(dS(w_app, dy), 1) for dy in (-0.03, -0.01, 0.01,
↪ 0.03)}).to_string())
```

```
{10: 0.493, 20: 0.507}
-3 pt   -48.3
-1 pt   -3.6
+1 pt   -2.5
+3 pt   -15.6
```

Étape 3 — Quelle paire de maturités protège le mieux ? Pour chaque paire (m_1, m_2), on apparie la duration cible, puis on mesure la **pire perte de surplus** sur des chocs parallèles entre -3 et $+3$ points.

```
chocs = np.linspace(-0.03, 0.03, 13)
res = []
for m1 in (2, 3, 5, 7, 10):
    for m2 in (15, 20, 25, 30, 40):
        d1, d2 = m1 / (1 + y0[m1 - 1]), m2 / (1 + y0[m2 - 1])
        w2 = (D_cible - d1) / (d2 - d1)
        if 0 <= w2 <= 1:
            w = {m1: 1 - w2, m2: w2}
            res.append((m1, m2, min(dS(w, dy) for dy in chocs)))
res = sorted(res, key=lambda r: -r[2])
print(pd.DataFrame(res[:5], columns=["m1", "m2", "pire variation du surplus
↪ (M€)"]).round(2).to_string(index=False))
```

```
print(pd.DataFrame(res[-3:], columns=["m1", "m2", "pire variation du surplus  
↪ (M€)"]).round(2).to_string(index=False))
```

```
m1 m2  pire variation du surplus (M€)
2  30                      0.0
2  40                      0.0
3  40                      0.0
5  40                      0.0
7  40                      0.0
m1 m2  pire variation du surplus (M€)
5  20                    -40.47
7  20                    -43.52
10 20                    -48.31
```

Lecture. Les meilleures paires sont les plus **éloignées** (un haltère très court et très long) : c'est la condition de convexité de Redington (la convexité de l'actif doit entourer celle du passif). Les paires proches du passif (par exemple 10 et 20 ans, celle de l'étape 2) laissent une perte de plusieurs dizaines de M€ pour un choc de 3 points.

Ici, la meilleure paire est (2 ; 30) ans, avec une pire variation de 0,00 M€ ; la moins bonne est (10 ; 20) ans, avec -48,3 M€.

Application 7.4 — Chocs de courbe et durations par maturité

Sections du livre : 7.1.5, 7.1.6. **Objectif :** lire l'exposition du passif nœud par nœud et la tester sur des chocs non parallèles, puis sur les pires mouvements de l'historique.

Étape 1 — Durations par maturité. On déplace un nœud de la courbe à neuf points de 1 point de base et on mesure la variation de valeur du passif.

```
noeuds = [0.25, 1, 2, 3, 5, 7, 10, 20, 30]
kr = {}
for k, m in enumerate(noeuds):
    cc = b.c0.copy(); cc[k] += 0.0001
    kr[m] = (0.vp(b.flux, 0.taux_annuels(cc)) - b.L0) / 1e3
print(pd.Series(kr, name="Δ passif (k€) pour +1 pb").round(0).to_string())
```

```
0.25    0.0
1.00   -2.0
2.00   -4.0
3.00   -9.0
5.00  -19.0
7.00  -32.0
10.00 -111.0
20.00 -196.0
30.00 -373.0
```

Étape 2 — Trois chocs. Parallèle (+1 pt), pentification (courts -0,5 pt, longs +1 pt) et aplatissement. On mesure la variation du surplus du portefeuille apparié de l'application 7.3.

```
def dS_courbe(w, ys):
    return (val_actif(w, ys) - b.A0 - (0.vp(b.flux, ys) - b.L0)) / 1e6
for nom, ys in (("parallèle +1", y0 + 0.01), ("pentification", 0.choc_pente(y0, -0.005, 0.01)),
               ("aplatissement", 0.choc_pente(y0, 0.01, -0.005))):
    print(f"{nom:15s} Δ surplus = {dS_courbe(w_app, ys):6.1f} M€")
```

```
parallèle +1  Δ surplus = -2.5 M€
pentification Δ surplus = -7.2 M€
aplatissement Δ surplus = 4.2 M€
```

Étape 3 — Les pires mouvements de l'historique. À partir de `courbe_taux.csv`, on calcule les

variations de la courbe sur 12 mois glissants, puis on revalorise le bilan avec la variation la **pire** pour le surplus.

```
C = b.courbe
var12 = C[12:] - C[:-12]
pertes = []
for v in var12:
    ys = 0.taux_annuels(np.maximum(b.c0 + v, 0.0))
    pertes.append(dS_courbe(w_app, ys))
i = int(np.argmax(pertes))
print("pire variation sur 12 mois :", round(pertes[i], 1), "M€ ; variation du taux à 5 ans :",
      ↪ round(100 * var12[i][4], 2), "pt")
```

```
pire variation sur 12 mois : -11.2 M€ ; variation du taux à 5 ans : -1.28 pt
```

Lecture. La pentification coûte plusieurs fois plus que le parallèle de même amplitude, parce qu'elle frappe les maturités longues où le passif est concentré. Le pire mouvement sur 12 mois est une **baisse** des taux : c'est le risque identifié en 7.1.7.

Ici, le nœud 30 ans porte 373 k€ par point de base contre 111 k€ pour le nœud 10 ans ; la pentification coûte -7,2 M€ contre -2,5 M€ pour le choc parallèle ; le pire mouvement sur 12 mois coûte -11,2 M€ (taux à 5 ans : -1,28 point).

Application 7.5 — La frontière efficiente sur cinq actifs

Sections du livre : 7.2.1 à 7.2.3. **Objectif :** estimer μ et Σ , tracer la frontière, comparer les portefeuilles de variance minimale et tangent, avec et sans vente à découvert.

Étape 1 — Paramètres annualisés.

```
ACT = list(rend.columns)
mu, S = 0.stats_annuelles(rend)
sd = np.sqrt(np.diag(S))
print(pd.DataFrame({"rendement (%)": 100 * mu, "volatilité (%)": 100 * sd},
  ↪ index=ACT).round(1).to_string())
```

	rendement (%)	volatilité (%)
actions_A	5.6	18.7
actions_B	4.3	22.8
obligations	4.0	4.4
immobilier	5.7	13.2
matieres	-2.9	22.0

Étape 2 — Variance minimale : formule fermée et optimisation. Sans contrainte de signe, $w = \Sigma^{-1}1 / (1^T \Sigma^{-1}1)$. Avec la contrainte « poids positifs », on passe par l'optimiseur.

```
un = np.ones(5)
w_libre = np.linalg.solve(S, un); w_libre /= w_libre.sum()
w_pos = 0.min_variance(S)
print(pd.DataFrame({"sans contrainte": w_libre, "poids positifs": w_pos},
  ↪ index=ACT).round(3).to_string())
print("volatilité :", round(100 * 0.vol_ptf(w_libre, S), 3), "%", round(100 * 0.vol_ptf(w_pos, S),
  ↪ 3), "%")
```

	sans contrainte	poids positifs
actions_A	0.019	0.019
actions_B	0.012	0.012
obligations	0.888	0.888
immobilier	0.062	0.062
matieres	0.019	0.019
volatilité :	4.09 %	4.09 %

Étape 3 — La frontière. Pour 30 rendements cibles, on cherche les poids de variance minimale (poids positifs) ; on trace la frontière avec les actifs.

```
cibles = np.linspace(w_pos @ mu, mu.max(), 30)
W = 0.frontiere(mu, S, cibles)
vols = np.sqrt(np.einsum("ij,jk,ik->i", W, S, W))
fig, ax = plt.subplots(figsize=(6, 3.6))
ax.plot(100 * vols, 100 * cibles); ax.scatter(100 * sd, 100 * mu, color="black")
for i, a in enumerate(ACT): ax.annotate(a, (100 * sd[i], 100 * mu[i]), fontsize=7)
ax.set_xlabel("écart-type (%)"); ax.set_ylabel("rendement (%)")
plt.close(fig) # dans un notebook : plt.show()
print("points de la frontière :", len(W), "; vol. min.", round(100 * vols[0], 2), "% ; vol. max.",
      ↪ round(100 * vols[-1], 2), "%")
```

points de la frontière : 30 ; vol. min. 4.09 % ; vol. max. 13.2 %

Étape 4 — Le portefeuille tangent. Sharpe maximal pour un taux sans risque de 1,5 %, avec et sans vente à découvert.

```
w_t = 0.tangent(mu, S, 0.015)
w_t_libre = np.linalg.solve(S, mu - 0.015); w_t_libre /= w_t_libre.sum()
print(pd.DataFrame({"poids positifs": w_t, "sans contrainte": w_t_libre},
  ↪ index=ACT).round(3).to_string())
sh = lambda w: (w @ mu - 0.015) / 0.vol_ptf(w, S)
print("Sharpe :", round(sh(w_t), 3), "(positifs),", round(sh(w_t_libre), 3), "(sans contrainte)")
```

	poids positifs	sans contrainte
actions_A	0.038	0.069
actions_B	0.000	-0.001
obligations	0.840	0.880
immobilier	0.122	0.151
matieres	0.000	-0.099

Sharpe : 0.666 (positifs), 0.734 (sans contrainte)

Lecture. Sans contrainte de signe, l'optimiseur vend à découvert des actifs au rendement estimé faible et lève de l'effet de levier sur ceux qu'il juge attractifs ; le Sharpe augmente, **sur l'historique**, au prix de poids extrêmes que personne ne tiendrait. L'interdiction de la vente à découvert est une **régularisation**.

Ici, la variance minimale est **la même** avec ou sans contrainte de signe (4,09 % de volatilité), parce que tous les poids de la formule fermée sont positifs (1 % à 89 %). Le portefeuille tangent sans contrainte a un Sharpe de 0,73 contre 0,67 avec poids positifs, mais des poids de -10 % à 88 %.

Application 7.6 — Contributions au risque et parité des risques

Sections du livre : 7.2.4. **Objectif :** calculer les contributions d'Euler, vérifier qu'elles s'additionnent, trouver le portefeuille de parité des risques par un algorithme simple.

Étape 1 — Contributions d'un portefeuille égal pondéré.

```
we = np.full(5, 0.2)
def rc(w): return w * (S @ w) / np.sqrt(w @ S @ w)
r = rc(we)
print(pd.DataFrame({"contribution": r, "part (%)": 100 * r / r.sum()}),
  ↪ index=ACT).round(4).to_string())
print("somme =", round(r.sum(), 6), "; volatilité =", round(0.vol_ptf(we, S), 6))
```

	contribution	part (%)
actions_A	0.0305	26.1926
actions_B	0.0381	32.7021
obligations	0.0002	0.1310

```

immobilier      0.0186   15.9192
matieres        0.0292   25.0551
somme = 0.116558 ; volatilité = 0.116558

```

Étape 2 — Parité des risques par itérations. On rapproche chaque contribution de la cible σ/n en multipliant les poids par (cible / contribution)^{0,5}, puis on renormalise.

```

w = np.full(5, 0.2)
for it in range(2000):
    r = rc(w)
    if np.abs(r / r.sum() - 0.2).max() < 1e-8:          # contributions égales à 1e-8 près : arrêt
        break
    w = w * (r.mean() / r) ** 0.5
    w /= w.sum()
print("itérations :", it, "; poids :", w.round(3))
print("contributions (%) :", (100 * rc(w) / rc(w).sum()).round(2))

```

```

itérations : 18 ; poids : [0.087 0.073 0.623 0.128 0.089]
contributions (%) : [20. 20. 20. 20. 20.]

```

Étape 3 — Comparaison. Rendement, volatilité et Sharpe des trois portefeuilles (égal pondéré, variance minimale, parité des risques).

```

w_min = 0.min_variance(S)
tab = {nom: [100 * (v @ mu), 100 * 0.vol_ptf(v, S), (v @ mu - 0.015) / 0.vol_ptf(v, S)] for nom, v in
        (("égal", we), ("variance min.", w_min), ("parité", w))}
print(pd.DataFrame(tab, index=["rendement (%)", "volatilité (%)", "Sharpe"]).T.round(2).to_string())

```

	rendement (%)	volatilité (%)	Sharpe
égal	3.36	11.66	0.16
variance min.	4.03	4.09	0.62
parité	3.78	5.78	0.40

Lecture. Le portefeuille de parité des risques est un compromis : plus de risque que la variance minimale, bien moins que l'égal pondéré, et il n'utilise aucune estimation de rendement (seulement Σ) : c'est son principal attrait.

Ici, la parité demande 62 % d'obligations pour une volatilité de 5,8 %, en 18 itérations ; dans le portefeuille égal pondéré, les obligations produisent 0,1 % du risque.

Application 7.7 — Estimation, instabilité et rétrécissement

Sections du livre : 7.2.6. **Objectif :** mesurer l'incertitude des poids optimaux par rééchantillonnage, puis l'effet du rétrécissement selon le nombre d'actifs.

Étape 1 — Rééchantillonnage par blocs d'un an. On tire 16 blocs de 250 jours (avec remise), on recalcule le portefeuille tangent, et on répète 100 fois.

```

rng = np.random.default_rng(0)
blocs = [rend.iloc[s:s + 250] for s in range(0, len(rend) - 250, 250)]
W = []
for _ in range(100):
    echantillon = pd.concat([blocs[i] for i in rng.integers(0, len(blocs), len(blocs))])
    m_, S_ = 0.stats_annuelles(echantillon)
    W.append(0.tangent(m_, S_, 0.015))
W = np.array(W)
print(pd.DataFrame({"moyenne": W.mean(axis=0), "écart-type": W.std(axis=0), "min": W.min(axis=0),
                    "max": W.max(axis=0)}, index=ACT).round(2).to_string())

```

	moyenne	écart-type	min	max
actions_A	0.05	0.06	0.00	0.21
actions_B	0.02	0.03	0.00	0.12
obligations	0.81	0.12	0.36	1.00
immobilier	0.12	0.13	0.00	0.64
matieres	0.00	0.01	0.00	0.07

Étape 2 — Même exercice pour la variance minimale.

```
Wm = np.array([0.min_variance(0.stats_annuelles(pd.concat([blocs[i] for i in rng.integers(0,
↳ len(blocs), len(blocs))]))[1]) for _ in range(100)])
print("écart-type des poids (variance min.) :", Wm.std(axis=0).round(3))
```

```
écart-type des poids (variance min.) : [0.011 0.007 0.015 0.01 0.006]
```

Étape 3 — Rétrécissement selon le nombre d'actifs. Volatilité vraie du portefeuille de variance minimale sans contrainte, construit sur 120 observations, pour n actifs simulés (modèle à facteurs) : covariance empirique, rétrécie (Ledoit–Wolf), poids égaux et optimum.

```
lignes = {}
for n in (10, 30, 60, 100):
    lignes[n] = 100 * 0.experience_retrencissement(n=n, T=120, reps=40, seed=1)
print(pd.DataFrame(lignes, index=["empirique", "Ledoit-Wolf", "poids égaux",
↳ "optimum"]).T.round(2).to_string())
```

	empirique	Ledoit-Wolf	poids égaux	optimum
10	1.98	1.98	5.39	1.91
30	1.35	1.30	9.92	1.18
60	1.03	0.90	10.28	0.73
100	1.60	0.80	11.25	0.64

Lecture. Les poids du portefeuille tangent varient énormément d'un rééchantillon à l'autre ; ceux de la variance minimale beaucoup moins. Quand n grandit avec $T = 120$ fixé, la covariance empirique se dégrade (au-delà de $n = T$ elle n'est même plus inversible) et le rétrécissement devient très utile.

Ici, l'écart-type du poids des obligations est de 0,12 pour le tangent et de 0,01 pour la variance minimale. À $n = 10$, la covariance empirique et la covariance rétrécie sont proches (1,98 % et 1,98 %) ; à $n = 100$, l'écart est net (1,60 % contre 0,80 %).

Application 7.8 — Surplus, VaR et simulation sur dix ans

Sections du livre : 7.3. Objectif : mesurer le risque du surplus de plusieurs allocations, optimiser sous un budget de risque, simuler le taux de couverture, puis tester l'effet d'une corrélation entre taux et actions.

Étape 1 — Scénarios annuels. Courbe et marchés tirés dans l'historique ; gain en euros de six instruments et variation du passif.

```
dY, ann = 0.tirages_annuels(b, 5000, 21)
X, dliab = 0.pnl_instruments(b, dY, ann)
S0 = b.A0 - b.L0
w_app_v = 0.poids_vecteur(w_app)
w_marche = 0.poids_vecteur({5: 0.40, "act": 0.45, "imm": 0.15})
for nom, w in (("apparié", w_app_v), ("marché", w_marche)):
    dSs = X @ w - dliab
    v, e = 0.var_es(dSs)
    print(f"{nom:8s} sd {dSs.std()/1e6:6.1f} VaR 99,5 % {v/1e6:6.1f} ES 99 % {e/1e6:6.1f} S0/VaR
↳ {S0/v:5.2f}")
```

apparié	sd	2.0	VaR 99,5 %	5.1	ES 99 %	5.9	S0/VaR	8.94
marché	sd	58.2	VaR 99,5 %	134.1	ES 99 %	138.6	S0/VaR	0.34

Étape 2 — Allocation optimale sous un budget de risque. On cherche l'allocation de gain espéré maximal dont l'écart-type du surplus est de 15 M€.

```
w_opt = 0.surplus_frontiere(X, dliab, [15e6])[0]
d_opt = X @ w_opt - dliab
print({k: round(float(v), 2) for k, v in zip(0.NOMS_INSTR, w_opt)})
print("gain espéré", round(d_opt.mean() / 1e6, 1), "M€ ; sd", round(d_opt.std() / 1e6, 1), "M€ ; VaR",
      ↪ round(0.var_es(d_opt)[0] / 1e6, 1), "M€")
```

```
{'ZC 5 ans': 0.29, 'ZC 10 ans': 0.0, 'ZC 20 ans': 0.0, 'ZC 30 ans': 0.55, 'actions': 0.07,
 ↪ 'immobilier': 0.09}
gain espéré 5.0 M€ ; sd 15.0 M€ ; VaR 28.4 M€
```

Étape 3 — Taux de couverture sur dix ans. 1 000 trajectoires, trois allocations.

```
w_marche_d = {5: 0.40, "act": 0.45, "imm": 0.15}
w_opt_d = {k: v for k, v in zip(0.INSTR, w_opt) if v > 1e-6}
lignes = {}
for nom, w in (("marché", w_marche_d), ("apparié", w_app), ("optimisé", w_opt_d)):
    FR = 0.simuler_aln(b, w, N=1000, annees=10, seed=3)
    lignes[nom] = [np.median(FR[:, 10]), 100 * (FR.min(axis=1) < 1).mean()]
print(pd.DataFrame(lignes, index=["médiane A/L à 10 ans", "P(A/L < 1 un jour)
      ↪ %"]).T.round(2).to_string())
```

	médiane A/L à 10 ans	P(A/L < 1 un jour) %
marché	1.43	46.0
apparié	1.14	0.2
optimisé	1.26	6.0

Étape 4 — Et si les taux et les actions étaient corrélés ? Les tirages sont indépendants. On impose une corrélation de rang ρ entre la variation du niveau de la courbe (taux à 10 ans) et le rendement des actions, en réordonnant les scénarios d'actions (copule gaussienne), puis on recalcule la VaR de l'allocation « marché ».

```
from scipy.stats import norm, rankdata
def avec_correlation(rho, seed=0):
    rng = np.random.default_rng(seed)
    u = norm.ppf(rankdata(dY[:, 6]) / (len(dY) + 1))           # rang des variations du taux à 10
    ↪ ans, en loi normale
    z = rho * u + np.sqrt(1 - rho**2) * rng.normal(size=len(dY))
    cible = np.argsort(np.argsort(z))                         # rang cible des rendements d'actions
    ann2 = ann.copy()
    ann2[:, :2] = np.sort(ann[:, :2], axis=0)[cible]
    ann2[:, 3] = np.sort(ann[:, 3])[cible]
    return 0.pnl_instruments(b, dY, ann2)
for rho in (-0.5, 0.0, 0.5):
    X2, dl2 = avec_correlation(rho)
    print(f"corrélation {rho:+.1f} : VaR 99,5 % du portefeuille marché = {0.var_es(X2 @ w_marche -
      ↪ dl2)[0] / 1e6:.1f} M€")
```

```
corrélation -0.5 : VaR 99,5 % du portefeuille marché = 110.0 M€
corrélation +0.0 : VaR 99,5 % du portefeuille marché = 144.8 M€
corrélation +0.5 : VaR 99,5 % du portefeuille marché = 166.5 M€
```

Lecture. L'allocation « apparié » a un risque de surplus minime, la « marché » un risque très supérieur au surplus initial. Dans l'étape 4, la corrélation entre les taux et les actions change la VaR : quand elle est **positive**, les baisses de taux (qui gonflent le passif) coïncident avec des baisses d'actions, ce qui cumule les mauvaises nouvelles du passif et de l'actif ; quand elle est

négative, elles se compensent. L'hypothèse d'indépendance du livre n'est donc pas neutre.

Ici, le surplus initial est de 46 M€ ; la VaR de l'allocation apparié est 5,1 M€, celle de l'allocation marché 134 M€. L'allocation optimisée à 15 M€ de risque a un gain espéré de 5,0 M€ et une VaR de 28 M€. Avec une corrélation de $-0,5$ la VaR « marché » vaut 110 M€, avec $+0,5$ 167 M€.

Exercices

Exercice 7.1 ★ — Prix et duration à la main (section 7.1.2 du livre)

Une obligation de nominal 100 € à deux ans verse un coupon de 5 € par an. Le taux est 4 %. Calculez à la main son prix, sa duration de Macaulay et sa duration modifiée.

Exercice 7.2 ★ — Variation du prix : duration et convexité (7.1.2)

Pour l'obligation de l'exercice 7.1, calculez la convexité, puis la variation relative du prix pour une hausse du taux de 1 point avec la duration seule, avec la duration et la convexité, et exactement.

Exercice 7.3 ★★ — Durations remarquables (7.1.2, 7.1.3)

- (a) Montrez que la duration de Macaulay d'un zéro-coupon de maturité m vaut m . (b) Montrez que celle d'une perpétuité de coupon annuel c , au taux y , vaut $(1 + y)/y$. (c) Que vaut-elle à 2 % ? Qu'en concluez-vous pour un passif de très longue durée ?

Exercice 7.4 ★★ — L'écart de duration en euros (7.1.4)

Un assureur a un actif de 110 M€ et un passif de 100 M€, de duration modifiée 8 ans. (a) Quelle duration modifiée doit avoir l'actif pour que le surplus soit insensible à un déplacement parallèle des taux ? (b) Si l'on égalise les durations (8 et 8), de combien varie le surplus pour une hausse de 1 point ? Pour une baisse de 1 point ?

Exercice 7.5 ★★ — Les conditions de Redington en chiffres (7.1.4)

Le passif est un paiement unique de 1 000 € dans 5 ans ; le taux plat est de 5 %. On le couvre avec des zéros-coupons de maturités 3 et 7 ans. (a) Quelles valeurs investir dans chacun pour égaler la valeur actuelle et la duration ? (b) Vérifiez la condition de convexité. (c) Calculez à la main la variation du surplus après un déplacement du taux de +2 points et de -2 points. Interprétez.

Exercice 7.6 ★ — Rendement et risque d'un portefeuille (7.2.1)

Deux actifs : $\mu = (8 \%, 4 \%)$, $\sigma = (25 \%, 8 \%)$, corrélation 0,3. Pour le portefeuille 40 %/60 %, calculez le rendement espéré et l'écart-type. Que vaudrait l'écart-type avec une corrélation de 1 ?

Exercice 7.7 ★★ — Variance minimale à deux actifs (7.2.2)

Avec les chiffres de l'exercice 7.6, (a) calculez le poids de variance minimale de l'actif risqué par la formule du livre. (b) À partir de quelle corrélation ce poids devient-il négatif ? (c) Pour $\rho = 0,5$, quel est le portefeuille de variance minimale avec et sans vente à découvert, et quelles volatilités ?

Exercice 7.8 ★★ — Contributions au risque à la main (7.2.4)

Trois actifs de volatilités 10 %, 20 %, 30 %, de corrélations $\rho_{12} = 0,5$, $\rho_{13} = 0,2$, $\rho_{23} = 0,1$, détenus à 50 %, 30 % et 20 %. Calculez la volatilité du portefeuille et la contribution de chaque actif ; vérifiez que la somme des contributions est la volatilité.

Exercice 7.9 ★★★ — Le portefeuille tangent (7.2.3)

- (a) Montrez, en maximisant le ratio de Sharpe, que le portefeuille tangent sans contrainte de signe est proportionnel à $\Sigma^{-1}(\mu - r_f \mathbf{1})$. (b) Vérifiez-le numériquement sur les cinq actifs (taux sans risque de 1,5 %) en comparant au résultat de l'optimiseur avec `long_only=False`.

Exercice 7.10 ★★ — Combien d'années pour distinguer deux actifs ? (7.2.6)

Deux actifs ont la même volatilité de 15 % et une corrélation de 0,5. Leur rendement espéré diffère de 3 points. Combien d'années d'observation faut-il pour que cette différence dépasse deux erreurs types ? Comparez à la longueur de l'historique du chapitre et vérifiez par simulation la probabilité d'estimer le bon signe avec 16 ans.

Exercice 7.11 ★★★ — Précision d'une VaR à 99,5 % (7.3.3)

Pour l'allocation « Mixte » (80 % apparié, 14 % d'actions, 6 % d'immobilier), estimez la VaR à 99,5 % du surplus avec $N = 500$, 2 000 et 8 000 scénarios, en répétant chaque estimation avec 8 graines. Calculez l'écart-type des estimations et commentez sa décroissance avec N .

Exercice 7.12 ★★ — Échéancier de refixation d'une banque (7.1.5, 7.1.6)

Reprenez l'échéancier du livre (actifs 200, 150, 400, 250 M€ ; passifs 430, 250, 170, 100 M€ ; parts de l'année restante 0,875, 0,375, 0, 0). (a) Calculez la variation de marge nette d'intérêt pour -1 point et $+1$ point. (b) Quel montant supplémentaire d'actif à taux variable (tranche « moins de 3 mois ») annule la sensibilité à un déplacement parallèle ? (c) Quel est l'effet, pour cette banque, d'une pentification (taux courts $+0,5$ point, longs $+1,5$ point, la tranche « 3 à 12 mois » à $+1$ point) ?

Corrigés**Corrigé 7.1**

Flux : 5 à $t = 1$, 105 à $t = 2$. Valeurs actuelles : $5/1,04 = 4,8077$ et $105/1,04^2 = 97,0784$, donc $P = 101,8861$ €. Duration de Macaulay : $(1 \times 4,8077 + 2 \times 97,0784)/101,8861 = 198,9645/101,8861 \approx 1,953$ an. Duration modifiée : $1,953/1,04 \approx 1,878$.

```
f = np.array([5.0, 105.0]); t = np.array([1, 2]); va = f / 1.04 ** t
P = va.sum(); D = (t * va).sum() / P
print(round(P, 4), round(D, 4), round(D / 1.04, 4))
```

```
101.8861 1.9528 1.8777
```

Corrigé 7.2

Convexité : $C = \sum t(t+1) \cdot VA / ((1+y)^2 P) = (2 \times 4,8077 + 6 \times 97,0784) / (1,0816 \times 101,8861) \approx 5,373$. Pour $\Delta y = +0,01$:

- **duration seule** : $-1,878 \times 0,01 = -1,878 \%$;
- **avec la convexité** : $-1,878 \% + \frac{1}{2} \times 5,373 \times 0,0001 = -1,851 \%$;
- **exact** : à 5 %, le prix vaut $5/1,05 + 105/1,05^2 = 100$ €, soit $100/101,8861 - 1 = -1,851 \%$.

À deux ans, la convexité suffit à retrouver le résultat exact au millième de point.

```
D_mod = D / 1.04
C = (t * (t + 1) * va).sum() / (1.04 ** 2 * P)
exact = (f / 1.05 ** t).sum() / P - 1
print(round(C, 3), round(-D_mod * 0.01, 5), round(-D_mod * 0.01 + 0.5 * C * 0.01**2, 5), round(exact, 5))
```

```
5.373 -0.01878 -0.01851 -0.01851
```

Corrigé 7.3

- (a) Un seul flux F à la date m : $D = m \cdot F v^m / (F v^m) = m$. (b) Avec $v = 1/(1+y)$, $P = c \sum v^t = c/y$ et $\sum t v^t = v/(1-v)^2 = (1+y)/y^2$. Donc $D = c (1+y)/y^2 \div (c/y) = (1+y)/y$. (c) À 2 % : $1,02/0,02 = 51$ ans. Un engagement qui n'a pas de terme (rente viagère, contrat vie entière) a une duration de plusieurs dizaines d'années quand les taux sont bas : une variation d'un point des taux modifie sa valeur de plus de 50 %. C'est ce qui rend le risque de taux des assureurs vie si sensible.

```
y = 0.02
t = np.arange(1, 5000)
va = 1 / (1 + y) ** t
print(round((t * va).sum() / va.sum(), 3), round((1 + y) / y, 3))
```

```
51.0 51.0
```

Corrigé 7.4

- (a) $D_A = D_L \times L/A = 8 \times 100/110 = 7,27$ ans. (b) Avec $D_A = D_L = 8$: $\Delta A \approx -8 \times 110 \times 0,01 = -8,8$ M€ et $\Delta L \approx -8 \times 100 \times 0,01 = -8,0$ M€, donc $\Delta S \approx -0,8$ M€ pour +1 point, et +0,8 M€ pour -1 point (à l'ordre un). L'égalité des durations laisse une sensibilité de 0,8 M€ par point, soit 10 % de la sensibilité du passif.

```
A, L, D = 110.0, 100.0, 8.0
print(round(D * L / A, 3), round(-D * A * 0.01 + D * L * 0.01, 3))
```

```
7.273 -0.8
```

Corrigé 7.5

- (a) VA du passif : $1\,000/1,05^5 = 783,53$ €. Duration modifiée du passif : $5/1,05 = 4,762$. Pour des zéros-coupons, la duration modifiée d'un titre de maturité m vaut $m/1,05$: avec des valeurs x_3 et x_7 , $x_3 + x_7 = 783,53$ et $(3x_3 + 7x_7)/1,05 = 4,762 \times 783,53$, d'où $3x_3 + 7x_7 = 3\,917,6$; avec $x_3 = 783,53 - x_7$: $2\,350,6 + 4x_7 = 3\,917,6$, soit $x_7 = 391,76$ et $x_3 = 391,76$: **moitié-moitié**. (b) Convexité : zéro-coupon de maturité m : $m(m+1)/1,05^2$; pour 3 et 7 ans : 10,884 et 50,794, moyenne à poids égaux 30,839 ; pour le passif, $5 \times 6/1,05^2 = 27,211$. L'actif est **plus convexe** : la condition (iii) est satisfaite. (c) Valeur de l'actif : $x_3 (1,05/(1,05+\Delta y))^3$

+ $x_7 (1,05/(1,05+\Delta y))^7$, valeur du passif : $1\,000/(1,05+\Delta y)^5$ — voir le code. Le surplus **augmente** dans les deux sens (+0,51 € pour +2 points et +0,64 € pour -2 points sur un passif de 783 €) : c'est exactement ce que promet Redington.

```
y, dy = 0.05, 0.02
L0 = 1000 / (1 + y) ** 5
x3 = x7 = L0 / 2
for d_ in (-dy, dy):
    A1 = x3 * ((1 + y) / (1 + y + d_)) ** 3 + x7 * ((1 + y) / (1 + y + d_)) ** 7
    L1 = 1000 / (1 + y + d_) ** 5
    print(f"Δy = {d_:+.2f} : ΔS = {A1 - L1:+.3f} €")
```

```
Δy = -0.02 : ΔS = +0.638 €
Δy = +0.02 : ΔS = +0.508 €
```

Corrigé 7.6

Rendement espéré : $0,4 \times 8 + 0,6 \times 4 = 5,6$ %. Variance : $0,4^2 \times 0,0625 + 0,6^2 \times 0,0064 + 2 \times 0,4 \times 0,6 \times 0,3 \times 0,25 \times 0,08 = 0,01 + 0,002304 + 0,00288 = 0,015184$, soit un écart-type de **12,32** %. Avec $\rho = 1$: $0,4 \times 25 + 0,6 \times 8 = 14,8$ % : la diversification enlève 2,5 points de volatilité.

```
w = np.array([0.4, 0.6]); s = np.array([0.25, 0.08])
for rho in (0.3, 1.0):
    Sg = np.array([s[0]**2, rho * s[0] * s[1]], [rho * s[0] * s[1], s[1]**2])
    print(rho, round(100 * np.sqrt(w @ Sg @ w), 2))
```

```
0.3 12.32
1.0 14.8
```

Corrigé 7.7

(a) $w^* = (\sigma_2^2 - \rho\sigma_1\sigma_2)/(\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2) = (0,0064 - 0,006)/(0,0625 + 0,0064 - 0,012) = 0,0004/0,0569 \approx 0,7$ %. (b) Le numérateur s'annule pour $\rho = \sigma_2/\sigma_1 = 0,08/0,25 = 0,32$; au-delà, $w^* < 0$. (c) Pour $\rho = 0,5$: numérateur $0,0064 - 0,01 = -0,0036$; dénominateur $0,0625 + 0,0064 - 0,02 = 0,0489$; $w^* = -7,4$ % (vente à découvert de l'actif risqué) ; avec cette position, la volatilité est de 7,83 % contre 8 % pour 100 % d'obligations (sans vente à découvert, on garde 100 % d'obligations). Le gain de la vente à découvert est ici de 0,17 point de volatilité.

```
s1, s2 = 0.25, 0.08
for rho in (0.3, 0.32, 0.5):
    w = (s2**2 - rho * s1 * s2) / (s1**2 + s2**2 - 2 * rho * s1 * s2)
    v = np.sqrt(w**2 * s1**2 + (1 - w)**2 * s2**2 + 2 * w * (1 - w) * rho * s1 * s2)
    print(rho, round(w, 4), round(100 * v, 3))
```

```
0.3 0.007 7.998
0.32 0.0 8.0
0.5 -0.0736 7.833
```

Corrigé 7.8

Covariances : $\sigma_{12} = 0,5 \times 0,1 \times 0,2 = 0,01$; $\sigma_{13} = 0,2 \times 0,1 \times 0,3 = 0,006$; $\sigma_{23} = 0,1 \times 0,2 \times 0,3 = 0,006$; variances 0,01, 0,04, 0,09. $\Sigma w = (0,0092 ; 0,0182 ; 0,0228)$. Variance : $0,5 \times 0,0092 + 0,3 \times 0,0182 + 0,2 \times 0,0228 = 0,01462$; écart-type **12,09** %. Contributions $w_i(\Sigma w)_i/\sigma$: $0,0046/0,1209 = 3,80$ points, $0,00546/0,1209 = 4,52$ points, $0,00456/0,1209 = 3,77$ points ; somme 12,09 points. Parts : 31,5 %, 37,3 %, 31,2 %. Les contributions sont presque égales alors que les poids ne le sont pas (50, 30, 20 %) : portefeuille proche de la parité des risques.

```
sg = np.array([0.1, 0.2, 0.3]); R = np.array([[1, .5, .2], [.5, 1, .1], [.2, .1, 1]])
Sg = R * np.outer(sg, sg); w = np.array([0.5, 0.3, 0.2])
vol = np.sqrt(w @ Sg @ w); rcs = w * (Sg @ w) / vol
print(round(100 * vol, 3), (100 * rcs).round(3), round(100 * rcs.sum(), 3), (100 * rcs /
↪ rcs.sum()).round(1))
```

```
12.091 [3.804 4.516 3.771] 12.091 [31.5 37.3 31.2]
```

Corrigé 7.9

- (a) On maximise $f(w) = (w^T m) / \sqrt{(w^T \Sigma w)}$ avec $m = \mu - r_f 1$ (le ratio est invariant par multiplication de w : la contrainte de somme 1 se réimpose ensuite). Le gradient s'annule quand $m / (w^T m) = \Sigma w / (w^T \Sigma w)$, soit $\Sigma w = \lambda m$ avec $\lambda = (w^T \Sigma w) / (w^T m)$: donc $w \propto \Sigma^{-1} m$, normalisé pour sommer à 1. (b) Le code compare les deux calculs ; l'optimiseur sans contrainte retrouve les poids de la formule (à la tolérance numérique).

```
mu, S = 0.stats_annuelles(rend)
w_f = np.linalg.solve(S, mu - 0.015); w_f /= w_f.sum()
w_o = 0.tangent(mu, S, 0.015, long_only=False)
print(w_f.round(3)); print(w_o.round(3)); print("écart max :", float(np.abs(w_f - w_o).max()))
```

```
[ 0.069 -0.001  0.88  0.151 -0.099]
[ 0.069 -0.001  0.88  0.151 -0.099]
écart max : 1.0728843480301009e-08
```

Corrigé 7.10

L'écart-type de la différence des deux rendements annuels est $\sigma_d = \sigma \sqrt{2(1 - \rho)} = 0,15 \times \sqrt{2 \times 0,5} = 15\%$. L'erreur type de la différence moyenne sur T années est $\sigma_d / \sqrt{T}$. Elle dépasse deux fois l'erreur type quand $3\% \geq 2 \times 15\% / \sqrt{T}$, soit $\sqrt{T} \geq 10$ et $T \geq 100$ ans. Avec 16 ans, l'erreur type vaut $15/4 = 3,75$ points : la différence de 3 points est **plus petite qu'une erreur type**. Probabilité d'estimer le bon signe : $P(N(3; 3,75) > 0) = \Phi(0,8) \approx 79\%$: on se trompe une fois sur cinq.

```
rng = np.random.default_rng(0)
sim = rng.normal(0.03, 0.15 / np.sqrt(16), 200000)
print(round((sim > 0).mean(), 3), round(0.15 / 4, 4))
```

```
0.787 0.0375
```

Corrigé 7.11

La VaR à 99,5 % repose sur la queue de l'échantillon (0,5 % des scénarios : 2,5 observations pour $N = 500$, 40 pour $N = 8\,000$). L'écart-type des estimations diminue en $1/\sqrt{N}$ environ, donc **lentement** : multiplier N par 16 (de 500 à 8 000) divise l'écart-type par 4 environ (huit répétitions ne permettent pas de le dire plus finement). Le code ci-dessous le mesure (les valeurs exactes dépendent des graines).

```
w_mix = 0.poids_vecteur({k: 0.8 * v for k, v in w_app.items()} | {"act": 0.14, "imm": 0.06})
lignes = {}
for N in (500, 2000, 8000):
    v = []
    for graine in range(8):
        dY_, ann_ = 0.tirages_annuels(b, N, 100 + graine)
        X_, dL_ = 0.pnl_instruments(b, dY_, ann_)
        v.append(0.var_es(X_ @ w_mix - dL_) [0] / 1e6)
    lignes[N] = [np.mean(v), np.std(v)]
```

```
print(pd.DataFrame(lignes, index=["VaR moyenne (M€)", "écart-type (M€)"]).T.round(2).to_string())
```

	VaR moyenne (M€)	écart-type (M€)
500	38.87	3.65
2000	39.81	1.74
8000	40.33	0.59

Résultat : l'écart-type de l'estimation passe de 3,65 M€ (N = 500) à 0,59 M€ (N = 8 000) pour une VaR de l'ordre de 40 M€.

Corrigé 7.12

- (a) $\Delta \text{MNI} = \sum \text{écart}_i \times \text{part}_i \times \Delta y$: pour +1 point, $(-230 \times 0,875 - 100 \times 0,375) \times 0,01 = -2,39$ M€ ; pour -1 point, +2,39 M€. (b) Un actif à taux variable de montant x dans la tranche « moins de 3 mois » ajoute $x \times 0,875 \times 0,01$: pour annuler -2,39, $x = 2,3875 / (0,875 \times 0,01) = 272,9$ M€. (Dans la pratique, on utilise un swap de taux plutôt qu'un actif au bilan.) (c) Chocs par tranche : +0,5 point (moins de 3 mois), +1 point (3 à 12 mois) : $\Delta \text{MNI} = -230 \times 0,875 \times 0,005 - 100 \times 0,375 \times 0,01 = -1,006 - 0,375 = -1,38$ M€ ; les taux longs (+1,5 point) n'affectent pas la marge de l'année (ils concernent des tranches qui ne se refixent pas dans l'année) mais affecteraient la valeur économique.

```
ecart = np.array([200 - 430, 150 - 250, 400 - 170, 250 - 100]); part = np.array([0.875, 0.375, 0, 0])
print(round((ecart * part).sum() * 0.01, 4), round(-(ecart * part).sum() * 0.01 / (0.875 * 0.01), 1))
choc = np.array([0.005, 0.01, 0.015, 0.015])
print(round((ecart * part * choc).sum(), 4))
```

```
-2.3875 272.9
-1.3812
```

Projet du volume et auto-évaluation

— exercices et applications

Ce chapitre du cahier clôt le volume V. Il contient le **projet du volume** : construire le **tarif d'une assurance automobile** pour la mutuelle, du jeu de données au rapport, en **validant** chaque étape hors période et en écrivant les **notes réglementaires** qui accompagneraient le modèle ; une **variante** reprend la démarche pour un **score de crédit**. Il ne demande aucune notion nouvelle : chaque étape s'appuie sur un chapitre du livre. Puis vient l'**auto-évaluation** (quarante questions).

Projet du volume

P.1 Le cahier des charges

La mutuelle prépare son **tarif 2025** pour l'assurance automobile. La direction technique pose six exigences :

1. **Un tarif lisible** : une prime = une prime de base $\times$ quelques coefficients (âge, zone, usage...), que les conseillers peuvent expliquer.
2. **Un tarif validé hors période** : le modèle est ajusté sur 2022–2023 et jugé sur 2024, jamais sur ses propres données.
3. **Un tarif équilibré** : le ratio sinistres sur primes attendu doit être proche de la cible (72 %), l'ensemble des primes couvrant sinistres, frais et marge.
4. **Un tarif robuste** : on connaît l'effet d'une inflation plus forte ou d'une fréquence plus élevée, et le capital nécessaire pour absorber une mauvaise année.
5. **Un tarif justifiable** : variables autorisées, absence de proxy douteux, documentation, avis de validation indépendant.
6. **Un tarif honnête** : on dit ce que le modèle ne sait pas.

La méthode suit huit étapes, chacune appuyée sur un chapitre du livre :

Étape	Question	Chapitre du livre
P.2 Auditer et découper	Les données sont-elles saines ? Comment valider hors période ?	2.1, 2.2
P.3 Fréquence	Combien de sinistres par année-police ?	2.1, 2.4
P.4 Sévérité	Combien coûte un sinistre ? Que faire des gros ?	2.1, 6.2

Étape	Question	Chapitre du livre
P.5 Prime pure et validation	La prime pure classe-t-elle bien les polices ?	2.2
P.6 Du modèle au tarif	Quelles primes commerciales, quels coefficients ?	2.2, 2.4
P.7 Robustesse et capital	Que se passe-t-il si l'année est mauvaise ?	3.1, 3.2, 4.2
P.8 Notes réglementaires	Le modèle est-il documentable, équitable, validable ?	3.4, 4.2
P.9 Le rapport	Peut-on publier ce tarif ?	tous

Les données. `donnees/polices_auto.csv` (100 000 lignes police-année, 2022 à 2024) et `donnees/sinistres_auto.csv` (un sinistre par ligne). Elles sont **simulées** : la vérité est programmée (`docstring` de `build/donnees5.py`) et nous la révélerons en fin d'étude. Aucune variable protégée (sexe, origine...) n'y figure.

P.2 Étape 1 : auditer les données et découper hors période

On vérifie d'abord ce que l'on a : volumes par année, expositions, fréquence observée, valeurs incohérentes. Puis on **découpe dans le temps** : 2022 et 2023 pour ajuster, 2024 pour juger (volume III, section 1.1).

```
import numpy as np, pandas as pd, statsmodels.api as sm, statsmodels.formula.api as smf
import warnings; warnings.filterwarnings("ignore")

pol = pd.read_csv("donnees/polices_auto.csv")
sin = pd.read_csv("donnees/sinistres_auto.csv")
audit = pol.groupby("annee").agg(lignes=("id_police", "size"), exposition=("exposition", "sum"),
    ↪ sinistres=("nb_sinistres", "sum"))
audit["frequence"] = (audit["sinistres"] / audit["exposition"]).round(4)
print(audit.round(0).astype({"lignes": int, "exposition": int, "sinistres":
    ↪ int}).assign(frequence=audit["frequence"]))
print("sinistres du fichier sinistres == somme des nb_sinistres :", len(sin) ==
    ↪ pol["nb_sinistres"].sum())
print("expositions hors ]0 ; 1] :", int(((pol["exposition"] <= 0) | (pol["exposition"] > 1)).sum()))
```

```
      lignes  exposition  sinistres  frequence
annee
2022    30244      26200      1679      0.0641
2023    32741      28321      1907      0.0673
2024    37015      32081      2136      0.0666
sinistres du fichier sinistres == somme des nb_sinistres : True
expositions hors ]0 ; 1] : 0
```

À retenir. La fréquence est **stable** d'une année à l'autre (autour de 6,6 %) : c'est ce qui autorise à juger sur 2024 un modèle ajusté sur 2022–2023. La **sévérité**, elle, dérive avec l'inflation : nous y reviendrons en P.4.

On regroupe aussi les variables continues en **classes** (âge, ancienneté du véhicule, bonus-malus). Les classes sont des choix de métier, que l'on documente ; elles rendent le tarif lisible et captent les effets non linéaires.

```
pol["classe_age"] = pd.cut(pol["age_conducteur"], [17, 24, 34, 49, 64, 70, 120], labels=["18-24",
    ↪ "25-34", "35-49", "50-64", "65-70", "71+"], astype(str)
pol["classe_veh"] = pd.cut(pol["age_vehicule"], [-1, 2, 5, 10, 30], labels=["0-2", "3-5", "6-10",
    ↪ "11+"], astype(str))
```

```

pol["classe_bm"] = pd.cut(pol["bonus_malus"], [0, 60, 80, 100, 200], labels=["<=60", "61-80",
↳ "81-100", ">100"]).astype(str)
sin = sin.merge(pol.drop(columns=["annee", "exposition", "nb_sinistres"]), on="id_police",
↳ how="left")
ajust = pol[pol["annee"] <= 2023].copy()
test = pol[pol["annee"] == 2024].copy()
print("lignes d'ajustement :", len(ajust), "| lignes de test :", len(test))

```

```
lignes d'ajustement : 62985 | lignes de test : 37015
```

P.3 Étape 2 : la fréquence

La fréquence suit une loi de Poisson dont la moyenne est proportionnelle à l'**exposition** (le temps pendant lequel la police est en risque) : c'est l'**offset** $\ln(\text{exposition})$ du GLM (volume II, section 2.3 ; livre, 2.1). On compare à une référence, la fréquence moyenne constante.

```

formule_f = "nb_sinistres ~ C(classe_age) + C(zone) + C(classe_bm) + puissance + C(usage) +
↳ C(carburant) + C(classe_veh)"
mod_f = smf.glm(formule_f, data=ajust, family=sm.families.Poisson(),
↳ offset=np.log(ajust["exposition"])).fit()
print("dispersion de Pearson :", round(mod_f.pearson_chi2 / mod_f.df_resid, 3))

def deviance_poisson(y, mu):
    y, mu = np.asarray(y, float), np.asarray(mu, float)
    return 2 * (np.where(y > 0, y * np.log(np.where(y > 0, y / mu, 1.0)), 0) - (y - mu)).sum()

mu_test = mod_f.predict(test, offset=np.log(test["exposition"]))
mu_ref = (ajust["nb_sinistres"].sum() / ajust["exposition"].sum()) * test["exposition"]
print("déviante test : modèle", round(deviance_poisson(test["nb_sinistres"], mu_test)), "|
↳ référence", round(deviance_poisson(test["nb_sinistres"], mu_ref)))
print("sinistres prévus 2024 :", round(mu_test.sum()), "| observés :",
↳ int(test["nb_sinistres"].sum()))

```

```

dispersion de Pearson : 1.028
déviante test : modèle 12056 | référence 12226
sinistres prévus 2024 : 2109 | observés : 2136

```

La dispersion de Pearson proche de 1 dit qu'un Poisson suffit à l'échelle de la police (l'hétérogénéité non observée existe, mais sa variance est petite devant la moyenne quand la fréquence annuelle est de 6,6 %) ; elle compte davantage à l'échelle d'un portefeuille (étape P.7).

```

rel_f = np.exp(mod_f.params).round(3)
rel_f = rel_f[rel_f.index != "Intercept"]
print(rel_f.to_string())

```

C(classe_age)[T.25-34]	0.559
C(classe_age)[T.35-49]	0.516
C(classe_age)[T.50-64]	0.554
C(classe_age)[T.65-70]	0.578
C(classe_age)[T.71+]	0.692
C(zone)[T.Zone B]	1.057
C(zone)[T.Zone C]	1.122
C(zone)[T.Zone D]	1.229
C(zone)[T.Zone E]	1.501
C(zone)[T.Zone F]	1.543
C(classe_bm)[T.81-100]	1.117
C(classe_bm)[T.<=60]	0.912
C(classe_bm)[T.>100]	1.300
C(usage)[T.professionnel]	1.152
C(carburant)[T.essence]	0.968

C(carburant)[T.hybride_electrique]	0.904
C(classe_veh)[T.11+]	0.790
C(classe_veh)[T.3-5]	0.922
C(classe_veh)[T.6-10]	0.863
puissance	1.058

Lecture. Les coefficients sont des **multiplicateurs** de la fréquence à modalité de référence (18-24 ans, zone A, bonus-malus de 61 à 80, usage privé, véhicule neuf). Un conducteur de 35 à 49 ans a une fréquence d'environ la moitié (0,52) de celle d'un jeune ; la zone F pèse 1,54 contre 1 pour la zone A ; un usage professionnel ajoute 15 %. La vérité programmée (docstring de `build/donnees5.py`) donnait un rapport de 1,73 entre jeunes et adultes (soit 0,58 dans ce sens) et de 1,65 entre les zones F et A : le modèle retrouve l'ordre de grandeur, avec l'écart d'estimation que l'on attend sur 63 000 lignes.

P.4 Étape 3 : la sévérité et les gros sinistres

Un sinistre coûte en moyenne quelques milliers d'euros, mais certains sinistres corporels coûtent plusieurs centaines de milliers : la moyenne est **tirée par la queue**. On sépare donc le coût en deux : une partie **écrêtée** à 100 000 €, que l'on modélise, et la partie **au-delà**, que l'on mutualise par un **coefficient de gros sinistres** unique (livre, 2.1 et 6.2). Regardons d'abord comment la queue se comporte d'une année à l'autre.

```
SEUIL = 100_000
sin["cout_ecrete"] = sin["montant"].clip(upper=SEUIL)
sin["cout_exces"] = sin["montant"] - sin["cout_ecrete"]
expo_an = pol.groupby("annee")["exposition"].sum()
par_an = sin.groupby("annee").agg(sinistres=("montant", "size"), gros=("montant", lambda m: int((m > SEUIL).sum())), exces=("cout_exces", "sum"))
par_an["exces_par_annee_police"] = (par_an["exces"] / expo_an).round(1)
print(par_an[["sinistres", "gros", "exces_par_annee_police"]])
```

	sinistres	gros	exces_par_annee_police
annee			
2022	1679	28	187.4
2023	1907	29	240.4
2024	2136	28	84.8

Le **nombre** de gros sinistres est stable, mais leur **coût total** varie du simple au triple : la queue est volatile, et c'est ce qui rend la sévérité difficile à estimer.

On ramène aussi les montants à un **niveau de prix commun** : les coûts augmentent avec le temps, donc un sinistre de 2022 coûte « moins cher » qu'un sinistre de 2024. Peut-on estimer cette **tendance** sur les années d'ajustement ? Essayons, sans toucher à 2024.

```
sin_aj = sin[sin["annee"] <= 2023]
tendance = smf.glm("cout_ecrete ~ annee", data=sin_aj,
    <- family=sm.families.Gamma(sm.families.links.Log())).fit()
ic = np.exp(tendance.conf_int().loc["annee"])
print("tendance estimée :", round(float(np.exp(tendance.params["annee"])) - 1, 3), "| intervalle à 95 % : [", round(ic[0] - 1, 3), ";", round(ic[1] - 1, 3), "]\n")
infl = 1.04 # hypothèse retenue : indice externe du coût des sinistres (la vérité programmée vaut aussi 0,04)
sin["cout_2024"] = sin["cout_ecrete"] * infl ** (2024 - sin["annee"])
sin["exces_2024"] = sin["cout_exces"] * infl ** (2024 - sin["annee"])
sin_aj = sin[sin["annee"] <= 2023]
```

```
tendance estimée : -0.092 | intervalle à 95 % : [ -0.239 ; 0.085 ]
```

Deux années de sinistres bruités ne suffisent pas à mesurer une tendance : l'**intervalle est large** et

contient des valeurs absurdes. On retient donc un **indice externe** (hypothèse écrite, à documenter dans le dossier), ce que ferait un actuair.

Reste à choisir la **complexité du modèle de sévérité**. On compare trois candidats par la déviance Gamma, en ajustant sur 2022 et en jugeant sur 2023 (le test de 2024 reste intact).

```
def deviance_gamma(y, mu):
    y, mu = np.asarray(y, float), np.asarray(mu, float)
    return 2 * (-np.log(y / mu) + (y - mu) / mu).sum()

candidats = {"constante": "cout_2024 ~ 1", "zone + puissance": "cout_2024 ~ C(zone) + puissance",
             "complet": "cout_2024 ~ C(classe_age) + C(zone) + puissance + C(usage) + C(classe_veh)"}
a22, a23 = sin[sin["annee"] == 2022], sin[sin["annee"] == 2023]
devs = {}
for nom, f in candidats.items():
    m = smf.glm(f, data=a22, family=sm.families.Gamma(sm.families.links.Log())).fit()
    devs[nom] = round(deviance_gamma(a23["cout_2024"], m.predict(a23)))
print(devs)
# règle de parcimonie : on ne complique que si la déviance baisse d'au moins 1 %
choix = next(nom for nom in candidats if devs[nom] <= 1.01 * min(devs.values()))
mod_s = smf.glm(candidats[choix], data=sin_aj,
               family=sm.families.Gamma(sm.families.links.Log())).fit()
print("modèle de sévérité retenu :", choix, "| sévérité écrêtée moyenne (prix 2024) :",
      round(mod_s.predict(sin_aj).mean()))
```

```
{'constante': 2837, 'zone + puissance': 2834, 'complet': 2815}
modèle de sévérité retenu : constante | sévérité écrêtée moyenne (prix 2024) : 5538
```

Les trois déviances diffèrent de moins de 1 % : aucune complication ne **se démarque du bruit**. La règle de parcimonie retient la **constante** : la sévérité est **mutualisée**, et la segmentation du tarif repose sur la fréquence.

Attention. Avec quelques milliers de sinistres, un modèle de sévérité riche **ne bat pas toujours une moyenne constante** : les coefficients estimés sont du bruit, et ce bruit se paie hors période. En pratique, beaucoup de tarifs mutualisent la sévérité et segmentent surtout la **fréquence**, qui s'estime bien mieux.

P.5 Étape 4 : la prime pure et sa validation hors période

La **prime pure** d'une police est l'espérance de sa charge annuelle : fréquence prévue × sévérité écrêtée prévue × (1 + coefficient de gros sinistres). Le coefficient est le rapport de l'excédent à la charge écrêtée sur les années d'ajustement, aux prix de 2024. On valide d'abord la partie **écrêtée**, qui porte l'essentiel du signal, puis on regarde les gros sinistres à part.

```
coef_gros = sin_aj["exces_2024"].sum() / sin_aj["cout_2024"].sum()
sev_test = mod_s.predict(test.assign(cout_2024=1.0))
test["prime_ecretee"] = mu_test.values * sev_test.values
test["prime_pure"] = test["prime_ecretee"] * (1 + coef_gros)
sin24 = sin[sin["annee"] == 2024]
test["charge_ecretee"] =
    test["id_police"].map(sin24.groupby("id_police")["cout_ecrete"].sum()).fillna(0.0)
test["charge_reelle"] =
    test["id_police"].map(sin24.groupby("id_police")["montant"].sum()).fillna(0.0)
print("coefficient de gros sinistres :", round(coef_gros, 3))
print("charge écrêtée par année-police : prévue", round(test["prime_ecretee"].sum() /
    test["exposition"].sum(), 1), "| réelle", round(test["charge_ecretee"].sum() /
    test["exposition"].sum(), 1))
print("charge totale par année-police : prévue", round(test["prime_pure"].sum() /
    test["exposition"].sum(), 1), "| réelle", round(test["charge_reelle"].sum() /
    test["exposition"].sum(), 1))
```

```

coefficient de gros sinistres : 0.624
charge écrêtée par année-police : prévue 364.1 | réelle 342.7
charge totale par année-police : prévue 591.3 | réelle 427.5

```

Lecture. Sur la partie écrêtée, la prévision est proche du réel : 364 € prévus contre 343 € par année-police, soit un écart de 6 %. La charge **totale** est beaucoup plus éloignée (591 € prévus contre 428 €) pour une seule raison : en 2024, les gros sinistres ont coûté 85 € par année-police alors que les deux années précédentes en avaient coûté 187 et 240. Ce n'est pas une erreur de modèle, c'est la **volatilité de la queue** : un seul exercice de validation ne permet pas de juger le coefficient de gros sinistres.

On juge ensuite le **classement** de la partie écrêtée : on trie les polices par prime pure, on regroupe en déciles d'exposition et on compare la charge réelle à la charge prévue (courbe de lift), puis on résume par un **coefficient de Gini** de la courbe de Lorenz ordonnée (livre, 2.2).

```

def table_deciles(df, score, reel, k=10):
    d = df.sort_values(score).copy()
    d["decile"] = np.minimum((d["exposition"].cumsum() / d["exposition"].sum() * k).astype(int), k -
↪ 1) + 1
    t = d.groupby("decile").agg(expo=("exposition", "sum"), prevue=(score, "sum"), reel=(reel,
↪ "sum"))
    return t.assign(ratio=t["reelle"] / t["prevue"])

def gini_lorenz(df, score, reel):
    d = df.sort_values(score)
    x = np.r_[0, (d["exposition"].cumsum() / d["exposition"].sum()).values]
    y = np.r_[0, (d[reel].cumsum() / d[reel].sum()).values]
    return 1 - 2 * np.trapezoid(y, x)

dec = table_deciles(test, "prime_ecretee", "charge_ecretee")
print(dec["ratio"].round(2).to_dict())
print("Gini de la prime écrêtée (2024) :", round(gini_lorenz(test, "prime_ecretee",
↪ "charge_ecretee"), 3))

```

```

{1: 1.13, 2: 0.97, 3: 1.15, 4: 0.89, 5: 1.0, 6: 0.76, 7: 0.77, 8: 0.85, 9: 1.1, 10: 0.89}
Gini de la prime écrêtée (2024) : 0.123

```

Lecture. Le Gini de 0,12 est modeste, et c'est normal : une charge de sinistres est dominée par le **hasard** (94 % des polices n'ont aucun sinistre dans l'année) : le signal à classer est faible devant le bruit. Les rapports réel/prévu par décile s'écartent de 1 de moins de 25 % ; ces écarts sont **du même ordre** que le bruit simulé ci-dessous.

Un décile est un petit échantillon (≈ 200 sinistres) : le rapport réel/prévu y fluctue de **plus ou moins 20 %** autour de 1 par pur hasard. Pour savoir si l'écart est du bruit, on le **simule** : on rejoue 300 fois la charge de chaque décile avec les coûts observés et la fréquence prévue (livre, 2.2).

```

rng0 = np.random.default_rng(7)
couts_ec = sin_aj["cout_2024"].values
sim = []
for k in range(1, 11):
    lam_k = dec.loc[k, "prevue"] / couts_ec.mean() # nombre moyen de sinistres du décile
    ch_k = np.array([rng0.choice(couts_ec, rng0.poisson(lam_k)).sum() for _ in range(300)])
    sim.append(ch_k.std() / ch_k.mean())
print("écart relatif typique dû au hasard, par décile :", np.round(sim, 2))

```

```

écart relatif typique dû au hasard, par décile : [0.22 0.23 0.21 0.23 0.21 0.19 0.2 0.19 0.17 0.15]

```

Le bruit typique d'un décile va de **15 à 23 %** : les écarts observés au tableau précédent sont donc **compatibles avec le hasard**. Un tarif ne se juge pas décile par décile sur une année ; on regarde

le classement global (Gini), la calibration d'ensemble et la tendance sur plusieurs années.

On compare enfin à une **référence plus flexible**, un boosting de gradient à perte de Poisson sur la seule fréquence (volume III, section 2.4), pour vérifier que le GLM ne laisse pas de signal sur la table.

```
import lightgbm as lgb
vars_m = ["age_conducteur", "age_vehicule", "puissance", "bonus_malus", "anciennete_permis"]
Xa = pd.concat([ajust[vars_m], pd.get_dummies(ajust[["zone", "usage", "carburant"]], dtype=float)],
               ↪ axis=1)
Xt = pd.concat([test[vars_m], pd.get_dummies(test[["zone", "usage", "carburant"]], dtype=float)],
               ↪ axis=1)
gbm = lgb.LGBMRegressor(objective="poisson", n_estimators=150, learning_rate=0.03, num_leaves=6,
                       ↪ min_child_samples=300,
                           subsample=0.8, subsample_freq=1, random_state=0, verbose=-1)
gbm.fit(Xa, ajust["nb_sinistres"] / ajust["exposition"], sample_weight=ajust["exposition"])
mu_gbm = gbm.predict(Xt) * test["exposition"].values
print("déviante test Poisson : GLM", round(deviance_poisson(test["nb_sinistres"], mu_test)), "|
     ↪ boosting", round(deviance_poisson(test["nb_sinistres"], mu_gbm)))
test = test.assign(freq_glm=mu_test.values, freq_gbm=mu_gbm)
print("Gini de la fréquence (2024) : GLM", round(gini_lorenz(test, "freq_glm", "nb_sinistres"), 3),
     ↪ "| boosting", round(gini_lorenz(test, "freq_gbm", "nb_sinistres"), 3))
```

```
déviante test Poisson : GLM 12056 | boosting 12073
Gini de la fréquence (2024) : GLM 0.125 | boosting 0.113
```

Lecture. Le boosting ne fait pas mieux que le GLM (déviante 12 073 contre 12 056, Gini 0,113 contre 0,125) : il n'y a **pas de signal oublié** par le tarif, et le GLM garde l'avantage de la **lisibilité**. C'est le résultat attendu ici : la vérité programmée est log-linéaire.

P.6 Étape 5 : du modèle au tarif

La prime commerciale ajoute à la prime pure les **frais** (acquisition, gestion) et une **marge**. Avec 25 % de frais et 3 % de marge, calculés sur la prime commerciale : $\text{prime} = \frac{\text{prime pure}}{1 - 0,25 - 0,03}$, soit un ratio sinistres/primes visé de 72 % (livre, 2.2). On vérifie ce ratio sur 2024, **hors gros sinistres** puis **avec**, et on regarde la **dispersion** des primes (une prime trop disparate fait fuir les bons risques).

```
FRAIS, MARGE = 0.25, 0.03
test["prime"] = test["prime_pure"] / (1 - FRAIS - MARGE)
sp_ecrete = test["charge_ecrete"].sum() / (test["prime_ecrete"].sum() / (1 - FRAIS - MARGE))
sp_total = test["charge_reelle"].sum() / test["prime"].sum()
print("ratio sinistres/primes visé : 0.72 | réalisé 2024 hors gros sinistres :", round(sp_ecrete, 3),
     ↪ "| avec gros sinistres :", round(sp_total, 3))
ap = test["prime"] / test["exposition"]
print("prime annualisée : min", round(ap.min()), "| médiane", round(ap.median()), "| max",
     ↪ round(ap.max()), "| rapport max/min", round(ap.max() / ap.min(), 1))
```

```
ratio sinistres/primes visé : 0.72 | réalisé 2024 hors gros sinistres : 0.678 | avec gros sinistres :
     ↪ 0.521
prime annualisée : min 351 | médiane 747 | max 3289 | rapport max/min 9.4
```

Lecture. Hors gros sinistres, le ratio réalisé (0,68) est proche de la cible de 0,72 (l'écart reflète les 6 % de la partie écrêtée). Avec les gros sinistres, il tombe à 0,52, pour la raison déjà vue : 2024 a été une bonne année pour la queue. Les primes annualisées vont de 351 € à 3 289 € (médiane 747 €) : un rapport de 9,4 entre la plus chère et la moins chère, un écart que la direction commerciale devra juger acceptable.

Un tarif se publie en **coefficients arrondis** : chaque modalité reçoit un multiplicateur par rapport

à la modalité de référence, arrondi à deux décimales, puis on rebalance la prime de base pour que le **total des primes reste inchangé**.

```
rel = np.exp(mod_f.params).drop("Intercept")
rel_arrondi = rel.round(2)
print("coefficients (extraits) :", rel_arrondi.loc[[i for i in rel_arrondi.index if "Zone F" in i or
↪ "professionnel" in i]].to_dict())
print("écart maximal dû à l'arrondi :", round((rel_arrondi / rel - 1).abs().max(), 3))
```

```
coefficients (extraits) : {'C(zone)[T.Zone F]': 1.54, 'C(usage)[T.professionnel]': 1.15}
écart maximal dû à l'arrondi : 0.009
```

Attention. Arrondir est un acte de **tarification** : un écart de 0,5 point sur un coefficient déplace la prime de milliers de clients. On le mesure (ici moins de 1 %) et l'on rebalance ; on ne l'ignore pas.

P.7 Étape 6 : robustesse et capital

Une prime **moyenne** suffisante ne dit pas ce qui arrive **une mauvaise année**. On simule la charge annuelle du portefeuille 2025 (à même exposition qu'en 2024) : le nombre de sinistres est **binomial négatif**, dont la variance additionne trois sources (le hasard de Poisson, l'hétérogénéité non observée entre polices — variance 0,4, vérité programmée — et un **choc commun** de 5 % qui touche toutes les polices à la fois, hypothèse), le coût de chaque sinistre est tiré dans les coûts observés remis au **niveau de prix 2025** (livre, 2.1 ; modèle collectif). On compare deux situations : **sans réassurance**, et avec un **traité en excédent de sinistre** qui prend en charge, pour chaque sinistre, la part au-delà de 250 000 € contre une prime égale à 135 % de l'espérance cédée (chargement d'exemple, livre 6.2 et 6.3).

```
rng = np.random.default_rng(2025)
lam = test["freq_glm"].sum()                                # nombre moyen de sinistres 2025 (même
↪ exposition)
theta, CHOC_COMMUN, PRIORITE, CHARG_REASS = 0.4, 0.05, 250_000, 1.35
var_nb = lam + theta * (test["freq_glm"] ** 2).sum() + (CHOC_COMMUN * lam) ** 2
print("nombre de sinistres 2025 : moyenne", round(lam), "| écart-type", round(var_nb ** 0.5), "| dont
↪ Poisson seul", round(lam ** 0.5))
prime_totale = test["prime"].sum() * infl                  # prime 2025 = tarif 2024 indexé :
↪ hypothèse simple

def simuler(freq_mult=1.0, infl_extra=0.0, n_sim=3000, seed=11):
    r = np.random.default_rng(seed)
    m_, v_ = lam * freq_mult, lam * freq_mult + theta * (test["freq_glm"] ** 2).sum() * freq_mult ** 2
    ↪ + (CHOC_COMMUN * lam * freq_mult) ** 2
    nb = r.negative_binomial(m_ ** 2 / (v_ - m_), m_ / v_, n_sim)
    cs = sin["montant"].values * (infl + infl_extra) ** (2025 - sin["annee"].values)
    brut = np.empty(n_sim); net = np.empty(n_sim)
    for i, k in enumerate(nb):
        c = r.choice(cs, k)
        brut[i], net[i] = c.sum(), np.minimum(c, PRIORITE).sum()
    cede_moy = lam * freq_mult * np.maximum(cs - PRIORITE, 0).mean()
    return brut, net + CHARG_REASS * cede_moy                # charge nette = gardée + prime de
    ↪ réassurance

brut, net = simuler()
for nom, ch in [("sans réassurance", brut), ("avec excédent de sinistre", net)]:
    print(f"{nom:27s} moyenne {ch.mean():1e6:5.2f} M€ | quantile 99,5 % {np.quantile(ch,
    ↪ 0.995):1e6:5.2f} M€ | capital (99,5 % - moyenne) {(np.quantile(ch, 0.995) -
    ↪ ch.mean()):1e6:5.2f} M€")
```

nombre de sinistres 2025 : moyenne 2109 | écart-type 115 | dont Poisson seul 46
 sans réassurance moyenne 17.57 M€ | quantile 99,5 % 25.60 M€ | capital (99,5 % - moyenne)
 ↪ 8.03 M€
 avec excédent de sinistre moyenne 18.79 M€ | quantile 99,5 % 22.29 M€ | capital (99,5 % - moyenne)
 ↪ 3.50 M€

Lecture. Au quantile 99,5 %, la charge annuelle est d'environ 25,6 M€ pour une moyenne de 17,6 M€ : le **capital de risque** (différence) est de 8,0 M€ sans réassurance et de 3,5 M€ avec le traité en excédent de sinistre, qui coûte environ 1,2 M€ de plus en moyenne (la prime de réassurance, chargée à 135 %). Le traité **stabilise** : il réduit le capital de plus de moitié pour un surcoût moyen d'environ 7 %.

Puis on **dégrade** les hypothèses : inflation de la sévérité plus forte de 6 points, fréquence plus élevée de 15 %. On compare les ratios sinistres/primes (charge nette de réassurance) obtenus, en moyenne et au quantile 99,5 %.

```
for nom, fm, ie in [("central", 1.0, 0.0), ("inflation +6 pts", 1.0, 0.06), ("fréquence +15 %", 1.15, 0.0), ("les deux", 1.15, 0.06)]:
    b_, n_ = simuler(fm, ie)
    print(f"{nom:18s} S/P net moyen {n_.mean()/prime_totale:.3f} | S/P net au quantile 99,5 %
          ↪ {np.quantile(n_, 0.995)/prime_totale:.3f} | sans réassurance : {np.quantile(b_,
          ↪ 0.995)/prime_totale:.3f}")
```

central	S/P net moyen 0.686 S/P net au quantile 99,5 % 0.813 sans réassurance : 0.934
inflation +6 pts	S/P net moyen 0.775 S/P net au quantile 99,5 % 0.910 sans réassurance : 1.050
fréquence +15 %	S/P net moyen 0.788 S/P net au quantile 99,5 % 0.936 sans réassurance : 1.063
les deux	S/P net moyen 0.891 S/P net au quantile 99,5 % 1.049 sans réassurance : 1.194

Lecture. Dans le scénario central, la charge nette moyenne représente 69 % des primes et 81 % au quantile 99,5 %. Si l'inflation est plus forte de 6 points **et** la fréquence plus élevée de 15 %, le ratio moyen monte à 89 % et dépasse 100 % au quantile 99,5 % (105 % avec le traité, 119 % sans) : la mutuelle perd de l'argent les mauvaises années. Ces scénarios ne sont **pas des prévisions** : ils montrent quelles hypothèses pèsent le plus, ici la fréquence (+15 %) un peu plus que l'inflation (+6 points).

Attention. Le capital de la queue dépend **entièrement** de l'ajustement de la queue. Avec une trentaine de gros sinistres par an et une loi à variance quasi infinie, la mauvaise année simulée ne reflète que les gros sinistres observés ; un autre échantillon donnerait un autre quantile. C'est ce que la réassurance achète : de la **stabilité**, et la possibilité de **borner** ce que l'on ne sait pas mesurer.

P.8 Étape 7 : les notes réglementaires

Le modèle n'est pas fini quand l'ajustement est fait : il faut pouvoir **le défendre**. Cette étape produit trois pièces. Les deux premières se **vérifient par le calcul**.

Variables autorisées et proxies. Aucune variable protégée n'est dans le modèle, mais une variable permise peut en être le **substitut** (un proxy). On mesure la dépendance de chaque variable de tarification avec l'âge, que beaucoup de textes encadrent, pour savoir ce que le tarif transmet indirectement (volume III, section 5.4).

```
cor_age = pol[["age_conducteur", "age_vehicule", "puissance", "bonus_malus",
↪ "anciennete_peris"]].corr()["age_conducteur"].drop("age_conducteur").round(2)
print(cor_age.to_string())
```

```
age_vehicule      0.00
puissance         0.00
bonus_malus      -0.26
anciennete_peris  0.99
```

Lecture. L'**ancienneté du permis** est corrélée à 0,99 avec l'âge : c'est un **substitut presque parfait**. Elle est exclue du tarif retenu (qui contient déjà l'âge) ; l'ajouter à un modèle qui n'aurait pas eu l'âge reviendrait à faire entrer l'âge **par la fenêtre**. La corrélation du bonus-malus avec l'âge (−0,26) est modérée : cette variable est **comportementale** (elle résume le passé de sinistres) mais elle transmet aussi, en partie, l'âge.

Stabilité du portefeuille. On compare la distribution des classes d'âge de 2024 à celle de l'ajustement par le **PSI** (volume IV, section 4.7) : un portefeuille qui change de forme invalide les coefficients.

```
def psi(attendu, actuel, eps=1e-4):
    p = np.clip(attendu / attendu.sum(), eps, None); q = np.clip(actuel / actuel.sum(), eps, None)
    return float(((q - p) * np.log(q / p)).sum())

for col in ["classe_age", "zone", "classe_bm"]:
    a = ajust.groupby(col)["exposition"].sum(); t =
↪ test.groupby(col)["exposition"].sum().reindex(a.index)
    print(f"PSI {col:11s}", round(psi(a.values, t.values), 4))
```

```
PSI classe_age  0.0004
PSI zone       0.0001
PSI classe_bm   0.0
```

Lecture. Les PSI sont presque nuls ($\ll 0,10$) : le portefeuille de 2024 a la même forme que celui de l'ajustement. C'est trop beau pour être vrai en réalité : ici la simulation tire chaque année dans la même population. Un PSI proche de 0,25 serait le signal qu'il faut reprendre l'étude (volume IV, section 4.7).

La fiche de validation. Un second regard, indépendant de l'équipe qui a construit le modèle, répond à une grille. Voici la nôtre, à remplir par les chiffres des étapes précédentes (livre, 3.4 et 4.2).

Question	Réponse tirée du projet
Les données sont-elles complètes et cohérentes ?	P.2 : fichiers cohérents, expositions valides
Le modèle a-t-il été jugé hors période ?	P.3 et P.5 : ajusté sur 2022–2023, jugé sur 2024
Surpasse-t-il une référence simple ?	P.3 : déviance de test plus basse que la fréquence constante
Un modèle plus flexible fait-il mieux ?	P.5 : comparaison au boosting
Le tarif est-il équilibré ?	P.6 : ratio S/P visé et réalisé

Question	Réponse tirée du projet
Que se passe-t-il en cas de stress ?	P.7 : quatre scénarios
Les variables sont-elles défendables ?	P.8 : pas de variable protégée ; corrélations avec l'âge
Qui est responsable, et quand revalide-t-on ?	à décider : propriétaire du modèle, revue annuelle, seuils de dérive

Rappel. Ces notes illustrent une **démarche** ; elles ne sont pas un avis juridique, comptable ou actuariel. Les exigences exactes dépendent du pays et du texte en vigueur : à faire vérifier par les fonctions compétentes.

P.9 Étape 8 : le rapport

On regroupe les critères en un **tableau de décision**, chaque critère étant fixé **avant** de regarder les résultats.

```
criteres = {
    "déviante de test inférieure à la référence constante": deviance_poisson(test["nb_sinistres"],
    ↪ mu_test) < deviance_poisson(test["nb_sinistres"], mu_ref),
    "sinistres prévus en 2024 à ±3 % des observés": abs(mu_test.sum() / test["nb_sinistres"].sum() -
    ↪ 1) < 0.03,
    "Gini de la prime écartée > 0,10": gini_lorenz(test, "prime_ecretee", "charge_ecretee") > 0.10,
    "ratio S/P 2024 hors gros sinistres dans [0,65 ; 0,80]": 0.65 <= sp_ecrete <= 0.80,
    "ratio S/P 2024 avec gros sinistres inférieur à 0,90": sp_total < 0.90,
    "ratio S/P net au quantile 99,5 % (scénario central) < 1,30": np.quantile(net, 0.995) /
    ↪ prime_totale < 1.30,
    "PSI de chaque variable < 0,10": all(psi(ajust.groupby(c)["exposition"].sum().values,
    ↪ test.groupby(c)["exposition"].sum().reindex(ajust.groupby(c)["exposition"].sum_
    ↪ ().index).values) < 0.10 for c in ["classe_age", "zone", "classe_bm"]),
}
for k, v in criteres.items():
    print("OK " if v else "KO ", k)
print("\nDécision :", "GO pour le tarif 2025" if all(criteres.values()) else "NO GO : revoir le tarif")
```

```
OK déviante de test inférieure à la référence constante
OK sinistres prévus en 2024 à ±3 % des observés
OK Gini de la prime écartée > 0,10
OK ratio S/P 2024 hors gros sinistres dans [0,65 ; 0,80]
OK ratio S/P 2024 avec gros sinistres inférieur à 0,90
OK ratio S/P net au quantile 99,5 % (scénario central) < 1,30
OK PSI de chaque variable < 0,10
```

Décision : GO pour le tarif 2025

Lecture. Les sept critères sont satisfaits : décision **GO**. Notez ce que la décision ne dit pas : elle ne dit pas que le tarif est « juste », elle dit qu'il satisfait des critères fixés **à l'avance**. Notez aussi que le ratio 2024 « avec gros sinistres » (0,52) est bas **par chance** (la queue a été clémentine), et que le critère de la queue est satisfait même sans traité (0,93), mais avec un capital plus de deux fois plus élevé (8,0 contre 3,5 M€).

Ce que dit la vérité programmée. Fréquence : le modèle a retrouvé les effets d'âge, de zone, d'usage, de bonus-malus (à l'incertitude près) ; sévérité : les vrais effets (puissance, zone) sont minuscules, ce qui justifie la sévérité mutualisée ; inflation : 4 % par an, que deux années de données ne permettaient pas d'estimer ; queue : le coefficient de gros sinistres est volatil parce que la queue de Pareto simulée a une variance quasi infinie. Dans une étude réelle, on ne dispose pas de cette vérité : seuls les contrôles hors période et les scénarios nous protègent.

À retenir. Un tarif n'est pas une prédiction : c'est une **décision** (un prix) prise sous incertitude et défendue par un dossier. La qualité d'un modèle de tarification se juge sur trois plans à la fois : **classer** les risques, **équilibrer** le total, **résister** au mauvais scénario.

P.10 Les limites de l'étude

- **Données simulées.** La fréquence, la sévérité et l'inflation obéissent à des lois simples ; la réalité apporte des ruptures (sinistres de masse, changements de comportement, fraude) que ce jeu ne contient pas.
- **Une seule branche.** Aucun lien avec les autres produits de la mutuelle, la réassurance (chapitre 6) ou la solvabilité complète (4.2) : le « capital de risque » de P.7 est un indicateur, pas un SCR.
- **Sévérité peu précise.** Quelques milliers de sinistres, des gros sinistres rares : l'écèlement à 100 000 € est un choix que l'on devrait revalider chaque année.
- **Inflation supposée constante.** L'indexation de P.7 est une hypothèse, pas une prévision.
- **Équité non étudiée en détail.** On a vérifié l'absence de variable protégée et mesuré des corrélations ; un audit d'équité complet demande des données sur les groupes concernés (volume III, section 5.4).

P.11 Variante : un score de crédit

La même démarche s'applique à un **score** de crédit (chapitre 1). Avec `donnees/credits_conso.csv` (prêts à la souscription, défaut à 12 mois), voici une version courte : découper, ajuster une logistique, juger par le **Gini** et le **KS**, puis vérifier la **calibration**.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, roc_curve

cr = pd.read_csv("donnees/credits_conso.csv")
cr["revenu_manquant"] = cr["revenu_annuel"].isna().astype(int)
cr["emploi_manquant"] = cr["anciennete_emploi"].isna().astype(int)
cr = cr.fillna({"revenu_annuel": cr["revenu_annuel"].median(), "anciennete_emploi":
    ↳ cr["anciennete_emploi"].median()})
cr["age_u"] = (cr["age"] - 47) * 2 / 100
cr["dti_coude"] = np.maximum(0, cr["taux_endettement"] - 0.5)
X = pd.concat([cr.drop(columns=["id_credit", "default_12m", "logement", "objet"]),
    ↳ pd.get_dummies(cr[["logement", "objet"]], drop_first=True, dtype=float), axis=1])
Xa, Xt, ya, yt = train_test_split(X, cr["default_12m"], test_size=0.3, random_state=0,
    ↳ stratify=cr["default_12m"])
mu_, sd_ = Xa.mean(), Xa.std()
lr = LogisticRegression(max_iter=2000).fit((Xa - mu_) / sd_, ya)
p = lr.predict_proba((Xt - mu_) / sd_)[:, 1]
fpr, tpr, _ = roc_curve(yt, p)
print("AUC :", round(roc_auc_score(yt, p), 3), "| Gini :", round(2 * roc_auc_score(yt, p) - 1, 3), "|
    ↳ KS :", round((tpr - fpr).max(), 3))
print("défaut observé :", round(yt.mean(), 4), "| défaut prévu moyen :", round(p.mean(), 4))
```

```
AUC : 0.782 | Gini : 0.563 | KS : 0.421
défaut observé : 0.0597 | défaut prévu moyen : 0.0596
```

Le **Gini** (0,56) et le **KS** (0,42) mesurent le classement ; l'égalité du taux de défaut prévu (5,96 %) et observé (5,97 %) mesure la calibration d'ensemble. Ajouter à la logistique les deux transformations (`age_u`, `dti_coude`) que la vérité programmée contenait est un acte de **connaissance du métier** : sans elles, la logistique linéaire passe à côté de l'âge en U et du coude de l'endettement (chapitre 1, 1.1 et 1.2).

Auto-évaluation

Mode d'emploi. Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections à relire sont indiquées dans le corrigé. Trente-cinq bonnes réponses sur quarante signalent un volume bien assimilé ; les questions des chapitres facultatifs (⊕ sections 1.4 à 1.6, 2.4 à 2.6, 3.3, 3.4, 4.4 à 4.6, chapitres 5 à 7) comptent si vous les avez lues. Les paramètres des exemples sont **illustratifs**.

Risque de crédit (chapitre 1)

1. Une AUC de 0,78 correspond à quel coefficient de Gini ? Que mesure-t-il, et que ne mesure-t-il pas ?
2. Dans une base de 9 400 bons payeurs et 600 mauvais payeurs, une classe contient 300 bons et 50 mauvais. Quel est son poids de l'évidence (WOE), et que dit son signe ?
3. Une grille est calée à 600 points pour une cote de 50 contre 1, avec 20 points pour doubler la cote. Combien de points pour une cote de 100 contre 1 ? De 200 contre 1 ?
4. Pourquoi une grille construite par classes bat-elle parfois une régression logistique « brute » sur les mêmes variables ?
5. Une perte attendue vaut $PD \times LGD \times EAD$. Calculez-la pour une PD à douze mois de 2 %, une LGD de 45 % et une exposition de 10 000 €, puis (sans actualiser) pour une PD sur la durée de vie de 5 %. À quelle étape IFRS 9 correspond chaque chiffre ?
6. Un modèle prédit 1 % de défaut pour 1 000 prêts ; on en observe 18. Le modèle est-il bien calibré ? Calculez l'écart réduit.
7. Avec la matrice de transition annuelle (3 états : bon, moyen, défaut) dont les lignes sont (0,90 ; 0,08 ; 0,02), (0,10 ; 0,80 ; 0,10) et (0 ; 0 ; 1), quelle est la probabilité de défaut sur deux ans d'un emprunteur « bon » ?
8. Pourquoi une matrice de transition estimée sur dix ans ne décrit-elle pas ce qui arrivera pendant une récession ?

Modélisation actuarielle (chapitre 2)

9. Une mutuelle observe 120 sinistres sur 2 000 années d'exposition. Quelle est la fréquence ? Combien de sinistres attend-on pour une police exposée six mois ?
10. La fréquence moyenne d'une police est de 0,07 par an et l'hétérogénéité non observée est de variance 0,4. Quelle est la variance du nombre de sinistres et le rapport variance sur moyenne ? Que dit ce rapport sur le choix entre Poisson et binomiale négative à l'échelle de la police ?
11. Une fréquence de 0,066 et une sévérité moyenne de 5 500 € : quelle est la prime pure ? Quelle prime commerciale pour un ratio sinistres/primes visé de 72 % ?
12. Dans le triangle cumulé (année 1 : 100, 150, 165 ; année 2 : 110, 168 ; année 3 : 120), quels sont les facteurs de développement du chain ladder et la provision totale ?
13. Pourquoi valide-t-on un tarif sur une période **postérieure** à celle de l'ajustement plutôt que sur un échantillon tiré au hasard ?
14. Un groupe de 500 années d'exposition a une fréquence de 5 % ; la moyenne générale est de 6,6 % et le point de crédibilité vaut $k = 1\,000$ années. Quelle est la fréquence créditée ?
15. Pourquoi écrête-t-on les sinistres avant de modéliser leur sévérité, et que fait-on de la partie écrêtée ?
16. En assurance santé, que sont la sélection adverse et l'aléa moral ? Donnez un symptôme de chacun dans les données.

Mesures de risque et stress tests (chapitre 3)

17. Les pertes quotidiennes d'un portefeuille suivent une loi normale centrée d'écart-type 2 %. Calculez la VaR à 99 % et l'expected shortfall à 99 %.
18. Quelle est la VaR à dix jours par la règle de la racine ? Quand la règle devient-elle fausse ?
19. Pourquoi dit-on que la VaR n'est pas une mesure de risque « cohérente » alors que l'expected shortfall l'est ?
20. Un modèle de VaR à 99 % compte 5 dépassements en 250 jours. Quelle zone du feu tricolore ? Le test de Kupiec rejette-t-il le modèle ? Et avec 10 dépassements ?
21. Qu'est-ce qu'un stress test inversé, et en quoi diffère-t-il d'un stress test classique ?
22. Pourquoi la VaR à 99,9 % d'une perte opérationnelle estimée sur dix ans de données est-elle peu fiable ?

Cadre réglementaire (chapitre 4)

23. Avec la formule IRB des « autres expositions de détail » (corrélation donnée par la formule du texte), calculez le besoin de capital K et le poids de risque pour une PD de 2 % et une LGD de 45 % (quantile 99,9 %).
24. Qu'est-ce qu'un ratio de fonds propres et quel risque la formule IRB a-t-elle pour but de couvrir ?
25. Trois modules de risque ont pour charges 100, 60 et 40 ; leurs corrélations sont de 0,25 entre le premier et chacun des autres et de 0,5 entre les deux derniers. Quelle est la charge agrégée ? Quelles hypothèses cache cette formule ?
26. Les fonds propres éligibles valent 330 M€ et le capital de solvabilité requis 254 M€. Quel est le ratio de solvabilité ? Que se passerait-il en dessous de 100 % ?
27. Dans le Takaful, quels sont les deux fonds, et qui supporte un déficit du fonds des participants ?
28. Dans le modèle wakala, les cotisations de l'année valent 10 M€, la commission de gestion 20 % et les sinistres 6 M€ : quel est l'excédent technique, et à qui revient-il ?
29. Une règle de détection de blanchiment lève 54 alertes dont 38 fausses. Quelle est la précision ? Pourquoi une fenêtre de temps améliore-t-elle la règle ?

Assurance vie (chapitre 5)

30. Avec un taux d'intérêt de 3 % et une rente viagère anticipée $\ddot{a}_x = 18$, quelle est la valeur du capital décès A_x ?
31. Un portefeuille compte 38 décès contre 50 attendus : quel est le rapport réel/attendu, et peut-on conclure que les assurés vivent plus longtemps que la population ?
32. Quelles contraintes d'identification impose-t-on au modèle de Lee-Carter, et pourquoi ?
33. Une baisse de la mortalité est-elle une bonne nouvelle pour un assureur de rentes ? Pour un assureur de capitaux décès ?

Réassurance (chapitre 6)

34. Une tranche de 200 000 € en excédent de 100 000 € s'applique à trois sinistres de 50 000, 150 000 et 400 000 €. Que cède-t-on ?
35. Une cession en quote-part de 30 % porte sur 10 M€ de primes et 7 M€ de sinistres : quelles sont les primes et les sinistres conservés ?
36. Pourquoi le *burning cost* d'une tranche haute sous-estime-t-il souvent le coût réel quand on dispose de peu d'années ?

Actif-passif et portefeuille (chapitre 7)

37. Un zéro-coupon de maturité 5 ans, taux de 5 % : quelles sont sa duration et sa duration modifiée ? De combien varie son prix si les taux montent d'un point ?
38. Un portefeuille est composé à 60 % d'un actif de volatilité 10 % et à 40 % d'un actif de volatilité 20 % ; leur corrélation est de 0,3. Quelle est la volatilité du portefeuille ? Comparez à la moyenne pondérée des volatilités.
39. Pourquoi l'égalité des durations de l'actif et du passif ne suffit-elle pas à immuniser le surplus quand l'actif ne vaut pas le passif ?
40. Pourquoi la diversification d'un portefeuille d'actions décevra-t-elle en période de crise ?

Corrigés des questions

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire.

```
import numpy as np
from scipy.stats import norm, binom, chi2

print("Q1 Gini =", round(2 * 0.78 - 1, 2))
print("Q2 WOE =", round(np.log((300 / 9400) / (50 / 600)), 3))
f = 20 / np.log(2); off = 600 - f * np.log(50)
print("Q3 points pour 100 :", round(off + f * np.log(100), 1), "| pour 200 :", round(off + f *
↳ np.log(200), 1))
print("Q5 ECL 12 mois :", round(0.02 * 0.45 * 10000), "€ | durée de vie :", round(0.05 * 0.45 *
↳ 10000), "€")
mu6, sd6 = 1000 * 0.01, np.sqrt(1000 * 0.01 * 0.99)
print("Q6 écart réduit :", round((18 - mu6) / sd6, 2), "| p unilatérale :", round(binom.sf(17, 1000,
↳ 0.01), 4))
P = np.array([[0.90, 0.08, 0.02], [0.10, 0.80, 0.10], [0, 0, 1.0]])
print("Q7 PD à 2 ans :", round((P @ P)[0, 2], 3))
print("Q9 fréquence :", 120 / 2000, "| six mois :", 0.5 * 120 / 2000)
print("Q10 variance :", round(0.07 + 0.4 * 0.07 ** 2, 5), "| rapport :", round((0.07 + 0.4 * 0.07 **
↳ 2) / 0.07, 3))
print("Q11 prime pure :", 0.066 * 5500, "| commerciale :", round(0.066 * 5500 / 0.72, 1))
f1, f2 = (150 + 168) / (100 + 110), 165 / 150
u = [165, 168 * f2, 120 * f1 * f2]
print("Q12 facteurs :", round(f1, 4), f2, "| provision :", round(u[1] - 168 + u[2] - 120, 1))
Z = 500 / (500 + 1000)
print("Q14 Z =", round(Z, 3), "| fréquence créditée :", round(Z * 0.05 + (1 - Z) * 0.066, 4))
z99 = norm.ppf(0.99)
print("Q17 VaR :", round(z99 * 2, 2), "% | ES :", round(2 * norm.pdf(z99) / 0.01, 2), "%")
print("Q18 VaR à 10 jours :", round(z99 * 2 * np.sqrt(10), 1), "%")
def kupiec(x, n=250, p=0.01):
    ph = x / n
    lr = -2 * ((n - x) * np.log(1 - p) + x * np.log(p) - (n - x) * np.log(1 - ph) - x * np.log(ph))
    return round(float(lr), 2), round(float(chi2.sf(lr, 1)), 4)
print("Q20 Kupiec (LR, p) : 5 dépassements", kupiec(5), "| 10 dépassements", kupiec(10))
pd_, lgd = 0.02, 0.45
w = (1 - np.exp(-35 * pd_)) / (1 - np.exp(-35)); rho = 0.03 * w + 0.16 * (1 - w)
K = lgd * (norm.cdf((norm.ppf(pd_) + np.sqrt(rho) * norm.ppf(0.999)) / np.sqrt(1 - rho)) - pd_)
print("Q23 corrélation :", round(rho, 4), "| K :", round(K, 4), "| poids de risque :", round(K * 12.5,
↳ 3))
c = np.array([100, 60, 40.]); R = np.array([[1, .25, .25], [.25, 1, .5], [.25, .5, 1]])
print("Q25 charge agrégée :", round(float(np.sqrt(c @ R @ c)), 1), "| somme :", c.sum())
print("Q26 ratio :", round(330 / 254 * 100), "%")
print("Q28 excédent :", 10 - 0.2 * 10 - 6, "M€")
print("Q29 précision :", round((54 - 38) / 54, 3))
d = 0.03 / 1.03
print("Q30 A_x =", round(1 - d * 18, 4))
print("Q31 A/E =", round(38 / 50, 2), "| intervalle de Poisson à 95 % :", round(chi2.ppf(0.025, 76) /
↳ 2 / 50, 2), "-", round(chi2.ppf(0.975, 76) / 2 / 50, 2))
```

```

cl = np.array([50e3, 150e3, 400e3])
print("Q34 cédé :", np.clip(cl - 100e3, 0, 200e3).tolist(), "= total", np.clip(cl - 100e3, 0,
↳ 200e3).sum())
print("Q35 primes conservées :", 10 * 0.7, "M€ | sinistres conservés :", round(7 * 0.7, 1), "M€")
print("Q37 duration modifiée :", round(5 / 1.05, 2), "| variation du prix :", round(-5 / 1.05, 2), "%")
wp, sg = np.array([.6, .4]), np.array([.10, .20])
var = (wp ** 2 * sg ** 2).sum() + 2 * wp[0] * wp[1] * 0.3 * sg[0] * sg[1]
print("Q38 volatilité :", round(np.sqrt(var) * 100, 2), "% | moyenne pondérée :", round(wp @ sg * 100,
↳ 1), "%")

```

```

Q1 Gini = 0.56
Q2 WOE = -0.96
Q3 points pour 100 : 620.0 | pour 200 : 640.0
Q5 ECL 12 mois : 90 € | durée de vie : 225 €
Q6 écart réduit : 2.54 | p unilatérale : 0.0138
Q7 PD à 2 ans : 0.046
Q9 fréquence : 0.06 | six mois : 0.03
Q10 variance : 0.07196 | rapport : 1.028
Q11 prime pure : 363.0 | commerciale : 504.2
Q12 facteurs : 1.5143 1.1 | provision : 96.7
Q14 Z = 0.333 | fréquence créditée : 0.0607
Q17 VaR : 4.65 % | ES : 5.33 %
Q18 VaR à 10 jours : 14.7 %
Q20 Kupiec (LR, p) : 5 dépassements (1.96, 0.1619) | 10 dépassements (12.96, 0.0003)
Q23 corrélation : 0.0946 | K : 0.0464 | poids de risque : 0.58
Q25 charge agrégée : 150.3 | somme : 200.0
Q26 ratio : 130 %
Q28 excédent : 2.0 M€
Q29 précision : 0.296
Q30 A_x = 0.4757
Q31 A/E = 0.76 | intervalle de Poisson à 95 % : 0.54 - 1.04
Q34 cédé : [0.0, 50000.0, 200000.0] = total 250000.0
Q35 primes conservées : 7.0 M€ | sinistres conservés : 4.9 M€
Q37 duration modifiée : 4.76 | variation du prix : -4.76 %
Q38 volatilité : 11.35 % | moyenne pondérée : 14.0 %

```

1. $Gini = 2 \times 0,78 - 1 = 0,56$. Il mesure le **pouvoir de classement** : la probabilité qu'un mauvais payeur reçoive un score plus risqué qu'un bon, rééchelonnée entre 0 et 1. Il ne dit rien de la **calibration** (les probabilités prédites sont-elles les bonnes ?) ni de la valeur économique d'un seuil (1.3.2).

2. $WOE = \ln \frac{300/9\,400}{50/600} \approx -0,96$ (code ci-dessus). Un WOE **négatif** signifie que la classe pèse moins parmi les bons (3,2 %) que parmi les mauvais (8,3 %) : elle est **plus risquée** que l'ensemble (1.1.4).

3. 620 points pour 100 contre 1 et 640 pour 200 contre 1 : chaque doublement de la cote vaut un PDO de 20 points (1.1.5).

4. Parce que les effets réels sont **non linéaires** (âge en U, coude de l'endettement) : un découpage en classes les capte, alors qu'une logistique linéaire sur la variable brute impose une pente unique (1.2.3).

5. 90 € à douze mois (**étape 1**), 225 € sur la durée de vie (**étape 2**, après une dégradation sensible du risque depuis l'octroi). Dans l'étape 3 (prêt en défaut), la PD vaut 100 % (1.5.1, 1.5.4).

6. On attend 10 défauts, avec un écart-type de 3,15 ; 18 défauts donnent un écart réduit de 2,54 (probabilité unilatérale d'environ 1,4 %) : le modèle **sous-estime** le risque. À nuancer : le test binomial suppose les défauts indépendants ; un facteur commun (la conjoncture) augmente la variance et rend l'écart moins surprenant (1.3.7).

7. $0,90 \times 0,02 + 0,08 \times 0,10 + 0,02 \times 1 = 0,046$, soit 4,6 % (1.6.4) : le défaut est un état **absorbant**,

on s'y retrouve en passant par « moyen ».

8. Une matrice estimée sur dix ans est une **moyenne** sur des années calmes et agitées. La conjoncture déforme la matrice (les baisses de note sont plus fréquentes en récession) : l'hypothèse d'**homogénéité** dans le temps est fautive, et il faut conditionner par la conjoncture (1.6.3, 1.6.5).

9. $120/2\,000 = 6\%$; $0,5 \times 0,06 = 0,03$ sinistre pour six mois : l'**exposition** multiplie la fréquence (2.1.2).

10. Variance = $0,07 + 0,4 \times 0,07^2 = 0,07196$; rapport 1,028. À l'échelle d'une police, la surdispersion est **discrète** : un Poisson suffit presque, la binomiale négative n'apporte qu'une correction de quelques pour cent (2.1.3).

11. Prime pure = $0,066 \times 5\,500 = 363\text{ €}$; prime commerciale = $363/0,72 \approx 504\text{ €}$ (2.2.1, 2.2.4).

12. $f_1 = (150 + 168)/(100 + 110) \approx 1,514$, $f_2 = 165/150 = 1,1$; charges ultimes 165, 184,8 et 199,9, soit une provision de $0 + 16,8 + 79,9 \approx 96,7$ (2.3.3).

13. Parce qu'un tarif sert à **prédire l'avenir**. Un échantillon tiré au hasard mélange passé et futur, et ignore ce qui change dans le temps (inflation, composition du portefeuille) : il rend le modèle meilleur qu'il ne sera (2.2.5).

14. $Z = 500/(500 + 1\,000) = 1/3$; fréquence créditée = $\frac{1}{3} \times 5\% + \frac{2}{3} \times 6,6\% \approx 6,07\%$: un groupe de taille modeste est ramené vers la moyenne (2.4.4).

15. Quelques sinistres très gros dominent la moyenne et rendent la sévérité **instable**. On modélise la partie **écrêtée** (stable) et l'on **mutualise l'excédent** par un coefficient, ou on le transfère à un traité de réassurance (2.1.5, 6.2).

16. La **sélection adverse** : les assurés qui s'attendent à consommer beaucoup choisissent les garanties riches (dans les données, la part des assurés en affection de longue durée croît avec le niveau). L'**aléa moral** : à état de santé égal, une meilleure couverture fait consommer davantage (le coût moyen croît avec le niveau, même hors ALD) (2.6.3).

17. $\text{VaR}_{99\%} = 2,326 \times 2\% \approx 4,65\%$; $\text{ES}_{99\%} = \sigma \varphi(z)/(1 - \alpha) \approx 5,33\%$ (3.1.3, 3.1.7).

18. $4,65\% \times \sqrt{10} \approx 14,7\%$. La règle suppose des pertes **indépendantes et de même loi** d'un jour à l'autre ; elle échoue quand elles sont autocorrélées ou que la volatilité change (3.1.6).

19. La VaR n'est pas **sous-additive** : on peut construire deux positions dont la VaR de la somme dépasse la somme des VaR (le contre-exemple à deux prêts de 3.1.8). L'expected shortfall l'est toujours, ce qui garantit que fusionner deux portefeuilles ne **crée** pas de risque.

20. 5 dépassements : zone **orange** (le vert va jusqu'à 4) ; Kupiec : $LR = 1,96$, $p \approx 0,16$: on **ne rejette pas**. Avec 10 : zone **rouge**, $LR \approx 12,96$, $p < 0,001$: rejet. Un test sur 250 jours a une faible **puissance** (3.4.2, 3.4.5, 3.4.6).

21. On part d'un **événement inacceptable** (capital sous le minimum, par exemple) et l'on cherche les scénarios qui y mènent, au lieu de prendre un scénario et de mesurer sa perte (3.2.7).

22. Le quantile à 99,9 % d'une perte agrégée dépend de la **queue**, ajustée sur quelques dizaines de grosses pertes : l'indice de queue a un intervalle très large et la VaR varie de plus de 50 % selon la graine du Monte-Carlo (3.3.3).

23. $\rho \approx 9,5\%$, $K \approx 4,6\%$, soit un poids de risque d'environ **58 %** ($K \times 12,5$) (4.1.4, 4.1.5). La formule couvre la **perte inattendue** au quantile 99,9 % à un an ; la perte attendue est provisionnée.

24. Le **ratio de fonds propres** est le rapport des fonds propres aux actifs pondérés par le risque ; il doit rester au-dessus de minima fixés par le cadre (4.1.2). La formule IRB sert à calculer le **capital** qui absorbe les pertes inattendues dans les cas extrêmes (4.1.4).

25. $\sqrt{c^T R c} \approx 150,3$, contre 200 pour la somme : le **bénéfice de diversification** vaut près de 25 %. La formule suppose des dépendances **linéaires et constantes** et n'est exacte que pour des pertes normales ; avec des marges asymétriques ou une dépendance de queue, la charge peut être plus élevée (4.2.4, 4.2.5).
26. $330/254 \approx 130$ %. Sous 100 %, les fonds propres ne couvrent plus le capital requis : le cadre prévoit des **mesures d'intervention** du contrôleur (plan de rétablissement, par exemple) (4.2.6) ; les seuils exacts relèvent du texte applicable.
27. Le **fonds des participants** (cotisations, sinistres) et le **fonds de l'opérateur** (commissions, capital). Un déficit du fonds des participants est comblé par un **prêt sans intérêt** (*qard hassan*) de l'opérateur, remboursé par les excédents futurs (4.3.2, 4.5.4).
28. Excédent technique = $10 - 0,2 \times 10 - 6 = 2$ M€ : la commission de 2 M€ revient à l'opérateur, et l'excédent appartient au **fonds des participants** (distribué ou mis en réserve selon les règles du contrat) (4.5.1, 4.5.2).
29. Précision = $(54 - 38)/54 \approx 0,30$. Le fractionnement est un **comportement rapproché dans le temps** : sans fenêtre, la règle accumule les dépôts de l'année entière et déclenche pour des commerces légitimes ; limitée à 14 jours, elle cible le schéma (4.6.3).
30. $A_x = 1 - d \ddot{a}_x = 1 - 0,0291 \times 18 \approx 0,476$, soit 0,476 € de capital décès par euro assuré (5.2.2).
31. $38/50 = 0,76$, mais l'intervalle de Poisson à 95 % (de 0,54 à 1,04) **contient 1** : avec si peu de décès, on ne peut pas conclure à une surmortalité ou à une sous-mortalité (5.1.5).
32. $\sum_x b_x = 1$ et $\sum_t k_t = 0$: sans ces contraintes, on peut multiplier b_x par une constante et diviser k_t par elle, ou translater k_t en corrigeant a_x (**non-identifiabilité**) (5.3.1).
33. Mauvaise nouvelle pour un assureur de **rentes** (on paie plus longtemps), bonne pour un assureur de **capitaux décès** (on paie plus tard et moins souvent) (5.2.5).
34. Les trois sinistres cèdent 0, 50 000 et 200 000 € : **250 000 €** au total (le troisième épuise la tranche : $400\,000 - 100\,000 > 200\,000$) (6.1.3).
35. On conserve 70 % : **7 M€** de primes et **4,9 M€** de sinistres (la réassurance proportionnelle ne change pas le ratio S/P, hors commissions) (6.1.2).
36. Une tranche haute n'est touchée que par des sinistres **rares** : sur peu d'années, l'expérience en contient peu ou pas, et l'estimation est très instable (une erreur de l'ordre de 30 % sur une tranche haute dans le chapitre) ; on s'appuie alors sur une loi de queue et sur la tarification par l'exposition (6.2.1, 6.2.3, 6.2.4).
37. Un zéro-coupon à 5 ans a une durée de **5 ans**, une durée modifiée de $5/1,05 \approx 4,76$; si les taux montent d'un point, le prix baisse d'environ **4,76 %** (approximation de premier ordre) (7.1.2).
38. $\sigma_p = \sqrt{0,6^2 \times 0,1^2 + 0,4^2 \times 0,2^2 + 2 \times 0,6 \times 0,4 \times 0,3 \times 0,1 \times 0,2} \approx 11,35$ %, contre 14 % pour la moyenne pondérée : c'est la **diversification** (7.2.1).
39. La variation du surplus est $A D_A - L D_L$ (en proportion du choc de taux) : elle s'annule si $D_A A = D_L L$, c'est-à-dire si les **sensibilités en euros** sont égales, pas les durations (7.1.4).
40. En crise, les **corrélations montent** : les actions chutent ensemble, et la diversification promise par des corrélations estimées en période calme disparaît (7.2.7, 3.2.8).

Votre grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer** / **je sais le faire** / **à revoir**.

Compétence	Où la retravailler
Construire une grille de score et lire une carte de score	1.1
Comparer des modèles de défaut et expliquer un refus	1.2
Mesurer un score : Gini, KS, calibration, stabilité	1.3
Calculer un WOE et une IV, regrouper de façon monotone	1.4
Chiffrer une perte attendue (PD, LGD, EAD, étapes IFRS 9)	1.5
Estimer et utiliser une matrice de transition	1.6
Modéliser fréquence et sévérité, gérer la queue	2.1
Construire et valider un tarif	2.2, 2.4
Provisionner par chain ladder et juger l'incertitude	2.3, 2.5
Raisonner sur l'assurance santé	2.6
Calculer et comparer VaR et ES	3.1
Concevoir un stress test	3.2
Chiffrer risque opérationnel, marché, liquidité	3.3
Backtester et valider un modèle	3.4
Appliquer la logique de Bâle (IRB)	4.1
Appliquer la logique de Solvabilité (SCR, solvabilité)	4.2
Expliquer le Takaful et répartir un excédent	4.3, 4.5
Situer IFRS 17 et les réformes de Bâle	4.4
Détecter le blanchiment et la fraude	4.6
Construire une table de mortalité, calculer primes et provisions de vie	5.1, 5.2
Modéliser la mortalité avec Lee-Carter	5.3
Tarifier et choisir un traité de réassurance	6
Gérer l'actif et le passif, optimiser un portefeuille	7
Mener un projet de tarification validé de bout en bout	Projet du volume